



Sérgio Luiz Banin

python3

CONCEITOS E
APLICAÇÕES
UMA ABORDAGEM DIDÁTICA





Sérgio Luiz Banin

python3

CONCEITOS E
APLICAÇÕES
UMA ABORDAGEM DIDÁTICA



Sérgio Luiz Banin

python 3

conceitos e aplicações
uma abordagem didática



Av. das Nações Unidas, 7221, 1º Andar, Setor B
Pinheiros – São Paulo – SP – CEP: 05425-902

SAC 0800-0117875

De 2ª a 6ª, das 8h às 18h

www.editorasaraiva.com.br/contato

Diretoria Executiva Flávia Alves Bravin

Diretoria Editorial Renata Pascual Müller

Gerência Editorial Rita de Cássia S. Pupo

Coordenação Editorial Rosiane Ap. Marinho Botelho

Aquisições Fernando Alves (Coord.)

Rosana Ap. Alves dos Santos

Edição Amanda Cordeiro da Silva

Paula Hercy Cardoso Craveiro

Silvia Campos Ferreira

Produção Editorial Camilla Felix Cianelli Chaves

Fábio Augusto Ramos

Kátia Regina Pereira

Serviços Editoriais Juliana Bojczuk Fermino

Kelli Priscila Pinto

Marília Cordeiro

Serviços de Edição Rosana Arruda da Silva

Preparação Rafael Faber Fernandes

Revisão Saphyra Editorial

Diagramação Book Maker Composição Editorial

Capa M10 Editorial

Impressão e acabamento

**DADOS INTERNACIONAIS DE CATALOGAÇÃO NA PUBLICAÇÃO
(CIP)**

ANGÉLICA ILACQUA CRB-8/7057

Banin, Sérgio Luiz

Python 3 : conceitos e aplicações : uma abordagem
didática / Sérgio Luiz Banin. -- São Paulo : Érica, 2018.

264 p.

Bibliografia

ISBN 978-85-365-2787-1

1. Python (Linguagem de programação de computador)

I. Título

18-0566 CDD-005.133

CDU-004.43

Índices para catálogo sistemático:

1. Python (Linguagem de programação de computador)

Copyright © 2018 Saraiva Educação

Todos os direitos reservados.

1ª edição

2018

Autor e Editora acreditam que todas as informações aqui apresentadas estão

corretas e podem ser utilizadas para qualquer fim legal. Entretanto, não existe qualquer garantia, explícita ou implícita, de que o uso de tais informações conduzirá sempre ao resultado desejado. Os nomes de sites e empresas, porventura mencionados, foram utilizados apenas para ilustrar os exemplos, não tendo vínculo nenhum com o livro, não garantindo a sua existência nem divulgação.

A Ilustração de capa e algumas imagens de miolo foram retiradas de <www.shutterstock.com>, empresa com a qual se mantém contrato ativo na data de publicação do livro. Outras foram obtidas da Coleção MasterClips/MasterPhotos[©] da IMSI, 100 Rowland Way, 3rd floor Novato, CA 94945, USA, e do CorelDRAW X6 e X7, Corel Gallery e Corel Corporation Samples. Corel Corporation e seus licenciadores. Todos os direitos reservados.

Todos os esforços foram feitos para creditar devidamente os detentores dos direitos das imagens utilizadas neste livro. Eventuais omissões de crédito e copyright não são intencionais e serão devidamente solucionadas nas próximas edições, bastando que seus proprietários contatem os editores.

Nenhuma parte desta publicação poderá ser reproduzida por qualquer meio ou forma sem a prévia autorização da Saraiva Educação. A violação dos direitos autorais é crime estabelecido na lei nº 9.610/98 e punido pelo artigo 184 do Código Penal.

CL 642043 CAE 628268

Fabricante

Produto: **Python Software Foundation**

9450 SW Gemini Dr.

ECM# 90772

Beaverton, OR 97008, USA

Site: <www.python.org/psf-landing/>.

Requisitos de Hardware e Software

Hardware Mínimo

Processador 32 bits (x86) de 800 MHz ou 64 bits (x64) de 800 MHz.

1 gigabytes (GB) de memória do sistema.

Disco Rígido com 1 GB livres.

Acesso à internet para baixar o instalador, no caso do ambiente Windows e eventuais atualizações, no caso de Linux e MacOS.

Sistema Operacional

No caso do Sistema Operacional Windows é exigida a versão Vista ou superior para o Python 3.6 em diante. Caso o leitor ainda utilize o Windows XP é possível instalar e usar o Python 3.4.

Quanto ao Sistema Operacional Linux a grande maioria das distribuições Linux existentes hoje já disponibilizam o interpretador Python pré-instalado ou pacotes binários que podem ser facilmente instalados. Para verificar, abra seu terminal e digite: `python -v`

Os computadores da Apple com o Mac OSX também já acompanham um interpretador Python pré-instalado que pode ser atualizado com a última versão disponibilizada no site oficial da linguagem Python na versão 3.6.

Dedicatória

À Adriana, companheira amada.

A Isabela e Murilo, a mais rica obra.

Agradecimentos

Agradeço à Adriana pela paciência e compreensão de todas as horas.

Aos meus pais, Luiz e Neide, que anos a fio se dedicaram ao meu crescimento, oferecendo oportunidades de estudo e aprendizado.

Ao colega professor Alan Carvalho, da Fatec São Caetano do Sul (SP), por ter me proporcionado o primeiro contato com a linguagem Python, e ao colega professor Hamilton Martins Viana, da Fatec São Paulo, que estimulou a adoção da linguagem Python na disciplina de Algoritmos e Lógica de Programação do primeiro semestre do curso de Análise e Desenvolvimento de Sistemas.

Sou muito grato também a todos os meus colegas professores e aos alunos, pelas experiências e vivências nestes mais de 20 anos na docência na área tecnológica.

Sobre o Autor

Sérgio Luiz Banin é tecnólogo em Processamento de Dados pela Fatec São Paulo e engenheiro naval e mestre em Engenharia pela Escola Politécnica da Universidade de São Paulo.

Atua como professor na Fatec São Paulo desde 1994, e como professor na Fatec São Caetano do Sul desde 2007, ambas ligadas ao Centro Estadual de Educação Tecnológica Paula Souza. Ministra aulas de Lógica, Algoritmos e Programação para os cursos de Análise e Desenvolvimento de Sistemas (Fatec SP e Fatec SCS) e no curso de Jogos Digitais (Fatec SCS).

Também atua como consultor e desenvolvedor de sistemas de informação voltado ao mercado de empresas privadas.

CV Lattes: <<http://lattes.cnpq.br/2462495053340379>>

Sumário

1. Python: uma Linguagem de Programação

1.1 Algoritmos e lógica de programação

1.2 A linguagem Python

1.3 Instalação do Python

1.4 Iniciando com o Python

1.5 A quem se destina este livro

1.6 Requisitos mínimos

2. Objetos e Comandos de Entrada e Saída em Python

2.1 Objetos e classes

2.2 Nomes de objetos: identificadores

2.3 Atribuições e expressões aritméticas

2.4 Funções matemáticas

2.5 Comando de exibição – print

2.6 Comando de entrada de dados – input

2.7 Funções de conversão entre tipos simples

2.8 Comentários no código

3. Controle de Fluxo

[3.1 Comando condicional](#)

[3.2 Comando de repetição](#)

[3.3 Tratamento de exceções](#)

4. Tipos Estruturados Sequenciais em Python

[4.1 Strings](#)

[4.2 Listas](#)

[4.3 Tuplas](#)

[4.4 O tipo *range*](#)

[4.5 O comando for](#)

5. Funções

[5.1 Direto ao ponto](#)

[5.2 A importância das funções](#)

[5.3 Definição e uso de funções](#)

[5.4 Recursividade](#)

6. Tipos Estruturados Não Sequenciais

[6.1 Hashable: o que é isso?](#)

[6.2 Conjuntos](#)

[6.3 Dicionários](#)

[7. Arquivos](#)

[7.1 Arquivos – conceitos iniciais](#)

[7.2 Arquivos em Python 3](#)

[8. Python 3 com Banco de Dados SQLite](#)

[8.1 Gerenciadores de bancos de dados](#)

[8.2 Python + SQLite](#)

[9. Projeto 1: Demanda de Mercadorias e Rentabilidade de Vendas](#)

[9.1 O problema](#)

[9.2 A solução – o programa apurador.py](#)

[9.3 A solução – o programa gerador.py](#)

[10. Projeto 2: Controle de Torneios Esportivos](#)

[10.1 Problema](#)

[10.2 A solução](#)

[Bibliografia](#)

Apresentação

Python é uma linguagem de programação de computadores que vem sendo desenvolvida, ampliada e utilizada desde os anos 1990. A comunidade

mundial de colaboradores e usuários de Python é grande, dinâmica e bastante engajada. A linguagem é simples e intuitiva por um lado, poderosa e robusta por outro. Aliar características assim não é nada fácil, e em Python isso foi obtido com grande sucesso e reconhecimento.

Por ser simples e intuitiva, ela atende bem ao propósito de ser uma linguagem inicial utilizada por estudantes de programação que precisam de uma ferramenta para implementar seus primeiros algoritmos.

Por ser poderosa e robusta, além de contar com uma grande e variada gama de bibliotecas aplicáveis a várias áreas da tecnologia da informação, ela pode ser adotada por profissionais de programação que necessitem de uma ferramenta que traga produtividade e qualidade aos projetos de software.

A proposta deste livro é abordar a linguagem de programação Python 3 e suas aplicações, assim como contempla um estudo sobre algoritmos e lógica de programação.

Ao leitor que é um iniciante no mundo da computação, este livro oferece um caminho que parte do básico e segue em um ritmo gradativo de conceitos, conteúdos e desafios.

Ao leitor que já domina outra linguagem e deseja aprender Python, este livro fornece aspectos muito próprios da linguagem de uma maneira clara, mostrando que não se trata de apenas mais uma linguagem, mas, sim, de uma linguagem dotada de grande flexibilidade, poder de processamento, consistência em seus paradigmas e solidez em seu modelo de implementação.

Python é uma linguagem de programação para todos. Talvez por isso tenha uma comunidade muito dinâmica e atuante tanto no Brasil como no mundo todo.

O conteúdo está organizado de modo que não se dependa de outras linguagens nem de recursos externos ao “mundo Python 3”. Os conceitos apresentados, seja na parte de lógica e algoritmos, seja na parte de Python,

contemplam exemplos, exercícios resolvidos, exercícios propostos e, ao final, dois projetos detalhados.

No Capítulo 1 são apresentados um breve histórico do desenvolvimento de Python 3, detalhes sobre a instalação, documentação e o ambiente de programação.

No Capítulo 2 é descrito o modelo de dados de Python e são apresentados os conceitos de objetos (simples e estruturados), as expressões, os operadores e as funções aritméticas. São feitos os primeiros usos do ambiente de programação. São apresentados e utilizados os comandos de entrada e saída de dados.

O Capítulo 3 é dedicado à lógica de programação em conjunto com os comandos de controle de fluxo do programa, a saber: condicional, de repetição e de tratamento de exceções, que são os pilares da construção de qualquer algoritmo. Muitos exemplos e exercícios são utilizados para ilustrar os tópicos do capítulo.

No Capítulo 4 é feito o detalhamento dos tipos estruturados sequenciais de Python 3, que são os tipos string, lista e tupla. Por meio de diversos exemplos e exercícios resolvidos e propostos é mostrada a diversidade de aplicações existentes para esses tipos de dados.

No Capítulo 5 são apresentadas as funções em Python 3 e é tratada a questão da programação modular, tão importante na organização dos programas de computador. Novamente, aqui os conceitos são explicados com muitos exemplos e exercícios.

O Capítulo 6 complementa o Capítulo 4 abordando os tipos estruturados não sequenciais: conjuntos e dicionários. Este último, em particular, é muito poderoso e importante na construção de soluções para problemas que envolvam grandes quantidades de dados.

Nos Capítulos 7 e 8 são tratadas duas formas distintas de persistência de

dados. Esse termo está associado à gravação em disco dos dados manipulados por um programa de computador e sua futura recuperação e uso. No Capítulo 7 isso é feito por meio de arquivos em disco e são apresentados os conceitos e técnicas de leitura e gravação de tais arquivos. Já no Capítulo 8 trabalha-se com banco de dados. São apresentados os conceitos dos sistemas gerenciadores de bancos de dados, seus comandos SQL para definição e manipulação dos dados, bem como ao modo como a linguagem Python se conecta ao banco de dados e o utiliza.

No Capítulo 9 é desenvolvido um projeto completo voltado à avaliação de demanda de mercadorias de uma empresa, bem como ao cálculo da rentabilidade das vendas desta. Essa aplicação é construída utilizando-se de praticamente todos os elementos de Python 3 vistos nos capítulos anteriores. Nessa aplicação a persistência dos dados é feita por meio de arquivos texto.

Um segundo projeto completo é detalhado no Capítulo 10, no qual a persistência de dados é feita por meio do uso do gerenciador de banco de dados SQLite 3. Nesse projeto é desenvolvida uma aplicação que permite controlar torneios esportivos por pontos corridos. Inspirado em um caso real ocorrido no início da carreira do autor, esse projeto traz ao leitor diversos elementos importantes, a saber: o uso de praticamente todos os elementos de Python 3 abordados nos demais capítulos, a conexão do Python 3 com o gerenciador de banco de dados, a geração de telas, menus de comandos, leitura e tratamento das entradas do usuário e execução dos comandos solicitados por ele, geração de resultados e geração de páginas HTML para apresentação e publicação das saídas do programa.

Boa leitura!

O autor



Python: uma Linguagem de Programação

Objetivos

Este capítulo apresentará uma introdução aos conceitos de algoritmos e lógica de programação, bem como à linguagem Python, sua base histórica, características, detalhes sobre a instalação, o ambiente IDLE, a documentação básica e requisitos de hardware e software. O objetivo é dar uma fundamentação sobre essa poderosa linguagem e, então, iniciar o estudo e a programação.

1.1 Algoritmos e lógica de programação

Este livro trata de programação de computadores, assunto que atrai muitos interessados, seja porque gostam de computadores, porque querem criar aquele aplicativo revolucionário, porque querem desenvolver websites ou porque ouviram dizer que trabalhar com isso resulta em bom salário. Seja qual for a motivação, muitos chegam a esse mundo da programação de computadores e logo se deparam com dois desafios:

- aprender lógica e, com ela, criar um algoritmo;
- aprender uma linguagem de programação e, com ela, fazer o algoritmo funcionar em um computador.

Esses dois desafios representam duas coisas que precisam ser aprendidas simultaneamente, uma vez que, se faltar uma das duas, não haverá programa

de computador.

Um **algoritmo** é uma sequência bem definida e ordenada de passos necessários à solução de algum problema. É comum que professores da área, normalmente na primeira aula, apresentem a seus alunos a ideia de que um algoritmo é como uma receita de bolo ou de ovo frito. É um bom ponto de partida. Porém, assim como algumas receitas exigem um bom chefe de cozinha para serem executadas, alguns algoritmos exigem bons programadores para serem escritos.

Essa sequência bem definida e ordenada supramencionada é aquilo a que se dá o nome de **lógica**. E não é só de passos em sequência que a lógica é feita. Há também a necessidade de escolher o caminho que será tomado, a depender de certa condição ser falsa ou verdadeira; há certos passos que devem ser repetidos um número de vezes ou até que algo ocorra. Um algoritmo, no entanto, não precisa ser um programa de computador, mas, sim, os passos necessários para a solução de um problema.

Para que o algoritmo se torne um programa de computador, é preciso escolher e utilizar uma linguagem de programação. Cada linguagem de programação existente foi criada com algum objetivo, tem características próprias, seu paradigma, um conjunto de comandos com determinada sintaxe etc. Houve um tempo em que se contava nos dedos o número de linguagens existentes, e são dessa época alguns nomes clássicos como *Assembly*, *Cobol*, *Fortran*, *PL/I*, *C* e *Pascal*. Atualmente, a quantidade de linguagens disponível é tão grande que há um enorme potencial para deixar perdido o iniciante no mundo da computação. Feita a escolha de uma linguagem e iniciados os estudos, começam as dúvidas referentes aos detalhes da linguagem, surgindo questões como: “Onde usar ponto e vírgula (;)?”, “É preciso pular a linha?”, “Tem parênteses ou não?”, e por aí vai.

Assim, considerando que para escrever um programa o estudante precisa

desenvolver o algoritmo e utilizar a linguagem de programação para implementá-lo, não há alternativa: é preciso aprender as duas coisas, e deve ser ao mesmo tempo. Existem registros de que a linguagem Python tem sido adotada em muitos cursos, no mundo todo, como linguagem de programação introdutória (GUO, 2014), em um esforço por parte das instituições de ensino superior em adotar uma linguagem que possa ser mais facilmente assimilada pelo estudante, que poderá, assim, concentrar-se melhor no aprendizado dos algoritmos. Por exemplo, informações disponíveis à época da redação deste livro indicam que Python é usada no Massachusetts Institute of Technology (MIT) em Boston, nas universidades paulistas USP e Unicamp, na Universidade Federal de São Carlos, entre outras. Recentemente, o corpo docente da Faculdade de Tecnologia de São Paulo (Fatec-SP), onde o autor deste livro atua como docente na área de programação, adotou também a linguagem Python como ferramenta na disciplina de Algoritmos do primeiro semestre do curso de Análise e Desenvolvimento de Sistemas.

Por outro lado, para o programador que já domina a lógica de programação e conhece pelo menos uma linguagem, aprender uma nova linguagem é um caminho mais rápido e suave. Nestes casos, aprender Python será muito prazeroso, pois é uma linguagem que conta com recursos poderosos que garantem uma produtividade e uma qualidade de software muito significativas.

1.2 A linguagem Python

1.2.1 Breve histórico e características

A linguagem Python foi concebida entre o fim de 1989 e o início dos anos 1990 como projeto pessoal de Guido van Rossum, que até hoje continua liderando seu desenvolvimento, contando com a colaboração de muitos

desenvolvedores ao redor do mundo. Em uma contínua trajetória evolutiva, ela reúne características relevantes, tais como:

- **Portabilidade:** seu código-fonte é escrito em linguagem ANSI C, e o interpretador Python, bem como suas bibliotecas-padrão, está disponível para um extenso leque de plataformas, que incluem Unix, Linux, Windows (todas as versões), macOS, BeOS, VMS, entre outras. Isso significa que um programa escrito em Python e que utilize apenas as bibliotecas-padrão será executado da mesma maneira em qualquer uma dessas plataformas.
- **Código livre (opensource):** o fato de Python ser *opensource* significa que pode ser utilizado e distribuído livremente. Ele pode ser utilizado por um programador para desenvolver e distribuir um software, assim como seu código-fonte pode ser baixado, adaptado e utilizado sem qualquer restrição.
- **Simplicidade com robustez:** Python é simples como as linguagens de programação de scripts como Perl e Scheme. Por outro lado, conta com recursos que a equiparam a linguagens como C, C++ e Java, permitindo o desenvolvimento de grandes projetos que podem ser constituídos por diversos módulos, que acessem bancos de dados, que enviem e recebam dados por meio de redes, trabalhem com recursos multimídia, entre outros. Python também dispõe de mecanismos que permitem a integração com softwares escritos em outras linguagens, como C.
- **Fácil de aprender:** dizer que Python é fácil de aprender implica entender como seria aprender uma linguagem considerada difícil. Ao iniciante que ainda não conhece linguagem alguma, talvez a primeira

seja algo difícil de conseguir entender. Nesse caso, surge a dúvida: será que é mesmo fácil de aprendê-la? Para responder a isso, cabe olhar para o artigo já mencionado (GUO, 2014), que mostra a grande adesão das universidades norte-americanas à Python como linguagem de programação introdutória.

- **Grande aplicabilidade:** o Python pode ser utilizado em um grande número de áreas do desenvolvimento de software, das quais se destacam: ferramentas para administração e interface com sistemas operacionais; aplicações que trabalhem com grandes volumes de dados armazenados em sistemas gerenciadores de bancos de dados, como Oracle, SQL Server, MySQL e outros; aplicações gráficas e multimídia; desenvolvimento de jogos digitais; programação para internet; desenvolvimento de software para engenharia; aplicações científicas.

1.2.2 Versões da linguagem Python

A linguagem Python conta com duas versões que coexistem. No período em que este livro foi redigido, as versões disponíveis eram: 2.7.14 e 3.6.3. As duas versões apresentam diferenças importantes, a ponto de a versão 3.x representar uma importante quebra de compatibilidade em relação à versão 2.x. Sobre isso, Guido van Rossum (2007) declarou:

Por um bom tempo não havia muito mais do que uma lista de arrependimentos e defeitos estruturais que eram impossíveis de corrigir sem quebrar a compatibilidade retroativa. A ideia era que Python 3000 seria o primeiro release do Python a desistir desta compatibilidade em favor de tornar-se uma linguagem melhor e evoluir.

Uma das características mais marcantes da comunidade de desenvolvedores Python é seu conservadorismo com relação a mudanças que possam causar incompatibilidade retroativa. O lançamento do Python 3.0, em 2008, foi um evento singular e cercado de grande cuidado, atenção e muitas e muitas horas de testes. A quantidade de aplicações e bibliotecas desenvolvidas em Python 2.x é tão grande que, passados quase dez anos do lançamento da versão 3.0, a versão anterior ainda é muito utilizada e permanece viva. Desse modo, tanto ao estudante que inicia seu aprendizado de Python quanto ao programador experiente que deseja aprender a linguagem vem a dúvida: a qual versão se dedicar?

No website oficial de Python (<www.python.org>) pode-se encontrar a seguinte afirmação: “Python 2.x é legado, Python 3.x é o presente e o futuro da linguagem” (PYTHON SOFTWARE FOUNDATION, 2017). Dessa afirmação o leitor pode tirar as próprias conclusões e decidir a qual versão dedicará seus esforços. Na mesma página onde se lê tal afirmação são descritas as diferenças entre as versões. Se desejar, pode consultar o site para obter mais informações.

Este livro é inteiramente dedicado ao Python versão 3.x.

1.3 Instalação do Python

A instalação do Python é um processo rápido, simples e seguro. No endereço <www.python.org/downloads> estão disponíveis os binários para diversas plataformas. As distribuições GNU/Linux costumam ter alguma versão de Python instalada por padrão. O usuário só precisará verificar qual a versão, 2.x ou 3.x. Para comprovar se o Python está instalado no Linux, pode-se usar o seguinte comando:

```
$ which python
```

Esse comando retornará à pasta onde o Python está instalado ou à mensagem “no python in...”, em caso contrário. Se a distribuição contiver a versão 2.x e desejar a versão 3.x, não há problema. Basta baixá-la e instalá-la em outra pasta.

Quanto ao sistema operacional Windows, é preciso baixar e instalar. O processo é rápido e fácil, sendo apenas necessário que se tenha acesso ao modo administrador do Windows para poder fazê-lo. Acesse a página de downloads do website oficial (Figura 1.1), baixe o instalador da versão desejada e execute-o. É possível executar a instalação-padrão, que incluirá o interpretador, o IDLE, as bibliotecas, a documentação e o gerenciador de bibliotecas Pip. Alternativamente, é possível usar a opção de instalação personalizada, por meio da qual poderá selecionar o que deseja e o que não deseja instalar.

Acesse o endereço <https://www.python.org/downloads/>



Figura 1.1 Instalação do Python.

A versão mais recente disponível quando este livro foi escrito é a 3.6.3. Desse modo, todo o material aqui disponível, incluindo exemplos e exercícios resolvidos, foi elaborado com base nessa versão, para o sistema

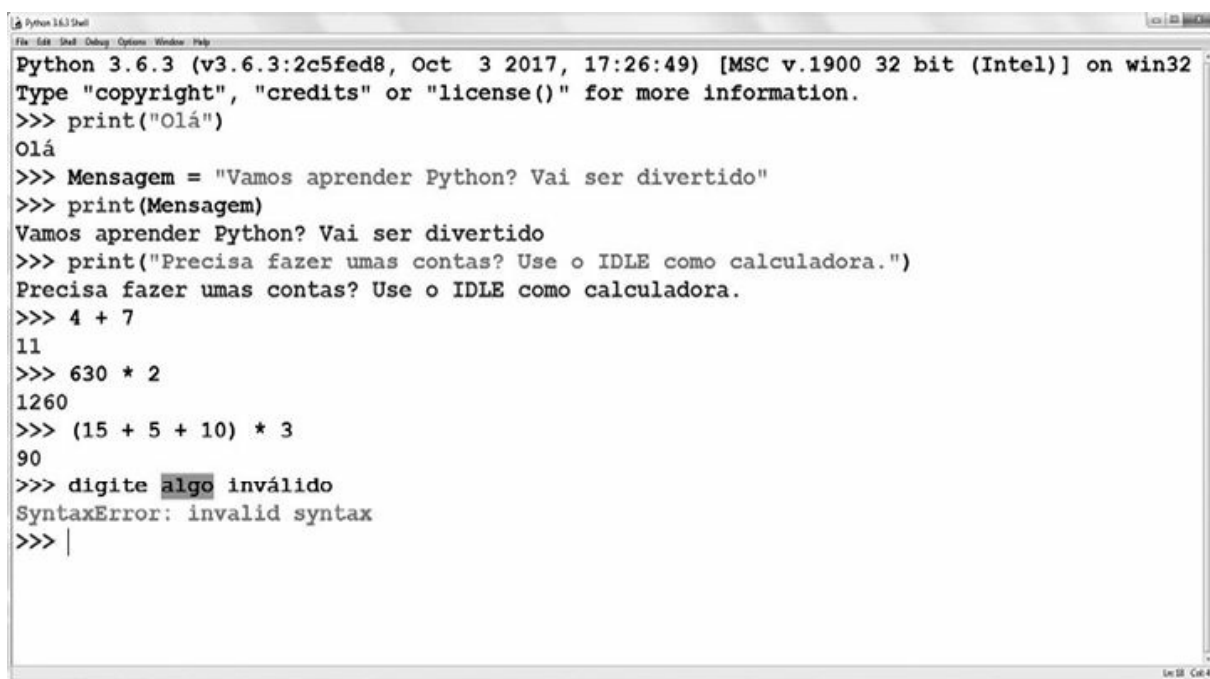
operacional Windows. Por ser multiplataforma, todo esse material poderá ser utilizado e testado em qualquer outra plataforma para a qual esteja disponível o interpretador Python 3.6.

1.4 Iniciando com o Python

1.4.1 O ambiente IDLE

Uma vez instalado, os primeiros passos podem ser dados com o IDLE. Esse nome refere-se a uma interface que o programador pode utilizar para interagir com o interpretador Python de uma maneira muito dinâmica.

Procure por ele em seu computador e abra-o. Será aberta uma janela, como mostrado na Figura 1.2, na qual há um *prompt* indicado pelo sinal “>>>”. No *prompt* é possível digitar comandos Python, e o interpretador imediatamente os executará. Faça alguns testes repetindo os comandos mostrados na figura.



```
Python 3.6.3 Shell
Python 3.6.3 (v3.6.3:2c5fed8, Oct  3 2017, 17:26:49) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print("Olá")
Olá
>>> Mensagem = "Vamos aprender Python? Vai ser divertido"
>>> print(Mensagem)
Vamos aprender Python? Vai ser divertido
>>> print("Precisa fazer umas contas? Use o IDLE como calculadora.")
Precisa fazer umas contas? Use o IDLE como calculadora.
>>> 4 + 7
11
>>> 630 * 2
1260
>>> (15 + 5 + 10) * 3
90
>>> digite algo inválido
SyntaxError: invalid syntax
>>> |
```

Figura 1.2 IDLE, a básica interface de usuário de Python.

Sempre que estiver em dúvida sobre o funcionamento de um comando,

método, função ou qualquer outro recurso da linguagem, poderá recorrer ao IDLE para testá-lo e entender como funciona e como utilizar o recurso. Se algo inválido, incorreto ou inexistente for digitado no IDLE, o interpretador avisará com uma mensagem. As mensagens emitidas são, em geral, autoexplicativas, contendo um grau de detalhe nas informações que ajudam bastante no entendimento e na solução do erro. Além disso, na internet há uma profusão de informações disponíveis, uma vez que a comunidade Python é grande e muito atuante.

Em todos os capítulos deste livro o IDLE é utilizado para exemplificar os comandos, conceitos e recursos abordados.

1.4.2 O editor de scripts

O IDLE é muito útil para testes e aprendizado, haja vista sua interatividade, porém, chegará o momento em que será necessário escrever um programa completo, salvá-lo em disco e posto a executar com começo, meio e fim, sem a interatividade.

Para isso, pode-se utilizar qualquer editor de textos simples como o Bloco de Notas ou o Notepad++. Porém, a opção mais simples é utilizar o editor de scripts que pode ser aberto a partir do IDLE, a partir do menu *File* → *New File*. Esse editor é integrado ao IDLE, de modo que é possível digitar seu programa, salvá-lo na pasta de sua preferência e, em seguida, pressionar a tecla de atalho F5 para executar o programa. Ao fazer isso, o programa é posto em execução no IDLE, e será possível testá-lo.

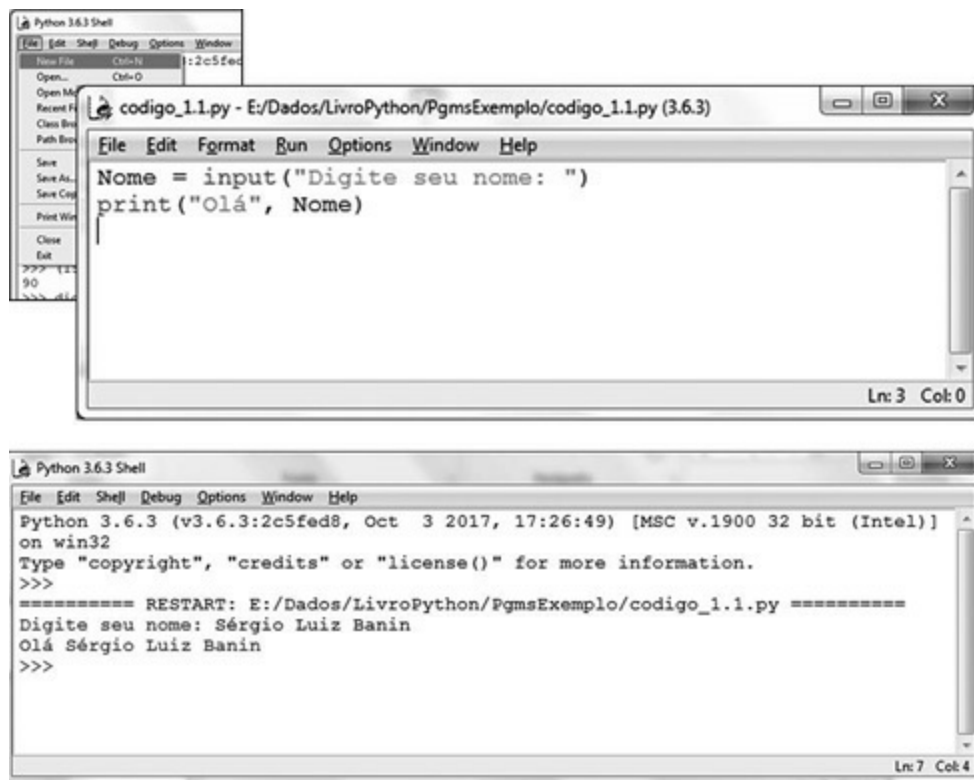


Figura 1.3 Editor de scripts do Python.

1.4.3 Outras IDEs

Os programadores já experientes e acostumados a trabalhar com outras linguagens talvez estranhem a simplicidade do ambiente IDLE e seu editor de scripts. Bem, o Python é assim mesmo. A filosofia promovida por Guido van Rossum e o time de desenvolvedores é uma filosofia que contém a simplicidade, a elegância, a coerência e a consistência entre seus atributos.

No entanto, com o passar do tempo e o aumento da utilização do Python ao redor do mundo, surgiram diversas opções de **ambientes integrados de desenvolvimento**, ou no termo pelo qual é de fato conhecido, mesmo no Brasil, IDE (*Integrated Development Environment*).

Um IDE é um programa de computador que contém recursos e funcionalidades direcionados ao desenvolvimento de programas de computador.

É possível encontrar na internet uma grande variedade de IDEs que podem ser utilizadas para escrever programas em linguagem Python. Uma rápida busca o levará a websites intitulados “Os top 10 IDEs para Python” ou “os 3 melhores IDEs para Python”. Listas assim refletem as preferências e necessidades de quem as elabora. Cada um desses ambientes terá recursos, características, vantagens e desvantagens próprios que não são objeto do texto deste livro. E, embora haja boas discussões técnicas acerca de benefícios e problemas de cada IDE, há também certa disputa de torcidas, mais ou menos aos moldes de “é biscoito ou bolacha?”. Há IDEs que são gratuitas e *opensource*, ao passo que outras são proprietárias e requerem o pagamento de licenças.

No Quadro 1.1 é oferecida uma lista parcial dos ambientes disponíveis e mais utilizados à época de redação deste livro, sem entrar no mérito de qual é melhor, mais poderoso, mais bonito ou qualquer outra adjetivação que envolva as palavras “mais” ou “menos”.

•Atom	•PyCharm
•Eclipse com PyDev	•Spyder
•Eric Python	•VIM
•Komodo IDE	•Visual Code Studio
•Ninja IDE	•Wing IDE

Quadro 1.1 Lista parcial de IDEs para escrever programas em Python em ordem alfabética (para uma lista mais completa, consulte <<https://wiki.python.org.br/IdesPython>>).

Nota

Neste livro, será utilizado exclusivamente o IDLE para criação dos exemplos, exercícios resolvidos e projetos.

1.4.4 Documentação e suporte

A comunidade Python, além de grande, é muito engajada, ativa e dinâmica. Com isso, não só a linguagem vem sofrendo constantes melhorias e acréscimos, como muito material de apoio tem sido produzido.

A documentação básica é o *Python Docs*, que está disponível no endereço <<https://docs.python.org/3/>> e que também é instalada na máquina do usuário, junto com o interpretador. No IDLE, aperte F1 para ter acesso a ele. O *Python Docs* é a fonte de referência primária para todo programador que trabalhe com a linguagem.

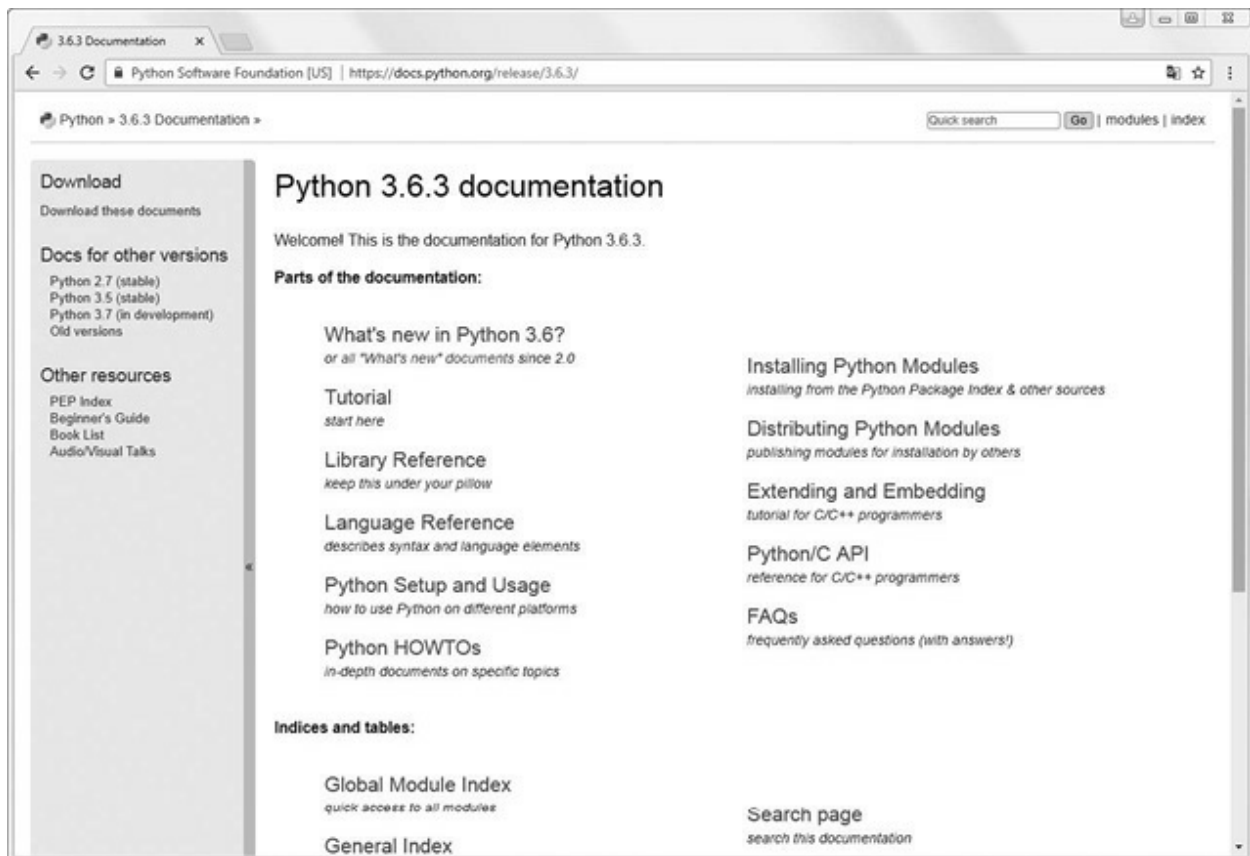


Figura 1.4 Python Docs.

Somando-se ao *Python Docs*, há um grande número de artigos, fóruns, listas de discussões, livros, blogs, e vídeos no *YouTube*, tudo isso acessível on-line em vários idiomas, inclusive português. Para os brasileiros, a comunidade Python Brasil (<<http://python.org.br>>) é um excelente recurso em português.

Outra referência importante e avançada é o conjunto de PEPs (*Python Enhancement Proposals*). PEP é um documento padronizado utilizado para formalizar a divulgação de informações à comunidade Python, para descrever uma nova funcionalidade, para que sejam apresentadas propostas de novos recursos, para a coleta de informações sobre problemas e para documentar as decisões de projeto que foram adotadas no Python.

1.5 A quem se destina este livro

O conteúdo foi pensado para atender desde o aluno iniciante até o programador experiente que necessita ou deseja aprender a linguagem Python.

Ao iniciante a linguagem Python propicia uma ferramenta simples, intuitiva e de fácil compreensão. Isso permite que o estudante não gaste seu tempo e energia com detalhes e especificidades da linguagem e dedique-os à resolução dos problemas de lógica, construindo algoritmos que representem a solução para tais problemas. Além disso, os diversos exemplos, exercícios resolvidos e exercícios propostos em cada capítulo ajudam o estudante a consolidar os conceitos apresentados, por meio da experimentação prática.

Ao programador que precisa aprender Python este livro oferece o modelo, os conceitos, elementos e detalhes da linguagem, com bom grau de profundidade, não encontrado em livros básicos. Tais elementos levam a uma compreensão de como o interpretador foi pensado para que todos os recursos existentes sejam coerentes, robustos e confiáveis, garantindo a qualidade do

produto final, que é o software escrito em Python.

1.6 Requisitos mínimos

Trabalhar com Python não requer um equipamento caro e cheio de recursos. Ao contrário, uma máquina mediana será capaz de executar o Python com folga. Um computador com processador i3 das primeiras gerações e com 2 GB de memória já é suficiente para isso. Quanto a software, todos os exemplos deste livro foram escritos e testados com uso de Python versão 3.6.3 para o sistema operacional Windows. No entanto, não foi utilizado nenhum recurso específico dessa plataforma, de modo que se utilizar outro sistema operacional não encontrará dificuldades.

1.6.1 Hardware

- Processador i3 de primeira geração 1220 MHz, ou compatível.
- 2 gigabytes (GB) de memória do sistema.
- Disco rígido com 1 GB livre.
- Acesso à internet para baixar o instalador, no caso do ambiente Windows e eventuais atualizações, no caso de Linux e macOS.

1.6.2 Software

No caso do sistema operacional Windows, é exigida a versão Vista ou superior para o Python 3.6 em diante. Caso ainda opte pelo Windows XP, é possível instalar e utilizar o Python 3.4.

Quanto ao sistema operacional Linux, a maioria das distribuições Linux existentes atualmente já disponibilizam o interpretador Python pré-instalado ou pacotes binários que podem ser facilmente instalados. Para verificar, abra

seu terminal e digite: `python -v`.

Os computadores da Apple com o macOS também já acompanham um interpretador Python pré-instalado que pode ser atualizado baixando a última versão disponibilizada no site oficial da linguagem Python na versão 3.6.

Neste livro, nos Capítulos 8 e 10, são utilizados dois softwares adicionais, o gerenciador de banco de dados SQLite 3 e o programa de manipulação de bancos de dados SQLite Studio. Ambos estão disponíveis para Windows, Linux e macOS.



Objetos e Comandos de Entrada e Saída em Python

Objetivos

Em essência, para ser útil, um programa de computador precisa ser capaz de receber dados de entrada, armazená-los em algum lugar, manipulá-los de algum modo, produzindo resultados, e, por fim, exibí-los de maneira apropriada, por meio de algum dispositivo.

Este capítulo tem por objetivo apresentar os mais básicos elementos da programação utilizando a linguagem Python, os quais permitem que as tarefas citadas sejam realizadas.

Desse modo, serão apresentados os objetos e os tipos de dados disponíveis na linguagem Python, os quais permitem o armazenamento de dados e sua manipulação, bem como serão vistos os comandos da linguagem utilizados para exibição em tela e leitura de dados do teclado.

2.1 Objetos e classes

2.1.1 Conceito de variáveis

Todo algoritmo que se possa construir utilizará conjuntos de dados. Tais dados podem ser, basicamente, números e caracteres isolados ou, de algum modo, agrupados.

Para que um algoritmo possa ser implementado em um computador, é preciso que exista um meio de armazenamento dos dados que serão manipulados. Assim, chega-se ao conceito existente em todas as linguagens de programação e que é usualmente designado pelo termo “variável”.

Em programação de computadores, uma variável é um elemento da linguagem que ocupa um ou mais bytes na memória RAM do computador. Esse local da memória é capaz de reter, ou seja, armazenar o elemento de dado. No programa, a variável é identificada por um nome ou identificador. Assim, pode-se entender que do ponto de vista do programador a variável é um nome que contém um dado, e do ponto de vista do computador a variável é um endereço de memória que retém um conjunto de bits que representam esse dado.

Por exemplo, imagine que se queira escrever um algoritmo capaz de calcular a área de um retângulo. Nesse algoritmo, haverá três dados, sendo dois de entrada – a base e a altura do retângulo –, e terceiro, o resultado, é a área calculada utilizando-se os outros dois. Assim sendo, pode-se esquematizar um rascunho de algoritmo que seja o seguinte:

```
1. Obter o valor da Base
2. Obter o valor da Altura
3. Calcula Area = Base x Altura
4. Exibir a Area calculada
```

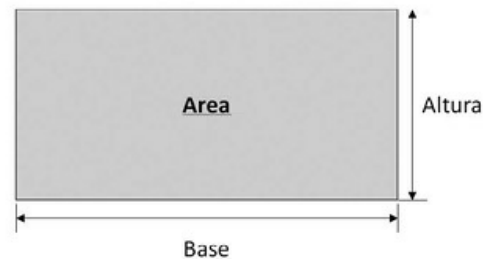


Figura 2.1 Um primeiro algoritmo utilizado para ilustrar o conceito de

variável.

Os quatro passos sequenciais aqui exibidos representam um algoritmo simples, e os identificadores Base, Altura e Area são variáveis.

2.1.2 Modelo de dados de Python

No contexto de linguagens de programação, a expressão “modelo de dados” diz respeito à abordagem, aos paradigmas e às técnicas adotadas no projeto conceitual da linguagem, visando definir a maneira como os dados serão mantidos em memória e acessados pelo conjunto de instruções.

O modelo de dados do Python (*Python Data Model*, em inglês) adota como paradigma que todo dado em um programa escrito com Python é representado por um **objeto**. Todo objeto Python tem três aspectos: um identificador, um tipo e um conteúdo.

O **identificador** é o nome que o objeto tem no programa.

O **tipo** do objeto determina não só a natureza dos dados que este armazena (por exemplo, um número inteiro, um texto), mas também as operações que são suportadas por ele. Cada objeto em Python é criado a partir de uma classe (*class*), que é um elemento do paradigma de programação conhecida como programação orientada a objetos. Será visto, nos capítulos posteriores, que os objetos, além do conteúdo, apresentam comportamentos associados. Assim, um objeto do tipo “*int*” (número inteiro) terá associado a si um conjunto de funções adequadas à manipulação de números inteiros; um outro objeto do tipo “*list*” (lista) terá outras funções que se adequam à manipulação de listas; e assim por diante. O **conteúdo** do objeto é o valor (ou conjunto de valores) armazenado nele.

Exemplo 2.1 Objetos em Python

```
>>> MeuObjeto = 10
```

```
>>> type(MeuObjeto)
```

```
<class 'int'>
```

No Exemplo 2.1, podem ser vistos os três aspectos mencionados: o identificador é `MeuObjeto`; o tipo é “int”, que representa números inteiros; e o conteúdo é o valor 10.

Considerando o que foi exposto, a conclusão imediata é que em Python não existem **variáveis**, como estas costumam ser conhecidas em outras linguagens. O que existe, de fato, são os **objetos**. Pode parecer uma simples questão de nomenclatura, mas não se trata disso, uma vez que cada objeto, além de ser utilizado para armazenar seu conteúdo, apresenta um comportamento próprio associado à classe a que pertence.

Assim sendo, deste ponto em diante, será dada preferência ao uso do termo “objeto” em detrimento de “variável” para referência aos elementos relacionados ao armazenamento de dados em programas escritos com Python.

A seguir, serão apresentados os tipos de objetos utilizados com maior frequência nos programas desenvolvidos por quem está iniciando o aprendizado de programação. Por questão de didática, será feita uma distinção entre tipos cujo conteúdo é indivisível, designados como “tipos simples”, daqueles cujos conteúdos representam coleções de elementos que podem ser acessados individualmente ou em grupo e que serão designados como “tipos estruturados”.

2.1.3 Tipos simples de dados

Embora nesse quesito haja muita semelhança entre as diversas linguagens existentes, cada uma tem suas peculiaridades. Em Python, estão disponíveis

os tipos simples relacionados a seguir.

- **Número inteiro (*int*):** capazes de armazenar números inteiros positivos, zero ou negativos.
- **Número real (*float*):** capazes de armazenar números reais positivos ou negativos, além do zero. O separador decimal é o ponto “.”.
- **Número complexo (*complex*):** armazena um número complexo do tipo $4 + 3j$ (note-se, na parcela imaginária, que é utilizada a letra “j” em vez da letra “i”). A linguagem Python tem suporte completo às operações aritméticas envolvendo números complexos. Essa característica é muito útil em programas voltados à solução de problemas de física e engenharia, nos quais há uma forte presença de números complexos, por exemplo, estudo de vibrações, circuitos elétricos e sistemas dinâmicos.

2.1.4 Tipos estruturados de dados

Em contraposição aos tipos simples, os tipos estruturados são compostos, ou seja, seu conteúdo é constituído por outros elementos. Assim sendo, tais tipos representam agregados de objetos que podem ser acessados e manipulados em conjunto ou isoladamente.

Os tipos compostos mais importantes para esta fase do aprendizado de lógica de programação serão objeto de estudo específico do Capítulo 4 deste livro. Desse modo, a proposta aqui é apenas listar e conceituar brevemente os tipos compostos existentes, dando uma visão geral das possibilidades da linguagem. O aprofundamento será estudado mais adiante.

- **Cadeia de texto (*str*):** também denominados strings, objetos deste tipo contêm qualquer sequência de caracteres, incluindo letras, algarismos e

caracteres especiais. Servem para armazenar nomes, endereços, texto em geral, bem como quaisquer dados aos quais não se aplicam ou não se realizam operações aritméticas. Utilizando strings em Python, é possível acessar todo o texto ou cada caractere individualmente. Este é um tipo imutável, de modo que não é possível alterar um caractere isoladamente. O programador poderá ter acesso individual a ele, para exibi-lo na tela, por exemplo, porém não será capaz de alterá-lo. Se tentar fazê-lo, receberá uma mensagem de erro.

- **Lista (list):** de todos os tipos disponíveis em Python, é um dos mais versáteis e poderosos. Uma lista caracteriza-se por ser um conjunto de itens entre colchetes e separados por vírgulas. Os itens não precisam ser todos do mesmo tipo e podem ser acessados e manipulados individualmente, todos de uma vez ou em grupos. Este é um tipo mutável.
- **Tupla (tuple):** são semelhantes às listas, no entanto, seus componentes não podem ser alterados. Este é um tipo imutável. Uma tupla caracteriza-se por ser um conjunto de itens entre parênteses e separados por vírgulas.
- **Conjunto (set e frozenset):** um conjunto é uma coleção não ordenada de elementos não duplicados e caracteriza-se por itens entre chaves e separados por vírgulas. Tem diversos usos possíveis e suporta operações matemáticas típicas de conjuntos, como união, interseção e diferença, muito úteis em alguns algoritmos. O tipo set é mutável, enquanto que frozenset é imutável.
- **Dicionário (dict):** também designado como tipo mapeado, um dicionário é uma coleção de pares “chave:valor” não ordenados, com a

obrigatoriedade de que as chaves sejam únicas (não duplicadas) dentro do dicionário. Enquanto as listas e tuplas são indexadas por um número inteiro, os dicionários são indexados pela chave associada ao valor. Dicionários são mutáveis.

Por fim, uma breve palavra sobre um conceito muito relevante em Python que está relacionado aos tipos de dados e com frequência deixa confuso o programador que já conhece outras linguagens e está iniciando seus estudos de Python.

Os objetos existentes na linguagem podem ser classificados como **imutáveis** (*immutable*) ou **mutáveis** (*mutable*). Um objeto imutável tem conteúdo fixo, ou seja, não pode ser alterado sem que o objeto seja reconstruído. Os tipos numéricos (int, float, complex), os strings, tuplas e frozenset são imutáveis, de modo que, quando um novo conteúdo for atribuído ao objeto, sua instância anterior é removida, e uma nova instância, criada. Os tipos lista, dicionário e set são mutáveis, de modo que podem ter seu conteúdo alterado, sem que sua instância seja recriada.

Esse conceito é realmente relevante em Python e será mais bem detalhado no Item 2.3.2. Ao programador iniciante a sugestão é que não se preocupe com esse assunto agora e se aprofunde mais no aprendizado da linguagem. Para os programadores experientes a sugestão é que não desistam do Python, pois no devido momento esse conceito, e diversos outros, muito típicos do Python, serão compreendidos e farão todo o sentido. Para ambos, convém dizer que será muito gratificante conhecer os detalhes e paradigmas do Python, pois são muito bem pensados e implementados.

2.1.5 Começando a trabalhar

Agora, é hora de começar a trabalhar com Python. Para isso, supondo que

já o tenha instalado, abra o ambiente IDLE e digite os comandos deste programa:

Exemplo 2.2 Um primeiro programa

```
>>> Base = 10
```

```
>>> Altura = 4
```

```
>>> Area = Base * Altura
```

```
>>> print(Area)
```

```
40
```

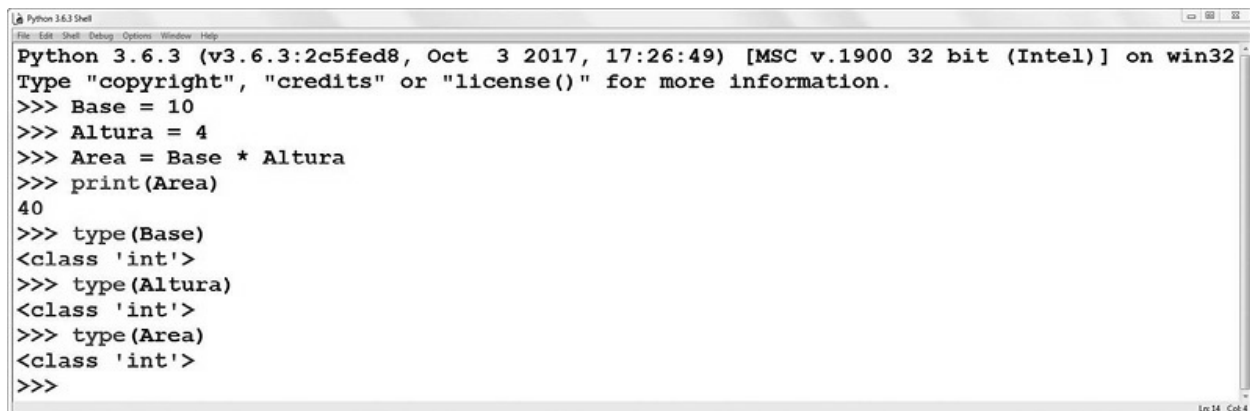
Os quatro comandos do Exemplo 2.2 são a tradução para Python do algoritmo da Figura 2.1. Os dois primeiros comandos atribuem valores fixos aos objetos `Base` e `Altura`. O terceiro comando contém, do lado direito, uma expressão aritmética de multiplicação cujo resultado é calculado e armazenado no objeto `Area`. Por fim, o comando `print` é utilizado para exibir na tela o conteúdo de `Area`, no caso, 40.

Nesse pequeno exemplo já fica evidente um aspecto importante do Python. Os objetos do programa não precisam ser explicitamente declarados para serem utilizados. Essa declaração explícita é necessária na maioria das linguagens, mas em Python, não. É frequente que esse aspecto da linguagem cause estranheza em quem já conhece alguma outra linguagem, como C, C++ ou Java.

Para que um objeto comece a existir, basta que a ele se atribua um valor inicial. Ao fazer isso, o Python cria seu identificador e reserva um espaço de memória para armazenar o dado contido.

Outra questão que surge como decorrência desse processo de criação do

objeto a partir da atribuição de valor inicial é quanto ao tipo, ou classe, do objeto. No caso desse exemplo, todos são números inteiros, formalmente: são do tipo “int”. Para constatar isso, utilize o comando *type*. Veja a Figura 2.2, em que foi utilizado o comando *type* três vezes, com o qual se verifica que, de fato, os objetos criados são do tipo “int”.

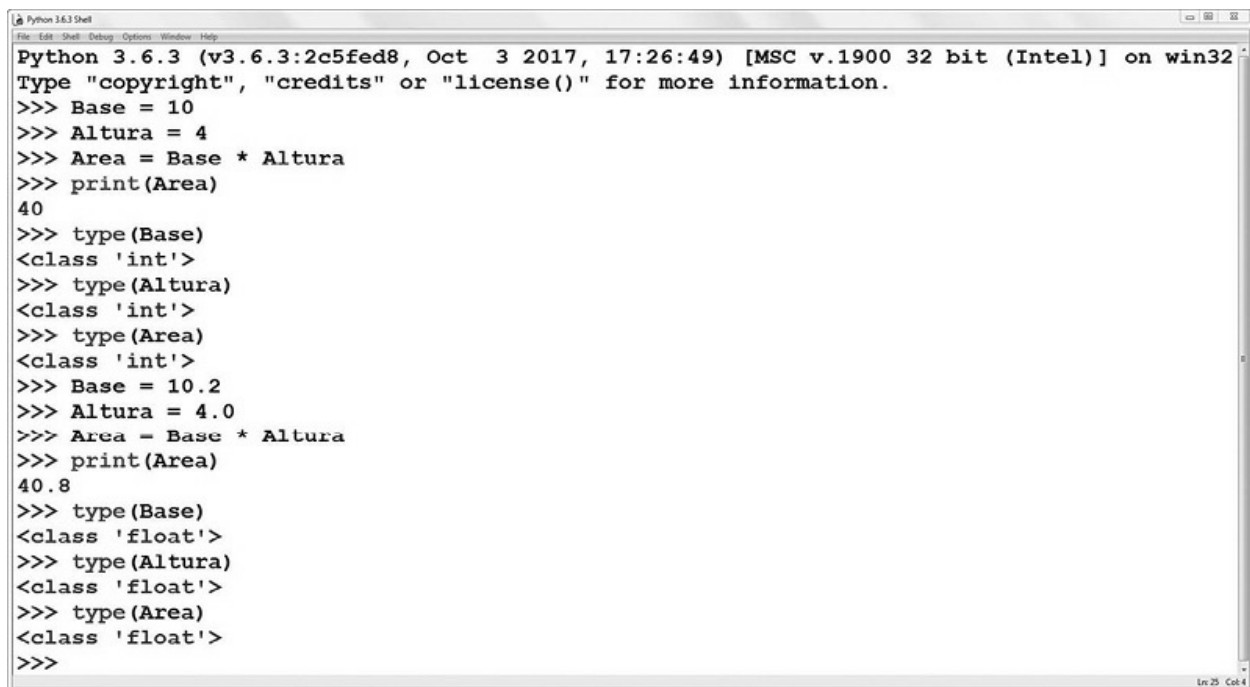
A screenshot of a Python 3.6.3 Shell window. The window title is "Python 3.6.3 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the following code and output:

```
Python 3.6.3 (v3.6.3:2c5fed8, Oct 3 2017, 17:26:49) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> Base = 10
>>> Altura = 4
>>> Area = Base * Altura
>>> print(Area)
40
>>> type(Base)
<class 'int'>
>>> type(Altura)
<class 'int'>
>>> type(Area)
<class 'int'>
>>>
```

The status bar at the bottom right indicates "Ln 14 Col 4".

Figura 2.2 Uso do comando *type*.

Caso se queira criar um objeto do tipo “float”, ou seja, capaz de conter um número real, basta atribuir a ele um valor que contenha a parte decimal. Pode-se fazer como mostrado na Figura 2.3.

A screenshot of a Python 3.6.3 Shell window. The window title is "Python 3.6.3 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the following code:

```
Python 3.6.3 (v3.6.3:2c5fed8, Oct 3 2017, 17:26:49) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> Base = 10
>>> Altura = 4
>>> Area = Base * Altura
>>> print(Area)
40
>>> type(Base)
<class 'int'>
>>> type(Altura)
<class 'int'>
>>> type(Area)
<class 'int'>
>>> Base = 10.2
>>> Altura = 4.0
>>> Area = Base * Altura
>>> print(Area)
40.8
>>> type(Base)
<class 'float'>
>>> type(Altura)
<class 'float'>
>>> type(Area)
<class 'float'>
>>>
```

The status bar at the bottom right shows "Ln 25 Col 4".

Figura 2.3 Uso do comando `type`.

No exemplo da Figura 2.3 foram utilizados, propositalmente, os mesmos nomes para os objetos e, em seguida, foi utilizado o comando `type` três vezes para verificar o tipo de cada um. Perceba que, se antes eles eram “int”, após o uso com números reais eles passaram a ser “float”. No momento de atribuição de um valor ao objeto, o interpretador verificará qual tipo mais adequado o utilizará.

Na linguagem Python, essa é uma característica relevante. Ao iniciante parece que os objetos podem mudar de tipo sempre que houver uma atribuição de valor. Na prática, o que ocorre é algo diferente: a cada operação de atribuição um novo objeto é criado em memória, sendo o anterior descartado.

Cada objeto criado tem uma **identidade** (*identity*), que é um número inteiro criado no momento em que ocorre a atribuição de valor.

Isso pode ser constatado por meio do uso do comando `id`, que retorna a identidade do objeto. Observe a Figura 2.4, na qual o mesmo nome foi

utilizado em três atribuições consecutivas. Cada uma das atribuições criou um objeto com identidade diferente, sendo que o nome identificador permaneceu o mesmo. Essa característica confere grandes flexibilidade e poder à programação Python. No momento, faltam elementos para comprovar tal afirmação, mas isso será mostrado mais adiante.

É importante não confundir dois termos utilizados até aqui: identificador e identidade. O primeiro é o nome do objeto utilizado ao se escrever o programa, e o segundo é um número inteiro criado quando o programa está em execução.

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main area shows a series of commands and their outputs: '>>> MeuObj = 25', '>>> id(MeuObj)', '1456855216', '>>> MeuObj = 3.7', '>>> id(MeuObj)', '47259360', '>>> MeuObj = 'Teste'', '>>> id(MeuObj)', '50236864', and '>>>'. The status bar at the bottom right shows 'Linha 36 Col 4'.

Figura 2.4 Verificação da identidade de um objeto.

Agora, faça uma experimentação. Abra o IDLE em seu computador e experimente as sugestões a seguir. Utilize a função **id()** também.

Exemplo 2.3 Teste o que já aprendeu

```
>>> X = 1.0
```

```
>>> type(X) # X é float
```

```
>>> Y = 18
```

```
>>> type(Y) # Y é int
```

```
>>> Z = X + Y
```

```
>>> type(Z) # Z é a soma de float com int. Qual é o tipo de Z?
```

```
>>> a = 5 + 3j
```

```
>>> type(a) # a é complex
```

```
>>> type(A) # Experimente usar o type com o objeto A (maiúsculo)
```

e verifique o que ocorre.

2.2 Nomes de objetos: identificadores

Quando o programador estiver escrevendo um programa, ele precisará atribuir nomes aos objetos, ou seja, definir o identificador que utilizará em cada um. Todas as linguagens têm regras para o estabelecimento desses identificadores.

É claro que tais regras vão variar de uma linguagem para outra, mas, em linhas gerais, os identificadores podem ser criados usando letras, números e o caractere *underscore* “_” (também denominado *underline*).

Em algumas linguagens, como Object Pascal, não faz diferença utilizar letras minúsculas ou maiúsculas, ou seja, os identificadores “Valor” e “valor” serão tratados como a mesma coisa e farão referência ao mesmo endereço de memória.

Em outras linguagens, como C, Java e Python, isso faz total diferença. Nesses casos, os identificadores “Valor” e “valor” vão se referir a diferentes endereços na memória do computador.

Em Python, os identificadores devem começar com uma letra, que pode ser

maiúscula ou minúscula, ou com o caractere *underscore*. Não é permitido que um identificador comece com um número.

É recomendável que os objetos sejam criados com nomes que ajudem a lembrar seu conteúdo. No Exemplo 2.2, os identificadores *Altura*, *Largura* e *Area* poderiam ser substituídos por *X*, *Y* e *Z*, por *v1*, *v2*, *v3* ou qualquer outra coisa, e ainda assim o programa funcionaria do mesmo modo. Porém, utilizar nomes que facilitam lembrar o que o objeto contém gera um ganho de produtividade durante o desenvolvimento do programa e, principalmente, facilita muito nas manutenções e atualizações futuras, quando é preciso lembrar o que o programa faz e como faz, para que as modificações possam ser desenvolvidas.

2.3 Atribuições e expressões aritméticas

2.3.1 Atribuições

Uma operação de atribuição, ou simplesmente “Atribuição”, já foi utilizada nos exemplos anteriores, e trata-se de uma expressão envolvendo o operador de atribuição “=”. Observe a linha a seguir:

Destino = Origem

Essa linha refere-se a uma atribuição, na qual *Destino* é um identificador de objeto e *Origem* podem ser diversos elementos, tais como um valor, um objeto, uma expressão aritmética, o retorno de uma função etc. Trata-se de algo simples de ser compreendido, porém, nos bastidores dessa operação simples estão implementados alguns conceitos e características importantes que serão agora descritos.

Em primeiro lugar, as atribuições criam um objeto e o associam a um nome identificador, conforme mostrado nas Figuras 2.2 a 2.4. É importante olhar

primeiro para o lado direito da atribuição, pois é a expressão ali contida que define o tipo de objeto que será criado. Uma vez avaliada a expressão e criado o objeto em memória, o identificador definido do lado esquerdo é associado ao objeto, como se uma etiqueta fosse colocada em uma peça produzida. Esse é o principal conceito de bastidor envolvendo as atribuições.

Os nomes de identificadores são criados quando atribuídos pela primeira vez. Caso sejam atribuídos novamente, há duas situações a considerar. Se a natureza do objeto é imutável, então, a instância anterior é destruída e uma nova instância é criada. É por esse motivo que, na Figura 2.4, a cada atribuição um novo *id* é criado. Porém, se a natureza do objeto for mutável, então, o objeto será mantido em memória e a alteração de seu conteúdo será feita sem mudança de *id*. Isso tem implicações que, se não forem bem compreendidas pelo programador Python, podem causar um entendimento errôneo do que está acontecendo em um programa. No Capítulo 4, serão descritas algumas situações em que isso é relevante.

Outro ponto importante: antes de serem referenciados em qualquer comando ou expressão, os identificadores precisam ser criados. Utilizar um identificador sem antes criá-lo com uma atribuição fará que o interpretador gere um erro e interrompa o programa.

2.3.2 Formas de atribuição

O Exemplo 2.4 exhibe um grupo contendo as mais simples e frequentes formas de atribuição utilizadas. Embora simples, o que é feito nos bastidores não é óbvio. Observe a criação do objeto B no exemplo. A atribuição $B = A$ faz que o identificador B passe a apontar para o mesmo objeto em memória para o qual aponta o objeto A. O conceito de bastidor aqui envolvido parte da ideia de que, se dois identificadores devem ter o mesmo conteúdo, então, é razoável que ambos apontem para o mesmo objeto, promovendo um melhor

uso da memória. Logo em seguida, é atribuído o valor 50 a B, e neste caso um novo objeto é criado em memória e o identificador B passa a apontar para ele, tendo seu *id* alterado. Esse comportamento ocorre porque os objetos A e B, do tipo “int”, são imutáveis.

O termo ***imutável*** já foi mencionado no item 2.1.4, e agora estão disponíveis os elementos para explicá-lo convenientemente. Quando um objeto é imutável e seu conteúdo é trocado, o objeto anterior é descartado e um novo é criado. No Exemplo 2.4, isso é mostrado por meio da verificação do *id* do objeto B antes e depois de receber o novo valor. Em contrapartida, um objeto ***mutável*** poderá ter seu conteúdo livremente alterado, ao mesmo tempo que seu *id* é mantido. Talvez o programador experiente agora compreenda melhor o que está acontecendo, mas talvez ainda não entenda por que variáveis simples, como um “int”, tenham de ser descartadas e recriadas a cada nova atribuição. Bem, então, o próximo passo no aprofundamento desse assunto será dado no início do Capítulo 6.

Seguindo no assunto de atribuições, observe-se o que ocorre com os identificadores L e M, que apontam para objetos do tipo lista. As listas são mutáveis de modo que uma alteração nos elementos contidos na lista não provoca a criação de um novo *id* e, por consequência, os dois identificadores L e M continuam apontando para o mesmo objeto que foi alterado. As listas são tipos sequenciais a serem estudados no Capítulo 4.

É frequente que programadores experientes em outras linguagens fiquem confusos com esse comportamento de Python, que lhes parece produzir resultados inesperados. Ao contrário de inesperado, estes são sólidos conceitos implementados em Python, e cabe ao programador interessado nessa linguagem buscar conhecê-los para poder fazer bom uso deles.

Exemplo 2.4 Formas simples de atribuição

```
>>> A = 10 # A é criado e recebe o valor 10
```

```
>>> id(A)
```

```
498390976 # A tem um id
```

```
>>> B = A # B é criado, recebendo A
```

```
>>> id(B) # observe que o id de B é o mesmo que o id de A
```

```
498390976 # foi criado o novo nome (B) que aponta para A
```

```
>>> B = 50 # nova atribuição para B
```

```
>>> id(B) # que passa a ter um novo id
```

```
498391616
```

```
>>> L = [12, 24, 36] # cria a lista L
```

```
>>> id(L)
```

```
48917320
```

```
>>> M = L # cria a lista M que passa a ter o mesmo id de L
```

```
>>> id(M)
```

```
48917320
```

```
>>> M[0] = 0 # altera-se um elemento de M
```

```
>>> print(M)
```

```
[0, 24, 36]
```

```
>>> print(L) # o elemento de L também foi alterado. Isto
```

```
[0, 24, 36] # ocorre porque listas são mutáveis.
```

```
>>> C = A * 2
```

```
>>> id(C)
```

```
498391136
```

```
>>> D = "TEXTO" # o objeto apontado por D é um tipo estruturado
```

```
>>> type(D) # string. A manipulação de strings em Python
```

```
<class 'str'> # é muito simples e poderosa
```

```
>>> from math import sqrt
```

```
>>> X = 25
```

```
>>> Y = sqrt(X) # o retorno de uma função também cria objetos
```

```
>>> Y
```

```
5.0
```

Continuando com o Exemplo 2.4, na criação do objeto C foi utilizada uma expressão aritmética envolvendo o objeto A e um valor. Em seguida, foi criado o objeto D com a atribuição de um string. Assim como as listas, os strings são tipos sequenciais, os quais serão estudados no Capítulo 4.

Por fim, foi feita a importação da função *sqrt* – raiz quadrada – do módulo

math, a qual foi utilizada para criar o objeto Y. Veja o Item 2.4 para mais informações sobre funções matemáticas.

O Python ainda suporta outros tipos de atribuição, mostradas no Exemplo 2.5. O caso 1 é o de atribuição múltipla, em que vários identificadores são criados simultaneamente. Se você já compreendeu os conceitos de bastidores implementados em Python deverá raciocinar que A, B e C são três identificadores distintos que apontam para o mesmo objeto em memória e, portanto, têm o mesmo *id*. E esse raciocínio está correto, como pode ser constatado no exemplo.

O caso 2 exemplifica a atribuição posicional. Cada objeto criado no lado direito da expressão é atribuído a cada identificador do lado esquerdo, segundo a posição relativa de cada um, de modo que $A = 1$, $B = 2$ e $C = 3$. Nesse caso, são objetos diferentes, portanto, cada identificador tem o próprio *id*. No Capítulo 4, esse tipo de atribuição será retomado, uma vez que essa operação envolve uma tupla de identificadores do lado esquerdo e uma tupla de objetos do lado direito da expressão.

O caso 3 também é uma atribuição de tuplas. Com essa forma de atribuição, é possível inverter os conteúdos de dois objetos. Essa é uma situação comum em muitos algoritmos, e em outras linguagens exige que uma variável intermediária seja utilizada na troca. Em Python, basta escrever essa forma de atribuição e os conteúdos serão invertidos. Esse caso pode ser generalizado para qualquer quantidade de objetos envolvidos.

Exemplo 2.5 Outros tipos de atribuição em Python

```
>>> A = B = C = 1 # caso 1: atribuição múltipla
```

```
>>> id(A) # A, B e C tem o mesmo id
```

```
498390832
```

```
>>> id(B)
```

```
498390832
```

```
>>> id(C)
```

```
498390832
```

```
>>> A, B, C = 1, 2, 3 # caso 2: atribuição posicional
```

```
>>> id(A)
```

```
498391008
```

```
>>> id(B)
```

```
498390848
```

```
>>> id(C)
```

```
498390864
```

```
>>> X, Y, Z = 0, -10, 10
```

```
>>> print(X, Y)
```

```
0 -10
```

```
>>> X, Y = Y, X # caso 3: inversão de objetos
```

```
>>> print(X, Y)
```

```
-10 0
```

```
>>> print(X, Y, Z)
```

```
-10 0 10
```

```
>>> X, Y, Z = Y, Z, X # é possível generalizar este caso
```

```
>>> print(X, Y, Z) # para qualquer quantidade de
```

```
0 10 -10 # objetos envolvidos
```

2.3.3 Expressões aritméticas

As expressões aritméticas são construídas utilizando-se objetos, operadores aritméticos e funções matemáticas, sendo que toda linguagem de programação permite a construção de operações aritméticas. Uma expressão aritmética é algo do tipo

$$R = A + B$$

em que: A e B são objetos numéricos e R recebe o resultado de sua adição. Nessa expressão, A e B são chamados de operandos e “+” é o operador aritmético de adição.

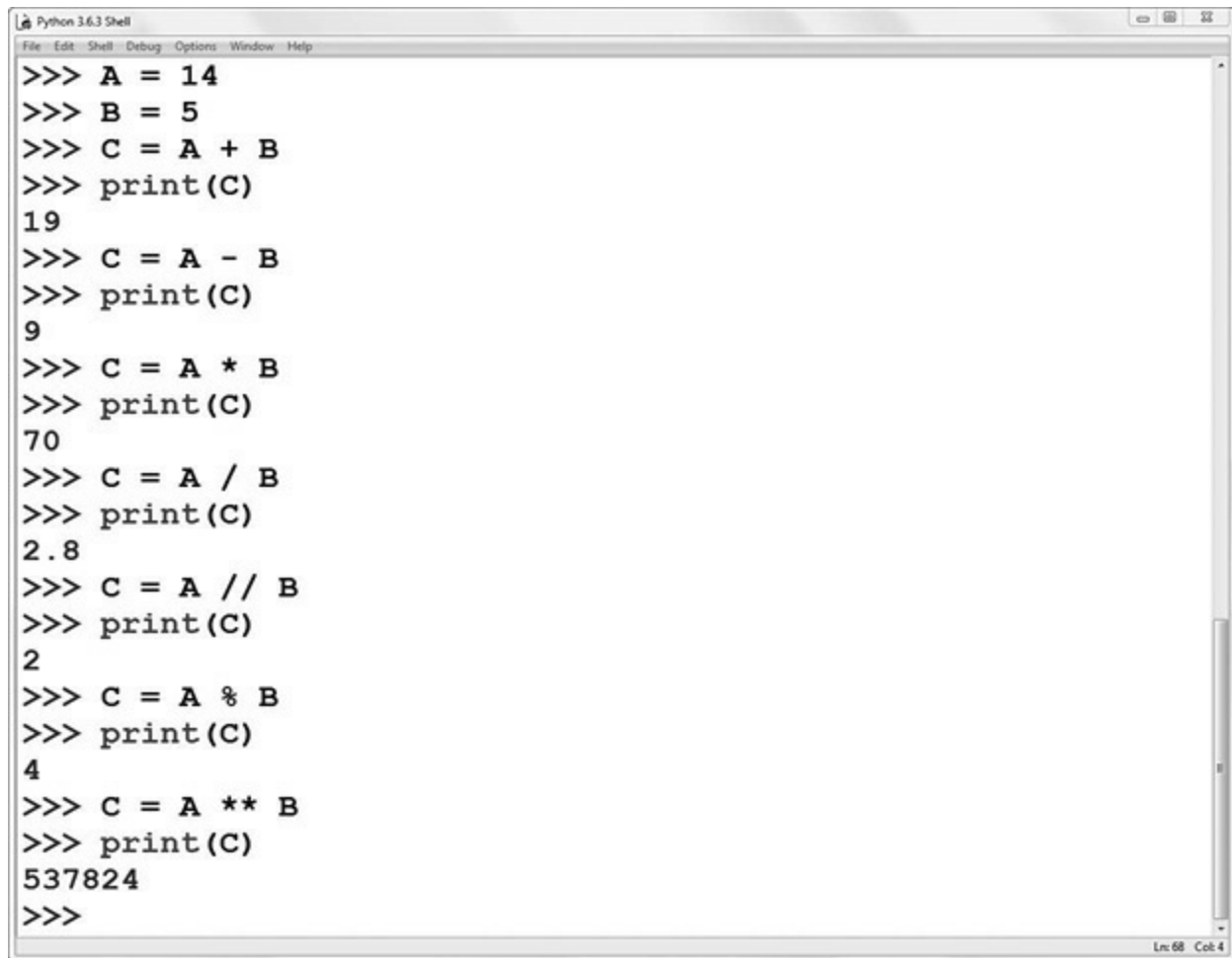
Como vimos, em Python estão disponíveis três tipos numéricos: inteiros, reais e complexos. Com esses três tipos é possível realizar operações aritméticas.

É possível misturar objetos de diferentes tipos numéricos em uma única expressão. Quando houver uma situação assim, o interpretador Python buscará a melhor maneira de resolvê-la. Havendo, em uma expressão, a mistura de operandos inteiros e reais, o resultado calculado será real. E quando houver inteiros, reais e complexos, o valor resultante será tratado como complexo.

Os operadores aritméticos disponíveis em Python são os indicados no Quadro 2.1. Execute todos os exemplos da tabela com os valores: $A = 14$ e $B = 5$. Obedeça ao esquema a seguir e compare os resultados que você obteve com os resultados esperados indicados no Quadro 2.1.

Operação	Operador	Exemplo	Resultado esperado
Adição	+	$C = A + B$	19
Subtração	–	$C = A - B$	9
Multiplicação	*	$C = A * B$	70
Divisão	/	$C = A / B$	2.8
Divisão inteira	//	$C = A // B$	2
Resto (módulo)	%	$C = A \% B$	4
– unário	–	$C = -A$	–14
Potenciação	**	$C = A ** B$	537824

Quadro 2.1 Operadores aritméticos em Python.

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main area shows a series of Python commands and their outputs. The commands are: A = 14, B = 5, C = A + B, print(C), C = A - B, print(C), C = A * B, print(C), C = A / B, print(C), C = A // B, print(C), C = A % B, print(C), and C = A ** B, print(C). The outputs are: 19, 9, 70, 2.8, 2, 4, and 537824. The status bar at the bottom right shows 'Ln: 68 Col: 4'.

```
>>> A = 14
>>> B = 5
>>> C = A + B
>>> print(C)
19
>>> C = A - B
>>> print(C)
9
>>> C = A * B
>>> print(C)
70
>>> C = A / B
>>> print(C)
2.8
>>> C = A // B
>>> print(C)
2
>>> C = A % B
>>> print(C)
4
>>> C = A ** B
>>> print(C)
537824
>>>
```

Figura 2.5 Uso dos operadores aritméticos.

É importante ressaltar que na linguagem Python, praticamente todos os operadores apresentados no Quadro 2.1 estão disponíveis para serem utilizados com os três tipos numéricos definidos na linguagem: int, float e complex.

Há duas exceções, no entanto, que são os operadores de cálculo de divisão de inteiros e resto que não estão definidos para os tipos complexos.

2.3.4 Construção de expressões aritméticas com múltiplos operadores

Em programação, é comum precisar escrever expressões aritméticas envolvendo dois ou mais operadores aritméticos. Nesses casos, a ordem de

prioridade entre os operadores deve ser observada. O operador de maior prioridade sempre será calculado antes. Na expressão a seguir, primeiro será calculada a multiplicação entre 2 e A, e ao resultado será adicionado B.

$$R = 2 * A + B$$

Onde for necessário, pode-se alterar a prioridade das operações utilizando parênteses de maneira apropriada. Assim, se o desejado para essa expressão fosse somar A e B primeiro e multiplicar o resultado dessa soma por 2 em seguida, então, a expressão deve ser escrita como:

$$R = 2 * (A + B)$$

Em programação, em qualquer linguagem, incluindo Python, as regras de precedência da álgebra são estritamente respeitadas e, se for preciso, pode-se abrir tantos parênteses quanto necessário.

Uma expressão aritmética muito utilizada nos algoritmos é aquela em que se toma o conteúdo de um objeto e a ele se soma (ou dele se subtrai etc.) um valor ou outra variável. Para efetuar uma operação, assim se escreve:

$$A = A + 1$$

Em Python, nestes casos, pode-se utilizar a operação de atribuição incremental, e a expressão ficará assim:

$$A += 1$$

Mais opções dessa operação estão exemplificadas no Exemplo 2.6.

Exemplo 2.6 Usos da atribuição incremental

```
>>> A = 10
```

```
>>> A
```

```
0
```

```
>>> A += 1 # atribuição incremental: adição
```

```
11
```

```
>>> A -= 5 # atribuição incremental: subtração
```

```
>>> A
```

```
6
```

```
>>> A *= 2 # atribuição incremental: multiplicação
```

```
>>> A
```

```
12
```

```
>>> A /= 4 # atribuição incremental: divisão
```

```
>>> A
```

```
3.0
```

```
>>> A = 10
```

```
>>> P = 4
```

```
>>> A += P # todas essas operações também
```

```
>>> A # podem ser feitas usando um objeto
```

14 # no lugar do valor literal

2.4 Funções matemáticas

Junto com os operadores mostrados no Exemplo 2.6, em Python pode-se utilizar uma gama muito grande de funções matemáticas. Parte dessas funções está na biblioteca-padrão (em inglês, denominada pelo termo *built-in*), e outra parte está nas bibliotecas de funções “math” e “cmath”, que fornecem ao programador uma grande variedade de funções matemáticas prontas.

A biblioteca-padrão está sempre disponível, e não é necessário utilizar nenhum comando específico para carregá-la.

A biblioteca “math” contém funções que suportam apenas tipos inteiros e reais e precisa ser carregada para ser utilizada.

A biblioteca “cmath” contém funções que suportam tipos complexos e precisa ser carregada para ser utilizada.

Para conhecer todas as funções da biblioteca-padrão consulte a seção 2 da referência “*The Python Standard Library*”, cujo caminho em *Python Docs* é:

Python » Documentation » The Python Standard Library » 2. Built-in Functions

Para conhecer todas as funções suportadas em math e cmath, consulte as Seções 9.2 e 9.3, respectivamente, da referência “*The Python Standard Library*” cujo caminho em *Python Docs* é:

Python » Documentation » The Python Standard Library » 9. Numeric and Mathematical Modules

Para utilizar tais bibliotecas, é necessário, primeiro, carregar a biblioteca desejada por meio do comando “from ... import ...”

```
>>> from math import sqrt
```

```
>>> x = 9
```

```
>>> r = sqrt(x)
```

```
>>> print(r)
```

3.0

O Quadro 2.2 relaciona algumas funções matemáticas importantes, indicando o que fazem e a qual biblioteca pertencem. Essa lista é um subconjunto do que existe. Consulte as referências indicadas para conhecer tudo o que está disponível.

Função	Descrição	Observação
abs(x)	Valor absoluto (módulo) de x.	Bib. padrão
int(x)	Converter x para inteiro eliminando sua parte decimal. O conteúdo de x deve ser real.	Bib. padrão
float(x)	Converte x para número real. O conteúdo de x deve ser inteiro.	Bib. padrão
round(x[, n])	Arredonda x com n dígitos decimais. Se n for omitido, o valor 0 é assumido.	Bib. padrão
trunc(x)	O valor x é truncado, ou seja, a parte decimal é eliminada. Na prática, equivale ao int(x).	Bib. math
floor(x)	Retorna o maior inteiro $\leq x$.	Bib. math
ceil(x)	Retorna o menor inteiro $\geq x$.	Bib. math
sqrt(x)	Calcula a raiz quadrada de x.	Bib. math e cmath

Função	Descrição	Observação
exp(x)	Retorna o exponencial de x, ou seja, e^x .	Bib. math e cmath
log (x[, base])	Retorna o logaritmo de x na base fornecida. Se a base for emitida, calcula o logaritmo natural.	Bib. math e cmath
		Bib. math e

<code>sin(x)</code>	Retorna o seno do ângulo x radianos.	<code>cmath</code>
<code>cos(x)</code>	Retorna o cosseno do ângulo x radianos.	Bib. <code>math</code> e <code>cmath</code>
<code>tan(x)</code>	Retorna a tangente do ângulo x radianos.	Bib. <code>math</code> e <code>cmath</code>
<code>rect(r, phi)</code>	Converte um número complexo expresso em coordenadas polares para sua representação retangular.	Bib. <code>cmath</code>
<code>polar(x)</code>	Retorna a representação de x em coordenadas polares. Retorna uma tupla com o par (r, phi), em que r é o módulo e phi é a fase.	Bib. <code>cmath</code>

Quadro 2.2 Lista de funções matemáticas.

2.5 Comando de exibição – `print`

O propósito do comando `print` é a exibição na tela de qualquer informação relevante ao usuário do programa. Pode-se raciocinar em termos de que o `print` é o mais básico comando existente para que o programa “se comunique” com quem o está utilizando.

Exemplo 2.7 Uso do comando `print`

```
>>> print("Este é o Capítulo 2 do livro") # caso 1
```

```
Este é o Capítulo 2 do livro
```

```
>>> A = 12
```

```
>>> print(A) # caso 2
```

```
12
```

```
>>> B = 19
```

```
>>> print(B) # outro caso 2
```

```
19
```

```
>>> print(A, B) # caso 3
```

```
12 19
```

```
>>> print("Valor de A =", A) # caso 4
```

```
Valor de A = 12
```

```
>>> print("Valor de A = {0} e valor de B = {1}".format(A, B)) # c.5
```

```
Valor de A = 12 e valor de B = 19
```

Com o `print`, é possível mostrar mensagens de texto, conteúdos de objetos ou uma combinação das duas coisas, como pode ser visto no Exemplo 2.7.

Nesse exemplo, o `print` do caso 1 exibe uma mensagem de texto. Note que o texto deve ser escrito entre aspas, que podem ser duplas (""") ou simples ('), para ser exibido como mensagem. A escolha do tipo de aspas cabe ao programador, é uma questão de gosto pessoal. O Python interpretará os dois tipos de aspas de maneira totalmente equivalente. Porém, é importante que não os misture, ou seja, se iniciou o texto com um tipo, deve finalizá-lo com ele.

Os prints identificados como caso 2 exibem um objeto cada um. Nesse caso, a diferença é que o nome do objeto é colocado no `print` sem o uso de aspas, e o que é exibido na tela é o conteúdo do objeto.

No `print` do caso 3, são exibidos simultaneamente os conteúdos de dois objetos. Isso faz que os valores sejam exibidos na mesma linha separados por um espaço em branco. Em casos assim, é possível alterar o caractere

separador especificando-se um ou mais caracteres alternativos por meio do parâmetro *sep*, como mostrado no Exemplo 2.8.

Exemplo 2.8 Uso do comando `print` com separador

```
>>> print(A, B, sep="-")
```

```
12-19
```

```
>>> print(A, B, sep=", ")
```

```
12, 19
```

No `print` do caso 4, é exibida uma mensagem seguida do conteúdo de um objeto, e como o parâmetro *sep* não foi especificado, foi inserido o espaço em branco padrão.

Por fim, no `print` do caso 5 do Exemplo 2.7, é mostrado como produzir uma saída formatada. Esse tipo de saída é muito útil para produzir exibições nas quais é possível controlar diversos detalhes dos elementos envolvidos.

Para produzir uma saída formatada, o primeiro passo é escrever a mensagem que se quer ver na tela, tomando o cuidado de utilizar os identificadores `{0}`, `{1}`, `{2}` etc. nos pontos da mensagem onde se deseja que apareça o conteúdo dos objetos envolvidos. O texto da mensagem deve ser seguido do método *format*, que conterà como argumentos os objetos que fornecerão os valores que substituirão os identificadores entre chaves.

A substituição dos identificadores pelos argumentos é feita seguindo-se o índice numérico, ou seja, nesse exemplo o conteúdo do objeto *A* substituirá o identificador `{0}`, porque *A* é o primeiro argumento, e o conteúdo de *B* substituirá o identificador `{1}`, independentemente do local em que esses identificadores estejam posicionados no texto.

É possível omitir o número dentro das chaves dos identificadores e utilizar apenas {}. Nesse caso, a associação entre identificador e objeto será feita pela ordem de ocorrência.

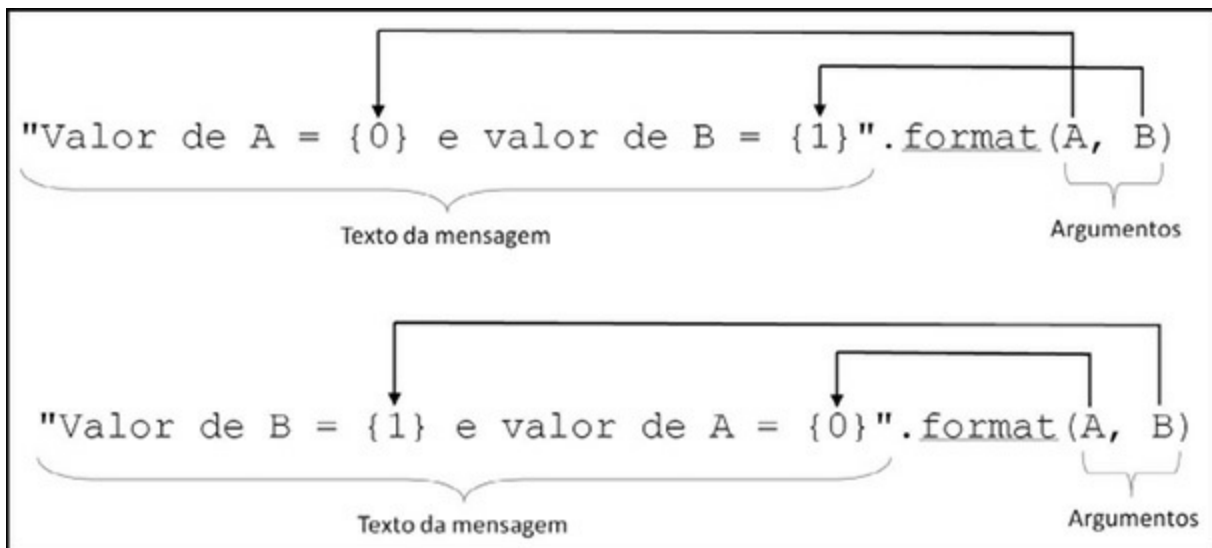


Figura 2.6 Exemplo de mensagem formatada.

Adicionalmente, os identificadores podem receber qualificadores de formatação que determinam como os dados devem ser apresentados. Isso se faz acrescentando “:”, um caractere de formatação e o tamanho, ficando assim: {0:d} ou {0:6.2f} no caso de identificadores numerados; e {:d} ou {:6.2f} no caso de numeração omitida.

Também é possível especificar se o dado será alinhado à esquerda, à direita ou centralizado, utilizando-se, respectivamente, os caracteres “<”, “>”, “^”.

Os tipos disponíveis são muito amplos, e no Quadro 2.3 são apresentados alguns de uso mais frequente. Para que se conheçam todas as opções com todos os detalhes existentes, é necessário recorrer à documentação oficial do Python referente à formatação de strings (disponível em <<https://docs.python.org/3.6/library/string.html>>).

Formatação	Resultado	Descrição

"Dado = {0:d}". format(A)	Dado = 9	d – número inteiro, em base 10.
"Dado = {0:5}". format(A)	Dado = 9	5d – número inteiro ocupando no mínimo 5 caracteres alinhado à direita.
"Dado = {0:f}". format(X)	Dado = 4.860000	f – número real, exibindo o padrão de 6 casas após a vírgula.
"Dado = {0:2f}". format(X)	Dado = 4.86	f – número real, exibindo 2 casas após a vírgula.
"Dado = {0:6.3f}". format(X)	Dado = 4.860	f – número real, ocupando no mínimo 6 caracteres e exibindo 1 casa após a vírgula.
"qq{:7d}qq". format(A)	qq 9qq	7d – número inteiro ocupando no mínimo 7 caracteres alinhado à direita.
"qq{:<7d}qq". format(A)	qq9 qq	7d – número inteiro ocupando no mínimo 7 caracteres alinhado à esquerda.
"qq{:^7d}qq". format(A)	qq 9 qq	7d – número inteiro ocupando no mínimo 7 caracteres centralizado.

Quadro 2.3 Formatação de exibição em tela.

Dica

Existe uma forma alternativa de trabalhar com strings formatados em Python. Ao fazer buscas pela internet, é muito provável que se depare com essa outra forma, que é muito parecida, porém, é diferente. Observe com atenção as duas linhas a seguir, sabendo de antemão que ambas produzem exatamente o mesmo resultado.

```
print("Valor de A = {0} e valor de B = {1}".format(A, B))
```

```
print("Valor de A = %d e valor de B = %d" % (A, B))
```

Nessa segunda opção utiliza-se “%d” no lugar dos identificadores {0} e {1}. E no lugar do método forma” utiliza-se o operador “%”.

Essa forma assemelha-se muito ao modo como a linguagem C e algumas outras linguagens formatam suas saídas. Por que utilizar a

primeira forma, então? A resposta encontra-se na documentação oficial do Python, na qual é declarado que se trata de uma forma obsoleta e que pode não ser suportada no futuro. Para saber, mais acesse a documentação:

Python » Documentation » The Python Standard Library » 2. Text Sequence Type » printf-style String Formatting

2.6 Comando de entrada de dados – input

Toda linguagem de programação apresenta um ou mais comandos relacionados à entrada de dados por meio do teclado. Tal tipo de comando tem como propósito permitir que o usuário digite o dado de entrada no teclado.

Em Python 3, o comando para isso é o *input*. Esse comando tem um parâmetro string opcional que é exibido na tela antes de iniciar a leitura, a qual, uma vez iniciada, será concluída ao pressionar a tecla *Enter*. O que tiver sido digitado é carregado em um objeto de destino. Caso o objeto de destino não exista, será criado nesse momento. Sua forma de uso é mostrada no Exemplo 2.9, em que o objeto “x” é criado e recebe o retorno do input.

Exemplo 2.9 Uso do comando input

```
>>> x = input("Digite algo: ")
```

```
Digite algo: teste de digitação
```

```
>>> x
```

```
'teste de digitação'
```

```
>>> n = input("Digite um número inteiro: ")
```

Digite um número inteiro: 2

```
>>> print(n)
```

2

```
>>> type(n)
```

```
<class 'str'>
```

```
>>> f = input("digite um número real: ")
```

digite um número real: 4.83

```
>>> print(f)
```

4.83

```
>>> type(f)
```

```
<class 'str'>
```

```
>>>
```

Nesse exemplo, o primeiro comando `input` apresenta a mensagem “Digite algo:”, indicando ao usuário o que deve ser feito. Qualquer coisa pode ser digitada e, após pressionar *Enter*, o objeto `x` é carregado com o que quer que tenha sido digitado. A leitura sempre resulta em uma cadeia de texto carregada no objeto de destino. Se forem digitados apenas algarismos, ainda assim a leitura resultará em uma cadeia de caracteres. Isso pode ser constatado por meio dos objetos `n` e `f`, nos quais, aparentemente, foram digitados o que seriam, respectivamente, um número inteiro e um real.

Porém, ao utilizar o comando *type*, verifica-se que ambos são do tipo string (str).

Em resumo, o comando input retorna exclusivamente cadeias de caracteres. Como fazer, então, caso se necessite ler números inteiros ou reais? A resposta para isso são as funções de conversão de tipo.

2.7 Funções de conversão entre tipos simples

Estas funções permitem realizar a conversão entre tipos de dados simples, conforme indicado no Quadro 2.4.

Função	Descrição
str (argumento)	Converte o argumento para cadeia de texto.
int (argumento)	Converte o argumento para um número inteiro, se for possível. Caso não seja possível, gera um erro.
float (argumento)	Converte o argumento para um número real, se for possível. Caso não seja possível, gera um erro.

Quadro 2.4 Funções de conversão de tipo.

O Exemplo 2.10 mostra diversos casos de conversão utilizando essas funções.

Exemplo 2.10 Uso das funções de conversão de tipo

```
>>> x = '19'
```

```
>>> type(x)
```

```
<class 'str'>
```

```
>>> a = int(x)
```

```
>>> type(a)
```

```
<class 'int'>
```

```
>>> x = '3.75'
```

```
>>> type(x)
```

```
<class 'str'>
```

```
>>> r = float(x)
```

```
>>> type(r)
```

```
<class 'float'>
```

```
>>> b = a + r
```

```
>>> print(b)
```

```
22.75
```

```
>>> x = str(b)
```

```
>>> print(x)
```

```
22.75
```

```
>>> type(x)
```

```
<class 'str'>
```

```
>>>
```

Unir essas funções com o comando input é uma possibilidade utilizada em Python para a leitura de objetos com conteúdo numérico, seja inteiro ou real, da seguinte maneira:

```
N = int(input("Digite um número inteiro"))
```

ou

```
F = float(input("Digite um número inteiro"))
```

2.8 Comentários no código

A inserção de comentários no código do programa é uma prática normal. Em função disso, toda linguagem de programação tem alguma maneira de permitir que comentários sejam inseridos nos programas. O objetivo é adicionar descrições em partes do código, seja para documentá-lo ou para adicionar uma descrição do algoritmo implementado. Os programadores também os utilizam para marcar que determinada linha, ou um conjunto de linhas, não devem ser processadas pelo interpretador, sem precisar excluí-las.

Em Python, existem duas maneiras de inserir comentários.

1.Opção para uma linha: a primeira forma usa o caractere # para comentar uma única linha. Não necessariamente esse caractere precisa ser posicionado no início da linha. Quando esse caractere é utilizado, o interpretador ignorará todo o restante da linha até o seu final. Veja a seguir:

```
# Esta linha inteira é um comentário e o interpretador a ignora
```

```
X = 25 # Daqui para a frente é comentário
```

```
print(X)
```

2.Opção para múltiplas linhas: a segunda forma possível utiliza três aspas duplas para abrir o bloco de comentário com muitas linhas e outras três aspas duplas para fechar o bloco. É possível obter o mesmo resultado colocando aspas simples no lugar das aspas duplas. Essa construção não é exatamente um bloco de comentário. É algo a mais, conhecido como *docstrings*, quando utilizado, por exemplo, dentro de funções (veja o Capítulo 5). Os *docstrings* devem acompanhar a indentação (isso é visto no Capítulo 3) do código. Os *docstrings* não são empregados pelo interpretador para gerar qualquer código executável, e é por esse motivo que são utilizados como comentários.

```
“””
```

Tudo o que estiver entre as três aspas não vai gerar código

pelo interpretador

```
“””
```

```
””
```

Podem ser aspas simples também

```
””
```

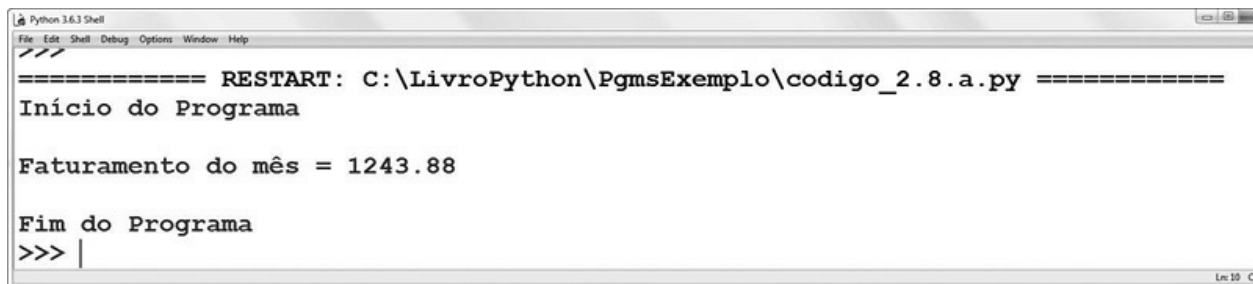
Exercícios resolvidos

1.Escreva um programa que calcule o faturamento de um representante comercial que recebe R\$ 500,00 fixos e 6% de comissão sobre as vendas do mês. Considere que ele fechou o mês com um valor de R\$

12.398,00 em vendas. Exiba o resultado com duas casas decimais.

Exercício resolvido 2.1

```
print("Início do Programa")
print(" ")
Fixo = 500.00
Vendas = 12398.00
Comissao = 6 / 100
Fat = Fixo + Vendas * Comissao
print("Faturamento do mês = {0:.2f}".format(Fat))
print(" ")
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_2.8.a.py =====
Início do Programa

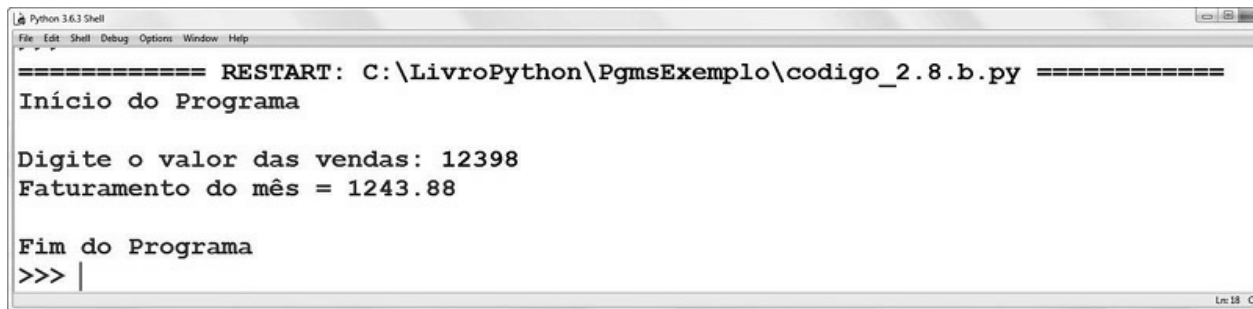
Faturamento do mês = 1243.88

Fim do Programa
>>> |
```

2. Reescreva o programa anterior alterando-o de modo que as vendas do mês sejam lidas do teclado.

Exercício resolvido 2.2

```
Fixo = 500.00
Vendas = float(input("Digite o valor de vendas: "))
Comissao = 6 / 100
Fat = Fixo + Vendas * Comissao
print("Faturamento do mês = {0:.2f}".format(Fat))
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
==== RESTART: C:\LivroPython\PgmsExemplo\codigo_2.8.b.py =====
Início do Programa

Digite o valor das vendas: 12398
Faturamento do mês = 1243.88

Fim do Programa
>>> |
Linha 18 C
```

Exercícios propostos

1. Faça $X = 0.0$ e $Y = 18$. Verifique o tipo de dado que o Python atribuiu a cada um. Faça $Z = X + Y$ e verifique o resultado calculado e armazenado em Z . Verifique com qual tipo de dado foi criado o objeto Z .
2. Atribua um valor qualquer a um objeto a (minúsculo). Utilize o comando *type* ou o comando *print* com o objeto A (maiúsculo). Relate o que aconteceu.
3. No IDLE, faça $A = \text{"Questão 3"}$, $B = 25$ e $C = 3.9$. Utilize o comando *type* para verificar qual é o tipo de dado dos objetos A , B e C .
4. Reproduza em um programa todos os casos de operações aritméticas do Quadro 2.1, para $A = 14$ e $B = 5$, e compare os valores obtidos por você com os valores esperados constantes do quadro.
5. Escreva a sequência de comandos necessária para o cálculo da área de um triângulo de base 9 e altura 6.
6. Refaça o exercício 5 alterando-o de modo que a base e a altura do triângulo sejam lidas do teclado. Considere-as números reais.
7. Escreva a sequência de comandos para calcular o salário bruto de um

profissional que ganha por hora, sabendo que ele ganha R\$ 14,25/h e trabalhou 163 horas normais e 20 horas extras (pagam o dobro).

8. Escreva em Python as seguintes expressões aritméticas para as fórmulas a seguir e teste as quatro primeiras para os valores $A = 4$, $B = 5$, $C = 1$ e a última para os valores $x_1 = 1$, $y_1 = 1$, $x_2 = 4$ e $y_2 = 5$.

8.a
$$R = \frac{A+B}{2}$$

8.b
$$x = \frac{-B + \sqrt{B^2 - 4AC}}{2A}$$

8.c
$$R = \frac{3A+2B}{A+B}$$

8.d
$$z = 7.6A - B^{1.7}$$

8.e
$$d = \sqrt{(x_1 - x_2)^2 + y_1^2 - y_2^2}$$

9. Escreva um programa que leia do teclado as coordenadas (x_1, y_1) do ponto 1 e (x_2, y_2) do ponto 2. Utilizando a expressão do item 8.e, determine a distância entre esses dois pontos e exiba-a na tela com três casas decimais. Teste-o com os dados da tabela a seguir.

x1	y1	x2	y2	d
0,0	0,0	3,0	4,0	5,000
2,0	1,0	0,0	5,0	4,472
-3,0	1,5	7,1	5,5	10,863
0,0	3,5	0,0	7,0	3,500

8,2 2,5 -5,0 -5,0 15,182
6,9 2,0 16,0 -1,8 9,862

10. Um vendedor ambulante vendeu os produtos indicados na tabela a seguir. Informe quanto ele faturou com cada produto e quanto ele faturou no total.

Produto	Quantidade vendida	Valor unitário R\$
Boneco Malandrinho	17	18,50
Spinner Pequeno	36	12,00
Cubo Mágico	7	5,90

Todos os dados devem ser lidos do teclado, sendo que o nome do produto é string, a quantidade vendida é um número inteiro e o valor unitário é um número real.



Controle de Fluxo

Objetivos

Neste capítulo, serão apresentados os comandos da linguagem Python que são essenciais para controlar o fluxo de execução dos programas, a saber: comando condicional if-else; comando de laço while; estrutura de tratamento de exceções try-except.

Por controle de fluxo em um programa entende-se a ordem lógica de execução dos comandos que o compõem, bem como os desvios nessa ordem necessários em função de certas condições que possam ocorrer.

O estudo deste capítulo é muito importante para a construção da lógica dos

programas.

3.1 Comando condicional

É comum que, em um algoritmo, seja necessária a tomada de decisões baseadas em valores contidos em objetos. Por exemplo, considere um algoritmo que tenha dois objetos de tipo inteiro A e B previamente carregados. Caso seja necessário calcular a divisão de A por B e o conteúdo do objeto B for zero, ocorrerá um erro, como pode ser visto no código a seguir. Isso ocorre porque divisões por zero não são permitidas.

Exemplo 3.1 Erro causado por uma divisão por zero

```
>>> A = 16
```

```
>>> B = 0
```

```
>>> R = A / B
```

Traceback (most recent call last):

File "<pyshell#2>", line 1, in <module>

```
R = A / B
```

ZeroDivisionError: division by zero

No caso desse algoritmo, a situação de erro é indesejável, então, é preciso tomar o cuidado de evitá-la. Uma das maneiras de se conseguir isso é utilizar o comando condicional if-else.

Ao utilizá-lo, será necessário formular uma condição cujo resultado será falso ou verdadeiro, e em função desse resultado o programa será escrito de

modo a executar diferentes comandos em cada caso. Então, pode-se formular a seguinte ideia: “se B for igual a zero, então apresente a mensagem ‘Não é possível calcular a divisão’, senão (ou seja, B é diferente de zero) calcule e apresente na tela A / B ”.

Agora, é preciso escrever isso em Python. Assim, tem-se o exemplo a seguir, em que é feita a leitura dos objetos A e B utilizando-se o comando `input`.

Exemplo 3.2 Uma possível maneira de evitar o erro mostrado no Exemplo 3.1

```
A = int(input("Digite um valor para A: ")) # linha 1
```

```
B = int(input("Digite um valor para B: ")) # linha 2
```

```
if B == 0: # linha 3
```

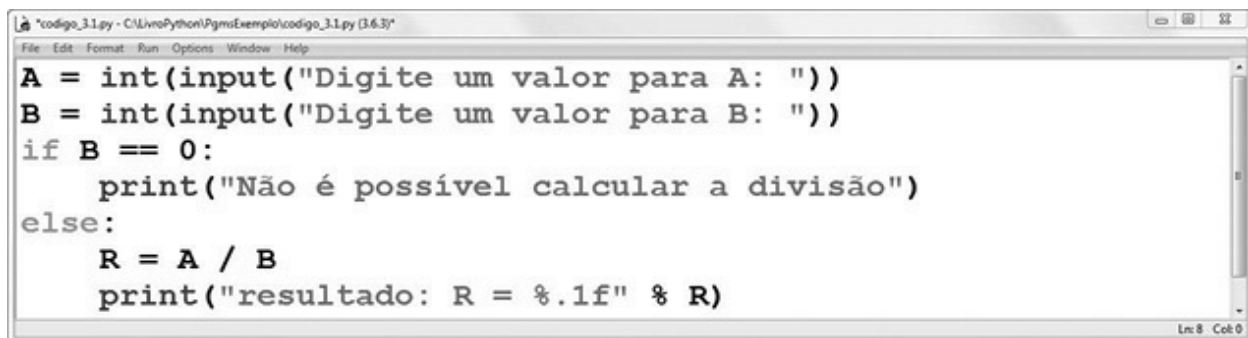
```
    print("Não é possível calcular a divisão") # linha 4
```

```
else: # linha 5
```

```
    R = A / B # linha 6
```

```
    print("resultado: R = %.1f" % R) # linha 7
```

Teste esse pequeno programa no Python escrevendo-o na forma de script. Para isso, abra o Python e acione o comando do menu *File* → *New File*, escreva o programa exibido no Exemplo 3.2, salve-o e execute-o (para executá-lo, utilize o comando *Run* ou a tecla de atalho F5). A Figura 3.1 mostra como ficará esse programa após sua digitação no editor do Python.

A screenshot of a Python IDE window titled "codigo_3.1.py - C:\Users\Python\PgmExemplo\codigo_3.1.py (3.6.3)". The window contains the following Python code:

```
A = int(input("Digite um valor para A: "))
B = int(input("Digite um valor para B: "))
if B == 0:
    print("Não é possível calcular a divisão")
else:
    R = A / B
    print("resultado: R = %.1f" % R)
```

The code prompts the user for two integers, A and B. If B is zero, it prints a message indicating that division is not possible. Otherwise, it calculates the division of A by B and prints the result formatted to one decimal place.

Figura 3.1 Uso do comando condicional if-else.

Na execução do Exemplo 3.2, caso seja fornecido o valor zero para B, será apresentada a mensagem “Não é possível calcular a divisão” e, caso B seja diferente de zero, então, será apresentado o resultado do cálculo. Rode algumas vezes esse programa, testando-o com diferentes valores para A e B.

3.1.1 Explicando em detalhes o Exemplo 3.2

Nas linhas 1 e 2, é feita a leitura dos objetos A e B, de modo que qualquer valor inteiro pode ser inserido para qualquer um deles.

Na linha 3 há o comando if (se) e sua condição. Nessa condição, a pergunta que se está fazendo é se o conteúdo de B é igual a zero. Para isso, foi utilizado o operador relacional “==”, que avalia se B que está do lado esquerdo contém um valor igual a zero, que está do lado direito. Essa condição será avaliada pelo processador e um resultado será gerado. Esse resultado pode ser falso ou verdadeiro. Caso seja verdadeiro, o programa seguirá para a linha 4 e executará o print. Caso seja falso, o programa desviará (pulará) a linha 4 e seguirá para a execução das linhas 6 e 7, que estão subordinadas ao else (senão) da linha 5.

Note que há um caractere “:” (dois-pontos) no final das linhas 3 e 5. Em Python, é obrigatória a inserção desse caractere no comando if-else, pois é por meio dele que o interpretador Python identifica o término do cabeçalho do comando e o início dos comandos que lhe estão subordinados.

Essa relação de subordinação é importante na lógica do algoritmo. Nesse exemplo, a linha 4 está subordinada ao if da linha 3 e as linhas 6 e 7 estão subordinadas ao else da linha 5.

3.1.2 Identação

O interpretador Python identifica a relação de subordinação descrita no parágrafo anterior pelo recuo que há na digitação das linhas do programa. Note que as linhas subordinadas estão digitadas com alguns espaços em branco à esquerda. Isso recebe o nome de indentação e, em Python, ela é obrigatória sempre que houver um ou mais comandos subordinados a outro. O else, por sua vez, não é indentado, e para que o programa fique correto é preciso que ele fique exatamente no mesmo alinhamento do if ao qual está associado.

Generalizando, em Python, todo conjunto de comandos subordinados deve estar indentado em relação ao seu comando proprietário. Isso vale para if-else, while, for, try, def e qualquer outro em que exista a relação de subordinação.

3.1.3 Construindo condições simples

No exemplo anterior, foi construída uma condição que avaliava se um valor contido no objeto B era igual a zero ou não. Para isso, utilizou-se a construção `B == 0`, onde B é um objeto e 0 é um número literal. A construções desse tipo dá-se o nome de condições simples.

Generalizando, as condições podem ser escritas da seguinte maneira:

{Expressão esquerda} {operador} {Expressão direita}

em que as **expressões** em ambos os lados podem ser:

- um literal (geralmente número ou texto);

- um objeto;
- uma fórmula (expressão aritmética);
- uma chamada de função (ver Capítulo 5).

O **operador** é um dos seis operadores relacionais exibidos no Quadro 3.1. No caso dos operadores que contêm dois caracteres, não é permitido haver espaço em branco entre eles.

Operador	O que ele faz
<code>==</code>	Igual a
<code>!=</code>	Diferente de
<code><</code>	Menor que
<code><=</code>	Menor ou igual a
<code>></code>	Maior que
<code>>=</code>	Maior ou igual a

Quadro 3.1 Operadores relacionais.

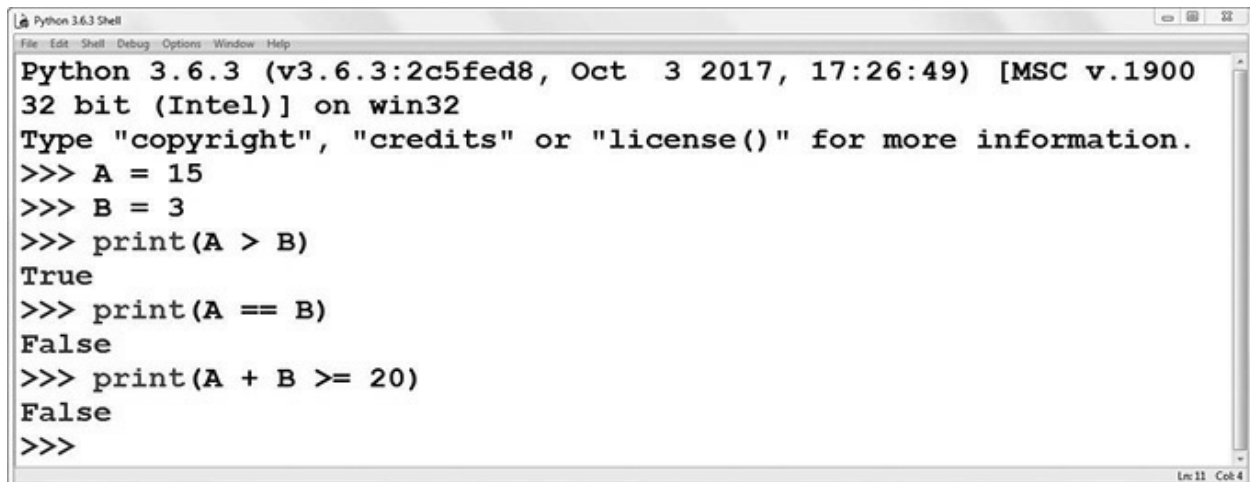
O Quadro 3.2 exemplifica a construção de condições simples.

Condição	Interpretação	Elementos envolvidos
<code>A > 0</code>	A maior que zero	Compara objeto com literal numérico (0).
<code>X <= Y</code>	X menor ou igual a Y.	Compara dois objetos.
<code>X != A + B</code>	X é diferente de A + B	Compara objeto com o resultado da expressão aritmética.
<code>C > 2*(A+B)</code>	C é maior que 2(A+B)	Compara os resultados de duas expressões aritméticas.
<code>10*A < 100*B</code>	10A é maior que 100B	Compara os resultados de duas expressões aritméticas.
<code>S == ""</code>	S igual a string vazio	Compara objeto com literal texto vazio.

<code>S != "SIM"</code>	S diferentes de "SIM"	Compara objeto com literal texto não vazio.
-------------------------	-----------------------	---

Quadro 3.2 Exemplos de condições simples.

No IDLE, atribua valores aos objetos sugeridos nos exercícios e utilize o comando `print` para exibir o resultado produzido pela condição, conforme exemplificado.



```

Python 3.6.3 (v3.6.3:2c5fed8, Oct  3 2017, 17:26:49) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> A = 15
>>> B = 3
>>> print(A > B)
True
>>> print(A == B)
False
>>> print(A + B >= 20)
False
>>>

```

Figura 3.2 Exemplo de testes de condições simples.

3.1.4 Construindo condições compostas

Muitas vezes, é preciso negar uma condição simples ou combinar duas ou mais condições simples em uma condição composta. Assim, as condições simples vistas anteriormente são a base para a construção desse novo tipo de condição.

A construção de uma condição composta tem uma das seguintes formas:

`not {Condição 1}`

`{Condição 1} {Operador Lógico and/or} {Condição 2}`

em que as condições 1 e 2 são duas condições simples, como as que já foram vistas no Item 3.1.3.

O operador lógico é um dos operadores apresentados no Quadro 3.3. Antes

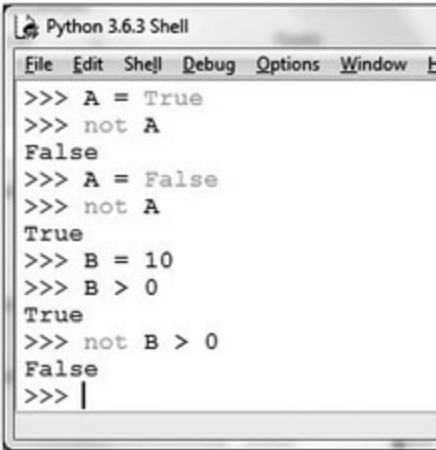
de seguir adiante, é preciso saber como avaliar expressões que contenham esses operadores not, and e or.

Operador	O que ele faz	Descrição
not	Negação	Nega a condição à qual é aplicado.
and	Conjunção operação lógica E	Resultado verdadeiro se forem verdadeiras as duas condições às quais é aplicado.
or	Disjunção operação lógica OU	Resulta verdadeiro se for verdadeira pelo menos uma das duas condições às quais é aplicado.

Quadro 3.3 Operadores lógicos.

A Figura 3.3 traz a forma de avaliação do operador lógico not e um exemplo de uso.

Tabela Verdade do operador not not C1 produz a negação do resultado de C1	
Condição C1	not C1
False	True
True	False



```

Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
>>> A = True
>>> not A
False
>>> A = False
>>> not A
True
>>> B = 10
>>> B > 0
True
>>> not B > 0
False
>>> |

```

Figura 3.3 Exemplo de uso do operador lógico not.

A Figura 3.4 mostra a forma de avaliação do operador lógico and e um exemplo de uso.

```

Python 3.6.3 (v3.6.3:2c5fed8, Oct 3 2017, 17:26:49) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> Base = 10
>>> Altura = 4
>>> Area = Base * Altura
>>> print(Area)
40
>>> type(Base)
<class 'int'>
>>> type(Altura)
<class 'int'>
>>> type(Area)
<class 'int'>
>>>

```

Figura 3.4 Exemplo de uso do operador lógico and.

Já a Figura 3.5 traz a forma de avaliação do operador lógico or e um exemplo de uso.

Tabela Verdade do operador or		
Para que or resulte verdadeiro pelo menos um dos dois, C1 ou C2, deve ser verdadeiro		
Condição C1	Condição C2	C1 or C2
False	False	False
False	True	True
True	False	True
True	True	True

```

Python 3.6.3 Shell
File Edit Shell Debug Options Window H
>>> print(False or False)
False
>>> print(False or True)
True
>>> print(True or False)
True
>>> print(True or True)
True
>>>

```

Figura 3.5 Exemplo de uso do operador lógico or.

Condição	Interpretação
$A > 0$ and $B > 0$	O resultado da condição será verdadeira se A e B forem ambos iguais a zero.
$X \leq Y$ and $Y \neq 0$	O resultado da condição composta será verdadeiro somente se X for menor ou igual Y, ao mesmo tempo que Y seja diferente de zero.
$X == 0$ or $X > 2000$	O resultado da condição composta será verdadeiro se X for igual a zero ou se X for maior que 1000.
$A < 0$ or $B < 0$	O resultado da condição composta será verdadeiro se pelo menos um dos objetos A e B contiver valor negativo.

not (X == 2)	O resultado da condição composta será verdadeiro se X for diferente de 2. Equivale a $X \neq 2$.
--------------	---

Quadro 3.4 Exemplos de condições compostas e sua interpretação.

3.1.5 Condições compostas mistas

Em uma única condição composta é possível misturar not, and e or. Quando isso ocorre, é necessário ter atenção à precedência com que esses operadores são considerados. Existe uma ordem de prioridade a ser respeitada. Essa prioridade obedece à seguinte ordem: not primeiro, and em seguida e or por último.

Assim, na avaliação de uma condição composta mista, a prioridade supracitada sempre será seguida. É necessário ter o devido cuidado ao construir condições assim e verificar que a falta de atenção pode levar a erros. Como exemplo, veja as duas expressões a seguir e avalie-as para $A = 15$, $B = 9$, $C = 9$.

$B == C$ or $A < B$ and $A < C$ Resultará Verdadeiro

$(B == C$ or $A < B)$ and $A < C$ Resultará Falso

O uso de parênteses serve para alterar a ordem de prioridade na avaliação de expressões lógicas. Uma vez inseridos, os parênteses estabelecem qual(is) parte(s) serão avaliada(s) primeiro.

3.1.6 Comando condicional completo

Retomando agora as explicações sobre o comando condicional, a seguir está sua forma completa.

```
if {condição 1}:
    {bloco de comandos 1}
elif {condição 2}:
    {bloco de comandos 2}
```

```
elif {condição 3}:  
    {bloco de comandos 3}  
...  
else:  
    {bloco de comandos do else}
```

As partes if e else já foram explicadas anteriormente. A parte elif permite que sejam utilizadas condições adicionais e confere a possibilidade de tomada de decisão entre múltiplas opções.

A execução desse comando inicia pela avaliação da {condição 1}, e se ela for verdadeira será executado o {bloco de comandos 1} e pulam-se todos os demais; caso a {condição 1} seja falsa, passa-se para a avaliação da {condição 2} e, caso seja verdadeira, será executado o {bloco de comandos 2}, pulando-se os demais, e assim sucessivamente. Ao final, se nenhuma das condições postas for verdadeira, então executa-se o {bloco de comandos do else}.

Não há limites para a quantidade de partes elif a serem utilizadas, de modo que o programador é livre para utilizar tantas dessas partes quanto for a necessidade do algoritmo.

E, por fim, é preciso dizer que as partes elif e else são opcionais, de modo que, se o programador não precisar incluí-las em seu algoritmo, elas podem simplesmente ser omitidas.

Exemplo 3.3 Uso da forma completa if-elif-else

```
PH = float(input("Digite um valor do PH: "))  
  
if PH < 7.0:  
  
    print("Solução ácida")
```

```
elif PH == 7.0:  
  
    print("Solução neutra")  
  
else:  
  
    print("Solução básica")
```

No Exemplo 3.3 é feita a leitura de um número real que representa o valor de pH de uma solução, para o qual há três possibilidades, segundo a química. Caso seja menor que 7,0, a solução é ácida; caso seja igual a 7,0, é neutra; e caso seja maior que 7,0, ela é básica. É exatamente isso que está implementado no código desse exemplo, em que foi empregada a construção if-elif-else para implementá-la.

3.1.7 Comandos condicionais aninhados

É frequente que existam situações em que é necessário colocar um if dentro de outro if. Isso pode ser feito normalmente, conforme mostrado no Exemplo 3.4. O único cuidado é que se respeite a indentação para que a relação de subordinação entre os comandos fique correta.

O que se deseja fazer no Exemplo 3.4 é que três números sejam carregados em A, B e C, e o programa deve mostrá-los em ordem crescente. Na solução, o primeiro if decide se A é o menor; caso seja, então, há um if aninhado a ele para decidir dentre os outros dois, B e C, qual é o menor. Se A não for o menor, então, o elif do primeiro if decide se B é o menor dos três e, caso seja, há um if aninhado a ele para decidir quem é o menor entre A e C. Por fim, o else do primeiro if será executado caso o menor seja C, e aí há um if aninhado para decidir quem é o menor entre A e B. Teste esse programa para as seis combinações possíveis de valor caso os três valores sejam diferentes entre si. Essas combinações são mostradas no Quadro 3.5. Em todos esses

casos, o programa deve exibir na tela a mensagem: “Ordem crescente: 1, 2, 3”.

Para os testes, convida-se o leitor a montar outras combinações em que dois valores sejam iguais e o terceiro seja diferente. Para isso, use os vazios do Quadro 3.5.

A	1	1	2	3	2	3						
B	2	3	1	1	3	2						
C	3	2	3	2	1	1						

Quadro 3.5 Possibilidades de valores para testar o Exemplo 3.4.

Exemplo 3.4 Comandos condicionais aninhados

```
A = int(input("Digite um valor para A: "))
```

```
B = int(input("Digite um valor para B: "))
```

```
C = int(input("Digite um valor para C: "))
```

```
if A <= B and A <= C: # A é o menor dos três
```

```
    if B <= C: # decide quem é o menor entre B e C
```

```
        print("Ordem crescente: {}, {}, {}".format(A, B, C))
```

```
    else:
```

```
        print("Ordem crescente: {}, {}, {}".format(A, C, B))
```

```
    elif B <= A and B <= C: # B é o menor dos três
```

```
        if A <= C: # decide quem é o menor entre A e C
```



```
print("Ordem crescente: {}, {}, {}".format(B, A, C))

else:

    print("Ordem crescente: {}, {}, {}".format(B, C, A))

else: # C é o menor dos três (opção que sobrou, por isso else)

    if A <= B: # decide quem é o menor entre A e B

        print("Ordem crescente: {}, {}, {}".format(C, A, B))

    else:

        print("Ordem crescente: {}, {}, {}".format(C, B, A))
```

3.2 Comando de repetição

É muito frequente que, para implementar determinada lógica, um programador precise repetir um trecho de programa um certo número de vezes. Isso pode ser realizado com o uso do comando de repetição `while`. Com ele, é possível repetir um conjunto de comandos enquanto uma condição especificada for verdadeira. Esse tipo de trecho repetitivo de código também é conhecido como laço ou, em inglês, como *loop*.

O comando `while` em Python tem a construção básica a seguir, que pode ser interpretada como: “enquanto a condição for verdadeira, execute o conjunto de comandos”.

```
while {condição}:

    {conjunto de comandos}
```

A condição segue exatamente as mesmas regras utilizadas nas condições já vistas quando foi abordado o comando if-else. Quanto ao conjunto de comandos subordinados ao while, podem ser quaisquer comandos válidos em Python, em quaisquer quantidade e extensão. Assim como no comando if-else, a indentação é importante, pois define a relação de subordinação entre o cabeçalho do comando e seu conjunto subordinado.

Para exemplificar a implementação de um laço, tem-se o código a seguir, no qual se quer exibir na tela todos os números inteiros entre 1 e 10, sendo um valor em cada linha.

Exemplo 3.5 Funcionamento do comando de repetição – laço contador

```
print("Início do Programa")
```

```
Cont = 1 # linha 1
```

```
while Cont <= 10: # linha 2
```

```
    print(Cont) # linha 3
```

```
    Cont = Cont + 1 # linha 4
```

```
print("Fim do Programa")
```

No Exemplo 3.5, na linha 1 é definido um objeto inteiro identificado por Cont e inicializado com o valor 1. Em seguida – linha 2 –, tem-se o comando while construído com a condição Cont <= 10. A avaliação dessa condição resulta em *True* (verdadeiro), de modo que o conjunto de comandos subordinado, constituído pelas linhas 3 e 4, é executado uma primeira vez. Com isso, o valor inicial de Cont é exibido e 1 é somado a Cont, que passará a ser 2. Após a execução da linha 4, o programa retorna para a linha 2, a

condição é avaliada e novamente resultará *True*, pois *Cont* é menor que 10. Isso fará que o `print` e a soma de 1 em *Cont* sejam executados uma segunda vez. Com isso, *Cont* passará a conter o valor 3 e o programa seguirá sucessivamente. Ao final, dez linhas terão sido exibidas na tela contendo os valores de 1 a 10.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
1
2
3
4
5
6
7
8
9
10
Fim do Programa
>>>
Ln: 17 Col: 4
```

Figura 3.6 Execução do programa do Exemplo 3.5.

Laços como esses são denominados “laços contadores”, pois seu controle é efetuado por meio de um objeto de controle que sofre um incremento a cada repetição. Nem todo laço é contador, como o que está implementado no Exemplo 3.6, descrito a seguir.

3.2.1 Lógica de funcionamento do laço `while`

No Exemplo 3.5 foi mostrado como utilizar o comando `while`. Trata-se de um comando de laço no qual o teste da condição é feito no início do laço. A Figura 3.7 ilustra essa situação, na qual a avaliação da condição é feita antes de se executar o conjunto de comando subordinado.



Figura 3.7 Diagrama ilustrativo da sequência de operações do comando de repetição.

Esse conceito é importante, porque, sempre que a condição for previamente falsa, o conjunto subordinado não será executado nenhuma vez, dado que a avaliação da condição é feita antes.

Todo laço, para ser implementado, requer quatro elementos: inicialização, condição, iteração e o corpo. Os três primeiros dizem respeito à construção e ao controle do laço. A inicialização constitui-se de todo código necessário para determinar a situação inicial do laço. A condição é uma expressão lógica, que pode ser simples ou composta, cujo resultado é avaliado em falso ou verdadeiro, que determina se o laço termina ou prossegue, respectivamente. A iteração é todo comando (pode ser um ou mais de um) que modifica os objetos envolvidos na condição, a cada execução do laço. Por fim, o corpo do laço é constituído pelos comandos que devem ser

executados repetidas vezes.

No Exemplo 3.5, a inicialização está na linha 1, a condição é a linha 2 e a iteração é a linha 4. O corpo do laço é a linha 3.

As condições usadas neste comando de repetição são exatamente iguais às usadas no comando if-else, vistas no Item 3.1.

No Exemplo 3.6 pede-se que se escreva um programa que permaneça em laço enquanto um valor X lido for diferente de zero. Para cada valor de X deve-se apresentar na tela se o mesmo é par ou ímpar.

Exemplo 3.6a Par ou ímpar

```
X = 1 # linha 1
```

```
while X != 0: # linha 2
```

```
X = int(input("Digite X: ")) # linha 3
```

```
if X % 2 == 0: # linha 4
```

```
print("%d é par" % X) # linha 5
```

```
else: # linha 6
```

```
print("%d é ímpar" % X) # linha 7
```

Nessa solução, o controle do laço não é implementado por meio de um contador que permitirá a repetição um certo número conhecido de vezes. Neste caso, não se sabe quantas vezes o laço repetirá. O que se sabe apenas é que termina quando zero for digitado para X.

Para garantir que o laço seja iniciado, o objeto X deve ser criado contendo qualquer valor diferente de 0, o que é feito na linha 1. Essa é a linha de

inicialização. A iteração é implementada na linha 3, que altera o valor do objeto X. O novo X lido logo no início do laço também é utilizado em seu corpo, que é constituído pelas linhas 4 a 7. O resto da divisão de X por 2 é calculado e comparado com zero. Se o resultado dessa comparação for verdadeiro, então, o número é par; caso contrário, é ímpar. Quando zero for digitado, o programa dirá que zero é par e terminará.

Outra solução possível para esse problema está implementada a seguir, no Exemplo 3.6b. Nessa segunda solução, a inicialização do objeto X é feita a partir da leitura do teclado. Caso X digitado seja diferente de zero, o laço é iniciado, o corpo do laço é executado e a leitura de um novo valor para X – linha 3 da solução anterior – foi transferida para o final do corpo do laço. Nela, X é lido e, em seguida, o laço retorna para seu cabeçalho para avaliar a condição e decidir se continuará ou não.

Exemplo 3.6b Par ou ímpar

```
X = int(input("Digite X: ")) # linha 1
```

```
while X != 0: # linha 2
```

```
    if X % 2 == 0: # linha 4
```

```
        print("%d é par" % X) # linha 5
```

```
    else: # linha 6
```

```
        print("%d é ímpar" % X) # linha 7
```

```
    X = int(input("Digite X: ")) # linha 3 (transferida para cá)
```

No próximo exemplo, pede-se que escreva um programa que mostre na tela os dez primeiros termos de uma progressão aritmética (PA) com primeiro

termo P e razão R. Os dois números P e R são inteiros e devem ser lidos do teclado.

Exemplo 3.7 Progressão aritmética

```
P = int(input("Digite o primeiro termo: ")) # linha 1
```

```
R = int(input("Digite a razão: ")) # linha 2
```

```
Cont = 0 # linha 3
```

```
while Cont < 10: # linha 4
```

```
    print(P) # linha 5
```

```
    P = P + R # linha 6
```

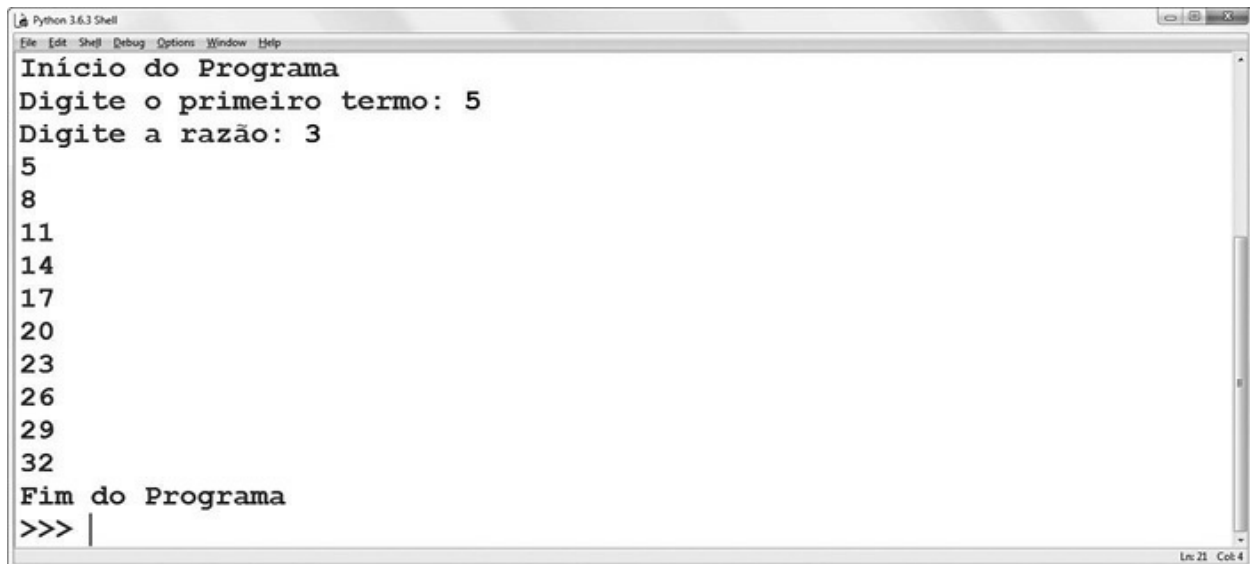
```
    Cont = Cont + 1 # linha 7
```

Nas linhas 1 e 2 são feitas as leituras dos dados de entrada. Na linha 3 é feita a inicialização do laço, atribuindo-se 0 ao objeto Cont. Na linha 4 está a condição de continuidade do laço que foi construída como $\text{Cont} < 10$. E a iteração é formada pela linha 7, na qual se soma 1 ao Cont. O corpo do laço é formado pelas linhas 5 e 6, nas quais é exibido o conteúdo de P e calculado o próximo termo a ser exibido, somando-se R ao objeto P e armazenando o resultado no próprio objeto P. Com isso, P é preparado para a próxima iteração. Na primeira vez em que o laço é executado será mostrado o primeiro termo da PA, contido em P. Ao somar R em P, este passa a conter o segundo termo. Quando o laço for executado pela segunda vez, será mostrado o segundo termo e calculado o terceiro. Ao mesmo tempo, 1 é somado em Cont a cada repetição, de modo que ele aumentará sempre até atingir o valor 10. Quando isso ocorrer, a condição da linha 4 será avaliada em False e o laço

terminará.

Propositadamente, no Exemplo 3.7 o objeto Cont foi inicializado com 0 e a condição foi escrita como $\text{Cont} < 10$. Compare-a com o Exemplo 3.5, em que o objeto Cont foi iniciado com 1 e a condição escrita como $\text{Cont} \leq 10$. São duas maneiras diferentes de implementar um laço que executa o mesmo número de vezes. Em casos assim, a escolha entre uma forma ou outra depende apenas do que o programador considerar mais apropriado.

Ao executar esse programa para um caso de teste em que $P = 5$ e $R = 3$, tem-se o resultado mostrado na Figura 3.8.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite o primeiro termo: 5
Digite a razão: 3
5
8
11
14
17
20
23
26
29
32
Fim do Programa
>>> |
```

Figura 3.8 Resultado da execução do Exemplo 3.8.

No Exemplo 3.8, vamos escrever um programa que permaneça em laço enquanto um valor X lido for diferente de zero. Totalize (some todos) e conte os valores digitados, exceto o zero, e apresente esses valores na tela. Use o caso de teste a seguir para verificar se o programa está correto::

Entrada	6 8 -30 9 -4 8 16 50 15 7 2 -5 0
Saída	Total dos valores digitados = 82 Quantidade de valores = 12

Tarefas adicionais

1. Altere o programa da PA para, em vez de dez elementos, apresentar na tela uma quantidade Q de elementos, onde Q é um inteiro lido do teclado.
2. Altere o programa da PA para também calcular e mostrar o somatório de todos os termos. Para isso, crie um novo objeto com o identificador Soma, atribuindo a ele o valor inicial 0, e a cada repetição do laço acrescente a ele um termo da PA.
3. Altere o programa da PA para exibir em cada linha uma frase como a seguir (exemplo: supondo que $P = 5$, $R = 3$ e $Q = 3$).

Termo 1 da PA = 5

Termo 2 da PA = 9

Termo 3 da PA = 13

Exemplo 3.8 Totalização de valores

Soma = 0

Qtde = 0

X = 1

while X != 0:

X = int(input("Digite X: "))

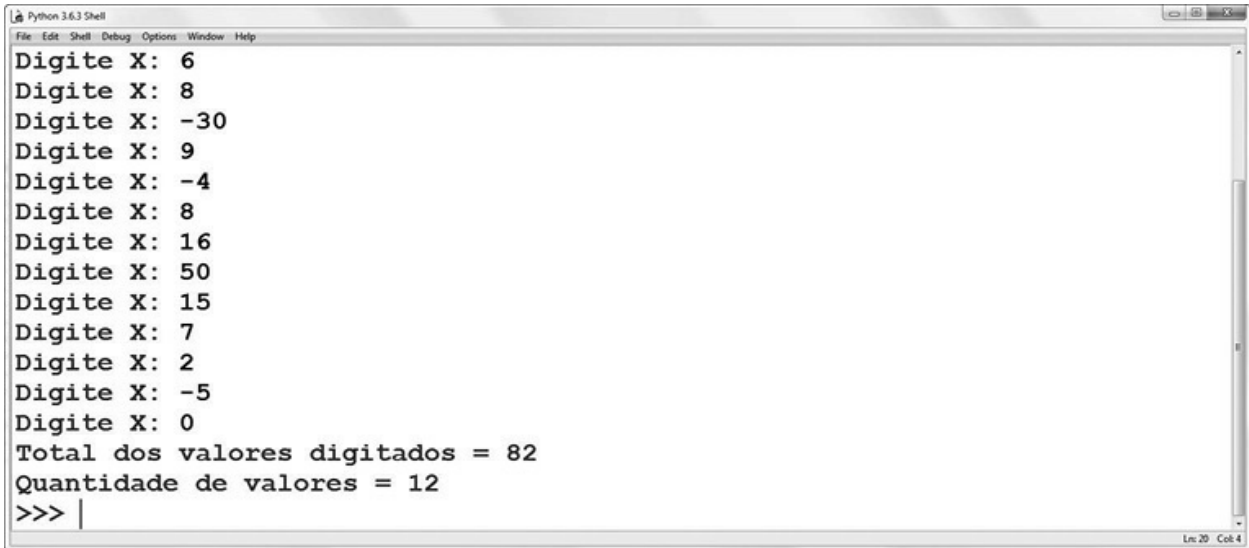
if X != 0: # Este if evita que o zero seja contado

Soma = Soma + X # Adiciona X na variável Soma

Qtde = Qtde + 1 # Soma 1 na quantidade

print("Total dos valores digitados = %d" % Soma)

```
print("Quantidade de valores = %d" % Qtde)
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Digite X: 6
Digite X: 8
Digite X: -30
Digite X: 9
Digite X: -4
Digite X: 8
Digite X: 16
Digite X: 50
Digite X: 15
Digite X: 7
Digite X: 2
Digite X: -5
Digite X: 0
Total dos valores digitados = 82
Quantidade de valores = 12
>>> |
```

Figura 3.9 Resultado da execução do Exemplo 3.8.

Observe que essa solução requer que dentro do laço seja escrito o comando `if X != 0`: para que, quando for digitado o valor zero para `X`, este não seja contado. Caso esse programa seja escrito utilizando a forma apresentada no Exemplo 3.6b, esse comando `if` não é necessário.

Tarefa adicional

Escreva uma nova solução para esse programa usando uma forma semelhante à apresentada no Exemplo 3.6.

3.2.2 Aspectos específicos dos comandos de repetição em Python

Até agora, o objetivo foi apresentar os conceitos básicos e gerais relativos a laços. Tais conceitos aplicam-se à maioria das linguagens de programação. A seguir, serão abordados alguns aspectos específicos que se aplicam à linguagem Python.

Existem dois comandos em Python que são utilizados no controle do fluxo de execução de laços. Estes comandos são `continue` e `break`, e só têm

significado se estiverem inseridos no bloco de comandos subordinado a um laço. Podem ser usados, tanto em laços while, quanto em laços for (que serão vistos no Capítulo 4) e em nenhum outro comando de Python.

3.2.2.1 continue

Encerra a iteração atual e desvia a execução do programa para o cabeçalho do while em que esteja inserido. Nova avaliação da condição será feita, e terá início uma nova iteração, caso a mesma seja verdadeira. No Exemplo 3.9, caso o valor lido para o objeto X seja menor ou igual a zero, a condição da linha 4 será True e o continue desviará a execução de volta para a linha 2, sem executar o bloco de comandos que inicia na linha 6.

Exemplo 3.9 Uso do comando continue

```
X = 1

while X != 0: # linha 2

    X = int(input("Digite um valor: "))

    if X <= 0: # linha 4

        continue

    # bloco de comandos # linha 6
```

3.2.2.2 break

Este comando termina o laço e desvia a execução para fora do mesmo. No Exemplo 3.10 foi criado um laço while True:, que é sempre verdadeiro e, por consequência, executará indefinidamente. Porém, na linha 4 há um comando break que encerrará esse laço quando X for igual a zero.

Exemplo 3.10 Uso do comando break

`while True: # linha 1 – a condição é sempre True`

`X = int(input("Digite um valor: "))`

`if X == 0: # linha 3`

`break # linha 4 - encerra o laço`

`# bloco de comandos # linha 5`

3.2.2.3 else em laços do Python

Outro aspecto a ser abordado aqui é o bloco `else` contido nos comandos de repetição de Python.

Esse é um recurso que causa muita estranheza nos programadores que já conhecem outras linguagens de programação. Todas as linguagens têm `else` associado ao comando `if`, para implementar o conceito de que “Se a condição for verdadeira faça isso, senão faça aquilo”. O `else`, nesse caso, é uma contraposição ao `if`.

Assim sendo, qual seria a função do `else` no comando `while`? O bloco de comandos subordinado ao `else` é executado se, e somente se, o `while` terminar normalmente, pelo fato de sua condição de continuidade se tornar falsa. Caso o `while` seja encerrado por um comando `break`, então, o `else` não é executado. Assim, esse bloco `else` implementa o conceito “Repita o laço e, quando este terminar normalmente, então, execute o `else`”. O `else`, neste caso, representa justamente o oposto do que significa a palavra “senão”.

Ramalho (2015) sugere que a escolha da palavra `else` foi infeliz e que a palavra `then` (então) teria sido uma opção melhor. No entanto, a palavra utilizada para esse bloco é `else` e não será alterada, de modo que o programador Python precisa se acostumar a essa ideia.

Em função da estranheza mencionada, muitos programadores, que construíram seu conhecimento de programação usando outras linguagens, subestimam esse recurso e tendem a não o utilizar, porém, essa não é a melhor das decisões. Seu uso torna o código mais simples e legível e evita a frequente situação em que é necessário criar objetos de sinalização (*flags*) e condições associadas para controlar se algo aconteceu ou não dentro de um laço e produzir um resultado.

O Exemplo 3.11 ilustra o uso do bloco `else` no comando `while`. Para que tenha uma ideia bem clara desse uso, serão desenvolvidas duas soluções, uma sem utilizar o bloco `else` e outra utilizando-o.

No Exemplo 3.11, escreva um programa que leia um número inteiro N e exiba na tela se ele é ou não primo.

Números primos são aqueles divisíveis apenas por 1 e por eles mesmos. Em termos práticos, visando escrever um programa que resolva esse problema, pode-se dizer que N não é divisível por nenhum valor contido no intervalo fechado $[2, N-1]$. A solução que será apresentada é a mais simples possível e também a menos eficiente. O objetivo aqui é um melhor entendimento por parte dos iniciantes, de modo que se pede licença aos mais experientes para que aceitem essa solução.

A solução consiste em criar um laço no qual se calcule o resto da divisão de N por todos os valores do intervalo $[2, N-1]$ e contar quantas vezes ocorreu resto igual a zero. Caso o laço termine com a contagem igual a zero, então, o número é primo, caso contrário, ele não é.

Exemplo 3.11 Verificar se um número é primo (versão A)

```
N = int(input("Digite N: "))
```

```
Cont = 0
```

```
i = 2

while i < N:

    R = N % i

    if R == 0:

        Cont += 1

    i += 1

    if Cont == 0: # linha 9

        print("{} é primo".format(N))

    else:

        print("{} não é primo".format(N))
```

Nessa solução, o objeto Cont é empregado como *flag*. Começa com zero e para toda ocorrência de R igual a 0 tem seu valor incrementado. Após o término do laço o if da linha 9, verifica o valor de Cont e exibe a mensagem apropriada.

Compare essa solução com a versão B. Ambas produzem o mesmo resultado, mas essa segunda versão é mais enxuta. Verifique-se que nela não existe o objeto Cont, ou seja, o flag tornou-se dispensável, e também não existe mais o if posterior ao comando while.

E não é só isso. Perceba que a segunda versão se tornou mais eficiente para os casos em que N não é primo, pois basta encontrar um primeiro resto igual a zero que o laço é encerrado com break. Caso todas as divisões sejam feitas,

a condição torna-se falsa quando i chega ao valor de N , e nesse caso a execução é desviada para o `else`, que exibe que o número é primo.

Exemplo 3.12 Verificar se um número é primo (versão B)

```
N = int(input("Digite N: "))

i = 2

while i < N:

    R = N % i

    if R == 0: # ao encontrar o primeiro R == 0

        print("{} não é primo".format(N)) # exibe que não é primo

        break # e encerra o laço while

    i += 1

else:

    print("{} é primo".format(N))
```

3.2.2.4 do-while não existe

Por fim, no estudo dos aspectos específicos do comando `while` em Python existe a questão da não existência de um comando de laço que se assemelhe ao do-while da linguagem C e está presente em boa parte das linguagens de programação.

Nota

Conforme visto, no comando *while* a condição é testada no início e,

caso seja verdadeira, o bloco de comandos é executado. Outra possibilidade é a existência de um comando de laço no qual a avaliação da condição seja feita após a execução do conjunto de comandos subordinado.

Em outras linguagens de programação, como C e Java, existe um segundo tipo de comando de laço, no caso é o *do-while*, no qual a avaliação da condição é feita **APÓS** a execução dos comandos subordinados.

Isso implica que o conjunto de comandos subordinado obrigatoriamente será executado pelo menos uma vez, mesmo que a condição de continuidade do laço seja previamente falsa. Isso é útil em algumas situações.

Em Python não existe essa opção.

Essa questão foi motivo de intenso debate na comunidade Python mundial. Conforme mencionado no Capítulo 1, essa comunidade é muito ativa e mantém o índice de PEPs (*Python Enhancement Proposal*), que é um banco de dados criado para, como o próprio nome diz, reunir todas as propostas de melhoria da linguagem Python.

Pois bem, a PEP 315 (ROSSUM, PEP-315) trata exatamente dessa questão, em que é feita a sugestão de criar uma alteração na sintaxe do comando *while* com o objetivo de que ele também pudesse ser utilizado, à semelhança de *do-while* (entenda-se: executar primeiro o bloco de comandos para depois verificar a condição).

Essa PEP já está encerrada e foi rejeitada. Uma mensagem enviada pelo próprio Guido von Rossum (ROSSUM, PEP315) recomenda a rejeição indicando que implementar tal comando não traria um benefício direto para a linguagem nem a tornaria mais legível ou fácil de se aprender. Além disso, tal resultado pode ser obtido com a seguinte estrutura:

while True: # este laço sempre inicia

<código> # executa o código do laço

if condição: # testa a condição e se for True

break # termina o laço

O comportamento desse laço é exatamente o que se espera de um do-while em C, então, a questão está resolvida.

3.3 Tratamento de exceções

O termo “exceção” em programas de computador diz respeito a uma situação em que algum evento ocorrido durante a execução do programa necessita de atenção e tratamento apropriado. Nessas situações, diz-se que a exceção precisa ser “tratada”.

3.3.1 O básico sobre exceções e seu tratamento

No Exemplo 3.1 foi mostrada uma situação em que uma mensagem de erro foi emitida indicando a ocorrência de uma situação de divisão por zero. Situações como essas são delicadas, pois fazem que o programa seja abortado no momento em que ocorre, se não houver um tratamento. No Exemplo 3.2 foi elaborada uma solução baseada em um comando if que evita o erro e cria uma alternativa a sua ocorrência, no caso, a exibição de uma mensagem.

Ocorre que, nos tempos atuais, os programas têm ficado cada vez maiores e mais complexos, de modo que usar condições para prever tudo o que pode dar errado em um programa torna-o muito mais complexo que o necessário e não garante que todas as possibilidades foram previstas e cobertas.

Assim, surgiu no campo da Ciência da Computação a necessidade de se

desenvolver um modo mais racional, organizado e bem estruturado de lidar com essas situações. A solução desenvolvida para dar atendimento a tal demanda é o **tratamento de exceções**. Trata-se de um recurso disponível na maioria das linguagens modernas, muito inteligente, poderoso e ao mesmo tempo simples.

O Exemplo 3.13 mostra como fica o código do Exemplo 3.1 caso se adote o tratamento de exceções como forma de resolver a questão do erro de divisão por zero. Ou seja, esse exemplo é uma alternativa à solução apresentada no Exemplo 3.2.

Exemplo 3.13 Tratamento de exceção

```
A = int(input("Digite um valor para A: "))
```

```
B = int(input("Digite um valor para B: "))
```

```
try: # linha 3
```

```
R = A / B
```

```
print("resultado: R = %.1f" % R)
```

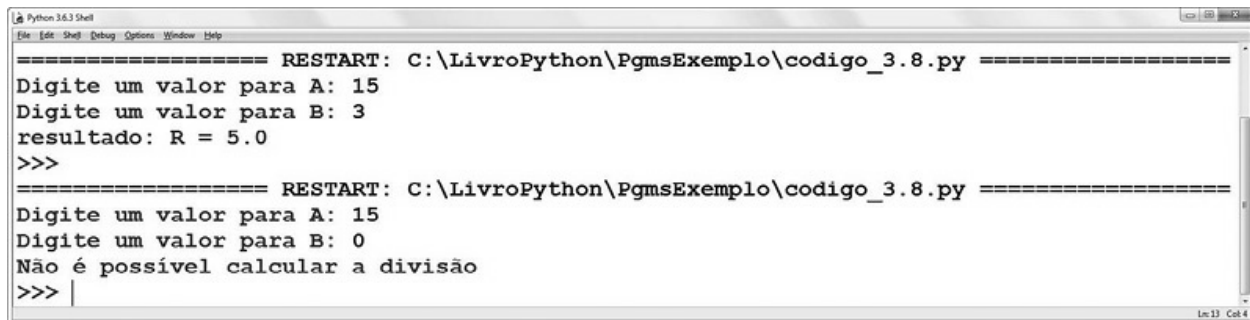
```
except: # linha 6
```

```
print("Não é possível calcular a divisão")
```

O comando try inicia o bloco de comandos que estará protegido pelo tratamento de exceções e a cláusula except contém o código que será executado em caso de erro.

Teste a execução desse programa fornecendo valores, como mostrado na Figura 3.10. Observe que quando B recebeu o valor zero a mensagem de

resultado foi omitida (pulada) e foi exibida a mensagem de exceção, pois o fluxo de execução do programa foi desviado do bloco try para o bloco except.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.8.py =====
Digite um valor para A: 15
Digite um valor para B: 3
resultado: R = 5.0
>>>

===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.8.py =====
Digite um valor para A: 15
Digite um valor para B: 0
Não é possível calcular a divisão
>>> |
```

Figura 3.10 Uso do tratamento de exceções.

3.3.2 Ampliando as possibilidades – exceções nomeadas

Toda exceção em Python tem um nome identificador, e isso pode ser utilizado pelo programador para fazer distinção entre diferentes tipos de exceções e dar um tratamento próprio a cada uma delas.

Considere-se que no caso do Exemplo 3.14 possam ocorrer outras situações que levem a erros. Se o usuário do programa digitar texto ou um número real no momento da leitura de A ou de B, como está sendo utilizada a função `int()` para converter o dado lido para número inteiro, isso causará erro.

Exemplo 3.14 Outra situação possível e indesejada

```
>>> A = int(input("Digite um valor para A: "))
```

Digite um valor para A: texto

Traceback (most recent call last):

File "<pyshell#2>", line 1, in <module>

```
A = int(input("Digite um valor para A: "))
```

ValueError: invalid literal for int() with base 10: 'texto'

Para evitar tal erro o programa pode ser escrito como mostrado no Exemplo 3.15, que é a mesma solução do Exemplo 3.13, com a única diferença de que as duas linhas de leitura dos objetos A e B foram transferidas para dentro do bloco protegido pelo try.

Exemplo 3.15 Ampliando o tratamento de exceção

```
try:
```

```
A = int(input("Digite um valor para A: ")) # a leitura está
```

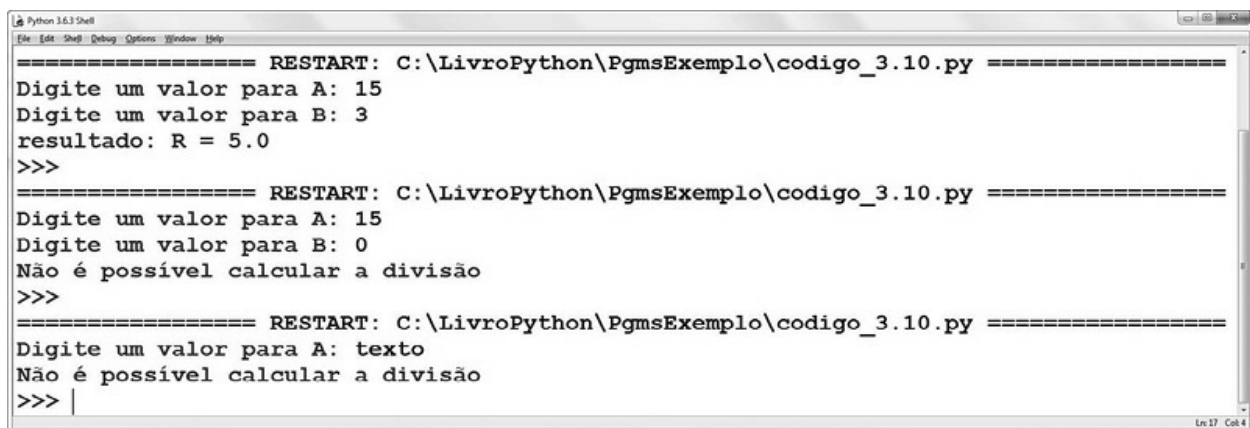
```
B = int(input("Digite um valor para B: ")) # protegida tb
```

```
R = A / B
```

```
print("resultado: R = %.1f" % R)
```

```
except: # linha 6
```

```
print("Não é possível calcular a divisão")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.10.py =====
Digite um valor para A: 15
Digite um valor para B: 3
resultado: R = 5.0
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.10.py =====
Digite um valor para A: 15
Digite um valor para B: 0
Não é possível calcular a divisão
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.10.py =====
Digite um valor para A: texto
Não é possível calcular a divisão
>>> |
```

Ao executar essa solução, as duas situações possíveis de erro já mencionadas provocam o desvio para o bloco except e a mesma mensagem

acaba sendo exibida. Nos programas modernos, convém oferecer a seus usuários uma solução mais precisa, em que cada exceção tenha tratamento próprio e diferenciado.

Assim, entram em cena as exceções nomeadas. Observe-se que, no caso anterior, a mensagem exibida no Exemplo 3.14 tem a identificação “ValueError” e o erro do Exemplo 3.1 é identificado por “ZeroDivisionError”. Estes são os nomes das exceções levantadas.

O Exemplo 3.16 mostra como usar esses nomes e tornar o programa mais preciso no tratamento de cada caso. Nas linhas 8 e 10 estão especificadas as exceções “ZeroDivisionError” e “ValueError” respectivamente. Na linha 12 foi mantida a cláusula except genérica (não nomeada) para capturar todas as outras exceções que não foram explicitamente previstas.

Nas linhas 5 e 6 foi propositalmente incluído um if no qual se pretende calcular o cosseno de A, caso A seja negativo. Porém, este cálculo vai falhar, com a exceção “NameError”, pois a biblioteca matemática não foi importada e a função cos() não será reconhecida pelo interpretador. Uma vez que não foi previsto tratamento específico para tal caso, quando isso ocorrer, o programa desviará para a exceção genérica na linha 12.

Não é obrigatório que o programador utilize a exceção genérica, e caberá a ele decidir se tal uso é ou não apropriado, em função das necessidades do programa que está desenvolvendo. Se não a usar e ocorrer uma exceção fora das previstas, então, o comportamento padrão será executado pelo interpretador e o programa será abortado.

Exemplo 3.16 Uso de exceções nomeadas

```
try:
```

```
A = int(input("Digite um valor para A: "))
```

```
B = int(input("Digite um valor para B: "))
```

```
R = A / B
```

```
if A < 0: # linha 5
```

```
C = cos(A)
```

```
print("resultado: R = %.1f" % R)
```

```
except ZeroDivisionError: # linha 8
```

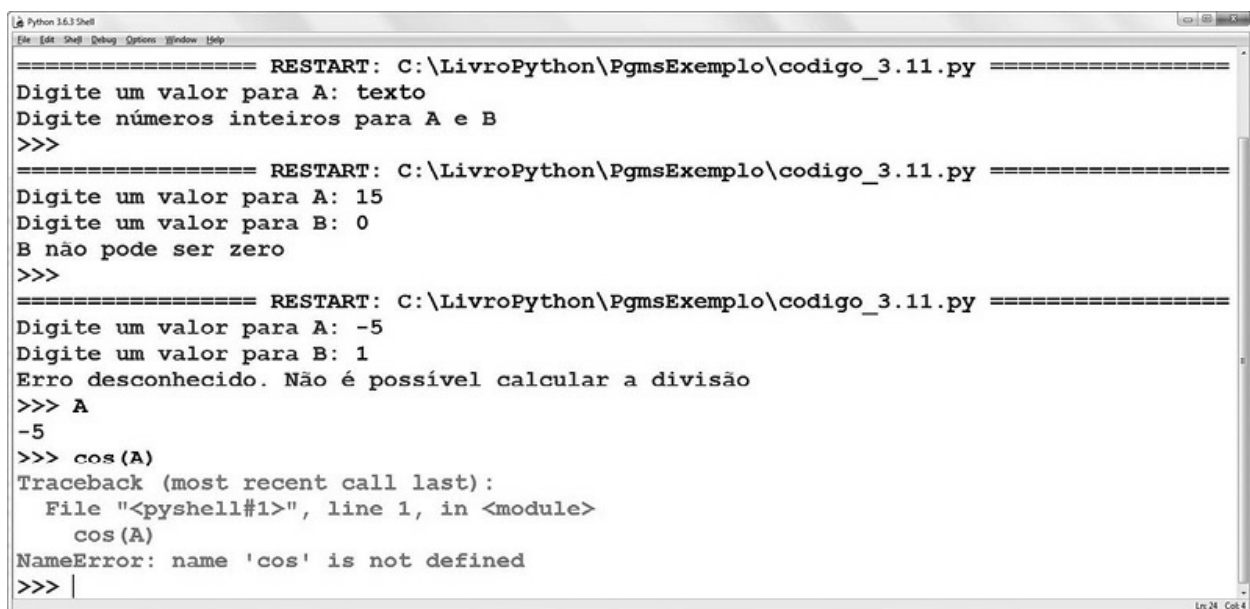
```
print("B não pode ser zero")
```

```
except ValueError: # linha 10
```

```
print("Digite números inteiros para A e B")
```

```
except: # linha 12
```

```
print("Erro desconhecido. Não é possível calcular a divisão")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.11.py =====
Digite um valor para A: texto
Digite números inteiros para A e B
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.11.py =====
Digite um valor para A: 15
Digite um valor para B: 0
B não pode ser zero
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_3.11.py =====
Digite um valor para A: -5
Digite um valor para B: 1
Erro desconhecido. Não é possível calcular a divisão
>>> A
-5
>>> cos(A)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    cos(A)
NameError: name 'cos' is not defined
>>> |
```

3.3.3 O formato completo do comando try

O comando try tem outros dois blocos ainda não mencionados, que serão vistos agora. Sua forma completa contém, além de try e dos vários possíveis except, os blocos else e finally, que são opcionais.

O bloco de comandos 1 é posto em execução. Caso nenhuma exceção ocorra e esse bloco termine normalmente, então, o bloco except (ou blocos, no caso de exceções nomeadas) é pulado. Se uma exceção ocorrer no bloco 1, então, todos os comandos posicionados abaixo da linha onde ocorreu a exceção são pulados, a execução do bloco 1 termina e o programa é desviado para o bloco 2 para tratar a exceção.

Caso não ocorra exceção e o try termine normalmente, então, será executado o bloco de comandos 3, que está subordinado ao else. Por fim, se o bloco finally estiver presente, ele sempre será executado, ocorra exceção ou não.

Além disso, é possível construir um bloco try que não contenha o except, mas contenha apenas try-finally. Esse tipo de construção permite ao programa ignorar eventuais exceções e executar um bloco de final (bloco de comandos 4) que faça algum tipo de tarefa de recuperação ou limpeza nos objetos do programa.

try:

{bloco de comandos 1}

except:

{bloco de comandos 2}

else:

```
{bloco de comandos 3}
```

```
finally:
```

```
{bloco de comandos 4}
```

No Exemplo 3.17, escreva um programa que leia um número inteiro no intervalo [100, 500]. Caso o usuário digite um número fora do intervalo ou um dado não numérico, utilize o tratamento de exceção para avisá-lo.

Exemplo 3.17 Tratamento de exceções

```
N = 0
```

```
while N < 100 or N > 500:
```

```
try:
```

```
S = input("Digite N no intervalo [100, 500]: ")
```

```
N = int(S)
```

```
except:
```

```
print("{} não é um número.".format(S))
```

```
N = 0
```

```
else:
```

```
if N < 100 or N > 500:
```

```
print("O valor lido {} está fora do \
```



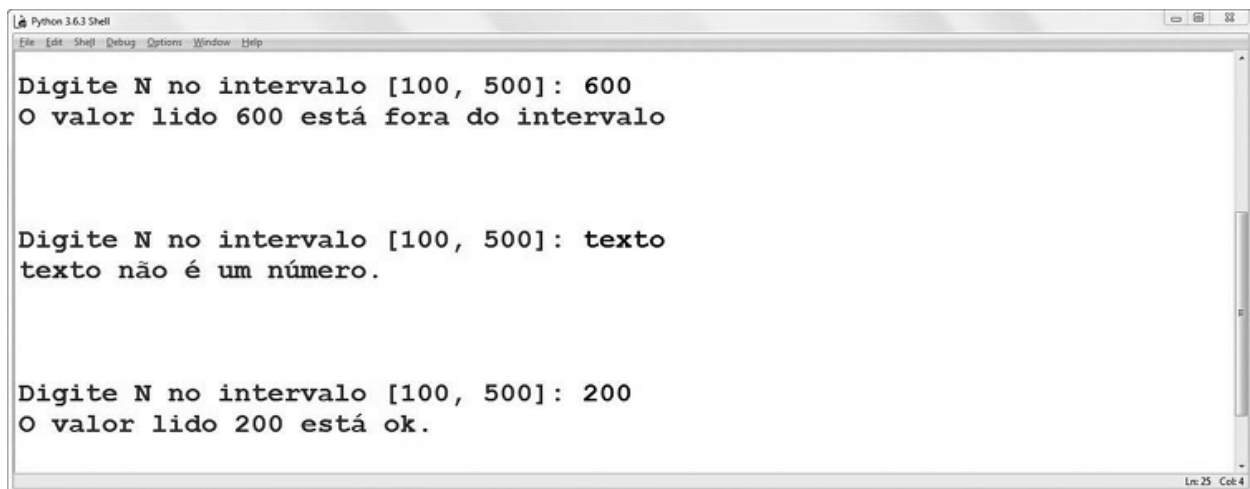
```
intervalo”.format(N))
```

```
else:
```

```
print(“O valor lido {} está ok.”.format(N))
```

```
finally:
```

```
print(“\n\n”)
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Digite N no intervalo [100, 500]: 600
O valor lido 600 está fora do intervalo

Digite N no intervalo [100, 500]: texto
texto não é um número.

Digite N no intervalo [100, 500]: 200
O valor lido 200 está ok.
```

Figura 3.11 Execução do Exemplo 3.17.

Nessa solução foi usada uma exceção genérica (não nomeada) para capturar a entrada não texto do usuário. Nessa cláusula é exibida a mensagem avisando que o dado digitado não é número e o objeto N é zerado. O bloco else será executado caso tenha sido digitado um dado numérico, e nele a mensagem avisa se N está ou não dentro do intervalo. No bloco finally são executados alguns pulos de linha para que a exibição de dados na tela fique mais espaçada. Todo esse bloco está dentro de um laço while, que só terminará quando N digitado estiver no intervalo pedido.

Exercícios resolvidos

1. Programa que totaliza um conjunto de valores

Escreva um programa que leia um número inteiro N e, em seguida, gere N números aleatórios no intervalo [1, 50] e totalize-os. Para gerar números aleatórios, use a função randint, disponível na biblioteca random.

Antes de começar: em sistemas computacionais, é frequente a necessidade de serem gerados números aleatórios (gerar um banco de dados de testes, fazer simulações, gerar dados para um jogo etc.). Em Python está disponível a biblioteca random, que contém um variado conjunto de funções destinadas a esse propósito. Para conhecer as possibilidades existentes, abra o IDLE e digite as duas linhas a seguir:

```
>>> import random
```

```
>>> help(random)
```

Como resultado da execução do comando help, serão listados todos os recursos contidos na biblioteca.

Aqui será utilizada a função randint(a, b). Essa função retorna um número inteiro tal que: $a \leq \text{randint}(a, b) \leq b$. Assim, tem-se:

Exercício resolvido 3.1 – Programa totalizador

```
from random import randint
```

```
N = int(input("Digite N: "))
```

```
Total = 0 # cria objeto Total zerado
```

```
i = 1
```

```
while i <= N:
```

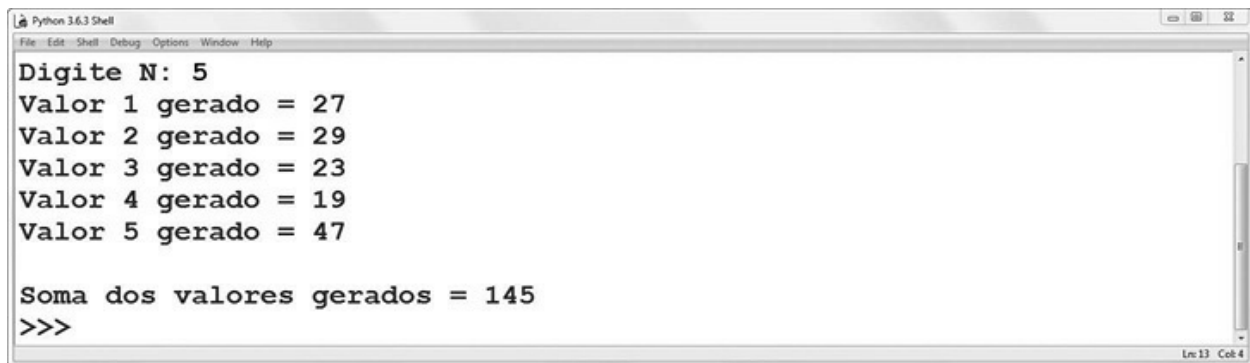
```
x = randint(1, 50) # gera um valor para x
```

```
print("Valor {} gerado = {}".format(i, x)) # exibe x na tela
```

```
Total = Total + x # acumula x em Total
```

```
i += 1
```

```
print("\nSoma dos valores gerados = {}".format(Total))
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Digite N: 5
Valor 1 gerado = 27
Valor 2 gerado = 29
Valor 3 gerado = 23
Valor 4 gerado = 19
Valor 5 gerado = 47

Soma dos valores gerados = 145
>>>
```

Nota

Todas as funções disponíveis na biblioteca random geram pseudoaleatórios e não aleatórios reais. Com isto tais funções geradoras jamais devem ser usadas em aplicações de segurança. Sobre isso, considere-se a conhecida citação de John von Neumann: *“Quem quer que seja que considere métodos aritméticos para produzir números aleatórios está, claro, num estado de pecado.”* (Dublin, 1993).

As referências DODIS (2013), VAZIRANI (1984) e NEUMANN (1951) tratam do assunto.

2. Programa que gera a sequência de Fibonacci

Escreva um programa que leia um número inteiro N e, em seguida, mostre na tela os N primeiros termos da sequência de Fibonacci. Faça o programa de modo que N seja no mínimo 2.

A sequência de Fibonacci é uma sequência de números inteiros que tem as seguintes regras de formação: os dois primeiros termos são 0 e 1; do terceiro em diante cada termo é a soma dos dois anteriores.

Se $N = 10$, então: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

A solução mais simples consiste em carregar dois objetos, A e B , com os valores iniciais e apresentá-los na tela. Após isso, inicie um laço no qual, na primeira repetição, seja calculado e exibido o terceiro termo e atualize A recebe o valor de B , que, por sua vez, recebe o valor calculado, preparando a próxima iteração. Então, tem-se:

Exercício resolvido 3.2 – Sequência de Fibonacci

```
print("Sequência de Fibonacci\n")
```

```
# leitura do número de termos
```

```
N = 0
```

```
while N < 2:
```

```
try:
```

```
N = int(input("Digite N(>1): "))
```

```
if N < 2:
```

```
    print("Digite N >= 2")
```

```
except:
```

```
    print("O dado digitado deve ser um número inteiro.")
```

```
A = 0
```

```
B = 1
```

```
print("0, 1, ", end="") # exibe os dois primeiros termos
```

```
i = 0
```

```
while i < N-2: # o laço tem que exibir N-2 termos
```

```
    C = A + B
```

```
    print("{}, {}".format(C), end="") # end="" suprime a mudança de
```

```
    A = B # linha na exibição em tela
```

```
    B = C
```

```
    i += 1
```

```
print("\n\nFim do Programa")
```

```

Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Sequência de Fibonacci

Digite N(>1): 15
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377,

Fim do Programa
>>>

```

Nessa solução foi utilizado um laço while em conjunto com try-except para garantir que o dado digitado seja numérico e no mínimo 2. O laço foi construído com a condição $i < N-2$, sendo que inicia em zero, porque os dois primeiros valores da sequência foram exibidos fora do laço. Nos prints foi utilizado o parâmetro `end=""` para suprimir o pulo de linha de modo a exibir todos os termos em uma única linha.

Exercícios propostos

1. Uso de condições simples: considerando os valores fornecidos, avalie cada condição e informe se o resultado é falso (False) ou verdadeiro (True). Avalie cada condição e anote o resultado que teste essas condições no IDLE do Python, conforme mostrado na Figura 3.2.

Valores para teste	Condição	Resultado
1 Para A = 0 e B = -3	$A > B$	
2 Para X = 3.7	$X \leq 10.0$	
3 Para A = 9 e B = 16	$A - B \geq 0$	
4 Para A = 2, B = 4 e N = 10	$A * B < N$	
5 Para A = 3, B = 9 e C = 5	$10 * A \geq B * C$	
6 Para A = 3, B = 6 e C = 5	$10 * A \geq B * C$	
7 Para N = 7	$N \% 2 == 0$	
8 Para N = 8	$N \% 2 == 0$	
9 Para T = "MORANGO"	$T == \text{"BANANA"}$	
10 Para T = "MORANGO"	$T > \text{"BANANA"}$	

Quadro 3.6 Exercícios de fixação de condições simples.

2. Uso de condições compostas: considerando os valores fornecidos, avalie cada condição composta e informe se o resultado é falso (False) ou verdadeiro (True). Faça o teste dessas condições no IDLE do Python.

	A	B	C	Condição	Resultado
1	10	15	4	$A < B$ and $A < C$	
2	10	15	4	$A < B$ or $A < C$	
3	1	9	0	$A \geq 0$ and $B == C$	
4	1	9	9	$A \geq 0$ and $B == C$	
5	1	9	0	$A \geq 0$ or $B == C$	
6	1	9	9	$A \geq 0$ or $B == C$	
7	0	0	0	$B != 0$ and $A != C$	
8	0	0	25	$B != 0$ and $A != C$	
9	0	0	0	$B != 0$ or $A != C$	
10	0	0	25	$B != 0$ or $A != C$	

Quadro 3.7 Exercícios de fixação de condições compostas.

3. Uso de condições mistas: considerando os valores fornecidos, avalie cada condição composta e informe se o resultado é falso (False) ou verdadeiro (True). Faça o teste dessas condições no IDLE do Python.

	A	B	C	Condição	Resultado
1	10	15	4	$A < B$ and $A < C$ or $C != 0$	
2	10	15	4	$A < B$ and ($A < C$ or $C != 0$)	
3	1	9	0	not ($A \geq 0$ and $B == C$)	
4	1	9	9	not ($A \geq 0$) and not ($B == C$)	
5	1	9	0	($A \geq 0$ or $B == C$) and $B > A$	
6	-2	0	2	not ($A \leq B$) or $C > B$	

7	-2	0	2	not (A <= 0 or C > B)	
8	0	1	0	A == 0 and B != 0 and C == 0	
9	5	0	0	A == 0 and B != 0 and C == 0	
10	5	0	0	A == 0 or B != 0 or C == 0	

Quadro 3.8 Exercícios de fixação de condições mistas.

4. Escreva um programa que leia um número inteiro do teclado e diga se esse número é positivo ou negativo.
5. Escreva um programa que leia um número inteiro do teclado e diga se esse número é par ou ímpar. Para saber se um número é par, deve-se verificar se o resto de sua divisão por 2 é igual a zero. Para calcular o resto da divisão de um número por outro deve-se utilizar o operador %. Por exemplo: ao escrever a expressão em negrito a seguir e supondo que A e B tenham conteúdo inteiro.

R = A % B, então, R é o resto da divisão de A por B

6. Escreva um programa que leia dois números quaisquer e mostre na tela qual é o menor e qual é o maior.
7. Escreva um programa que leia três números reais e informe se eles constituem os lados de um triângulo. Em caso afirmativo, informe se o triângulo é equilátero, isósceles ou escaleno. Para que três números formem um triângulo, a soma dos dois lados menores deve ser maior que o lado maior. Uma boa solução para esse problema envolve o uso dos operadores and e or.
8. Escreva um programa que leia o nome de um lutador e seu peso. Em seguida, informe a categoria a que pertence o lutador, conforme o Quadro 3.9 (note que o quadro foi criado para efeito deste exercício e

não condiz com qualquer categoria de luta). A saída do programa deve exibir na tela uma frase com o padrão descrito a seguir:

Nome fornecido: Pepe Jordão

Peso fornecido: 73.4

Frase a ser exibida: O lutador Pepe Jordão pesa 73,4 kg e se enquadra na categoria Ligeiro

Peso	Categoria
Menor que 65 kg	Pena
Maior ou igual a 65 kg e menor que 72 kg	Leve
Maior ou igual a 72 kg e menor que 79 kg	Ligeiro
Maior ou igual a 79 kg e menor que 86 kg	Meio-médio
Maior ou igual a 86 kg e menor que 93 kg	Médio
Maior ou igual a 93 kg e menor que 100 kg	Meio-pesado
Maior ou igual a 100 kg	Pesado

Quadro 3.9 Categorias para a questão 5.

9. Escreva um programa que leia o valor hora que um profissional ganha na empresa onde trabalha. Leia também as quantidades de horas normais e horas extras trabalhadas em um mês. Calcule o valor a ser recebido pelo profissional nesse mês, sabendo que nas horas extras o pagamento é dobrado.
10. Escreva um programa que mostre na tela todos os números inteiros de 1 a 10. Para fazer esse programa, será necessário usar o comando de repetição *while*.
11. Escreva um programa que leia um número inteiro e, em seguida,

apresente na tela a tabuada de 0 a 10 para esse número fornecido. Siga o formato apresentado a seguir (supondo que foi digitado 4):

$$4 \times 1 = 4$$

$$4 \times 2 = 8$$

$$4 \times 3 = 12$$

...

$$4 \times 10 = 40$$

12. Escreva um programa que leia um número inteiro N e, em seguida, leia N números reais, calculando a soma de todos os valores positivos fornecidos e ignorando os negativos. Este exercício pode ser elaborado a partir do Exercício resolvido 3.1, no qual, em vez de gerar números aleatórios, os valores sejam lidos do teclado.

13. Escreva um programa que leia valores numéricos inteiros e totalize separadamente os positivos e os negativos até que o usuário digite 0. Ao final, mostre na tela esses dois totais.

14. Escreva um programa que calcule os N primeiros termos de uma PG com razão R e o primeiro termo P1 fornecidos pelo usuário. Também deve ser calculada e apresentada a soma desses N termos.

15. Escreva um programa que apresente todos os valores inteiros divisíveis por 5 situados no intervalo fechado [Min, Max], em que Min e Max são fornecidos pelo usuário. É obrigatório que o valor Max seja maior que Min e, se isso não ocorrer, o programa deve

exibir uma mensagem de aviso ao usuário e inverter os valores.

16. Escreva um programa que leia um número inteiro N e, em seguida, leia N números reais, separando o menor e o maior, apresentando-os na tela.
17. Reescreva um programa do exercício 16 ignorando os números negativos fornecidos pelo usuário.
18. Elabore um programa que efetue a leitura de valores positivos inteiros até que zero ou um valor negativo seja informado. Ao final, devem ser apresentados o maior e menor valores informados pelo usuário, a quantidade de valores, a soma e a média de todos.
19. Escreva um programa que contenha um laço que será executado enquanto o número digitado for diferente de zero. Para cada número digitado pelo usuário, mostrar na tela apenas os que forem divisíveis por 2 e por 3.
20. Elabore um programa que apresente o somatório dos valores pares existentes na faixa entre 1 e N , em que N é um número digitado pelo usuário e que deve ser no mínimo 100 (obrigatório garantir esse requisito).
21. Reescreva o programa do Exercício resolvido 3.2 – Sequência de Fibonacci fazendo a seguinte alteração: leia N que será a quantidade de termos a ser exibida e leia um número inteiro adicional chamado Prim. Essa versão do programa deverá apresentar N termos da sequência de Fibonacci imediatamente maiores que Prim.



Tipos Estruturados Sequenciais em Python

Objetivos

Neste capítulo, serão apresentados os elementos da linguagem Python que são conhecidos como tipos estruturados sequenciais. São eles: strings, listas e tuplas. Os não sequenciais, por sua vez, são os conjuntos e dicionários, que serão abordados no Capítulo 6.

Tais tipos têm as próprias características e usos, porém, todos têm um aspecto em comum: são compostos por outros elementos, de modo que seu conteúdo não é indivisível, como no caso dos tipos simples vistos no Capítulo 2. O conteúdo dos tipos sequenciais é uma coleção de outros elementos arranjada de maneira sequencial e que podem ser acessados, utilizados e, em alguns casos, alterados individualmente ou em grupo.

Os tipos estruturados conferem à linguagem Python grandes flexibilidade e versatilidade. Cada um desses tipos, com suas características e funcionalidades próprias, fornece ao programador importantes ferramentas que podem ser utilizadas no desenvolvimento dos algoritmos. São designados como estruturados, pois seu conteúdo é formado por elementos que podem ser acessados individualmente ou em grupo.

Entre esses tipos estruturados, há aqueles cujo conteúdo é organizado de maneira sequencial e serão designados como tipos sequenciais. São eles: as cadeias de texto (*string*), as listas (*list*) e as tuplas (*tuple*). A principal característica dos tipos sequenciais é que seus elementos são mantidos em uma organização baseada em um índice numérico crescente da esquerda para a direita, que começa em zero e sofre o incremento de um a um. Com isso, é

possível ao programador acessar individualmente seus elementos por meio do uso de índices especificados entre colchetes: [].

Por outro lado, existem outros tipos cujo conteúdo não seguem uma organização como essa, e neste texto serão vistos dois deles: os conjuntos (*set*) e os tipos mapeados (*dictionary*).

Uma característica inerente a todos esses tipos estruturados é que eles podem ser utilizados como iteráveis (*iterables*). Um iterável é definido como um objeto capaz de retornar seus elementos um de cada vez dentro de um processo de repetições sucessivas. Os iteráveis têm um papel muito importante na programação Python, e isso será visto em detalhes no final deste capítulo.

4.1 Strings

Strings são comuns em todas as linguagens de programação e são utilizados para armazenar cadeias de caracteres. Nos capítulos anteriores, os strings já foram utilizados, e o objetivo aqui é formalizar o conceito, bem como apresentar aspectos que ainda não foram vistos.

Um string em Python é uma sequência composta por quaisquer caracteres delimitada por aspas simples ou duplas. No Exemplo 4.1 são definidos os strings S e D, cada um utilizando um tipo diferente de aspas. Ao utilizar o comando *type* com essas variáveis, verifica-se que ambas são da classe “str”, ou seja, string.

A função *len* permite descobrir o tamanho do string, pois retorna a quantidade de caracteres de seu conteúdo.

Exemplo 4.1 Primeiros usos de strings

```
>>> S = 'Cadeia de texto definido com aspas simples'
```

```
>>> D = "Cadeia de texto definido com aspas duplas"
```

```
>>> print(S)
```

Cadeia de texto definido com aspas simples

```
>>> print(D)
```

Cadeia de texto definido com aspas duplas

```
>>> type(D)
```

```
<class 'str'>
```

```
>>> type(S)
```

```
<class 'str'>
```

```
>>> len(S)
```

```
42
```

```
>>> S[0] # Primeiro elemento do string S
```

```
'C'
```

```
>>> S[1] # Segundo elemento do string S
```

```
'a'
```

```
>>> S[41] # Último elemento do string S
```

```
'S'
```

```
>>> S[42] # Elemento inexistente no string S. Gera erro.
```

Traceback (most recent call last):

File "<pyshell#7>", line 1, in <module>

```
print(S[42])
```

IndexError: string index out of range

```
>>>
```

Como também pode ser visto nesse exemplo, o uso de um índice entre colchetes que permite acesso individual aos caracteres que o compõem. Uma vez que o índice do primeiro caractere é zero, o índice do último será $len - 1$. No caso do string S do exemplo S[41] é o último caractere, visto que tem dimensão 42. Caso seja utilizado um índice além do limite, o interpretador gera uma mensagem de erro.

Todos os tipos sequenciais em Python também aceitam indexadores negativos, os quais são interpretados como contagem da direita para a esquerda, em que o índice -1 é o do último elemento, -2 do penúltimo, e assim sucessivamente, conforme mostrado no Exemplo 4.2.

Exemplo 4.2 Uso de indexação negativa em tipos sequenciais

```
>>> X = 'ABCD'
```

```
>>> X[-1]
```

```
'D'
```

```
>>> X[-2]
```

```
'C'
```

```
>>> X[-3]
```

```
'B'
```

```
>>> X[-4]
```

```
'A'
```

```
>>>
```

4.1.1 Manipulação de strings

4.1.1.1 Atribuição de valor aos strings

A qualquer momento é possível alterar o conteúdo de um objeto do tipo string. Porém, não é possível alterar individualmente um caractere que o compõe. Isso ocorre porque em Python os strings são objetos imutáveis. Isso significa que cada operação de atribuição de valor a um string, na verdade, produz um novo objeto. Isso pode ser comprovado com o uso da função *id*, vista no Capítulo 2. A cada atribuição de valor o *id* do objeto se altera, indicando ser um outro objeto.

Observe e repita as operações feitas no Exemplo 4.3. Percebe-se que, ao tentar atribuir um caractere ao elemento `V[0]`, ocorre uma mensagem de erro indicando a impossibilidade de completar o comando.

Exemplo 4.3 Manipulação de strings

```
>>> V = '' # String vazio
```

```
>>> len(V)
```


0

```
>>> type(V)
```

```
<class 'str'> # De fato V é string, mas está vazio
```

```
>>> id(V)
```

```
1742944 # id de V
```

```
>>> V = 'Novo'
```

```
>>> id(V)
```

```
49492576 # Novo id de V
```

```
>>> V = 'Outro'
```

```
>>> id(V)
```

```
49492320 # mais um
```

```
>>> V[0] = 'P'
```

```
Traceback (most recent call last):
```

```
File "<pyshell#81>", line 1, in <module>
```

```
V[0] = 'P'
```

```
TypeError: 'str' object does not support item assignment
```

4.1.1.2 Concatenação e multiplicação de strings

O Exemplo 4.4 também mostra que é possível utilizar os operadores

concatenação e “+” e multiplicação “*” com objetos string.

A concatenação aplica-se a dois operandos do tipo string e produz a junção dos dois. Por outro lado, se o programador tentar utilizar esse operador misturando string com outro tipo de objeto, ocorrerá um erro, uma vez que tais combinações não são suportadas.

Exemplo 4.4 Concatenação de strings

```
>>> S = 'Festa' # carrega S com algum texto
```

```
>>> T = ' na Vila' # faz o mesmo com T
```

```
>>> U = S + T # o operador '+' está definido em Python
```

```
>>> U # para efetuar a concatenação de strings
```

```
'Festa na Vila'
```

```
>>> U = 'Hoje tem ' + U
```

```
>>> U
```

```
'Hoje tem Festa na Vila'
```

```
>>> S = 'Festa' + 1000 # tenta concatenar string com outro tipo
```

```
Traceback (most recent call last):
```

```
File "<pyshell#115>", line 1, in <module>
```

```
S = 'Festa' + 1000
```

```
TypeError: must be str, not int
```

O operador multiplicação de string está definido para uso com um operador string e outro numérico inteiro, sem importar a ordem em que ambos aparecem na expressão. Dados um string S e um número N, ao utilizar esse operador com ambos, será gerado um string resultante em que S ocorre N vezes.

Exemplo 4.5 Multiplicação de strings

```
>>> S = "repete."
```

```
>>> T = S * 3
```

```
>>> T
```

```
'repete.repete.repete.'
```

4.1.2 Fatiamento

Fatiamento, ou *slicing*, é um recurso disponível nos tipos sequenciais existentes em Python, strings, listas e tuplas. O fatiamento é utilizado para selecionar partes específicas de um tipo sequencial e trata-se de um recurso mais poderoso do que muitos programadores imaginam. Um pouco desse poder será mostrado no Capítulo 5, no qual serão resolvidos exercícios utilizando funções recursivas que operam com listas fatiadas.

Seja um string S, o fatiamento utiliza a notação S[início:final] para selecionar o substring de S, que começa na posição dada pelo indexador início e termina na posição dada pelo indexador final – 1.

O Exemplo 4.6 inicializa o string S com as quinze primeiras letras minúsculas do alfabeto. Em seguida, é realizado o fatiamento S[3:10], que seleciona desde o caractere cujo índice é 3 até o caractere cujo índice é 9 (10 – 1). Assim sendo, a parte selecionada será “defghij”, conforme mostrado.

Caso os valores definidos para o par indexador dados por [início:final] sejam incoerentes, o fatiamento retornará um string vazio. Para que esse par seja coerente, é necessário que ocorra: início < final. Assim, S[3:4] seleciona o substring “d”.

O retorno produzido pelo fatiamento também pode ser atribuído a um novo objeto, como foi feito no exemplo com o objeto P.

Os índices de fatiamento também podem ser fornecidos por meio de objetos do tipo número inteiro em substituição aos valores numéricos fixos, conforme exemplificado a seguir com os objetos i e f.

Exemplo 4.6 Fatiamento de strings

```
>>> S = 'abcdefghijklmno' # Define o string S com 15 caracteres
```

```
>>> print(S)
```

```
abcdefghijklmno
```

```
>>> len(S)
```

```
15
```

```
>>> S[3:10] # Substring de S das posições 3 a 9
```

```
'defghij'
```

```
>>> S[0:5] # Substring de S das posições 0 a 4
```

```
'abcde'
```

```
>>> P = S[3:10] # Atribui a P o substring de S de 3 a 9
```

```
>>> print(P)

defghij

>>> len(P)

7

>>> i = 3

>>> f = 10

>>> S[i:f] # Uso de objetos como índices

'defghij'

>>> i = 0

>>> f = 5

>>> S[i:f]

'abcde'
```

O fatiamento também pode omitir um dos índices da faixa de seleção. Quando isso acontece, a interpretação é feita segundo as opções contidas no Quadro 4.1.

Exemplo 4.7 Fatiamento de strings com omissão de início ou final

```
>>> S = 'abcdefghijklmno' # Define o string S com 15 caracteres

>>> S[:5] # Substring de S das posições 0 a 4
```

```
'abcde'
```

```
>>> S[5:] # Substring de S das posições 5 ao final
```

```
'fghijklmno'
```

```
>>>
```

Por fim, o fatiamento pode apresentar um terceiro parâmetro, que é o passo. Para compreender como esse terceiro parâmetro é utilizado, suponha-se que ele tenha o valor P. Assim sendo, o string será dividido em substrings de tamanho P e apenas será selecionado o primeiro caractere de cada subdivisão. No Exemplo 4.8 isso é demonstrado.

Exemplo 4.8 Fatiamento de strings com terceiro parâmetro

```
>>> S = 'abcdefghijklmno' # Define o string S com 15 caracteres
```

```
>>> S[0:15:4] # Do início ao fim seleciona 1 a cada 4
```

```
'aeim'
```

```
>>> T = '9pula8pula7pula6pula5'
```

```
>>> T[0:21:5] # Do início ao fim seleciona 1 a cada 5
```

```
'98765' # Quando se quer trabalhar com todo o string
```

```
>>> T[::5] # é possível omitir os delimitadores
```

```
'98765' # e se produz o mesmo resultado
```

Forma de fatiamento	Interpretação
------------------------	---------------

S[ini:fim]	O fatiamento tem início e final, então, seleciona-se o substring desde a posição ini até a posição fim –1.
S[:fim]	Neste caso, foi emitido o índice inicial da faixa. O interpretador assume que deve selecionar o string a partir do primeiro caractere até a posição especificada, ou seja, de 0 a fim –1.
S[ini:]	Nesta caso, foi emitido o índice final da faixa. O interpretador assume que deve selecionar o string a partir da posição especificada até o final dele, ou seja, de ini até o final de S.
S[ini:fim:p]	Nesta opção estão colocados três parâmetros para fatiamento. Os dois primeiros definem o início e o final do substring. O terceiro define o passo, ou seja especifica que de cada p caracteres toma-se apenas o primeiro para compor o substring.

Quadro 4.1 Opções de fatiamento de tipos sequenciais.

4.1.3 Métodos da classe “str”

Em termos simples e iniciais, pode-se dizer que **métodos** são funções específicas contidas em uma classe de objetos. Em adição aos operadores vistos anteriormente, a classe string tem um conjunto de métodos úteis ao programador. Um desses métodos – o format – já foi visto no Item 2.5, em que foi explicado o comando print, utilizado para exibição de dados em tela.

Não há espaço aqui para explicar em detalhes cada um dos métodos disponíveis, porém, no Exemplo 4.9, serão mostrados os usos de alguns deles.

	Para os exemplos a seguir seja S = "abc_123_XYZ"	
Método	Descrição	Retorno
S.lower	Retorna um string com todas as letras minúsculas e não afeta os demais caracteres.	abc_123_xyz
S.upper	Retorna um string com todas as letras maiúsculas e não afeta os demais caracteres.	ABC_123_XYZ
	Retorna um string invertendo as letras	

S.swapcase	maiúsculas e minúsculas e não afeta os demais caracteres.	ABC_123_xyz
S.title	Retorna um string com a primeira letra maiúscula e as demais minúsculas, para cada sequência de letras.	Abc_123_xyz
S.capitalize	Retorna um string com a primeira letra maiúscula e as demais minúsculas.	Abc_123_xyz
S.find("123") S.find("m")	Pesquisa um substring em S e retorna um número inteiro, indicando a posição se o encontrar, ou -1 caso não encontre.	4 -1
S.count["_"]	Conta quantas vezes um substring está contido em S.	2

S.replace("_", "*")	Este método recebe dois parâmetros do tipo string. O primeiro é procurado dentro de S e, caso seja encontrado, é substituído pelo segundo substring.	abc*123*XYZ
S.isalnum	Retorna <i>True</i> caso o string contenha apenas letras e números. Caso contrário, retorna <i>False</i> .	
S.isalpha	Retorna <i>True</i> caso o string contenha apenas letras. Caso contrário, retorna <i>False</i> .	
S.isnumeric	Retorna <i>True</i> caso o string contenha apenas números. Caso contrário, retorna <i>False</i> . Útil para testar a entrada de dados numéricos digitados no teclado.	
S.partition("_")	Pesquisa S em busca do substring passado e, caso o encontre, retorna três strings: a parte antes do substring, o próprio substring e o resto de S. Caso não o encontre, retorna o próprio S mais dois strings vazios.	
S.split("_")	Retorna uma lista de strings separados a partir de S, utilizando o substring passado como parâmetro como delimitador da separação. Se ele não for fornecido, o espaço	["abc", "123", "XYZ"]

em branco é utilizado como delimitador.

Quadro 4.2 Alguns métodos disponíveis na classe “str”.

Exemplo 4.9 Métodos da classe “str”

```
>>> S = 'abc_123_XYZ'
```

```
>>> S.lower() # retorna todas as letras em minúsculas
```

```
'abc_123_xyz'
```

```
>>> S.upper()
```

```
'ABC_123_XYZ'
```

```
>>> S.title()
```

```
'Abc_123_Xyz'
```

```
>>> S.swapcase()
```

```
'ABC_123_xyz'
```

```
>>> S.find('123')
```

```
4
```

```
>>> S.find('m')
```

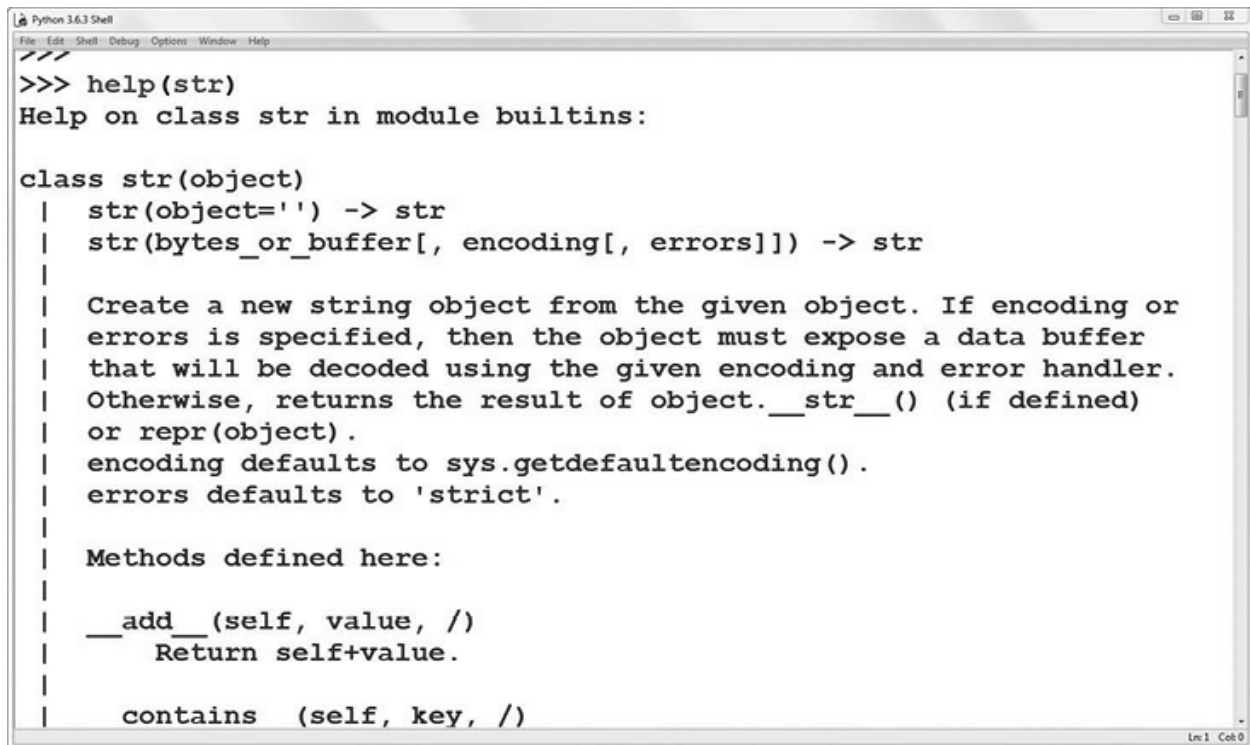
```
-1
```

```
>>> S.count('_')
```

```
2
```

```
>>> S.replace('_', '*')
'abc*123*XYZ'
>>> S.replace('123', 'xpto')
'abc_xpto_XYZ'
>>> S.partition('_')
('abc', '_', '123_XYZ')
>>> S.partition('*')
('abc_123_XYZ', '', '')
>>> S.split('_')
['abc', '123', 'XYZ']
```

Para obter um resumo de todos os métodos disponíveis na classe string, utilize os comandos ***dir*** e ***help*** no IDLE ou consulte a documentação da linguagem.

The image shows a screenshot of the Python 3.6.3 Shell window within the IDLE environment. The window title is "Python 3.6.3 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The command prompt shows the user entering ">>> help(str)". The output displays the help text for the 'str' class in the 'builtins' module. It starts with "Help on class str in module builtins:", followed by the class signature "class str(object)", and then the constructor "str(object='') -> str" and "str(bytes_or_buffer[, encoding[, errors]]) -> str". A detailed description follows: "Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer that will be decoded using the given encoding and error handler. Otherwise, returns the result of object.__str__() (if defined) or repr(object). encoding defaults to sys.getdefaultencoding(). errors defaults to 'strict'." Below this, it lists "Methods defined here:" and shows two methods: "__add__(self, value, /)" with the description "Return self+value." and "contains (self, key, /)". The status bar at the bottom right indicates "Ln:1 Col:9".

```
>>> help(str)
Help on class str in module builtins:

class str(object)
| str(object='') -> str
| str(bytes_or_buffer[, encoding[, errors]]) -> str
|
| Create a new string object from the given object. If encoding or
| errors is specified, then the object must expose a data buffer
| that will be decoded using the given encoding and error handler.
| Otherwise, returns the result of object.__str__() (if defined)
| or repr(object).
| encoding defaults to sys.getdefaultencoding().
| errors defaults to 'strict'.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| contains (self, key, /)
```

Figura 4.1 Exemplo de uso do comando help no IDLE.

4.1.4 Exercícios resolvidos utilizando o tipo string

1. Validando a entrada de dados numérica

Escreva um programa que leia um string que deve conter, obrigatoriamente, um número inteiro e, caso isso não aconteça, emita uma mensagem de erro.

Exercício resolvido 4.1

```
S = input("Digite um número inteiro: ")
```

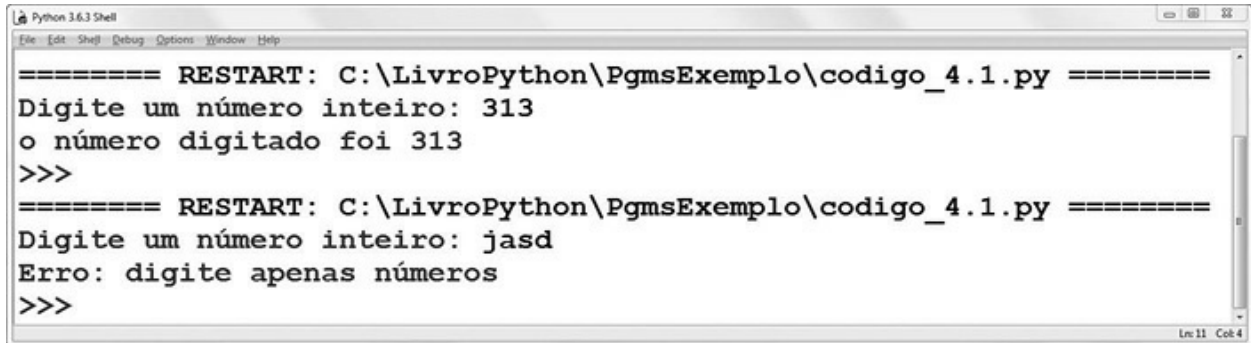
```
if S.isnumeric():
```

```
N = int(S)
```

```
print("o número digitado foi {0}".format(N))
```

else:

print('Erro: digite apenas números')



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_4.1.py =====
Digite um número inteiro: 313
o número digitado foi 313
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_4.1.py =====
Digite um número inteiro: jasd
Erro: digite apenas números
>>>
```

2.Leitura de vários números em uma mesma linha

Faça a leitura de uma linha de dados que contenha quatro números separados por espaços em branco e carregue os objetos A, B, C com os valores individuais.

Exemplo: linha a ser digitada 35 22 87

o programa deve carregar: A com 35, B com 22, C com 87

Exercício resolvido 4.2

```
S = input("Digite três números inteiros: ") # Lê o string
```

```
L = S.split(" ") # Faz a separação gerando a lista L
```

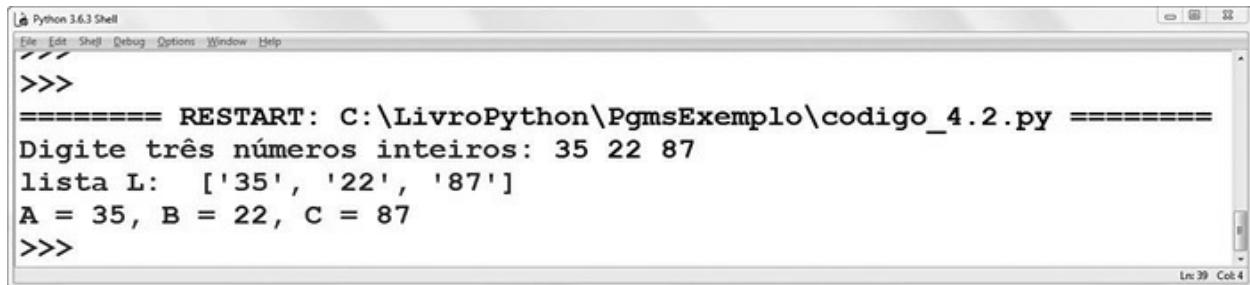
```
print("lista L: ", L) # Exibe na tela a lista L
```

```
A = int(L[0]) # converte o elemento L[0] para inteiro
```

```
B = int(L[1]) # converte o elemento L[1] para inteiro
```

```
C = int(L[2]) # converte o elemento L[2] para inteiro
```

```
print("A = {}, B = {}, C = {}".format(A, B, C))
```



```
>>>
==== RESTART: C:\LivroPython\PgmsExemplo\codigo_4.2.py =====
Digite três números inteiros: 35 22 87
lista L: ['35', '22', '87']
A = 35, B = 22, C = 87
>>>
```

Nessa solução, foi utilizado o método `split()` para separar as partes do string `S`, utilizando o espaço em branco como delimitador. O retorno do `split` é uma lista (o próximo assunto deste capítulo) que foi atribuída ao objeto `L` e exibida na tela. Em seguida, os elementos 0, 1, 2 de `L` foram convertidos para número inteiro e armazenados, respectivamente, nos objetos `A`, `B` e `C`. Não foi feita uma validação individual das partes digitadas, de modo que, se o usuário digitar caracteres não numéricos, o programa gerará erro no momento da conversão utilizando a função `int()`.

4.2 Listas

Durante o desenvolvimento de software, independentemente de plataforma e linguagem, é comum a necessidade de criar, manter e manipular conjuntos de dados. Tais conjuntos são muito variados, tanto quanto à natureza dos dados como com relação às quantidades envolvidas.

Na linguagem Python, o tipo lista é a ferramenta disponível para atender a essa demanda e representa o mais genérico, versátil e poderoso tipo sequencial. Por exemplo, as listas podem ser empregadas para armazenar coleções de números inteiros ou reais, palavras, nomes ou quaisquer outras informações necessárias à solução de algum problema computacional. Muitas vezes, tais conjuntos são muito grandes, e é preciso mantê-los e manipulá-los de maneira segura e eficiente na memória, bem como gravá-los em disco ou

enviá-los de um computador para outro em uma rede.

Assim como o `string`, é um tipo sequencial composto por elementos organizados de modo linear, na qual cada um pode ser acessado a partir de um índice que representa sua posição na coleção, iniciando em zero. Em função disso, a lista suporta muitas das mesmas operações que já foram vistas para o tipo `string` no Item 4.1. Então, tem-se que as listas apresentam os mesmos mecanismos de indexação e fatiamento, suportam os operadores de concatenação “+” e multiplicação “*” e têm comprimento variável, que pode ser descoberto com o uso da função `len`.

Por outro lado, algumas de suas principais características são completamente opostas às dos `strings`. Quanto ao conteúdo, os elementos de uma lista podem ser qualquer tipo de objeto. Além disso, as listas são mutáveis, de modo que seus elementos podem ser alterados livremente e a qualquer momento pelo programador.

4.2.1 Operações básicas com listas

O Exemplo 4.10 mostra as operações básicas possíveis de ser efetuadas com as listas. É possível criar uma lista atribuindo-se um conjunto de dados entre colchetes [] a um identificador de objeto. Se não houver dados entre os colchetes, cria-se uma lista vazia.

O acesso individual aos elementos da lista é feito por meio de seu índice e cada elemento é mutável, podendo, portanto, ser alterado. Além disso, esses elementos podem ser de quaisquer tipos, compondo uma lista heterogênea.

Caso se queira excluir um elemento da lista, basta utilizar o comando `del`, passando como parâmetro o elemento a ser excluído.

Exemplo 4.10 Operações básicas com listas

```
>>> L = [] # cria uma lista vazia
```

```
>>> type(L) # mostra o tipo do objeto L
```

```
<class 'list'>
```

```
>>> print(L) # exibe a lista L
```

```
[]
```

```
>>> L = [10, 15, 20, 25, 30] # L passa a conter 4 elementos
```

```
>>> print(L) # Exibe a lista. No IDLE pode-se
```

```
[10, 15, 20, 25, 30] # omitir o print
```

```
>>> L[0] # primeiro elemento: índice 0
```

```
10
```

```
>>> L[4] # último elemento: índice 4
```

```
30
```

```
>>> len(L) # comprimento de L
```

```
5
```

```
>>> L[0] = 8 # L é mutável
```

```
>>> L
```

```
[8, 15, 20, 25, 30] # L[0] foi alterado para 8
```

```
>>> A = [3, 7.5, 'txt'] # Nova lista elementos heterogêneos
```

```
>>> A
```

```
[3, 7.5, 'txt']
```

```
>>> type(A)
```

```
<class 'list'>
```

```
>>> type(A[0]) # o primeiro elemento é 'int'
```

```
<class 'int'>
```

```
>>> type(A[1]) # o segundo elemento é 'float'
```

```
<class 'float'>
```

```
>>> type(A[2]) # o terceiro elemento é 'str'
```

```
<class 'str'>
```

```
>>> del(A[1]) # Exclui o segundo elemento de A
```

```
>>> A # no caso, o valor 7.5
```

```
[3, 'txt']
```

No Exemplo 4.11, deve-se ter o cuidado de interpretar o resultado produzido pelo operador de adição “+”. Quando os operadores envolvidos forem elementos da lista, a operação será definida em função dos tipos desses elementos. No caso do exemplo, trata-se de tipos numéricos, e o resultado é a adição dos valores neles contidos. Se o mesmo operador for aplicado a duas listas, então, o resultado será uma nova lista concatenando-as, gerando uma terceira.

Exemplo 4.11 Uso do operador aditivo “+” com listas

```
>>> X = L[0] + A[1] # Soma o primeiro elemento de L com
```

```
>>> X # o segundo elemento de A, ou seja,
```

```
15.5 # 8 + 7.5 = 15.5
```

```
>>> X = L + A # Cuidado: aqui é diferente. Como
```

```
>>> X # não foram usados os índices
```

```
[8, 15, 20, 25, 30, 3, 7.5, 'txt']
```

```
>>> type(X) # o resultado foi a concatenação
```

```
<class 'list'> # das duas listas L e A.
```

O Exemplo 4.12 ilustra o fatiamento de listas, que segue o conceito já visto para strings, porém, neste caso, produzindo como resultado uma nova lista. Utiliza-se a mesma notação `L[início:final]` para selecionar o sublista de `L` que começa na posição dada pelo indexador início e termina na posição dada pelo indexador final – 1. Também é possível utilizar a notação que inclui o passo: `L[início:final:passo]`, que, dentro do intervalo `[início:final]`, seleciona o primeiro elemento de cada subintervalo dado pelo valor contido no passo.

Exemplo 4.12 Fatiamento de listas

```
>>> L = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23]
```

```
>>> L[0:3] # elementos de 0 a 2
```

```
[1, 3, 5]
```

```
>>> L[4:10] # elementos de 4 a 9
```

```
[9, 11, 13, 15, 17, 19]
```

```
>>> L[:5] # elementos de 0 a 4
```

```
[1, 3, 5, 7, 9]
```

```
>>> L[5:] # elementos de 5 ao último
```

```
[11, 13, 15, 17, 19, 21, 23]
```

```
>>> L[0:8:3] # elementos de 0 a 7 e
```

```
[1, 7, 13] # retorna o primeiro a cada 3
```

```
>>> L[::4] # considera a lista toda e
```

```
[1, 9, 17] # retorna o primeiro a cada 4
```

Quando aplicado a listas, o operador multiplicativo “*” necessita de uma lista de origem e um número inteiro e produz uma nova lista com diversas repetições da lista original. Assim, conforme mostrado no Exemplo 4.13, se a lista for [3, 7] e o número inteiro for 3, será produzida a lista [3, 7, 3, 7, 3, 7].

Exemplo 4.13 Uso do operador multiplicativo “*” com listas

```
>>> A = [3, 7] * 3
```

```
>>> A
```

```
[3, 7, 3, 7, 3, 7]
```

```
>>> L = [0] * 10
```

```
>>> L
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Outra operação que pode ser útil em muitos casos é a conversão de um string em uma lista. Anteriormente, foi mostrado o uso do método `split()` pertencente à classe “str”, que é capaz de separar um string em elementos. É possível também separar um string fazendo que cada caractere seja um elemento em uma lista resultante, conforme mostrado no Exemplo 4.14.

Exemplo 4.14 Conversão de string em lista

```
>>> S = 'Um texto.'
```

```
>>> L = list(S) # uso da função list para separar
```

```
>>> L # um string
```

```
['U', 'm', ' ', 't', 'e', 'x', 't', 'o', '.']
```

```
>>> S = '5 7 8.8 12' # string com espaços em branco
```

```
>>> L = S.split() # separa S usando espaço em branco
```

```
>>> L # como delimitador
```

```
['5', '7', '8.8', '12']
```

```
>>> S = '5;7;8.8;12' # string com o caractere ';'
```

```
>>> L = S.split(';') # separa S usando ';'
```

```
>>> L # como delimitador
```

```
['5', '7', '8.8', '12']
```

```
>>>
```

4.2.2 Operador in

O operador *in* permite ao programador verificar se um valor está presente em uma lista. Ou, então, pode-se utilizá-lo em conjunto com o operador lógico *not* (*not in*) para verificar o contrário. Em ambos os casos, o resultado produzido é *True* (verdadeiro) ou *False* (falso).

Esse operador não se aplica apenas às listas, pelo contrário, ele pode ser utilizado em todos os tipos estruturados existentes em Python. O Exemplo 4.15 ilustra alguns de seus usos. Mais adiante, será visto que há outras formas de utilizá-lo.

Exemplo 4.15 Operadores *in* e *not in*

```
>>> L = [3, 6, 9]
```

```
>>> 9 in L # 9 está em L
```

```
True
```

```
>>> 5 in L # 5 não está em L
```

```
False
```

```
>>> 5 not in L # como 5 não está em L o operador
```

```
True # not in resulta em True
```

```
>>> Caes = ['Labrador', 'Poodle', 'Terrier']
```

```
>>> a = 'Collie' # testar se 'Collie' está na lista
```

```
>>> if a in Caes:
```

```
... print('Boa escolha')
```

```
... else:
```

```
... print('Não temos essa raça')
```

```
Não temos essa raça # este é o resultado do if-else
```

```
>>>
```

4.2.3 Métodos da classe “list”

A classe “list” apresenta um conjunto de métodos que podem ser utilizados pelos programadores para executar tarefas típicas envolvendo listas. O Quadro 4.3 apresenta todos esses métodos e os descreve. O Exemplo 4.16 ilustra seu uso.

Para obter um resumo de todos os métodos disponíveis na classe “list”, use os comandos *dir(list)* e *help(list)* ou consulte a documentação da linguagem.

	Considere-se disponível a lista L
Método	Descrição
L.append(object)	Acrescenta um objeto à lista. Exemplo: L.append(5)
L.clear()	Limpa a lista, removendo todos seus elementos. Exemplo: L.clear()
L.copy()	Produz uma cópia da lista L. Exemplo: Nova - L.copy()
L.count(object)	Retorna o número de ocorrências de um objeto dentro da lista Exemplo: Qtde - L.count(5)
L.extend(iterable)	Expande a lista L, acrescentando a ela todos os elementos contidos no objeto iterável passado como parâmetro.

	Exemplo: L.extend(OutraLista)
L.index(value, [start, stop])	Retorna o índice da primeira ocorrência do valor “value” dentro da lista. Se start e stop (opcionais) forem fornecidos, o método considera apenas seu intervalo. Caso “value” não esteja na lista, é gerado um erro. Exemplo: posição = L.index(5)
L.insert(index, object)	Insere o objeto fornecido na posição dada por “index”, deslocando todos os demais para a direita. Exemplo: L.insert(2, 30)
L.pop(index)	Retorna o elemento que está na posição dada por “index” e o remove da lista. Exemplo: L.pop(0)
L.remove(value)	Remove da lista a primeira ocorrência do valor “value”. Se o valor não estiver na lista, gera um erro. Exemplo: L.remove(5)
L.reverse()	Inverte a posição dos elementos dentro da lista: o primeiro valor passa a ser o último, o segundo passa a penúltimo, e assim por diante. Não retorna nada, pois inverte a própria lista. Exemplo: L.reverse()
L.sort(...)	Ordena a lista, colocando-a em ordem crescente ou decrescente. Não retorna nada, pois ordena a própria lista. Exemplo: L.sort() # ordena em ordem crescente L.sort(reverse=True) # ordena em ordem decrescente Observações: 1. Se desejar preservar a lista L e gerar uma cópia ordenada dela, utilize a função sorted em vez deste método. 2. O método sort não funciona com listas heterogêneas que misturem números com strings.

Quadro 4.3 Métodos da classe “list” disponíveis ao programador.

Exemplo 4.16 Métodos da classe “list”

```
>>> L = [3, 6, 9]
```

```
>>> L.append(5) # insere novo objeto no final de L
```

```
>>> L
```

```
[3, 6, 9, 5]
```

```
>>> L.append(2)
```

```
>>> L
```

```
[3, 6, 9, 5, 2]
```

```
>>> L.insert(2, 15) # insere novo objeto na posição 2
```

```
>>> L
```

```
[3, 6, 15, 9, 5, 2]
```

```
>>> L.insert(99, 21) # insere novo objeto na posição 99,
```

```
>>> L # porém a lista não tem tais posições
```

```
[3, 6, 15, 9, 5, 2, 21] # então insere no final
```

```
>>> L.append(6)
```

```
[3, 6, 15, 9, 5, 2, 21, 6]
```

```
>>> L.count(6) # conta as ocorrências do valor 6
```

2

```
>>> L.count(45) # conta as ocorrências do valor 45
```

0

```
>>> L.index(15) # retorna o índice de 15
```

2

```
>>> L.index(6) # retorna o índice da primeira
```

1 # ocorrência de 6

```
>>> L.index(45) # 45 não está na lista, gera erro
```

Traceback (most recent call last):

File "<pyshell#76>", line 1, in <module>

L.index(45)

ValueError: 45 is not in list

```
>>> L.pop(3) # retorna o elemento de índice 3 e o
```

9 # remove da lista

```
>>> L
```

[3, 6, 15, 5, 2, 21, 6]

```
>>> L.remove(6) # remove a primeira ocorrência de 6
```



```
>>> L
```

```
[3, 15, 5, 2, 21, 6]
```

```
>>> A = [22, 32, 42]
```

```
>>> L.extend(A) # acrescenta a lista A em na lista L
```

```
>>> L
```

```
[3, 15, 5, 2, 21, 6, 22, 32, 42]
```

```
>>> L.reverse() # inverte a lista
```

```
>>> L
```

```
[42, 32, 22, 6, 21, 2, 5, 15, 3]
```

```
>>> L.reverse() # inverte novamente
```

```
>>> L
```

```
[3, 15, 5, 2, 21, 6, 22, 32, 42]
```

```
>>> L.sort() # ordena em ordem crescente
```

```
>>> L
```

```
[2, 3, 5, 6, 15, 21, 22, 32, 42]
```

```
>>> L.sort(reverse=True) # ordena em ordem decrescente
```

```
>>> L
```

```
[42, 32, 22, 21, 15, 6, 5, 3, 2]
```

```
>>> L.clear() # limpa a lista, deixando-a vazia
```

```
>>> L = ['dado', 'uva', 'caixa', 'lata', 'casa']
```

```
>>> L.sort() # ordenação de lista com strings
```

```
>>> L
```

```
['caixa', 'casa', 'dado', 'lata', 'uva']
```

Não é possível usar o método sort com a lista heterogênea abaixo

```
>>> L = [23, 7.7, 3.9, 3, 35, 'txt', '3em1']
```

```
>>> L.sort() # se usar ocorre erro
```

Traceback (most recent call last):

File "<pyshell#28>", line 1, in <module>

L.sort()

TypeError: '<' not supported between instances of 'str' and 'float'

4.2.3 Exercícios resolvidos utilizando o tipo lista

3.Criação de uma progressão aritmética

Escreva um programa que leia três dados de entrada: o primeiro termo, a razão e a quantidade de termos de uma P.A., todos números inteiros. O programa deve calcular todos os termos, colocando-os em uma lista, e exibi-

la no final.

Observação

Este exercício já foi resolvido e explicado no Capítulo 3 (veja Exercício resolvido 3.2). A diferença aqui é que se está utilizando uma lista para armazenar os diversos termos antes de exibi-los.

Exercício resolvido 4.3

```
P = int(input("Digite o primeiro termo: "))
```

```
R = int(input("Digite a razão da PA: "))
```

```
Qtde = int(input("Digite a quantidade de termos: "))
```

```
L = [] # inicia uma lista vazia
```

```
cont = 0
```

```
while(cont < Qtde):
```

```
    L.append(P) # acrescenta P em L
```

```
    P = P + R # ou pode ser usada a forma P+=R
```

```
    cont+=1
```

```
print("PA resultante: ", L)
```

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
>>>
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_4.3.py =====
Digite o primeiro termo: 7
Digite a razão da PA: 5
Digite a quantidade de termos: 8
PA resultante: [7, 12, 17, 22, 27, 32, 37, 42]
>>>
```

4.Criação de uma lista contendo números inteiros

Escreva um programa que permaneça em laço lendo números inteiros enquanto os valores digitados forem diferentes de zero. Cada número não zero digitado deve ser incluído em uma lista. Ao final, exiba a lista e seu tamanho.

Exercício resolvido 4.4

```
L = [] # inicia uma lista vazia
```

```
x = int(input("Digite um inteiro: ")) # lê o primeiro valor
```

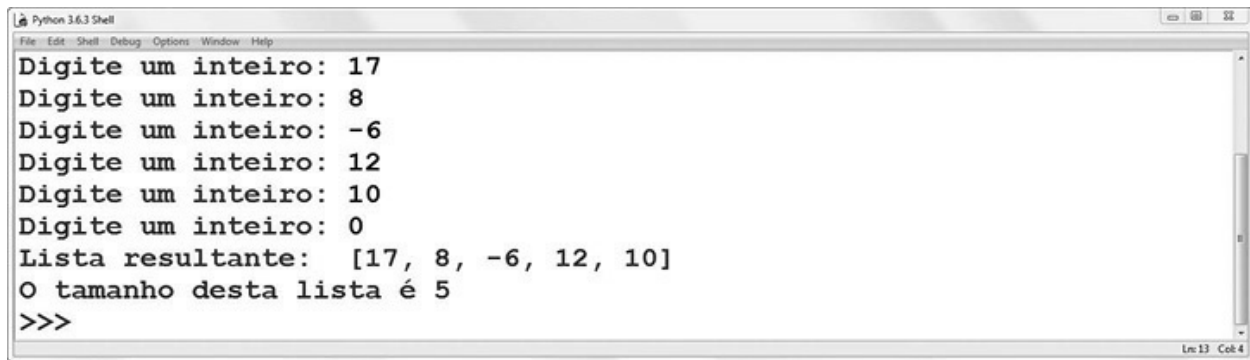
```
while(x != 0): # enquanto x não for zero
```

```
L.append(x) # insere na lista
```

```
x = int(input("Digite um inteiro: ")) # lê o próximo
```

```
print("Lista resultante: ", L)
```

```
print("O tamanho desta lista é {}".format(len(L)))
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Digite um inteiro: 17
Digite um inteiro: 8
Digite um inteiro: -6
Digite um inteiro: 12
Digite um inteiro: 10
Digite um inteiro: 0
Lista resultante: [17, 8, -6, 12, 10]
O tamanho desta lista é 5
>>>
```

5.Criação de uma lista contendo números inteiros

Escreva um programa que leia um número inteiro N e gere uma lista com N elementos aleatórios (utilize a função *randint* explicada no Exercício resolvido 3.7) entre 10 e 50. Exiba a lista gerada e exiba também a mesma lista com seus elementos ordenados em ordem crescente.

Tarefa adicional

Essa solução tem um problema. Caso o dado digitado não seja um número inteiro, o uso da função de conversão *int* causará um erro. Faça um teste. Isso pode ser resolvido com o comando *try-except* (Capítulo 3, Item 3.3). Como desafio, sugere-se ao leitor que faça tal alteração.

Exercício resolvido 4.5

```
from random import randint # importa a função randint
```

```
N = int(input("Digite o tamanho da lista: ")) # lê N
```

```
L = []
```

```
i = 0
```

```
while(i < N):
```

```
    x = randint(10, 50) # gera um aleatório entre 10 e 50
```

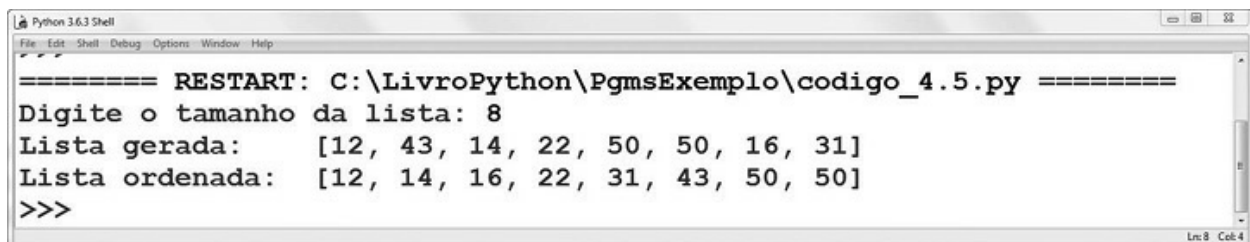
```
L.append(x) # acrescenta na lista
```

```
i+=1
```

```
print("Lista gerada: ", L) # exibe a lista gerada
```

```
L.sort() # ordena
```

```
print("Lista ordenada: ", L) # exibe a lista ordenada
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:\LivroPython\PgmsExemplo\codigo_4.5.py =====
Digite o tamanho da lista: 8
Lista gerada:      [12, 43, 14, 22, 50, 50, 16, 31]
Lista ordenada:    [12, 14, 16, 22, 31, 43, 50, 50]
>>>
```

Tarefa adicional

Como foi visto, a execução desse programa para $N = 8$ gerou uma lista contendo duas ocorrências do valor 50. Nada foi dito no enunciado sobre isso. A tarefa é alterar esse programa de modo que não haja valores duplicados na lista gerada. Dica: lembre-se dos operadores *in* e *not in*.

4.2.5 Listas vinculadas

A seguir é abordada uma característica das listas. Em Python é possível utilizar o operador de atribuição “=” para atribuir uma lista existente a outro identificador. No Exemplo 4.17 foi criada a lista L com cinco elementos e, em seguida, foi utilizado o operador de atribuição para vincular V a L: $V = L$. De fato, o termo correto aqui é **vincular** e não **copiar**. Observe as linhas seguintes do exemplo: o primeiro elemento de V foi alterado de 2 para 15 e a lista L também foi alterada.

Isso ocorre porque, ao utilizar o operador de atribuição “=”, o efeito produzido foi fazer que o novo identificador V aponte para os mesmos dados

contidos em L, ou seja, após o comando $V = L$, os dois identificadores passam a ter o mesmo id, portanto, fazendo referência ao mesmo local da memória onde está armazenado o conteúdo da lista.

Caso precise de uma cópia da lista original, deve utilizar o método *copy*, como mostrado no final deste exemplo.

Exemplo 4.17 Listas vinculadas

```
>>> L = [2, 4, 6, 8, 10]
```

```
>>> V = L # cria o vínculo entre V e L
```

```
>>> V
```

```
[2, 4, 6, 8, 10]
```

```
>>> V[0]
```

```
2
```

```
>>> V[0] = 15 # de modo que ao alterar V altera-se
```

```
>>> V # L e vice-versa
```

```
[15, 4, 6, 8, 10]
```

```
>>> L
```

```
[15, 4, 6, 8, 10]
```

```
>>> id(L) # |
```

```
48849904 # | Verifique que L e V
```

```
>>> id(V) # | tem o mesmo id

48849904 # |

>>> C = L.copy() # cria a lista C como uma cópia de L

>>> C

[15, 4, 6, 8, 10]

>>> id(C) # C tem outro id

48889048

>>> C[0] = 2 # alterações em C não alteram L

>>> C # e vice-versa

[2, 4, 6, 8, 10]

>>> L

[15, 4, 6, 8, 10]
```

4.2.6 Listas aninhadas

Foi dito anteriormente que uma lista pode conter qualquer tipo de objeto disponível em Python. No entanto, até o momento foram apresentados exemplos de listas contendo números e strings. Aqui, o objetivo é mostrar outras opções, tais como listas e tuplas dentro de listas. O termo “listas aninhadas” (em inglês, *nesting*) é utilizado para a situação em que há listas dentro de uma lista.

No Exemplo 4.18 a lista L é uma lista aninhada que contém, inicialmente,

duas sublistas (termo informal que aqui será utilizado para referenciar as listas que estão contidas em outra lista).

Para se ter acesso a um elemento de uma lista aninhada, é preciso utilizar dois índices entre colchetes e posicionados lado a lado da seguinte maneira: $L[i][j]$. O primeiro índice, i , seleciona a sublista i e o segundo índice, j , seleciona o elemento j dentro da sublista i . Com esse recurso, é possível implementar matrizes em programas escritos em Python.

Exemplo 4.18 Listas aninhadas

```
>>> L = [[1, 2, 3], [4, 5, 6]] # L é uma lista aninhada
```

```
>>> L[0]
```

```
[1, 2, 3]
```

```
>>> type(L[0]) # L[0] é o primeiro elemento de L
```

```
<class 'list'> # e é uma lista
```

```
>>> L[1]
```

```
[4, 5, 6]
```

```
>>> type(L[1]) # L[1] é o segundo elemento de L
```

```
<class 'list'> # e também é uma lista
```

```
>>> L[0][0] # primeiro elemento da sublista 0
```

```
1
```

```
>>> L[1][0] # primeiro elemento da sublista 1
```

4

```
>>> L[1][2] # terceiro elemento da sublista 1
```

6

```
>>> A = [7, 8, 9, 10] # seja a matriz A com 4 elementos
```

```
>>> L.append(A) # pode-se usar append e incluir a em L
```

```
>>> L # L fica assim
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9, 10]]
```

É possível utilizar o método *append* de uma lista para acrescentar a ela uma nova lista, como está exemplificado no final do Exemplo 4.18.

Além disso, as sublistas podem ser de variados tamanhos e conteúdos. Tudo pode ser aninhado em Python, conforme a vontade e a necessidade do programador.

Outra característica é que o grau de aninhamento não tem limite de profundidade, ou seja, é possível ter uma lista, dentro de outra lista, dentro de outra maior ainda, e assim por diante, tantos níveis quantos forem necessários, para resolver algum problema computacional. O Exemplo 4.19 mostra isso, com a lista L, que apresenta grau de profundidade 4.

Exemplo 4.19 Listas aninhadas com profundidade 4

```
>>> L = [[1, 2, ['3.1', '3.2', ['3.3.1', ['3.3.2.1', '3.3.2.2'], '3.3.3']], [4, 5, 6]]
```

```
>>> L[0]
```

```
[1, 2, ['3.1', '3.2', ['3.3.1', ['3.3.2.1', '3.3.2.2'], '3.3.3']]]
```

```
>>> L[0][2]
```

```
['3.1', '3.2', ['3.3.1', ['3.3.2.1', '3.3.2.2'], '3.3.3']]
```

```
>>> L[0][2][2]
```

```
['3.3.1', ['3.3.2.1', '3.3.2.2'], '3.3.3']
```

```
>>> L[0][2][2][1] # a lista mais aninhada tem 4 índices
```

```
['3.3.2.1', '3.3.2.2']
```

```
>>>
```

4.2.7 Exercícios resolvidos utilizando listas aninhadas

6.Programa para criar e exibir uma matriz

Escreva um programa que leia dois números inteiros Lin e Col, que representam, respectivamente, a quantidade de linhas e colunas em uma matriz. Utilizando listas aninhadas, crie uma representação para essa matriz, utilizando a função *randint* para gerar números para cada posição da matriz. Apresente-a na tela com uma aparência matricial.

Exercício resolvido 4.6

```
from random import randint
```

```
Lin = int(input("Quantidade de linhas = "))
```

```
Col = int(input("Quantidade de colunas = "))
```

```
M = []
```

```
i = 0 # ver Detalhe 1
```

```
while i < Lin: # ver Detalhe 2
```

```
M.append([]) # ver Detalhe 4
```

```
j = 0
```

```
while j < Col:
```

```
M[i].append(randint(0, 20)) # ver Detalhe 3
```

```
j+=1 # incrementa o índice de coluna
```

```
i+=1 # incrementa o índice da linha
```

```
print("\nEsta é a lista M gerada") # exibe M se formatação
```

```
print('M =', M)
```

```
print("\nExibindo como matriz fica assim")
```

```
i = 0
```

```
while i < Lin: # laço externo varia o índice de linhas
```

```
j = 0
```

```
print('|', end='')
```

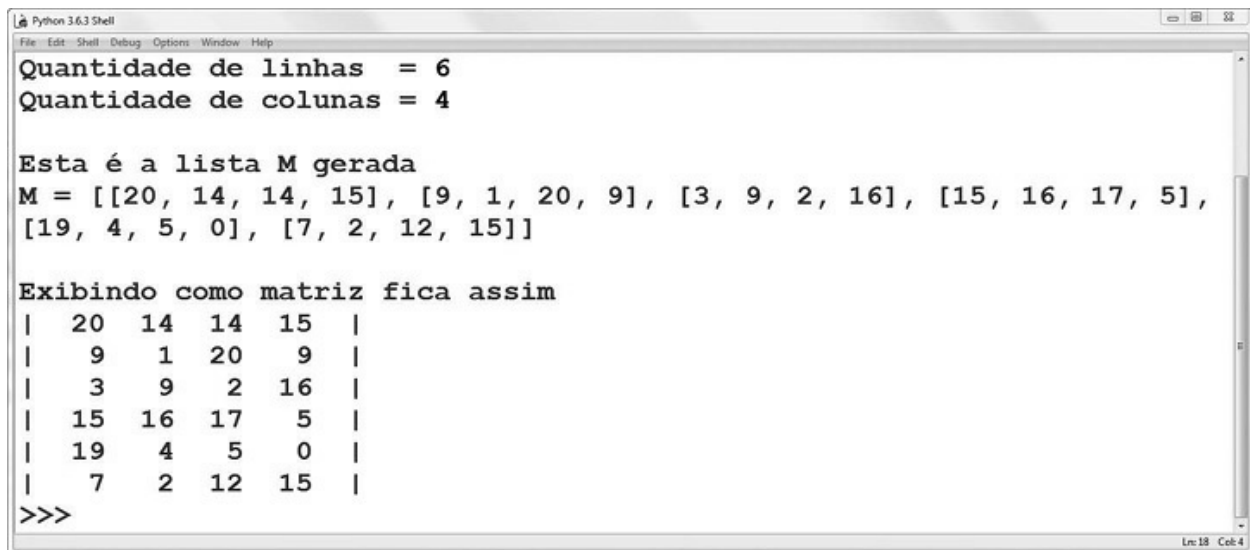
```
while j < Col: # laço interno varia o índice de cols
```

```
print("{0:4}".format(M[i][j]), end='') # e exibe cada elemento
```

```
j+=1
```

```
print(' |')
```

```
i+=1
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Quantidade de linhas = 6
Quantidade de colunas = 4

Esta é a lista M gerada
M = [[20, 14, 14, 15], [9, 1, 20, 9], [3, 9, 2, 16], [15, 16, 17, 5],
[19, 4, 5, 0], [7, 2, 12, 15]]

Exibindo como matriz fica assim
| 20 14 14 15 |
| 9 1 20 9 |
| 3 9 2 16 |
| 15 16 17 5 |
| 19 4 5 0 |
| 7 2 12 15 |
>>>
```

Detalhes dessa solução:

1. São necessários dois índices, i e j, para escrever esse programa, em que i será utilizado como índice de linha, e j, como índice de coluna da matriz. Na prática, i será o primeiro indexador da lista, e j, o segundo.
2. Ao utilizar M[i] no programa, está se fazendo referência à sublista i da lista M. E, ao utilizar o método M[i].append(numero), está sendo acrescentado um elemento a essa sublista. Para que a lista tenha Lin linhas (sublistas), é preciso que i varie de 0 a Lin-1 no laço externo.
3. Para que a lista tenha Col colunas, é preciso que o segundo índice, j,

varie de 0 a Col -1 no laço interno. Dentro deste é que é utilizado o método `M[i].append(numero)` descrito anteriormente.

4. Antes de iniciar o laço interno, é preciso garantir que a sublista `M[i]` exista. Se não existir, ocorrerá erro no programa. É por isso que esse comando `M.append([ ])` foi incluído nesse ponto do programa. Ele garante a existência de uma lista `M[i]` vazia e que será preenchida no laço interno.

4.2.8 Agora, um erro comum

Quem está iniciando o aprendizado em Python e já aprendeu muitos aspectos da linguagem pode se sentir tentado a gerar a matriz utilizando o operador multiplicativo “*” para depois preenchê-la. Isso é algo natural para o estudante, mas pode levá-lo a um resultado confuso, como mostrado a seguir.

Exemplo 4.20 Uso do operador multiplicativo “*” para produzir uma matriz – um erro comum

```
>>> Lin = 5 # quantidade de linhas

>>> Col = 3 # quantidade de colunas

>>> A = [0] * Col # lista temporária A, ela será base

>>> A # para as sublistas de M

[0, 0, 0] # como fica A

>>> A[0] = 5

>>> A[1] = 12
```

```
>>> A[2] = 15
```

```
>>> A
```

```
[5, 12, 15]
```

```
>>> M = [A] * Lin # M contém Lin sublistas [A]
```

```
>>> M # resultado produzido em M
```

```
[[5, 12, 15], [5, 12, 15], [5, 12, 15], [5, 12, 15], [5, 12, 15]]
```

```
# como visto acima M tem 5 sublistas e agora pode ter preenchidos
```

```
# seus demais elementos
```

```
>>> M[1][0] = 9 # primeiro elemento de M[1] = 9
```

```
>>> M # este é o resultado
```

```
[[9, 12, 15], [9, 12, 15], [9, 12, 15], [9, 12, 15], [9, 12, 15]]
```

```
# todas as sublistas de M agora tem seu primeiro elemento = 9
```

O que aconteceu foi que o operador multiplicativo “*”, quando aplicado na linha `M = [A] * Lin`, produziu múltiplas referências ao objeto `A`, cujo conteúdo continua a ser único na memória, ou seja, o `id` de todas as sublistas de `M` é o mesmo, pois são listas vinculadas, como visto no Item 4.2.5. Por outro lado, quando foi feito `A = [0] * Col`, foi possível alterar individualmente os elementos da lista `A`. De fato, como os elementos de `A` são do tipo “int”, eles são imutáveis, e a cada atribuição `A[i] = ...` o objeto anterior foi destruído, e um novo, criado. Já no caso da lista `M`, seus objetos

constituintes são também listas, as quais são mutáveis, de modo que o interpretador Python aplicou a eles o conceito de referência dinâmica, resultando nessa indesejada troca de todos os primeiros elementos das sublistas.

Portanto, a solução inicial apresentada para esse exercício resolvido é a que melhor se aplica.

4.3 Tuplas

A tupla é um tipo sequencial em muitos aspectos semelhante à lista, porém, imutável como um string. As tuplas são definidas, atribuindo uma lista de dados separados por vírgulas ou, como é mais frequente, encapsulando os dados em parênteses. Uma vez criada, a tupla não pode ser modificada.

Tuplas são capazes de conter quaisquer outros tipos definidos em Python, números, strings, listas, outras tuplas etc. Como são sequenciais, o acesso aos elementos se dá por meio de índices, para os quais valem as mesmas regras de strings e listas. Aceitam os operadores de concatenação “+” e multiplicativo “*” e aplicam-se a elas as operações de fatiamento.

As tuplas são muito utilizadas para encapsular o retorno de funções. Esse assunto será visto em detalhes no Capítulo 5.

4.3.1 Operações básicas com tuplas

O Exemplo 4.21 ilustra as operações básicas frequentemente realizadas com tuplas.

Exemplo 4.21 Operações básicas em tuplas

```
>>> V = () # define uma tupla vazia
```

```
>>> V
```


()

```
>>> len(V) # o tamanho de V é zero
```

0

```
>>> P = 3, 6, 9 # define uma tupla
```

```
>>> P
```

(3, 6, 9)

```
>>> T = (17, 3, 'txt', 3.8) # define uma tupla
```

```
>>> type(T)
```

<class 'tuple'>

```
>>> len(T) # contém 4 elementos
```

4

```
>>> T[0]
```

17

```
>>> T[2]
```

'txt'

```
>>> T[3]
```

3.8

```
>>> T = T + (15, 16) # concatenação com outra tupla
```

```
>>> T
```

```
(17, 3, 'txt', 3.8, 15, 16)
```

```
>>> P = (0, 1)
```

```
>>> P = P * 6 # uso do operador multiplicativo '*'
```

```
>>> P
```

```
(0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1)
```

É necessário que o programador tenha atenção e tome cuidado ao definir uma tupla que contenha um único elemento numérico. O Exemplo 4.22 mostra que, ao se tentar definir uma tupla com um único valor, define-se um objeto de tipo “int”. Isso se deve ao fato de que os parênteses também são utilizados para expressões aritméticas como $(2 + X) * 2$. O Python, haja vista seu esquema de prioridades, interpreta a expressão (14) como uma expressão aritmética, e não como uma tupla. Para se definir corretamente uma tupla com um único objeto, deve-se usar um caractere vírgula “,” dentro dos parênteses e após o objeto, como mostrado.

Exemplo 4.22 Detalhes sobre as tuplas

```
>>> T = (14) # essa sintaxe define um 'int' e não
```

```
>>> type(T) # uma tupla
```

```
<class 'int'>
```

```
>>> T = (14,) # desta forma se define uma tupla
```

```
>>> type(T) # contendo um único objeto
```

```
<class 'tuple'>
```

4.3.2 Atribuições envolvendo tuplas

O operador de atribuição tem sido largamente utilizado neste livro, sem que em qualquer momento tenha sido explicado de maneira formal. Trata-se de uma operação um tanto óbvia e intuitiva, e por esse motivo tal falta de formalização pode ter passada despercebida.

Quando se escreve $X = 10$, está se atribuindo o conteúdo 10 ao identificador X. Quando se escreve $Y = (2 + X) / 3$, o resultado da expressão aritmética está sendo calculado e atribuído ao identificador Y. E, ao escrever $L = [9, 18, 27]$, está sendo criada uma lista com três elementos e atribuída ao identificador L. Em expressões assim, deve começar sua leitura pelo lado direito, pois é ele que gera o objeto resultante em memória. Em seguida, o identificador explicitado do lado esquerdo é aplicado a esse objeto.

É utilizando esse mecanismo que as atribuições são processadas pelo interpretador Python. O mesmo ocorre quando se trata de tuplas, porém, com elas há uma característica que difere dos demais tipos de objetos da linguagem.

As tuplas podem estar **dos dois lados** da operação de atribuição. A primeira atribuição feita no Exemplo 4.23 mostra esse conceito. Seu lado direito contém uma tupla de números inteiros definida sem o uso de parênteses. E o lado esquerdo contém uma tupla de identificadores. A associação é feita segundo a sequência com que os identificadores e os valores aparecem na expressão. Assim, o identificador a recebe o valor 10, b recebe 15 e c recebe 20. Para que essa atribuição múltipla funcione, é necessário que em ambos os lados da expressão haja o mesmo número de identificadores e valores.

O Exemplo 4.23 mostra outras situações. Observe-as com atenção. O último caso, em particular, mostra como fazer a inversão de dados entre duas variáveis, algo muito comum em programas. A expressão `a, b = b, a` faz que o valor contido em `b` seja colocado em `a`, e vice-versa.

Exemplo 4.23 Expressões de atribuição múltipla

```
>>> a, b, c = 10, 15, 20 # atribuição múltipla
```

```
>>> a
```

```
10
```

```
>>> b
```

```
15
```

```
>>> c
```

```
20
```

```
>>> d, e = a*2, b*3 # do lado esquerdo são permitidas
```

```
>>> d # expressões aritméticas quaisquer
```

```
20 # segundo as regras válidas para
```

```
>>> e # esse tipo de expressão
```

```
45
```

```
>>>
```

```
>>> x = 5 # x contém o valor 5
```

```
>>> x, y = 10, x*3 # neste caso x receberá 10 e
```

```
>>> x # y receberá 5 multiplicado por 3
```

```
10 # ou seja, é usado o valor anterior
```

```
>>> y # de x, e não o novo valor
```

```
15
```

```
>>> L, m = [3, 6, 9], 14 # L recebe uma lista e m recebe o
```

```
>>> L # número inteiro 14
```

```
[3, 6, 9]
```

```
>>> m
```

```
14
```

```
>>> a, b, c = 2, 4, 6, 8 # gera erro, quantidades são diferentes
```

Traceback (most recent call last):

File "<pyshell#238>", line 1, in <module>

```
a, b, c = 2, 4, 6, 8
```

ValueError: too many values to unpack (expected 3)

```
>>> a, b = 17, -9 # a recebe 17 e b recebe -9
```

```
>>> a, b = b, a # inverte os valores de a e b
```

```
>>> a
```

```
-9
```

```
>>> b
```

```
17
```

4.3.3 Tuplas são imutáveis

No entanto, ao se tentar alterar um objeto escrevendo uma expressão do tipo `T[0] = 29`, será gerado um erro.

```
>>> T[0] = 29 # erro ao tentar alterar o elemento
```

Traceback (most recent call last):

File "<pyshell#148>", line 1, in <module>

```
T[0] = 29
```

TypeError: 'tuple' object does not support item assignment

Dada essa característica de imutabilidade, muitos programadores que estão aprendendo Python questionam sua utilidade. Na realidade, elas não são tão utilizadas como as listas, porém, sua característica de imutabilidade tem sua importância. É muito comum nos programas que dados sejam passados de um módulo para outro. Se isso for feito com listas, dado que são mutáveis, elas podem acabar sendo alteradas em algum ponto. Ao fazer essas mesmas transferências utilizando tuplas, sua imutabilidade garante uma integridade e consequente consistência. Essa garantia de integridade é bastante conveniente nos programas em geral e, em particular, naqueles que são muito grandes e

contém muitos módulos.

4.3.4 Listas aninhadas em tuplas

Por serem capazes de conter objetos de qualquer outro tipo, as tuplas podem contar com uma ou mais listas aninhadas. Quando isso ocorre, embora a tupla seja imutável, a lista aninhada continua sendo mutável. O Exemplo 4.24 ilustra esse aspecto. Nele foi definida a tupla P, cujo terceiro elemento – P[2] – é uma lista. Pode-se alterar os elementos da lista aninhada, porém, não é possível remover a lista de dentro da tupla.

Exemplo 4.24 Listas aninhadas em Tuplas

```
>>> P = (14, 26, [0, 0, 0], 31)
```

```
>>> P[2]
```

```
[0, 0, 0]
```

```
>>> type(P[2]) # o terceiro elemento de P é uma lista
```

```
<class 'list'>
```

```
>>> P[2][1] = 39 # então é possível alterar um elemento
```

```
>>> P # da lista P[2]
```

```
(14, 26, [0, 39, 0], 31)
```

```
>>> P[2].append(16) # é possível aumentar a lista P[2]
```

```
>>> P
```

```
(14, 26, [0, 39, 0, 16], 31)
```

```
>>> P[2].remove(0) # é possível diminuí-la
```

```
>>> P
```

```
(14, 26, [39, 0, 16], 31)
```

```
>>> P[2].clear() # e até mesmo limpá-la
```

```
>>> P
```

```
(14, 26, [], 31)
```

```
# porém não é possível alterar o objeto
```

```
>>> P[2] = 'outra coisa' # P[2] da tupla P para outro tipo
```

```
Traceback (most recent call last):
```

```
File "<pyshell#198>", line 1, in <module>
```

```
P[2] = 'outra coisa'
```

```
TypeError: 'tuple' object does not support item assignment
```

4.3.5 Tuplas como registros (records) que contêm dados

Outro uso para as tuplas é tratá-las como um repositório de dados inter-relacionados. Neste livro, será empregado o termo “registro” para fazer referência a esse tipo de construção.

Por hipótese, imagine-se um conjunto de dados inter-relacionado, ou seja, imagine-se um registro que contenha o código de um produto, seu nome, a quantidade em estoque e o preço unitário de compra. Esse conjunto pode ser representado pela tupla exibida na primeira linha do Exemplo 4.25. Esse

conjunto pode ser utilizado de muitas maneiras: pode ser passado para uma função, pode retornar de uma função, pode ser incluído em uma lista, pode ser gravado em ou lido de um arquivo etc. São muitas as possibilidades. Vejam as linhas seguintes do exemplo, em que são montados mais dois conjuntos, e todos são inseridos em uma lista L. Ao fazer isso, a lista está se tornando um banco de dados de produtos. E fica claro que, assim como é possível inserir esses registros em uma lista, também é possível usar outra tupla no lugar de tal lista, caso se queira.

Exemplo 4.25 Uso de tupla como registro

```
>>> P = (12336, 'Sabão', 1337, 1.37)
```

```
>>> L = []
```

```
>>> L.append(P)
```

```
>>> P = (13446, 'Arroz 1kg', 3554, 2.65)
```

```
>>> L.append(P)
```

```
>>> P = (13956, 'Fubá 500g', 439, 1.19)
```

```
>>> L.append(P)
```

```
>>> L
```

```
[(12336, 'Sabão', 1337, 1.37), (13446, 'Arroz 1kg', 3554, 2.65), (13956, 'Fubá 500g', 439, 1.19)]
```

Uma vez construído esse conjunto de dados contido na lista, podem-se recuperá-los e usá-los de várias maneiras. O Exemplo 4.26 mostra uma

possibilidade dentre muitas.

Exemplo 4.26 Uso de tupla como registro

```
>>> L # seja a lista L carregada no Exemplo 4.25
```

```
[(12336, 'Sabão', 1337, 1.37), (13446, 'Arroz 1kg', 3554, 2.65), (13956,  
'Fubá 500g', 439, 1.19)]
```

```
# deseja-se recuperar os dados do segundo elemento de L (L[1])
```

```
# fazendo que variáveis separadas recebam os valores contidos
```

```
# na tupla. Então, tem-se:
```

```
>>>Codigo, Nome, Qtde, PcUnit = L[1]
```

```
>>>Codigo
```

```
13446
```

```
>>>Nome
```

```
'Arroz 1kg'
```

```
>>>Qtde
```

```
3554
```

```
>>>PcUnit
```

```
2.65
```

4.4 O tipo range

Muitos textos sobre a linguagem Python fazem referência a *range* como uma função, quando, na verdade, segundo a documentação oficial – Seção 4.6.6 do Capítulo 4 da *The Python Standard Library* – trata-se de um tipo sequencial imutável. Por esse motivo, neste livro será utilizada a expressão **tipo range** em vez de **função range**.

Basicamente, o tipo *range* produz uma sequência imutável contendo números inteiros, sendo comumente empregada em laços implementados com o comando `for`, que será visto a seguir. Pode-se afirmar que ele é um gerador de números inteiros que seguem uma regra de progressão aritmética definida pelos parâmetros utilizados. A sintaxe para uso desse tipo tem duas opções, em que a diferença são os parâmetros necessários.

1. *class range*(stop)

2. *class range*(start, stop, [step])

Na opção 1 é fornecido apenas o limite final da progressão aritmética, assumindo-se que o valor inicial é 0 e o incremento é 1.

Na opção 2 são fornecidos o valor inicial e o limite final. Opcionalmente, pode ser fornecido também um terceiro parâmetro: o passo.

Em ambos os casos, os parâmetros devem, obrigatoriamente, ser números inteiros. Quanto a terminologia utilizada, há uma diferença entre os termos valor e limite empregados nas definições anteriores. O valor inicial sempre estará incluído na sequência gerada, ao passo que o limite final nunca estará incluído.

No Exemplo 4.27 são mostrados vários casos de uso do *range*. Ao utilizá-lo no IDLE, é preciso converter seu retorno para uma lista utilizando a sintaxe: `list(range( ... ))`

Exemplo 4.27 Uso do tipo *range*

```
>>> list(range(10)) # forma do caso 1
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> list(range(5, 15)) # forma do caso 2, omitindo o passo
```

```
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
>>> list(range(-3, 4)) # os parâmetros podem ser negativos
```

```
[-3, -2, -1, 0, 1, 2, 3]
```

```
>>> list(range(5, 15, 3)) # forma do caso 2, com os 3 parâmetros
```

```
[5, 8, 11, 14]
```

```
>>> list(range(10, 0, -1)) # o passo pode ser negativo
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
>>> list(range(6, -1, -2))
```

```
[6, 4, 2, 0]
```

```
>>> list(range(10, 3, 1)) # dados inconsistentes geram uma
```

```
[] # sequência vazia
```

4.5 O comando for

Na introdução deste capítulo, foi dada a definição do conceito de objeto iterável. Strings, listas e tuplas são iteráveis, e agora é necessário mostrar como fazer uso deste conceito ao mesmo tempo simples e poderoso.

No Capítulo 3 foi apresentado o comando de laço while, e até aqui muitos programas exemplo foram escritos utilizando-o. Agora, será apresentado o comando for, que é uma segunda opção disponível em Python para a construção de laços de repetição.

Esse comando vale-se fortemente do conceito de iterável, pois é utilizado para a iteração sobre os tipos sequenciais. O Exemplo 4.27 mostra o uso do comando for iterando com a lista L. Os comandos subordinados ao for serão repetidos cinco vezes, pois a lista L contém cinco elementos. A cada repetição, o objeto x recebe um valor dentre os contidos em L, segundo a sequência da lista. Desse modo, na primeira vez x recebe 2, na segunda vez recebe 4, e assim sucessivamente, até o término do laço.

Isso é um laço iterador, muito utilizado na programação com Python. No lugar da lista L seria possível utilizar uma tupla, um string ou um conjunto (que ainda será visto).

Exemplo 4.28 Uso do comando for

```
print("Inicio do Programa")

L = [2, 4, 6, 8, 10]

i = 0

for x in L:

    print("Elemento {0} = {1}".format(i, x))

    x = 0 # foi colocado 0 em x, mas isso não afeta o laço

    i+=1
```

```
print("Fim do Programa")
```

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the following output:

```
Início do Programa  
Elemento 0 = 2  
Elemento 1 = 4  
Elemento 2 = 6  
Elemento 3 = 8  
Elemento 4 = 10  
Fim do Programa  
>>>
```

The status bar at the bottom right shows 'Ln: 12 Col: 4'.

Suponha-se, agora, que por algum motivo o valor do objeto `x` tenha sido alterado durante a iteração. Por exemplo, como mostrado no Exemplo 4.28, após o `print` foi feito `x = 0`. Isso não afeta nem o laço nem a lista `L`. O valor de `x` pode, sim, ser alterado dessa maneira, porém, na próxima iteração `x` receberá o próximo valor contido em `L`.

4.5.1 Formato geral do comando `for`

Os laços em Python iniciam uma linha de cabeçalho na qual se especifica um objeto iterador (`objctrl`) que receberá, um a um, os valores contidos em um objeto de tipo sequencial (`objseq`), que será denominado **sequência iterável**. Para cada valor atribuído ao objeto de controle, os comandos contidos no <bloco 1> serão executados. Desse modo, o laço será executado um certo número de vezes, que depende exclusivamente da quantidade de elementos contidos em `objseq`.

Assim como o comando `while`, o `for` também suporta a opcional cláusula `else`, que funciona exatamente da mesma maneira, como já foi visto para o comando `while`. Se o laço terminar normalmente, sem que uma instrução `break` seja executada, então, o <bloco 2> de comandos será executado.

```
for objctrl in objseq:
```

```
<bloco 1>
```

```
else:
```

<bloco 2>

Os comandos `break` e `continue` podem ser utilizados aqui do mesmo modo como visto para o `while`: `break` que termina o laço imediatamente e `continue` que termina a iteração atual e segue para a próxima.

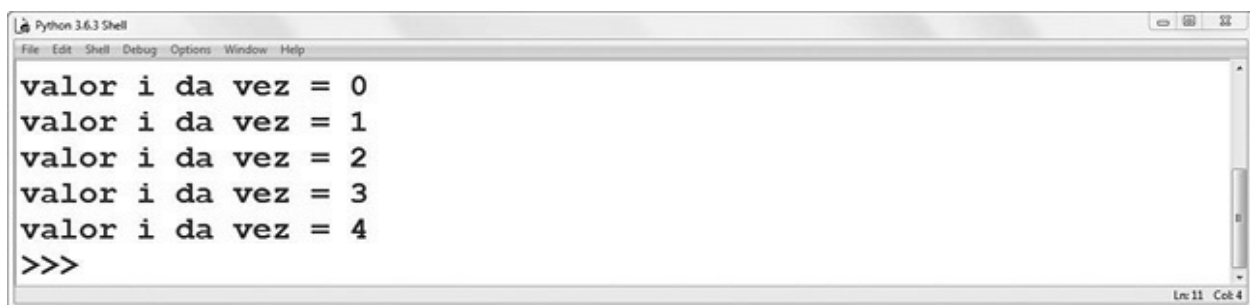
Com relação aos motivos de existir a cláusula `else` em um comando de laço, valem exatamente as mesmas considerações feitas quando foi visto o comando `while`.

4.5.2 Exemplos de uso

A seguir serão vistos diversos exemplos de uso do comando *for*. O primeiro é o Exemplo 4.29, no qual o tipo *range* é utilizado para gerar a sequência iterável. Neste caso, o objeto iterador recebe os valores de 0 a 4 gerados por `range(5)`.

Exemplo 4.29 Uso do comando `for` em conjunto com *range*

```
for i in range(5):  
  
    print("valor i da vez = {}".format(i))
```



```
Python 3.6.3 Shell  
File Edit Shell Debug Options Window Help  
valor i da vez = 0  
valor i da vez = 1  
valor i da vez = 2  
valor i da vez = 3  
valor i da vez = 4  
>>>
```

O Exemplo 4.30 mostra a execução de um `for` que utiliza um string como sequência iterável. Para cada caractere contido no string é feita uma saída em tela, seguida de um caractere hífen “-”

Exemplo 4.30 Comando `for` em conjunto com string

```
S = 'Programe em Python'
```

```
for x in S:
```

```
    print(x, end='-')
```

```
# Este código produz a saída
```

```
P-r-o-g-r-a-m-e- -e-m- -P-y-t-h-o-n-
```

O uso do comando `for` com tuplas seria semelhante ao já visto com listas, *range* e string. Porém, o uso de tuplas ou listas contendo outras tuplas abre possibilidades mais amplas. O Exemplo 4.31 contém uma tupla *T* constituída de três elementos que também são tuplas, cada uma com dois números. O comando `for` foi construído de modo que o iterador (`objctrl`) é uma tupla de objetos (*a*, *b*). Isso faz com que, a cada tupla contida em *T*, o objeto *a* receba um valor e *b* receba o outro.

Exemplo 4.31 Comando *for* em conjunto com uma tupla de tuplas

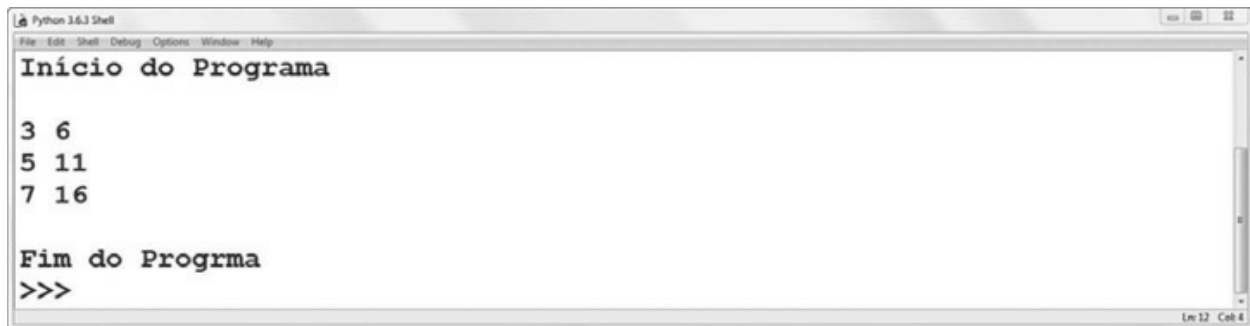
```
print("Início do Programa\n")
```

```
T = ((3, 6), (5, 11), (7, 16))
```

```
for (a, b) in T:
```

```
    print(a, b)
```

```
print("\nFim do Progrma")
```

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa

3 6
5 11
7 16

Fim do Programa
>>>
Ln 12 Col 4
```

O Exemplo 4.32 mostra um caso semelhante ao 4.31, agora, ilustrando-o com a lista de produtos gerada no Exemplo 4.25, visto anteriormente. É possível notar o poder dessas iterações, uma vez que a lista constituída de tuplas fornece, a cada vez, um conjunto completo de dados sobre um produto. Dentro do bloco de comandos do laço é possível efetuar qualquer operação que seja necessária com esses dados. No caso, foi feita uma simples exibição em tela e foi calculado o valor total gasto com cada produto em estoque.

Os dados foram carregados de modo literal nas primeiras linhas desse programa. Essas linhas podem ser substituídas por uma leitura de arquivo em disco ou acesso a um banco de dados, que forneceriam os dados para o algoritmo.

Exemplo 4.32 Comando *for* em conjunto com uma tupla listas de tuplas

```
L = []
```

```
P = (12336, 'Sabão', 1337, 1.37)
```

```
L.append(P)
```

```
P = (13446, 'Arroz 1kg', 3554, 2.65)
```

```
L.append(P)
```

```
P = (13956, 'Fubá 500g', 439, 1.19)
```

```
L.append(P)
```

```
print("Lista de Produtos em Estoque\n")
```

```
for (Cod, Nome, Qtde, PcUnit) in L:
```

```
    print("Identificação do Produto:", Cod)
```

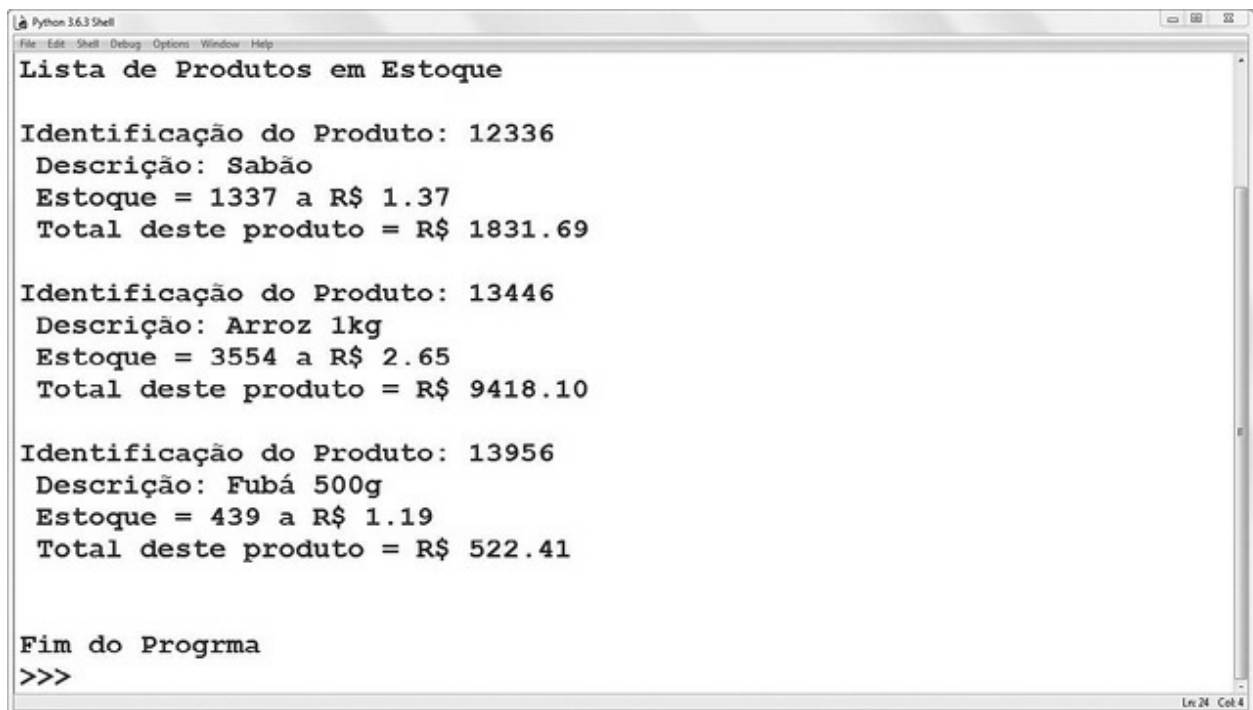
```
    print(" Descrição:", Nome)
```

```
    print(" Estoque = {0} a R$ {1:.2f}".format(Qtde, PcUnit))
```

```
    print(" Total deste produto = R$ {0:.2f}".format(Qtde*PcUnit))
```

```
    print()
```

```
print("\nFim do Progrma")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Lista de Produtos em Estoque

Identificação do Produto: 12336
Descrição: Sabão
Estoque = 1337 a R$ 1.37
Total deste produto = R$ 1831.69

Identificação do Produto: 13446
Descrição: Arroz 1kg
Estoque = 3554 a R$ 2.65
Total deste produto = R$ 9418.10

Identificação do Produto: 13956
Descrição: Fubá 500g
Estoque = 439 a R$ 1.19
Total deste produto = R$ 522.41

Fim do Progrma
>>>
```

4.5.3 Considerações finais sobre o comando for

As várias formas de uso observadas do comando `for`, não esgotam todas as possibilidades existentes, embora tenham sido abordadas as principais e mais frequentes.

A implementação desse comando é especialmente otimizada para efetuar as iterações utilizando o iterador e a sequência iterável. Como consequência direta, tem-se que os laços `for` rodam mais rápido que laços construídos com o comando `while`. Muitas fontes de consulta disponíveis na bibliografia, por exemplo, Lutz (2009), recomendam que o programador privilegie o uso do comando `for` em detrimento do `while`, sempre que isso for possível.

Além disso, o código escrito com o comando `for` resulta em ser mais simples, de mais fácil leitura e menos sujeito a erros do programador, quando comparado ao código escrito utilizando-se `while`. Por exemplo, não é necessário que o programador tenha de controlar contadores de laço nem com a inicialização dos objetos com os quais tal contagem será feita. Na prática, esses contadores nem estarão no código se a escolha recair sobre o comando `for`.

Quando a sequência iterável for uma lista, o programador deve lembrar-se de que a lista é um objeto mutável e tomar cuidado com o que ocorre com ela durante as repetições do laço. Não é nada recomendável que a lista seja alterada durante o laço. Se isso ocorrer, podem ser verificados resultados incoerentes no processamento das repetições. Basta pensar que o laço está construído tendo por base uma lista que é alterada a cada iteração, o que leva a uma condição imprevisível e instável. Essa situação é indesejada e deve ser evitada sempre.

Essas considerações dizem respeito exclusivamente a Python. Em outras linguagens, os comandos de laço têm outras formas de implementação, de modo que cada caso é diferente do outro e precisa ser devidamente estudado pelo programador para que este descubra quais opções trazem as maiores

vantagens.

Exercícios resolvidos

A seguir serão trabalhados problemas com dois propósitos: implementar e explicar algoritmos variados e fixar os conceitos da linguagem Python. O primeiro propósito é endereçado ao leitor iniciante em programação e que deseja compreender como os algoritmos são construídos, ao passo que o segundo propósito busca atender o leitor que já conhece programação e deseja conhecer Python.

7. Programa que gera uma lista com a sequência de Fibonacci

A sequência de Fibonacci já foi explicada no Exercício resolvido 3.7. Escreva um programa que gere os N primeiros termos dessa sequência utilizando uma lista para armazená-los. N é um número inteiro a ser lido do teclado e deve, obrigatoriamente, ser maior ou igual a 2.

Exercício resolvido 4.7 – Sequência de Fibonacci utilizando lista

```
print("Sequência de Fibonacci\n")
```

```
# leitura do número de termos
```

```
N = 0
```

```
while N < 2:
```

```
try:
```

```
N = int(input("Digite N(>1): "))
```

```
if N < 2:
```

```
print("Digite N >= 2")

except:

print("O dado digitado deve ser um número inteiro.")

# criação da lista com a sequência de Fibonacci

L = [0, 1] # L inicializada com [0, 1]

for i in range(N-2): # range produz a seq 0, 1, 2, 3...

L.append(L[i] + L[i+1])

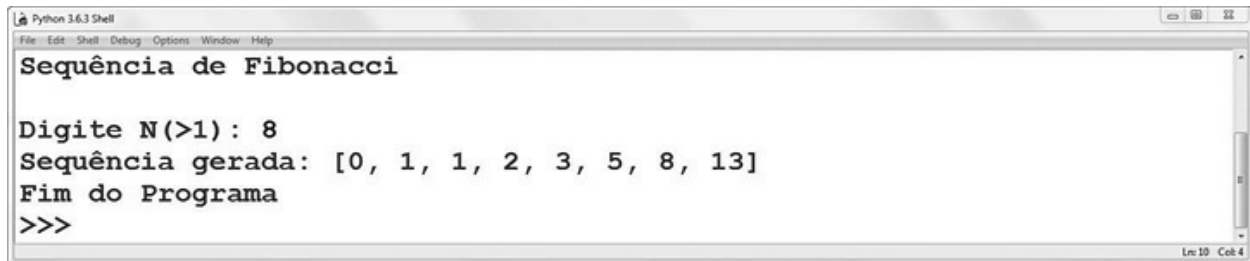
print("Sequência gerada:", L) # Exibe a lista

print("Fim do Programa")
```

Essa solução tem como principal objetivo exemplificar os recursos de Python e tem duas partes: a leitura do dado de entrada N e a montagem e exibição da lista com os elementos da sequência. A leitura de N já foi explicada na solução do Exercício resolvido 3.8.

A segunda parte dessa solução utiliza recursos típicos de Python que permitem que seja escrita em apenas três linhas. Dado que o valor de N é no mínimo 2, a lista L é inicializada com os dois primeiros termos: 0 e 1. Em seguida, o comando for, combinado com o tipo range(N-2), permite a criação de um laço que será responsável por incluir na lista mais N-2 elementos. Se N for 8, então, N-2 será 6 e o tipo *range* gerará a sequência (0, 1, 2, 3, 4, 5). Cada um desses valores será assumido pelo iterador i. Quando i for 0, a expressão aritmética $L[i] + L[i+1]$ corresponderá a $L[0] + L[1]$, que é a soma dos dois primeiros elementos

já presentes na lista. O resultado dessa soma é adicionado à lista L, que passará a ter três elementos. Em seguida, i passa a ser 1 e o próximo valor adicionado à lista é dado por $L[1] + L[2]$, e assim por diante, até o último valor de i, que será 5.

A screenshot of a Python 3.6.3 Shell window. The window has a title bar that says "Python 3.6.3 Shell" and a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area contains the following text:

```
Sequência de Fibonacci  
  
Digite N(>1): 8  
Sequência gerada: [0, 1, 1, 2, 3, 5, 8, 13]  
Fim do Programa  
>>>
```

The status bar at the bottom right shows "Linha 10 Col 4".

8. Programa que gera uma lista em ordem crescente

Escreva um programa que permaneça em laço lendo números inteiros enquanto os valores digitados forem diferentes de zero. Para cada valor digitado, adicione-o a uma lista na posição imediatamente anterior ao primeiro elemento da lista que seja maior ou igual a ele. Exiba a lista no final.

Exercício resolvido 4.8 – Gerador de lista em ordem crescente

```
print("Gera lista em ordem crescente\n")
```

```
L = []
```

```
x = int(input("Digite um valor: "))# lê o primeiro x
```

```
while x != 0: # entra no laço se x != 0
```

```
p = 0
```

```
while p < len(L) and L[p] < x: # laço de pesquisa da posição 'p'
```

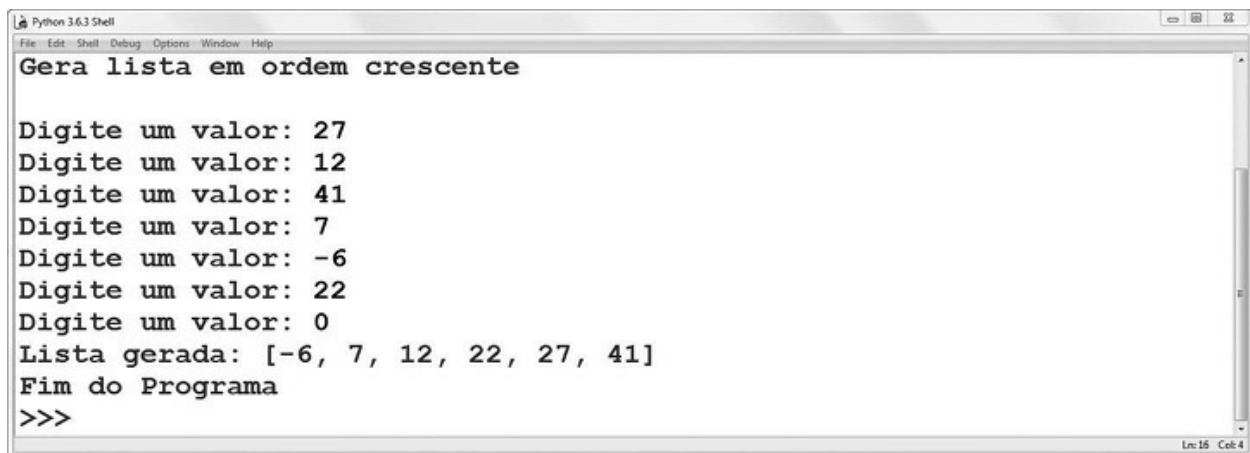
```
p+=1; # de inserção de x em L.
```

```
L.insert(p, x)
```

```
x = int(input("Digite um valor: "))
```

```
print("Lista gerada:", L) # Exibe a lista
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Gera lista em ordem crescente

Digite um valor: 27
Digite um valor: 12
Digite um valor: 41
Digite um valor: 7
Digite um valor: -6
Digite um valor: 22
Digite um valor: 0
Lista gerada: [-6, 7, 12, 22, 27, 41]
Fim do Programa
>>>
```

O objetivo principal dessa solução é exemplificar a lógica do algoritmo adotado. Observe o resultado de sua execução, em que os valores foram digitados em qualquer ordem, e ao final a lista está ordenada. Isso ocorre nesse algoritmo porque a inclusão de x na lista é feita com o uso do método `insert`, em uma posição dada por “ p ” que foi pesquisada antes.

O laço de pesquisa é o elemento-chave da lógica desse algoritmo. Ele é construído com duas condições unidas pelo operador lógico `and`, e para permanecer em laço é necessário que ambas sejam verdadeiras: p menor que o tamanho da lista e o elemento $L[p]$ menor que x . Ela se tornará falsa quando o tamanho da lista for alcançado ou for encontrado o primeiro elemento da lista que seja maior ou igual a p . Em ambos os

casos, o conteúdo do objeto `p` indica a posição de `L` em que `x` deve ser inserido.

9. Busca sequencial de um valor em uma lista

Escreva um programa que leia um número inteiro `N` e gere uma lista com números pares de 2 até `N`. Se `N` for par, deve estar incluído na lista. Em seguida, inicie um laço que deve permanecer em execução enquanto `x` for diferente de zero. Para cada valor de `x` fornecido, o programa deve informar se `x` está ou não na lista.

A solução desse problema tem duas partes: a geração da lista e a pesquisa de `x`.

A primeira parte tem esta solução: `L = list(range(2, N+1, 2))`

Isso mesmo! Uma única linha de código é capaz de criar a lista pedida, pois se vale dos recursos disponíveis no tipo *range*.

A segunda parte será resolvida de duas maneiras:

a) Privilegiando o uso dos recursos de Python:

Exercício resolvido 4.9 – Busca de valor em uma lista (versão A)

```
print("Pesquisa sequencial\n")
```

```
N = int(input("Digite N: "))
```

```
L = list(range(2, N+1, 2))
```

```
print("Lista gerada:", L)
```



```
x = int(input("Digite x: "))

while x != 0:

    if x in L: # operador in de Python resolve o problema

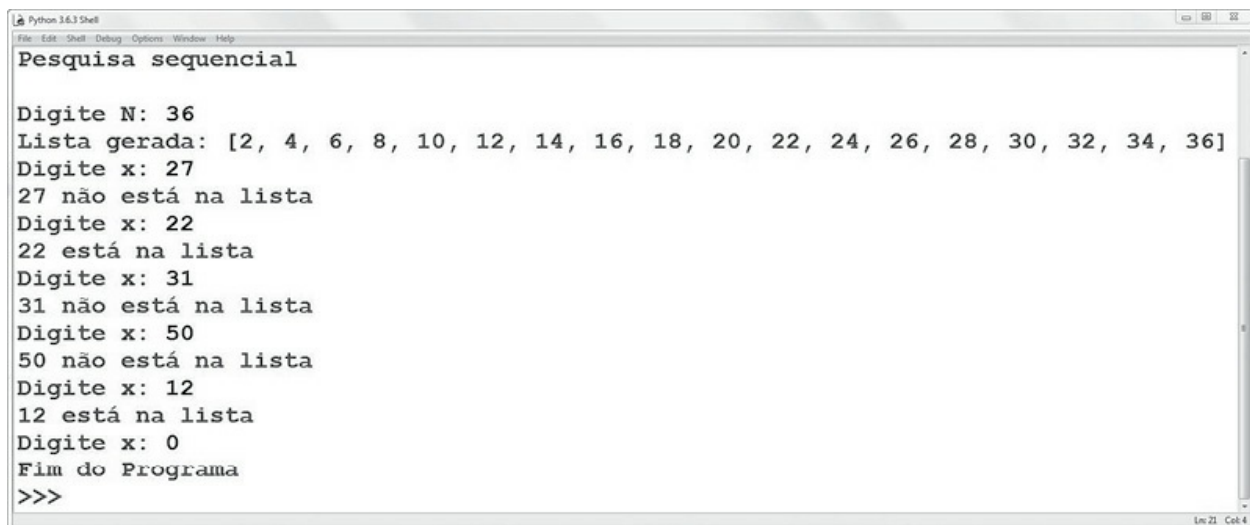
        print("{0} está na lista".format(x))

    else:

        print("{0} não está na lista".format(x))

    x = int(input("Digite x: "))

print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Pesquisa sequencial

Digite N: 36
Lista gerada: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36]
Digite x: 27
27 não está na lista
Digite x: 22
22 está na lista
Digite x: 31
31 não está na lista
Digite x: 50
50 não está na lista
Digite x: 12
12 está na lista
Digite x: 0
Fim do Programa
(>>>)
```

b) Privilegiando a lógica do algoritmo construída apenas com comandos básicos, disponíveis em qualquer linguagem.

Nessa segunda solução, o objetivo é mostrar a implementação do algoritmo de busca sequencial. Com ele, sempre é possível determinar se um valor está ou não contido em uma lista de valores. Além disso,

para aplicá-lo não há nenhum requisito prévio quanto à ordem da lista.

O algoritmo consiste em um laço controlado por meio de um contador “i”. No laço percorre-se a lista e, em cada repetição, verifica-se se o valor procurado é o elemento atual. Caso ocorra a igualdade $L[i] == x$, o laço termina com i menor que o tamanho da lista. Isso indica que o valor foi encontrado.

Observando o código seguinte, versão B do Exercício resolvido 4.9, parece que não há muita diferença em relação à solução proposta no Exercício resolvido 4.9 versão A, porém, há um laço a mais. Na prática, aqui foi feito um laço que substitui o uso do operador “in” utilizado na versão A.

Exercício resolvido 4.9 – Busca sequencial de valor em uma lista (versão B)

```
print("Pesquisa sequencial\n")
```

```
N = int(input("Digite N: "))
```

```
L = list(range(2, N+1, 2))
```

```
print("Lista gerada:", L)
```

```
x = int(input("Digite x: "))
```

```
while x != 0:
```

```
    i = 0
    while i < len(L) and L[i] != x:
        i += 1
    if i < len(L):
```

} Esta é a parte diferente.

```
print("{0} está na lista".format(x))

else:

print("{0} não está na lista".format(x))

x = int(input("Digite x: "))

print("Fim do Programa")
```

O resultado da execução das duas versões, A e B, do Exercício resolvido 4.9 é exatamente a mesma.

10. Busca binária de um valor em uma lista

Reescreva o programa do exercício anterior, trocando o algoritmo de busca sequencial pelo algoritmo de busca binária.

O algoritmo de busca binária é significativamente mais rápido que o de busca sequencial quando aplicado a grandes conjuntos de dados. Porém, ele requer que a lista esteja ordenada. A ideia básica implementada nesse algoritmo é verificar se o valor procurado “x” está na posição central da lista. Se estiver, então, o valor foi encontrado e o algoritmo termina. Caso não esteja e x seja menor que o valor central, então, a busca prossegue na metade à esquerda do centro; caso seja maior, a busca prossegue na metade à direita.

A solução a seguir mostra a implementação dessa ideia.

Exercício resolvido 4.10 – Algoritmo de busca binária

```
print("Pesquisa sequencial\n")
```

```
N = int(input("Digite N: "))

L = list(range(2, N+1, 2))

print("Lista gerada:", L)

x = int(input("Digite x: "))

while x != 0:

    ini = 0 # índice do primeiro elemento

    fim = len(L)-1 # índice do último elemento

    meio = (ini+fim) // 2 # calcula o índice do meio

    while ini <= fim: # laço de pesquisa

        if x == L[meio]: # se for igual achou

            print("{0} está na lista".format(x)) # exibe e

            break # termina o laço

        if x < L[meio]: # se x for menor

            fim = meio - 1 # atualiza o índice fim para meio-1

        else: # se x for maior

            ini = meio + 1 # atualiza o índice ini para meio+1

        meio = (ini+fim) // 2 # calcula o novo índice do meio
```

else:

```
print("{0} não está na lista".format(x))
```

```
x = int(input("Digite x: "))
```

```
print("Fim do Programa")
```

Nessa solução, a lista é dividida na metade a cada execução do laço e a busca vai ficando cada vez mais restrita a um conjunto menor de valores. A alteração dos objetos “ini” ou “fim” a cada repetição é a responsável por isso. E a cada alteração de um desses objetos é necessário calcular o novo “meio”. O algoritmo termina ou porque x foi encontrado ou quando “ini” passa a ser maior que fim, e isso significa que x não foi encontrado. Para melhor compreensão dessa lógica, verifique a Figura 5.2, no Capítulo 5. Nele, esse mesmo algoritmo é implementado utilizando-se uma função recursiva (Exercício resolvido 5.4).

11. Programa que ordena uma lista não ordenada

Escreva um programa que ordene uma lista não ordenada utilizando o algoritmo Bubble Sort.

Em Python, para ordenar uma lista de qualquer tamanho, basta utilizar o método `sort` ou a função `sorted`, que estão prontos e já foram mencionados neste capítulo. Portanto, o objetivo deste exercício é mostrar e explicar a lógica do algoritmo para o leitor que está iniciando seus estudos de programação.

Nesse algoritmo, a ideia é percorrer a lista comparando dois elementos

vizinhos e verificando se o elemento à esquerda é maior que o da direita. Caso seja, eles devem ser trocados de posição. Considere-se o exemplo a seguir:

ao comparar 17 com 4 será feita a troca, e fica assim: 4, 17, 23, 8, 19, 12

ao comparar 17 com 23 nada muda

ao comparar 23 com 8 será feita a troca, e fica assim: 4, 17, 8, 23, 19, 12

ao comparar 23 com 19 será feita a troca, e fica assim: 4, 17, 8, 19, 23, 12

ao comparar 23 com 12 será feita a troca, e fica assim: 4, 17, 8, 19, 12, 23

Ao final da primeira passagem pela lista completa, seus elementos ficam da forma como acabamos de ver. O maior de todos já está em seu lugar correto, porém, os demais não necessariamente.

Todo esse processo deve, então, ser repetido tantas vezes quando necessário, até que a lista esteja em ordem. Essa condição é detectada pelo algoritmo quando nenhuma troca tenha sido feita.

17 4 23 8 19 12 Ponto de partida

4 17 8 19 12 23 Ao final da 1ª passada

4 8 17 12 19 23 Ao final da 2ª passada

4 8 12 17 19 23 Ao final da 3ª passada

Exercício resolvido 4.11 – Algoritmo de ordenação Bubble Sort

```
print("Ordenação Bolha\n")
```

```
L = [17, 4, 23, 8, 19, 12] # poderia ter sido gerada uma lista
```

```
print("Lista gerada:", L) # com números aleatórios com randint()
```

```
Trocou = 1 # flag que indica se houve troca ou não

while Trocou: # enquanto trocou é diferente de zero

    Trocou = 0 # faça Trocou = 0

    i = 0

    while i < len(L)-1: # laço que percorre a lista fazendo

        if L[i] > L[i+1]: # as trocas quando necessário

            L[i], L[i+1] = L[i+1], L[i] # faz a troca

            Trocou = 1 # se houve troca então Trocou = 1

        i+=1

    print(" estado parcial de L:", L)

    print("\nSituação final")

    print("Lista ordenada:", L)

    print("Fim do Programa")
```

O objeto de controle do laço interno “i” deve ter seu primeiro valor igual a zero e o último igual ao tamanho de L – 2, para que não ocorra erro de indexação na referência L[i + 1].

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Ordenação Bolha

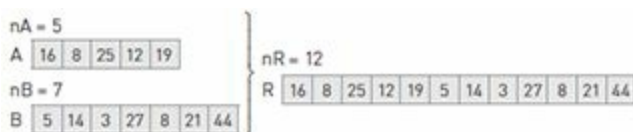
Lista gerada: [17, 4, 23, 8, 19, 12]
estado parcial de L: [4, 17, 8, 19, 12, 23]
estado parcial de L: [4, 8, 17, 12, 19, 23]
estado parcial de L: [4, 8, 12, 17, 19, 23]
estado parcial de L: [4, 8, 12, 17, 19, 23]

Situação final
Lista ordenada: [4, 8, 12, 17, 19, 23]
Fim do Programa
>>>
```

Este é o resultado da execução desse algoritmo.

Exercícios propostos

1. Escreva um programa que leia do teclado uma lista com tamanho de 10 elementos e exiba-a na tela na ordem inversa à ordem de leitura.
2. Escreva um programa que leia do teclado duas listas com tamanho 10, com números inteiros. Em seguida, o programa deve juntar as duas listas em uma única com o tamanho 20.
3. Escreva um programa que preencha com números inteiros duas listas denominadas A e B com diferentes tamanhos n_A e n_B , respectivamente. Em seguida, o programa deve juntar as duas em uma única lista com o tamanho $n_A + n_B$. Exibir na tela a lista resultante. Veja o exemplo:



4. Escreva um programa que leia uma lista com N números inteiros, em que N é um número inteiro previamente digitado pelo usuário. O

programa não deve aceitar um número digitado que já esteja inserido na lista, sendo que, quando essa situação ocorrer, uma mensagem deve ser dada ao usuário. Por fim, exibir na tela a lista resultante.

5. Escreva um programa que leia do teclado dois números inteiros nA e nB e leia também duas listas denominadas A e B com os tamanhos nA e nB , respectivamente. Na leitura de cada uma das listas é obrigatório que não sejam aceitos valores repetidos. Em seguida, o programa deve juntar as duas em uma única lista R (resultante), tomando o cuidado de que R não tenha valores duplicados. Veja o exemplo:



6. Escreva um programa que leia três dados de entrada: o primeiro termo, a razão e a quantidade de termos de uma P.A., todos números inteiros. O programa deve calcular todos os termos, colocando-os em uma lista, e exibi-la no final. Esse exercício já foi resolvido e explicado no Capítulo 3 (veja Exercício resolvido 3.2). A diferença, aqui, é que se pede para utilizar uma lista para armazenar os diversos termos antes de exibi-los.
7. Escreva um programa que leia um número N obrigatoriamente entre 0 e 50 e, em seguida, leia N números reais em uma lista A. O programa deve separar os valores lidos em A em outras duas listas NEG e POS: a primeira contendo somente os valores negativos e a segunda contendo os valores positivos e zero. Apresentar na tela as listas NEG e POS e a quantidade de valores contidos em cada uma.

8. Escreva um programa que leia um número N (entre 0 e 50) e, em seguida, defina uma lista V preenchendo-a com N números inteiros aleatórios (utilizar a função *randint*). Exiba-a na tela. Inicie um laço no qual será feita a leitura de um número X e que termina quando X for zero. Pesquise se X está ou não na lista V e, caso esteja, elimine todas as suas ocorrências.
9. O programa deverá ler dois inteiros chamados Min e Max. Min pode ser qualquer valor e Max, obrigatoriamente, deve ser maior que Min. Em seguida, preencher uma lista com todos os valores divisíveis por 7 contidos no intervalo fechado [Min, Max]. Exibir a lista resultante na tela.
10. Escreva um programa que leia do teclado uma lista com N elementos. Em seguida, o programa deve eliminar os elementos que estiverem repetidos, mantendo apenas a primeira ocorrência de cada. Apresentar a lista resultante na tela. Os valores eliminados devem ser armazenados em outra lista que também deve ser exibida.
11. Faça um programa que leia um número inteiro N bem grande (acima de 5.000). Preencha uma lista de tamanho N com números inteiros aleatórios positivos. Em seguida, inicie um laço de pesquisa, no qual o valor a ser pesquisado deve ser lido do teclado, e o programa deve dizer se tal valor está ou não contido na lista, bem como dizer sua posição. No caso de várias ocorrências, exibir todas. O laço de pesquisa termina quando for digitado o zero. Use o algoritmo de busca sequencial.
12. Escreva um programa que leia do teclado duas matrizes de dimensões 2×2 e mostre na tela a soma dessas duas matrizes.

13. Escreva um programa que leia do teclado duas matrizes de dimensões 2×2 e mostre na tela a multiplicação dessas duas matrizes.

14. A matriz a seguir mostra o custo unitário de cada produto e a quantidade de cada um dos produtos no estoque de três lojas de uma rede. Escreva um programa que exiba na tela as respostas para as perguntas. Na solução desse problema, elabore uma maneira de armazenar seus dados utilizando lista e sublistas. Os dados da matriz devem ser lidos do teclado.

	Custo Unitário	Loja 1	Loja 2	Loja 3
Produto A	R\$ 72,35	373	558	358
Produto B	R\$ 43,93	1228	1448	907
Produto C	R\$ 17,84	4135	2059	3122
Produto D	R\$ 23,19	1139	1450	843

a) Qual é o valor total de estoque em cada uma das lojas?

b) Qual é o valor total de estoque para cada produto disponível na rede?

c) Qual é o valor total de estoque da rede?



Funções

Objetivos

Funções, também conhecidas como subprogramas ou subrotinas, são

pequenos blocos de código aos quais se dá um nome, desenvolvidos para resolver tarefas específicas. Tais funções constituem um elemento de fundamental importância na moderna programação de computadores, a ponto de ser possível afirmar que atualmente nenhum programa de computador é desenvolvido sem o uso desse recurso.

Neste capítulo, esse assunto será abordado, apresentando-se os conceitos gerais envolvidos e suas aplicações, que valem para a maioria das linguagens de programação. Também são apresentadas as características e peculiaridades da linguagem Python relativas ao assunto.

5.1 Direto ao ponto

Vamos apresentar o uso de funções em programas escritos em Python. Trata-se de um assunto importante que envolve muitos conceitos, os quais, em um primeiro momento, podem parecer demasiadamente abstratos ao programador iniciante. Assim sendo, a abordagem aqui adotada é a de primeiro mostrar como é feito, para depois aprofundar os conceitos.

Observe com atenção o código a seguir, bem como a Figura 5.1, que contém o resultado de sua execução.

Exemplo 5.1 Criação e uso de funções

```
def Soma(X, Y): # linha 1

    R = X + Y

    return R

a = int(input("Digite um valor para a: "))

b = int(input("Digite um valor para b: ")) # linha 2

c = int(input("Digite um valor para c: "))
```

```
s = Soma(a, b) # linha 3

print("a + b = {0}".format(s))

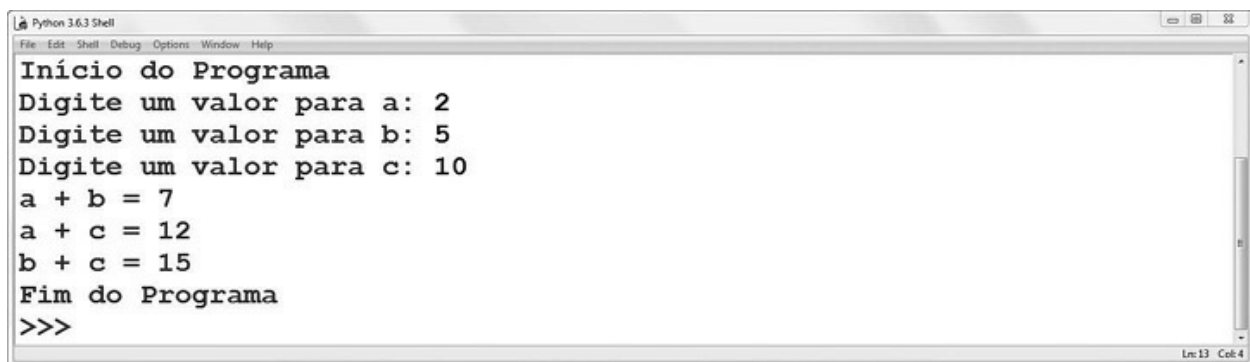
s = Soma(a, c) # linha 4

print("a + c = {0}".format(s))

s = Soma(b, c) # linha 5

print("b + c = {0}".format(s))

print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite um valor para a: 2
Digite um valor para b: 5
Digite um valor para c: 10
a + b = 7
a + c = 12
b + c = 15
Fim do Programa
>>>
Lin: 13 Col: 4
```

Figura 5.1 Exemplo de uso de função.

Nesse código está definida uma função chamada Soma, que calcula e retorna a soma dois valores a ela fornecidos, por meio dos parâmetros X e Y. Note que essa função foi chamada três vezes no programa, nas linhas 3, 4 e 5. Em cada uma das chamadas foram passados diferentes valores, e a função calculou e retornou a soma dos mesmos.

Toda vez que uma função é chamada, o fluxo de execução dos comandos é interrompido no local da chamada e a execução é transferida para os comandos internos da função. Se houver parâmetros – como nesse exemplo há X e Y –, estes são devidamente carregados com os valores passados na

chamada, antes de se executar o primeiro comando interno. Ao término da função, ou seja, após a execução de todos os comandos internos, a execução retorna para a instrução imediatamente seguinte ao ponto em que ocorreu a chamada.

Como fica claro neste primeiro exemplo, existem dois aspectos relevantes referentes ao uso de funções em programação:

1. A **definição** da função, que é o ponto do programa no qual ela é criada.
2. O **uso** da função, que são os pontos onde ela é usada (ou chamada). A chamada de uma função pode ocorrer na parte principal do programa ou dentro de outra função.

No Exemplo 5.1 a função calcula algo, uma soma, que poderia ser feita com uma simples expressão algébrica. A simplicidade desse exemplo pode levar o leitor iniciante nos estudos de programação a questionar qual é a importância de seu uso. Na prática, em uma leitura superficial, fica-se com a impressão de que o programa ficou mais complicado que o necessário. Bem, isso é verdade, ficou mesmo mais complicado. No entanto, este é apenas um exemplo inicial, em seguida serão apresentados conceitos que deverão responder a tal questionamento e compreender que o uso de funções na programação moderna, mais do que importante, é fundamental.

5.2 A importância das funções

Em programação de computadores, o termo “função” tem um significado totalmente diferente daquele empregado em outras áreas do conhecimento, como matemática, biologia ou química.

No contexto de programação, uma função é um conjunto de comandos, ou

bloco de código, ao qual se atribui um nome identificador que executa certa tarefa, e pode produzir e retornar algum resultado.

Um programa pode conter tantas funções quanto se queira, bastando ao programador desenvolvê-las conforme julgue adequado.

Além disso, as funções podem ser desenvolvidas, testadas e agrupadas em bibliotecas de modo a ficar disponíveis para uso em mais de um programa diferente, sempre que o programador necessite delas.

5.2.1 Dividir para conquistar

A expressão “dividir para conquistar” ilustra um aspecto central relativo ao uso de funções que consiste em dividir um problema maior e mais complexo, em partes menores e mais simples. Em seguida, implementar a solução das partes simples a partir da criação de uma função para cada parte e, por fim, montar a solução completa juntando e justapondo as tais funções de modo apropriado.

Essa abordagem tem sido utilizada ao longo de décadas, e em todo este tempo se mostra comprovadamente eficaz. Para o programador iniciante, à primeira vista, pode parecer que o uso de funções é um complicador desnecessário na elaboração de um programa. Deve-se lembrar, no entanto, que os programadores iniciantes estão apenas dando os primeiros passos neste mundo da programação de computadores, e os programas que desenvolvem são pequenos contendo dezenas ou, no máximo, umas poucas centenas de linhas de código.

5.2.2 Reúso de código e eliminação da redundância

Um segundo aspecto fundamental relativo ao uso de funções em programas diz respeito à eliminação da redundância possibilitada pelo reúso de um código pronto.

É comum que um programador necessite executar certa tarefa diversas vezes no mesmo programa. Exatamente como foi feito no Exemplo 5.1, em que a função Soma foi utilizada três vezes. Suponha que essa função tivesse uma centena de linhas de código em vez de apenas duas. Fica claro o ganho em uma situação assim, pois, em vez de repetir um extenso código nos vários pontos do programa, escreve-se uma função que efetua a tarefa e a chama sempre que for preciso.

Nas demais seções deste capítulo serão apresentados e exemplificados todos os aspectos relativos à criação de funções e seu uso.

5.3 Definição e uso de funções

5.3.1 Definição de funções

Em Python, uma função é definida por meio de um cabeçalho que contém quatro elementos: a palavra reservada `def`, um nome válido, parâmetros entre parênteses e o caractere `:`. Esse cabeçalho é sucedido por um bloco de comandos identados que constitui o corpo da função. Todos esses elementos estão presentes no Exemplo 5.1.

O nome pode ser qualquer identificador válido segundo as regras vistas no Item 2.2. Após o nome da função, entre parênteses, são fornecidos os parâmetros que ela receberá, se existirem. Por fim, o caractere `:` indica ao interpretador o término do cabeçalho. Todos os comandos internos à função devem estar identados para que o interpretador reconheça que são comandos subordinados ao cabeçalho e, portanto, pertencentes à função.

5.3.2 Parâmetros de funções

Os parâmetros representam dados de entrada a serem utilizados pela função e são opcionais. No Exemplo 5.1 a função Soma necessita receber como

dados de entrada os valores a serem somados. No caso, foram, então, incluídos dois parâmetros, X e Y. Não existe qualquer limite para a quantidade de parâmetros que uma função pode receber, de modo que o programador é livre para criar a função com tantos parâmetros quanto necessário.

Por outro lado, haverá funções que não necessitam de qualquer valor de entrada. Nesses casos, ainda assim os parênteses devem estar presentes no cabeçalho. O Exemplo 5.2 ilustra uma função desse tipo. Ela faz a leitura do teclado e o que for digitado é convertido para um número inteiro e retornado.

Exemplo 5.2 Função sem parâmetro de entrada

```
def LerInteiro():  
  
    n = int(input("Digite um número inteiro: "))  
  
    return n  
  
x = LerInteiro()  
  
print("Valor lido na função = {0}".format(x))
```

No Exemplo 5.1 a função Soma foi utilizada para somar números inteiros. No entanto, os parâmetros X e Y que recebem os valores não têm qualquer tipificação. Em outras palavras, seria possível passar números reais, complexos ou quaisquer outros tipos de dados para eles. Uma vez que o operador de adição “+” esteja definido para os dados que forem passados, a função deverá funcionar normalmente. Observem-se os Exemplos 5.3 e 5.4.

Exemplo 5.3 Função Soma com números reais passados como parâmetros

```
def Soma(X, Y):
```

```
R = X + Y
```

```
return R
```

```
print("Início do Programa")
```

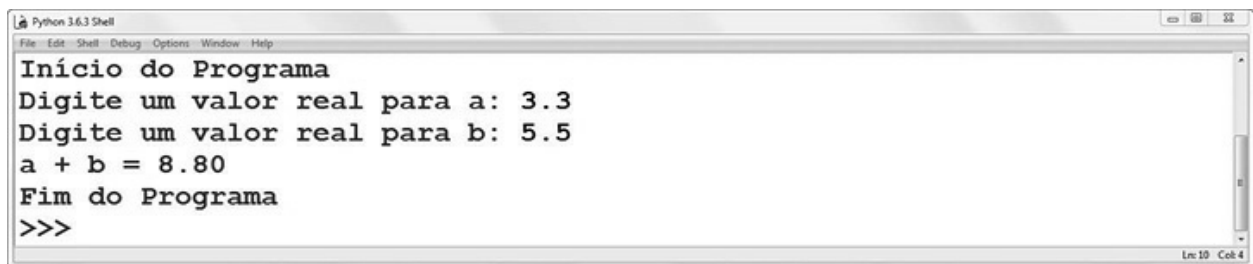
```
a = float(input("Digite um valor real para a: "))
```

```
b = float(input("Digite um valor real para b: "))
```

```
s = Soma(a, b)
```

```
print("a + b = {0:.2f}".format(s))
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite um valor real para a: 3.3
Digite um valor real para b: 5.5
a + b = 8.80
Fim do Programa
>>>
```

Exemplo 5.4 Função Soma com strings passados como parâmetros

```
def Soma(X, Y):
```

```
R = X + Y
```

```
return R
```

```
print("Início do Programa")
```

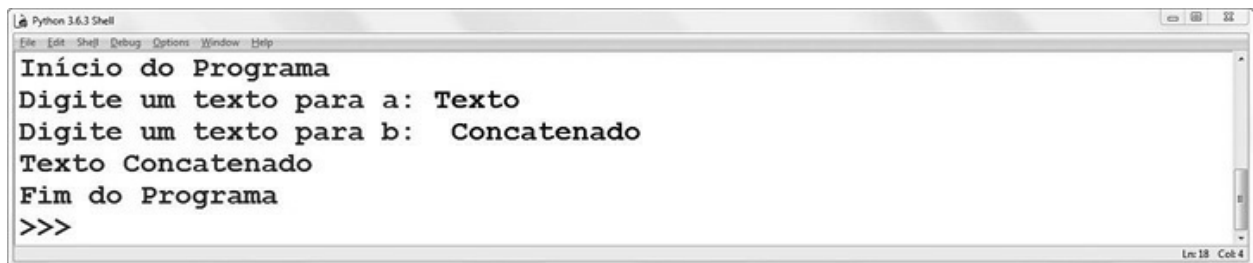
```
a = input("Digite um texto para a: ")
```

```
b = input("Digite um texto para b: ")
```

```
s = Soma(a, b)
```

```
print(s)
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite um texto para a: Texto
Digite um texto para b: Concatenado
Texto Concatenado
Fim do Programa
>>>
```

Sem qualquer alteração no cabeçalho ou no corpo da função Soma, e apenas fazendo adaptações nos dados de entrada lidos fora da função, foi possível executar o programa sem qualquer problema. Essa possibilidade de X e Y receberem números inteiros, no Exemplo 5.1, números reais, em 5.2, e strings, em 5.3, mostra que a função Soma pode funcionar de maneira polimórfica, ou seja, o interpretador busca adequar o comportamento do programa ao tipo de dado que está sendo utilizado na operação.

A seguir, outro exemplo envolvendo listas. O operador “+” está definido para listas e é capaz de produzir uma lista resultante juntando as duas listas passadas.

Exemplo 5.5 Função Soma com listas passadas como parâmetros

```
def Soma(X, Y):
```

```
    R = X + Y
```

```
    return R
```

```
print("Início do Programa")
```

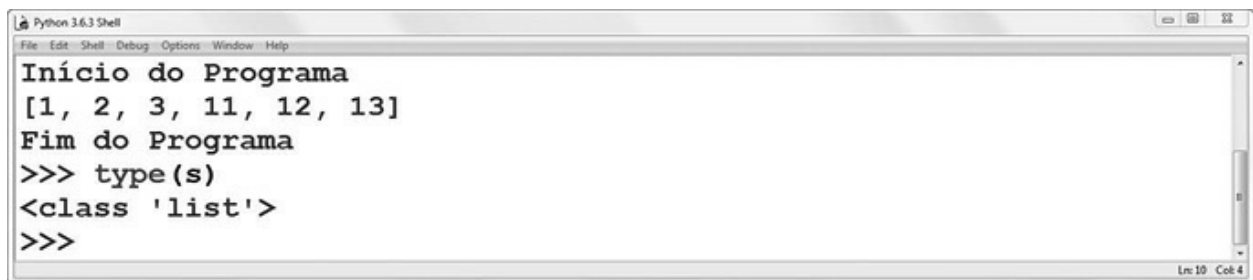
```
a = [1, 2, 3]
```

```
b = [11, 12, 13]
```

```
s = Soma(a, b)
```

```
print(s)
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
[1, 2, 3, 11, 12, 13]
Fim do Programa
>>> type(s)
<class 'list'>
>>>
```

Isto tudo é possível porque Python é uma linguagem que utiliza tipagem dinâmica. E, uma vez que o operador “+” esteja definido para o tipo de dado que é passado, então, o código da função será executado de maneira correta.

Se tal operador não estiver definido, então, o interpretador levantará uma exceção que poderá ser tratada, conforme visto no Item 3.3.

Programadores experientes em outras linguagens, como C, C++ ou Java, podem estranhar um recurso como este, pois tais linguagens utilizam o conceito de tipagem estática, segundo o qual a função é criada com parâmetros de tipos bem definidos. No entanto, os tempos atuais apresentam demandas que exigem das linguagens flexibilidade com concisão, e a tipagem dinâmica é uma resposta eficaz a essa demanda. Por esse motivo, Python, bem como PHP, JavaScript e outras linguagens mais recentes a adotam.

5.3.3 Parâmetros com valores-padrão

Os parâmetros podem apresentar valores-padrão – *default* – atribuídos na definição da função. Quando um parâmetro tem valor-padrão, ele se torna opcional na chamada da função, e caso seja omitido o valor-padrão é utilizado.

Exemplo 5.6 Função soma com parâmetro Y com valor-padrão

```
def Soma(X, Y = 1):  
  
    R = X + Y  
  
    return R  
  
print("Início do Programa")  
  
a = int(input("Digite um valor para a: "))  
  
b = int(input("Digite um valor para b: "))  
  
s = Soma(a, b)  
  
print("a + b = {0}".format(s))  
  
s = Soma(a)  
  
print("a + 1 = {0}".format(s))  
  
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite um valor para a: 2
Digite um valor para b: 10
a + b = 12
a + 1 = 3
Fim do Programa
>>>
```

Neste caso, o parâmetro Y tem valor-padrão igual a 1. Assim sendo, na chamada da função, caso o segundo parâmetro seja omitido, o valor 1 será assumido como valor de Y. Na execução do Exemplo 5.6 foram digitados os valores: 2 para o objeto a e 10 para o objeto b.

Na primeira chamada da função Soma, foram passados a e b:

```
s = Soma(a, b) # s resulta igual a 12 pois a = 2 e b = 10
```

Já na segunda chamada da função Soma, foi passado apenas a:

```
s = Soma(a) # s resulta igual a 3 pois a = 2 e o segundo
# parâmetro foi omitido, então, Y assumiu o valor 1.
```

Ao definir o cabeçalho de uma função, é preciso respeitar a regra de que primeiro devem ser relacionados todos os parâmetros que não apresentam valor-padrão e, depois, aqueles que os apresentam. O interpretador Python não aceita que um parâmetro sem valor-padrão seja declarado após outro que o contém.

5.3.4 Parâmetros nomeados

Outra característica de Python é a possibilidade de utilizar parâmetros nomeados.

Até o momento, por ser algo intuitivo, nada foi dito sobre a ordem de atribuição dos parâmetros. E a forma que se vem utilizando até aqui é a **passagem posicional**. Nos vários exemplos anteriores admitiu-se que a ordem de atribuição dos objetos passados na chamada da função é posicional,

ou seja, o primeiro objeto passado na chamada é assumido pelo primeiro parâmetro relacionado no cabeçalho, o segundo passado na chamada é assumido pelo segundo relacionado no cabeçalho, e assim por diante. Essa suposição está correta, sendo assim mesmo.

```
def Soma(X, Y):
```

```
... ↑ ↑
```

```
s = Soma(a, b) # a é passado para X e b é passado para Y
```

No entanto, a ordem pode ser trocada, desde que se faça referência ao nome do parâmetro que receberá o valor passado, ficando assim:

```
s = Soma(Y = b, X = a) # a é passado para X e b é passado para Y
```

```
# porém, as ordens estão trocadas
```

Essa forma de fazer a passagem de parâmetros é conhecida como **passagem nomeada** e pode ser utilizada em qualquer chamada de função. Nestes casos, a ordem não faz diferença, desde que, na chamada, todos os parâmetros sejam nomeados.

Aqui, há de se tomar um cuidado. Misturar parâmetros posicionais e nomeados em uma mesma chamada é possível, porém, deve-se respeitar a regra de que os primeiros podem ser posicionais, porém, após o primeiro parâmetro nomeado, todos devem ser nomeados.

```
def Funcao(M, N, O, P): # esta função recebe 4 parâmetros
```

```
...
```

```
Funcao(3, 6, 9, 12) # chamada Ok.
```

Os parâmetros são posicionais

Funcao(N=6, P=12, M=3, O=9) # chamada Ok.

Os parâmetros são nomeados

Funcao(3, 6, P=12, O=9) # chamada Ok.

neste caso os dois primeiros são posicionais e os outros dois

são nomeados

Funcao(3, N=6, 9, 12) # chamada incorreta. Gera erro.

Funcao(3, N=6, O=9, P=12) # chamada Ok.

5.3.5 Empacotamento e desempacotamento de parâmetros

Por fim, existe uma terceira alternativa para passagem de parâmetros que é utilizada com um pouco menos de frequência, por se aplicar a situações mais específicas. Trata-se de uma opção muito útil nos casos em que é preciso escrever uma função sem saber exatamente quantos parâmetros serão passados.

Esse número arbitrário será encapsulado em uma tupla que será passada para a função. Dentro da função, essa tupla poderá ser utilizada de qualquer maneira que o programador precise.

Exemplo 5.7 Função com empacotamento de parâmetros

```
>>> def Soma(*valores):
```

```
    r = 0
```



```
for i in valores:
```

```
    r += i
```

```
return r
```

```
>>> Soma(3, 9)
```

```
12
```

```
>>> Soma(1, 2, 3, 4)
```

```
10
```

```
>>> Soma(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1)
```

```
12
```

```
>>> Soma()
```

```
0
```

```
>>> Soma(5)
```

```
5
```

No Exemplo 5.7 foi definida a função `Soma`. Essa função recebe um parâmetro que está qualificado com o operador “*”, e o interpretador Python assumirá que está recebendo uma tupla. Quando a função é chamada, todos os parâmetros presentes na chamada, independentemente da quantidade, são coletados e convertidos em uma nova tupla que associa o objeto “valores” a essa tupla.

O caso inverso também é possível. Veja o Exemplo 5.8. A função

ExibeFormatado() deve receber três parâmetros posicionais. É possível utilizar uma lista ou uma tupla para passar tais parâmetros, porém, há duas condições:

- É preciso utilizar o operador “*” para informar ao interpretador que a lista (ou tupla) deve ser desempacotada.
- É preciso que a lista (ou tupla) tenha exatamente o número de parâmetros posicionais esperados pela função. Caso isso não aconteça, ocorrerá erro, como mostrado a seguir.

Exemplo 5.8 Função com desempacotamento de parâmetros

```
>>> def ExibeFormatado(a, b, c):
```

```
    print("1º valor = {}".format(a))
```

```
    print("2º valor = {}".format(b))
```

```
    print("3º valor = {}".format(c))
```

```
>>> L = [31, 77, 193]
```

```
>>> ExibeFormatado(*L)
```

```
1º valor = 31
```

```
2º valor = 77
```

```
3º valor = 193
```

```
>>> L = [43, 22, 323, 31]
```

```
>>> ExibeFormatado(*L)
```

Traceback (most recent call last):

File "<pyshell#32>", line 1, in <module>

```
ExibeFormatado(*L)
```

TypeError: ExibeFormatado() takes 3 positional arguments but 4 were given

A escolha entre utilizar um método ou outro de chamada de função é do programador. Há aqueles que preferem uma forma e outros que preferem outra. Em ambos os casos, haverá argumentos a favor e contra, e não é objetivo deste capítulo discuti-los. Fica aqui o convite para que teste as várias formas e adote a que considerar mais adequada ao seu estilo e às necessidades de seus algoritmos.

5.3.6 Retornos de funções

Em todas as linguagens é possível existir uma função que não retorna qualquer valor, bem como é possível retornar um ou muitos valores. Em Python isso também é assim.

Para que uma função tenha retorno basta utilizar a instrução `return`, que produz dois efeitos: retorna o objeto que é colocado à sua frente e encerra a função imediatamente.

Em todos os exemplos anteriores a função `Soma` retornava um resultado contido no objeto `R` e terminava a função. Porém, nesses exemplos o `return` era a última linha do código da função. É possível, no entanto, que exista um `return` em qualquer ponto da função, como mostrado no Exemplo 5.9. Nesse exemplo há `return` em dois pontos distintos do código. No primeiro, se `X` for

negativo, então, a função retornará o valor -1 e será imediatamente encerrada. Caso contrário, seguirá sua execução até chegar à última linha, gerando um retorno igual a 0.

Exemplo 5.9 Instrução *return* em pontos diversos dentro da função

```
def FuncaoF(X):  
  
    ... # vários comandos aqui  
  
    if X < 0:  
  
        return -1  
  
    else  
  
        ... # mais comandos aqui, subordinados ao else  
  
        ... # mais um pouco de comandos  
  
    return 0
```

Em funções que não têm retorno a instrução *return* não é utilizada. Nestes casos, uma vez chamada, sua execução prosseguirá desde a primeira até a última instrução de seu bloco de código. Funções sem valor de retorno são chamadas de um modo diferente, bastando escrever seu nome como se fosse um comando e passar os parâmetros apropriados, conforme ilustrado no Exemplo 5.10.

Exemplo 5.10 Função que não retorna valor

```
def ExibeLista(L):
```

```
for x in L:
```

```
    print(x)
```

```
Pares = [2, 4, 6, 8, 10]
```

```
print("Exibição da lista, sendo um elemento por linha")
```

```
ExibeLista(Pares)
```

Por sua vez, as funções que retornam valor podem ser chamadas do mesmo modo mostrado no Exemplo 5.10. Nesse caso, a função é executada, e sua ação interna, qualquer que seja, terá os devidos efeitos, porém, seu retorno não seria aproveitado.

Normalmente, porém, as funções que produzem retornos têm estes devidamente aproveitados. Tal aproveitamento pode ocorrer de diversas maneiras, dependendo do tipo de retorno que se tem. A seguir, são apresentadas algumas, porém, não todas, as possibilidades.

- Atribuído a um objeto:

```
s = Soma(a, b) # válido para qualquer caso
```

- Utilizado em meio a uma expressão aritmética:

```
s = 2 * Soma(a, b) / 10 # para retorno numérico
```

- Utilizado em uma condição:

```
if Soma(a, b) > 0: # p/ retorno passível de comparação
```

- Utilizado como iterador:

```
for x in Operacoes(a, b) # se o retorno for um iterador
```

```
# veja o Exemplo 5.11 a seguir
```

5.3.7 Retorno de múltiplos valores

Até agora, os exemplos utilizados retornavam um único valor. Porém, é possível escrever uma função que retorne múltiplos valores, como mostrado no Exemplo 5.11. Na função `Operacoes` desse exemplo são calculadas, respectivamente, a adição, a subtração, a multiplicação e a divisão dos dois parâmetros `X` e `Y` passados à função. O retorno é produzido escrevendo o comando `return` sucedido dos quatro valores calculados pela função.

O interpretador encapsula os vários elementos de retorno em uma tupla que é atribuída ao identificador “`s`”, o qual recebe o retorno da chamada da função. Essa tupla “`s`” pode ser utilizada da maneira que o programador desejar, utilizando os recursos vistos no Capítulo 4.

Exemplo 5.11 Função com múltiplos retornos

```
def Operacoes(X, Y):
```

```
    ad = X + Y
```

```
    su = X - Y
```

```
    mu = X * Y
```

```
    di = X / Y
```

```
    return ad, su, mu, di
```

```
print("Início do Programa")
```

```
a = int(input("Digite um valor para a: "))
```

```
b = int(input("Digite um valor para b: "))
```

```
s = Operacoes(a, b)
```

```
print(s)
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite um valor para a: 12
Digite um valor para b: 4
(16, 8, 48, 3.0)
Fim do Programa
>>> type(s)
<class 'tuple'>
>>>
```

Alternativamente, é possível utilizar o recurso de atribuição múltipla do Python, como mostrado a seguir. Os objetos r1 a r4 recebem o retorno da função Operacoes segundo a posição relativa de cada uma, de modo que r1 recebe a adição, r2, a subtração, r3, a multiplicação, e r4, a divisão.

```
>>> r1, r2, r3, r4 = Operacoes(a, b)
```

```
>>> print(r1)
```

```
16
```

```
>>> print(r2)
```

```
8
```

```
>>> print(r3)
```

48

```
>>> print(r4)
```

3.0

5.3.8 Escopo de funções

Com o que foi visto até agora, pode-se depreender que em um programa escrito em Python existem dois ambientes distintos:

1. Ambiente externo à função – que será chamado de Global.
2. Ambiente interno à função – que será chamado de Local.

Escopo diz respeito ao estudo desses ambientes e da maneira como eles se relacionam.

Durante a execução de um programa, todos os objetos criados fora de qualquer função são denominadas globais e todos os objetos criados dentro de uma função são denominadas locais.

Os objetos locais existem apenas enquanto a função está em execução. Quando uma função é chamada, seus objetos internos são criados, passam a existir, ocupando parte da memória do computador, e podem ser utilizados plenamente. Quando a função termina, esses objetos são removidos da memória, deixam de existir e os dados que continham são descartados.

Os valores de retorno da função também deixam de existir, porém, antes de serem descartados são atribuídos aos objetos que os recebem na chamada da função.

Assim, recorrendo ao Exemplo 5.1 verifica-se que os objetos *a*, *b* e *s* ali presentes são globais e existem durante todo o tempo em que o programa estiver em execução, **inclusive dentro das funções**. Por sua vez, *X*, *Y* e *R*

são locais e só existem durante a execução da função.

Agora, recorrendo ao Exemplo 5.12, cabe aprofundar um pouco mais o entendimento de escopo em Python. Nesse exemplo o objeto global X foi definido com o valor 10. Em seguida, a função EstudaEscopo foi chamada e dentro dela é feito o print de X, que, por ser global, está disponível dentro da função e pode ser utilizado no comando print.

Exemplo 5.12 Escopo de funções

```
def EstudaEscopo():
```

```
    print("X global existe dentro função: valor = {0}".format(X))
```

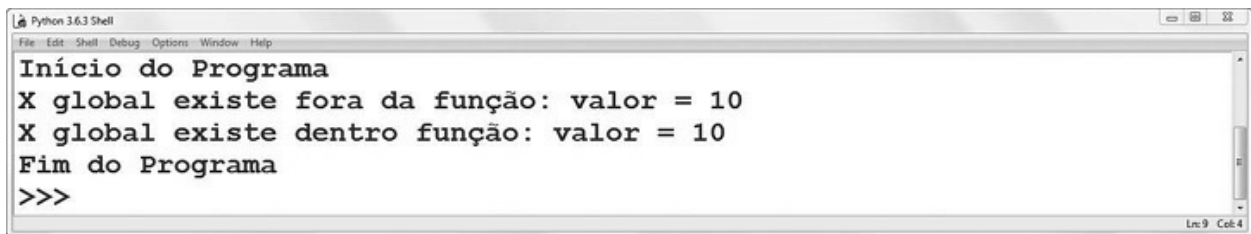
```
print("Início do Programa")
```

```
X = 10
```

```
print("X global existe fora da função: valor = {0}".format(X))
```

```
EstudaEscopo()
```

```
print("Fim do Programa")
```



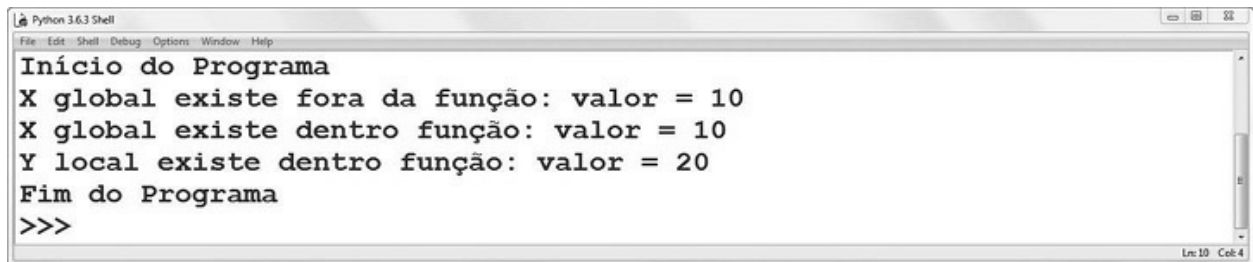
```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
X global existe fora da função: valor = 10
X global existe dentro função: valor = 10
Fim do Programa
>>>
```

Portanto, desse exemplo se constata que, de fato, o objeto X está disponível dentro e fora da função. Qualquer objeto que seja criado dentro da função terá escopo local. Fazendo uma pequena alteração a esse exemplo tem-se uma nova situação em que foi incluído o objeto local Y e que recebe o valor X *

2.

Exemplo 5.13 Escopo de funções

```
def EstudaEscopo():  
  
    Y = X * 2  
  
    print("X global existe dentro função: valor = {0}".format(X))  
  
    print("Y local existe dentro função: valor = {0}".format(Y))  
  
    print("Início do Programa")  
  
    X = 10  
  
    print("X global existe fora da função: valor = {0}".format(X))  
  
EstudaEscopo()  
  
print("Fim do Programa")
```



```
Python 3.6.3 Shell  
File Edit Shell Debug Options Window Help  
Início do Programa  
X global existe fora da função: valor = 10  
X global existe dentro função: valor = 10  
Y local existe dentro função: valor = 20  
Fim do Programa  
>>>
```

Com o Exemplo 5.13 chega-se à situação que se quer discutir neste momento. Do modo como está construído o exemplo, se for acrescentada a linha `X = 39` dentro da função, como mostrado a seguir, o leitor iniciante pode ser levado a deduzir que está sendo feita uma alteração no valor do objeto global `X`. Porém, ao executar esse programa, terá uma surpresa ao notar que `X` global continua com o conteúdo 10, embora dentro da função

exiba o valor 39.

```
def EstudaEscopo():
```

```
    X = 39
```

```
    Y = X * 2
```

```
    print("X global existe dentro função: valor = {0}".format(X))
```

```
    print("Y local existe dentro função: valor = {0}".format(Y))
```

Como o interpretador cria os objetos em tempo de execução, o que ocorreu com essa alteração é que foi criado o objeto local com o identificador X, e a partir daí, dentro da função, quaisquer referências a X dizem respeito ao objeto local, e não mais ao global.

Caso o desejo do programador seja alterar o conteúdo do global X dentro da função, então, deve-se recorrer à diretiva “global” para informar o interpretador que se quer alterar o objeto global dentro da função, em vez de criar um objeto X local. O código fica assim:

Exemplo 5.14 Escopo de funções

```
def EstudaEscopo():
```

```
    global X
```

```
    X = 19 # aqui está sendo alterado o objeto X global
```

```
    Y = X * 2
```

```
    print("X global existe dentro função: valor = {0}".format(X))
```

```
print("Y local existe dentro função: valor = {0}".format(Y))
```

```
print("Início do Programa")
```

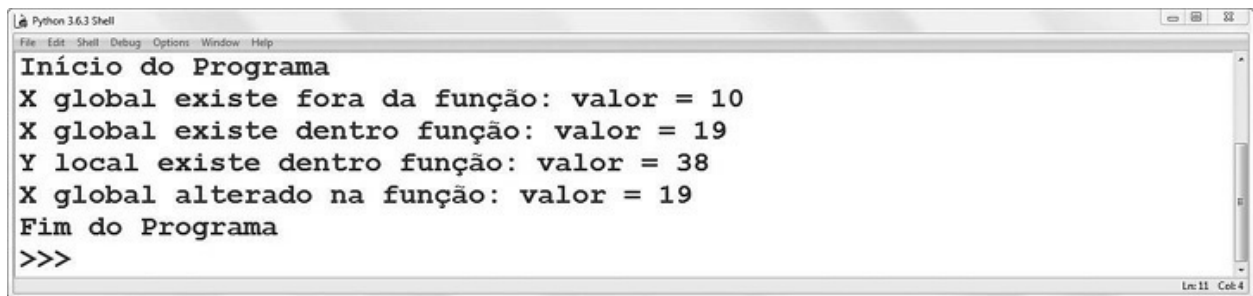
```
X = 10
```

```
print("X global existe fora da função: valor = {0}".format(X))
```

```
EstudaEscopo()
```

```
print("X global alterado na função: valor = {0}".format(X))
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
X global existe fora da função: valor = 10
X global existe dentro função: valor = 19
Y local existe dentro função: valor = 38
X global alterado na função: valor = 19
Fim do Programa
>>>
```

A questão levantada com o uso do objeto X só aconteceu porque houve uma coincidência de nomes de objetos, sendo um de escopo global, e outro, de escopo local. Recomenda-se fortemente evitar tal situação. A medida simples que pode ser tomada e que evita que isso aconteça é jamais utilizar objetos globais e locais com nomes idênticos. Essa recomendação não se restringe ao Python. Em geral, ela é válida na maioria das linguagens de programação.

5.3.9 Documentação de funções

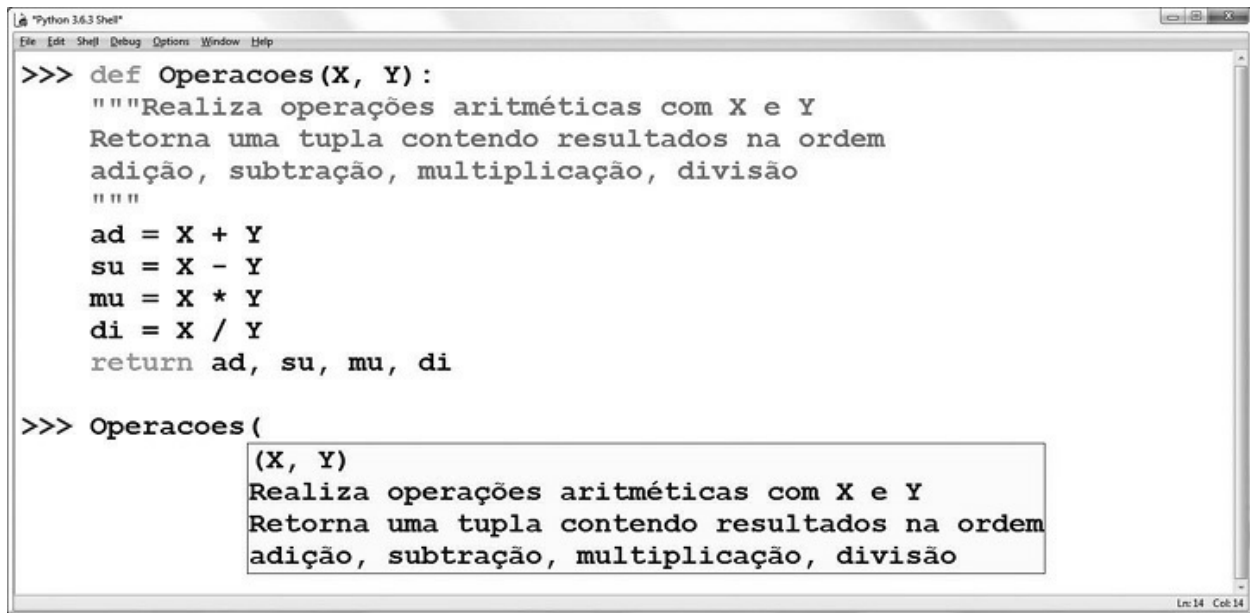
A comunidade Python estimula e encoraja os programadores a sempre criar documentação apropriada para as funções que desenvolvem. Essa

documentação é feita no próprio código do programa utilizando o recurso conhecido como *docstring*, que foi mencionado no Capítulo 2, Item 2.8. Observe-se o Exemplo 5.15, no qual foi inserido um *docstring* para a função Operacoes. Note que ele deve, obrigatoriamente, ser o primeiro elemento dentro da função e deve acompanhar a indentação.

Uma vez definido o *docstring* para a função, ele será utilizado para exibir a caixa de dica no momento de usá-la no IDLE, ou quando for utilizado o comando `help`. As PEPs 8 e 257 tratam desse assunto com mais profundidade.

Exemplo 5.15 Uso de docstrings para documentar uma função

```
def Operacoes(X, Y):  
    """Realiza operações aritméticas com X e Y  
  
    Retorna uma tupla contendo resultados na ordem  
    adição, subtração, multiplicação, divisão  
    """  
  
    ad = X + Y  
  
    su = X - Y  
  
    mu = X * Y  
  
    di = X / Y  
  
    return ad, su, mu, di
```

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main area shows a Python function definition and its execution. The function is named 'Operacoes' and takes two arguments, 'X' and 'Y'. It has a docstring in Portuguese: 'Realiza operações aritméticas com X e Y. Retorna uma tupla contendo resultados na ordem adição, subtração, multiplicação, divisão'. The function body calculates 'ad = X + Y', 'su = X - Y', 'mu = X * Y', and 'di = X / Y', then returns a tuple '(ad, su, mu, di)'. Below the definition, the function is called with 'Operacoes(' followed by a tooltip box. The tooltip box contains the function signature '(X, Y)' and the same docstring as the function definition. The status bar at the bottom right shows 'Ln: 14 Col: 14'.

```
>>> def Operacoes(X, Y):  
    """Realiza operações aritméticas com X e Y  
    Retorna uma tupla contendo resultados na ordem  
    adição, subtração, multiplicação, divisão  
    """  
    ad = X + Y  
    su = X - Y  
    mu = X * Y  
    di = X / Y  
    return ad, su, mu, di  
  
>>> Operacoes(  
    (X, Y)  
    Realiza operações aritméticas com X e Y  
    Retorna uma tupla contendo resultados na ordem  
    adição, subtração, multiplicação, divisão
```

5.4 Recursividade

Funções recursivas são aquelas que chamam a si mesmas. É um dos exemplos clássicos de função recursiva é o cálculo do fatorial de um número. No Capítulo 3 foi resolvido um exercício que calculava $N!$ usando um laço `while`.

Agora, será resolvido esse mesmo problema usando uma função recursiva. Toda função recursiva tem uma condição de parada. Essa condição determina em que ponto ela não mais chama a si mesma, iniciando o processo de saída das sucessivas chamadas.

Exemplo 5.16 Função recursiva – cálculo de $N!$

```
def Fatorial(N): # linha 1
```

```
    if N <= 1: # linha 2
```

```
        return 1 # linha 3
```

```
    else: # linha 4
```

```
return N * Fatorial(N-1) # linha 5

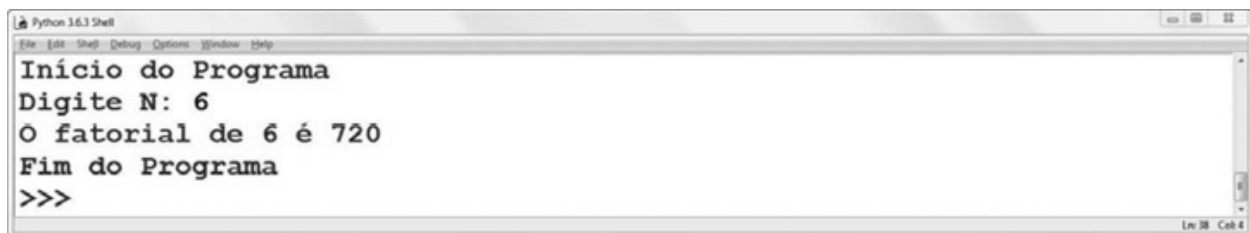
print("Início do Programa")

X = int(input("Digite N: "))

F = Fatorial(X)

print("O fatorial de {0} é {1}".format(X, F))

print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Digite N: 6
O fatorial de 6 é 720
Fim do Programa
>>>
```

Na execução do Exemplo 5.16 foi fornecido como dado de entrada o valor 6. Sabe-se que o $6! = 720$, portanto, verifica-se que o programa está correto. O ponto agora é explicar o que está sendo feito na função Fatorial.

O programa foi executado com o valor 6 fornecido para X, que é o objeto usado na leitura. Assim, na chamada de Fatorial o valor passado para o parâmetro N foi 6. Na linha 5 da função encontra-se o ponto-chave. Nessa linha o comando `return N * Fatorial de (N-1)` provoca uma segunda chamada à própria função, porém, passando como parâmetro o valor N-1, ou seja, 5. Ao ser chamada pela terceira vez o parâmetro será 4, e assim por diante, até que ocorra uma chamada com parâmetro $N = 1$. Quando isso ocorrer, a condição do comando `if` na linha 2 avaliará como verdadeiro e a função retornará à chamada anterior, retornando o valor 1. Esse retorno será multiplicado por $N = 2$ e retornará para a chamada anterior, e assim sucessivamente, conforme ilustrado no Quadro 5.1.

Chamada nº	Condição na entrada	Ciclo de entrada com N * Fatorial (N-1)	Ciclo de saída das chamadas
0	–	Função principal	Recebe 720 da chamada 1
1	N = 6	6 * Fatorial (5)	Calcula 6 * 120 = 720 e retorna 720
2	N = 5	5 * Fatorial (4)	Calcula 5 * 24 = 120 e retorna 120
3	N = 4	4 * Fatorial (3) ↓	Calcula 4 * 6 = 24 e retorna 24 ↑
4	N = 3	3 * Fatorial (2)	Calcula 3 * 2 = 6 e retorna 6
5	N = 2	2 * Fatorial (1)	Calcula 2 * 1 = 2 e retorna 2
6	N = 1	return 1	Retorna 1 à chamada anterior.

Quadro 5.1 Ciclo de chamadas recursivas da função fatorial.

Outra maneira de ver o que está ocorrendo dentro de uma função recursiva é incluir nela um ou mais prints exibindo na tela os dados com os quais a função trabalha.

Exercícios resolvidos

1. Funções comuns – números primos

Escreva uma função que receba como parâmetro de entrada um número inteiro N. Ela deve retornar 1 se N for primo ou 0, caso não seja. Esse problema já foi discutido no Exercício resolvido 3.4.

Considerações para a solução: na função serão considerados apenas números maiores que 1. O número 2 é o único par que é primo. Segue a solução:

Exercício resolvido 5.1 – Função que determina se um número é primo ou não

```
def EPrimo(P):  
  
    if P <= 1:  
  
        return "Erro" # retorna Erro se P <= 1  
  
    elif P == 2: # se P é 2 retorna 1 indicando  
  
        return 1 # que é primo  
  
    elif P % 2 == 0: # dado que P não é dois, se ele for  
  
        return 0 # par, então, retorna 0, não é primo  
  
    else: # P é ímpar, então, é necessário  
  
        raiz = P ** 0.5 # implementar um laço de teste  
  
        R = 1  
  
        i = 3  
  
        while i <= raiz and R != 0:  
  
            R = P % i  
  
            i+=2  
  
        return R
```

Os casos em que P é menor ou igual a 2 ou um número par são triviais e

estão comentados ao lado do código. Quando P é ímpar é necessário testar o resto de todas as divisões por valores ímpares entre 3 e a raiz quadrada de P . Utilizando esse recurso economiza-se muito tempo de processamento, e ele é válido pois o resultado que efetivamente interessa é o resto da divisão. Considere que P seja 17. Pode-se dividir 17 por 3 e o quociente será 5 com resto 2. Assim, é desnecessário dividir 17 por 5, pois o quociente será 3 com o mesmo resto 2. Fazer as duas operações resultaria em redundância e desperdício de tempo de processamento.

Como P pode ser escrito na forma $P = A \times B$ quando A é pequeno B é grande, à medida que A cresce, B diminui, e isso pode prosseguir até que se igualem. A partir daí, se A continuar crescendo assumirá valores que já foram de B , e vice-versa. Assim, o limite de crescimento de A é a raiz quadrada de P .

No algoritmo que acabamos de ver o laço é processado enquanto o contador i for menor que raiz (P) e o resto R diferente de zero. Caso P não seja primo, em algum momento R será zero, o laço termina e a função retorna R (que é zero). Se o número não for primo, então, R nunca será zero e o laço terminará com i atingindo seu limite máximo e, nesse caso, ao retornar R , a função estará retornando algum valor diferente de zero.

2. Funções comuns – interseção de listas

Escreva uma função que receba duas listas $L1$ e $L2$ como parâmetro de entrada e retorne uma lista que seja a interseção de $L1$ e $L2$, em que uma lista interseção é aquela que contém os elementos que estejam presentes em ambas, $L1$ e $L2$.

Exercício resolvido 5.2 – Função que retorna a lista Interseção das listas $L1$ e $L2$

```
def Intersecao(L1, L2):  
  
    LR = [] # define a lista resultante vazia  
  
    for e in L1: # percorre L1 do início ao fim  
  
        if e in L2: # se elemento e de L1 estiver em L2  
  
            LR.append(e) # insere e em LR  
  
    return LR # retorna LR
```

3. Função recursiva – soma dos elementos de uma lista

Suponha que uma lista está carregada com diversos números inteiros. Escreva uma função recursiva que calcule a soma desses valores. Para testar essa função, use a lista $L = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$, cuja soma resulta = 55.

Exercício resolvido 5.3 – Soma recursiva dos valores contidos em uma lista

```
def SomaLista(L):  
  
    print(L)  
  
    if L == []:  
  
        return 0  
  
    else:  
  
        return L[0] + SomaLista(L[1:])
```

```
print("Início do Programa")
```

```
Lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

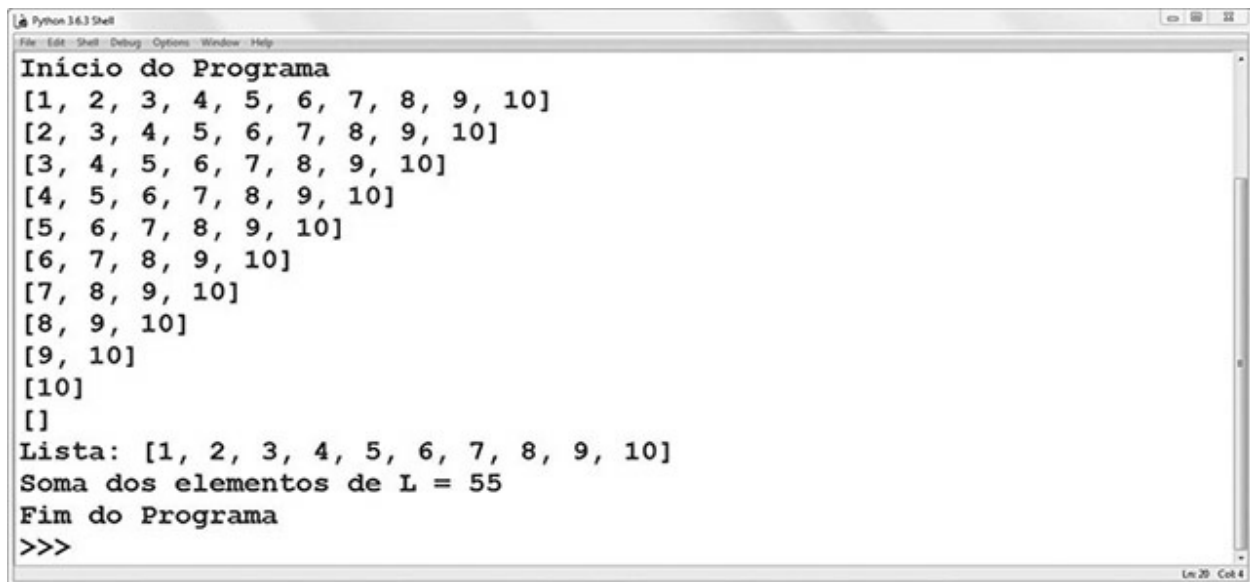
```
S = SomaLista(Lista)
```

```
print("Lista: ", end="")
```

```
print(Lista)
```

```
print("Soma dos elementos de L = {0}".format(S))
```

```
print("Fim do Programa")
```

A screenshot of a Python 3.6.3 Shell window. The window title is "Python 3.6.3 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The output of the program is displayed in the main area:

```
Início do Programa
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[2, 3, 4, 5, 6, 7, 8, 9, 10]
[3, 4, 5, 6, 7, 8, 9, 10]
[4, 5, 6, 7, 8, 9, 10]
[5, 6, 7, 8, 9, 10]
[6, 7, 8, 9, 10]
[7, 8, 9, 10]
[8, 9, 10]
[9, 10]
[10]
[]
Lista: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Soma dos elementos de L = 55
Fim do Programa
>>>
```

The status bar at the bottom right shows "Ln 20 Col 4".

Nessa solução a condição de parada da recursividade é a `L == []`, ou seja, quando a lista recebida pela função estiver vazia, retornando 0 neste caso. As novas chamadas da função são realizadas com o comando `L[0] + SomaLista(L[1:])`, ou seja, o primeiro elemento da lista (`L[0]`) somado à soma do resto de `L`.

Relembrando: dada uma lista `L` e considerando que as listas em Python têm

o primeiro elemento com índice 0, a sintaxe `L[1:]` produz uma lista que inicia com o segundo elemento de `L` e contém todos os demais elementos até o final, uma vez que o segundo parâmetro de fatiamento foi omitido. Esse é um uso interessante para o recurso de fatiamento de sequências visto no Capítulo 4.

A seguir foi inserido um `print` no início da função que permite visualizar como a lista `L` entra em cada chamada.

4. Função recursiva – busca binária em lista ordenada

Escreva uma função que recebe dois parâmetros: uma lista `L` contendo números inteiros e organizada em ordem crescente; um número inteiro `N`. Essa função deve verificar se `N` está contido em `L` utilizando o algoritmo de busca binária e retornar a posição em que ele se encontra ou retornar 0 caso `N` não esteja na lista.

Considerações preliminares

O algoritmo de busca binária é clássico na programação de computadores e todo programador deve conhecê-lo. Trata-se de um algoritmo capaz de determinar se um valor está ou não presente em uma grande coleção de dados, partindo do pressuposto de que a coleção está ordenada, seja de maneira crescente ou decrescente. Esse algoritmo é bem mais eficiente – ou seja, mais rápido – que o algoritmo de busca sequencial visto no Capítulo 4 (exceção feita aos poucos casos em que o valor procurado está entre os primeiros da sequência).

A busca binária baseia-se no paradigma da divisão e conquista, no qual a coleção é sucessivamente dividida ao meio, reduzindo-se o espaço de busca e sempre comparando-se o valor buscado com o elemento que está no meio desse espaço de busca. Se o valor procurado for igual ao

elemento do meio, então, a função retorna com sucesso. Se o valor procurado for menor a busca continua, porém, restrita à metade inicial da coleção, e caso seja maior a busca continua restrita à metade final.

Valor Buscado: 42

Lista

3	8	11	14	16	19	25	29	31	37	42	46	53	58	60	63	71	82
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Primeira tentativa – Elemento do meio 31 é menor que 42: a busca prossegue pela metade superior

3	8	11	14	16	19	25	29	31	37	42	46	53	58	60	63	71	82
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Primeira tentativa – Elemento do meio 58 é maior que 42: a busca prossegue pela metade inferior

37	42	46	53	58	60	63	71	82
----	----	----	----	----	----	----	----	----

Primeira tentativa – Elemento do meio 42 é o valor procurado: a busca termina com sucesso

37	42	46	53
----	----	----	----

Observação sobre os índices usados. O índice do elemento do meio é calculado com matemática de números inteiros. Assim, se o índice do início é 9 e o índice do final é 12 o índice do meio será:

$\text{meio} = (\text{início} + \text{final}) // 2$ que resulta meio = 10

Figura 5.2 Ilustração do algoritmo de busca binária.

O código do Exercício resolvido 5.4 traz a implementação em Python desse algoritmo por meio de uma função recursiva, bem como de um programa que o utiliza para teste da função.

Exercício resolvido – 5.4 Implementação do algoritmo de busca binária

```
def BuscaBin(L, N, ini, fim):
```

```
    if ini > fim:
```

```
return 0
```

```
meio = (ini + fim) // 2
```

```
if X == L[meio]:
```

```
return meio
```

```
elif X < L[meio]:
```

```
return BuscaBin(L, X, ini, meio-1)
```

```
else:
```

```
return BuscaBin(L, X, meio+1, fim)
```

```
print("Início do Programa")
```

```
Lista = [3,8,11,14,16,19,25,29,31,37,42,46,53,58,60,63,71,82]
```

```
X = int(input("Digite um valor para pesquisa na lista: "))
```

```
while X != 0:
```

```
Pos = BuscaBin(Lista, X, 0, len(Lista))
```

```
if Pos != 0:
```

```
print("{0} está na posição {1} da lista".format(X, Pos));
```

```
else:
```

```
print("{0} não está na lista".format(X));
```

```
X = int(input("Digite um valor para pesquisa na lista: "))
```

```
print("Fim do Programa")
```

Exercícios propostos

1. Escreva uma função que recebe um número inteiro como parâmetro de entrada e retorna o texto “PAR” ou “ÍMPAR”
2. Utilize a função EPrimo desenvolvida no Exercício resolvido 5.1 para carregar uma lista contendo os N primeiros números primos, em que N é um número inteiro fornecido pelo usuário.
3. Escreva uma função que receba dois números inteiros A e B como parâmetros de entrada e retorne 1 se A for divisível por B e 0 caso contrário.
4. Escreva uma função que receba um número inteiro N e retorne uma lista com os bits 0 e 1, que representam N convertido para binário. Não use nenhuma função Python de conversão para binários. Em vez disso, elabore uma lógica baseada no processo de divisões sucessivas.
5. Escreva uma função que receba como parâmetro de entrada uma lista L de números inteiros e um valor. A função deve retornar quantas vezes o valor está contido na lista. Caso ele não esteja em L, retorne 0.
6. Escreva um programa que receba como parâmetro de entrada um número inteiro de 5 dígitos no intervalo fechado [10000, 30000] que represente códigos de produtos vendidos em uma loja. A função deve calcular e retornar o dígito verificador utilizando regra de cálculo explicada a seguir. Considere o código 21853, em que cada dígito é

multiplicado por um peso começando em 2, os valores obtidos são somados, e do total obtido calcula-se o resto de sua divisão por 7.

Dígito	2	1	8	5	3	
Peso	2	3	4	5	6	
Multiplicação	4	3	32	25	18	Soma todos = 82
						Resto de 82 por 7 = 5

7. Escreva uma função que receba como parâmetro de entrada dois números reais Min e Max. Essa função deve ler do teclado um número real e retorná-lo caso esteja dentro do intervalo fechado [Min, Max]. Caso contrário, a função deve exibir uma mensagem de erro e ler um novo valor.
8. Escreva uma função que receba uma lista como parâmetro de entrada e retorne uma tupla contendo quatro valores na seguinte ordem: a soma, a média, o menor e o maior valor dentre todos os elementos nela contidos. Considere que nessa lista ocorram apenas números reais. Escreva um programa para testar essa função, exibindo na tela os resultados. Neste exercício, evite utilizar as funções prontas existentes no Python, como sum, min e max.
9. Escreva uma função que receba uma lista L e elimine os eventuais elementos repetidos contidos na mesma, deixando na lista resultante apenas uma ocorrência de cada elemento. Escreva um programa para testar essa função, o qual deve ler do teclado os elementos que farão parte da lista. (veja o Exercício proposto 4.10)
10. Escreva uma função que receba duas listas L1 e L2 como parâmetro de entrada e retorne uma lista contendo todos os elementos de L1 que

não estão em L2. Escreva um programa para testar essa função.

- 11.** Escreva uma função que receba como parâmetro de entrada uma lista L e retorne uma lista organizada em ordem crescente. Para fazer a ordenação, use o Algoritmo de Ordenação Bolha (Bubble Sort). Crie uma segunda versão dessa função que retorne uma lista organizada em ordem decrescente.

Escreva um programa para testar essas duas funções. Esse programa deve ler um número inteiro N e gerar uma lista com N números inteiros aleatórios utilizando a função *randint()*. Use as duas funções de ordenação e exiba na tela as listas ordenadas crescente e decrescente.

- 12.** Escreva um programa que leia um número inteiro Q e exiba na tela os Q primeiros termos da sequência de Fibonacci, utilizando uma função recursiva para determinar o elemento da sequência a ser exibido. A sequência de Fibonacci, já vista nos Exercícios resolvidos 3.7 e 4.7, caracteriza-se por um termo ser a soma dos dois anteriores, sendo que os dois primeiros termos são 0 e 1. Assim, os dez primeiros termos são: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

- 13.** Dada uma lista contendo números inteiros, escreva uma função recursiva para calcular a multiplicação de todos os elementos. Exiba o resultado na tela.

- 14.** Faça uma pesquisa sobre o Algoritmo de Ordenação Quicksort. Implemente uma função recursiva que use esse algoritmo para organizar a lista L de forma crescente. Escreva um programa para testar a função.

15. Escreva um programa que contenha duas funções de ordenação diferentes: uma que implemente o algoritmo Bubble Sort (criada no Exercício 11) e outra que implemente o algoritmo Quicksort (criada no Exercício 14). Escreva um programa que gere uma lista com um tamanho bem grande e utilize-a para testar o desempenho das duas funções.

Sugestões:

Faça esse programa gerar a lista grande contendo Q elementos, na qual Q é digitado pelo usuário.

Esse valor de Q deve ser bem grande, por exemplo, de 10 a 50 mil elementos, ou mais, dependendo da capacidade de seu computador.



Tipos Estruturados Não Sequenciais

Objetivos

No Capítulo 4 foram vistos os tipos estruturados sequenciais, que se caracterizam por ter seu conteúdo composto por distintos elementos que podem ser acessados a partir de um índice numérico, sequencial e que inicia em zero. Neste capítulo serão vistos os tipos estruturados não sequenciais, os quais também se caracterizam pelo conteúdo composto, porém, a individualização dos elementos não se dá pelo uso de um índice.

No entanto, para que os conceitos de conjunto e das chaves dos dicionários possam ser apresentados de maneira apropriada, é necessário apresentar os

conceitos de objetos *hashable* e não *hashable*, os quais estão vinculados com a imutabilidade e a mutabilidade desses objetos.

Após essa conceituação, serão vistos dois tipos de objetos, que são o propósito deste capítulo: os conjuntos (***set***) e os dicionários (***dictionary***).

6.1 Hashable: o que é isso?

Antes de começar, duas considerações: este é um assunto que poderia ter sido tratado no Capítulo 2, mas foi deixado para este capítulo para evitar que o programador iniciante fosse abarrotado com conceitos específicos e aprofundados de Python antes de dominar seus aspectos mais básicos; o termo técnico *hashable*, do inglês, não tem uma tradução em português, de modo que esse será o termo utilizado daqui para a frente.

Diz-se que um objeto é *hashable* quando ele tem um valor *hash* que não se altera durante o ciclo de vida do objeto e que pode ser comparado ao *hash* de outros objetos.

Neste contexto, *hash* é um número inteiro calculado a partir do conteúdo de um objeto e pode ser verificado com o uso da função *hash*, como mostrado no Exemplo 6.1. Nele, são criadas duas variáveis string (poderia ser qualquer outro tipo, com qualquer conteúdo) contendo o mesmo conjunto de caracteres. Os números id de cada objeto são exibidos com o propósito de mostrar que são ids diferentes. E a função *hash* é usada com os dois objetos para mostrar que ambos têm o mesmo valor de *hash*. Isto ocorre porque esse número é calculado a partir do conteúdo do objeto. Quando se escreve uma condição de igualdade ($T==Z$) ou diferença ($T!=Z$), o interpretador usa os valores de *hash* de cada um dos objetos para avaliar o resultado.

Da definição dada anteriormente e considerando que todos os tipos imutáveis não podem ter seu valor alterado, então, conclui-se que eles são *hashable*, ao passo que os objetos mutáveis não o são. A única exceção é que

as tuplas que contêm tipos mutáveis não são *hashable*.

Exemplo 6.1 Exemplo de *hash* de objetos

```
>>> T = 'Texto'
```

```
>>> id(T)
```

```
48527360
```

```
>>> hash(T)
```

```
-1151134949
```

```
>>> Z = 'Texto'
```

```
>>> id(Z)
```

```
48527360
```

```
>>> hash(Z)
```

```
-1151134949
```

```
>>> T == Z
```

```
True
```

```
>>> x = (3, 6, 9) # tuplas são hashable
```

```
>>> type(x)
```

```
<class 'tuple'>
```

```
>>> hash(x)
```

```
-149728741
```

```
>>> y = (3, 6, 9, [2, 4]) # tuplas que contêm listas não são
```

```
>>> type(y) # hashable
```

```
<class 'tuple'>
```

```
>>> hash(y)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#141>", line 1, in <module>
```

```
hash(y)
```

```
TypeError: unhashable type: 'list'
```

```
>>> L = [1, 2, 3] # listas não são hashable
```

```
>>> hash(L)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#143>", line 1, in <module>
```

```
hash(L)
```

```
TypeError: unhashable type: 'list'
```

6.2 Conjuntos

O tipo conjunto é uma coleção não ordenada de elementos não repetidos. Esse tipo suporta as operações características da Teoria dos Conjuntos, um ramo da matemática, tais como união, interseção e diferença.

Por definição, dentro do conjunto, só haverá uma ocorrência de cada elemento, independentemente de quantas vezes se tente adicioná-lo. Um conjunto pode ser criado de duas maneiras: utilizando dados entre chaves, { } ou a função set. É possível existir um conjunto vazio. E apenas objetos *hashable* podem ser membros de um conjunto.

Em Python existem dois tipos de conjunto: o set e o frozenset. Em praticamente tudo eles são iguais, a única e fundamental diferença entre ambos é que o frozenset é imutável e, uma vez criado, não pode ter seus membros alterados, incluídos ou removidos.

O Exemplo 6.2 ilustra diversas situações possíveis para a criação de conjuntos. No caso 1, foram utilizadas as chaves e, dentro delas, valores numéricos. Com isso, o conjunto “a” resultante contém objetos numéricos. No caso 2 foi utilizada a função set com um string de parâmetro e o resultado obtido é o desmembramento do string, de modo que cada caractere seja um elemento do conjunto. No caso 3 foi utilizado um string entre chaves e não houve desmembramento, resultando em um conjunto com um único elemento.

Exemplo 6.2 Criação de conjunto

```
>>> a = {1, 2, 3, 4, 5} # caso 1
```

```
>>> a
```

```
{1, 2, 3, 4, 5} # cria um set com cinco elementos
```

```
>>> type(a)
```

```
<class 'set'>
```

```
>>> len(a) # a função len() pode ser usada
```

```
5
```

```
>>> b = set('12345') # caso 2
```

```
>>> b
```

```
{'1', '4', '5', '3', '2'} # cria um novo set com cinco elementos
```

```
>>> type(b)
```

```
<class 'set'>
```

```
>>> c = {'12345'} # caso 3
```

```
>>> c
```

```
{'12345'} # cria um conjunto com um elemento
```

```
>>> type(c)
```

```
<class 'set'>
```

```
>>> d = {} # este comando não cria um conjunto
```

```
>>> type(d) # vazio, mas sim um dicionário
```

```
<class 'dict'>
```

```
>>> d = set() # conjuntos vazios são criados assim
```



```
>>> type(d)
```

```
<class 'set'>
```

É possível criar um conjunto vazio e adicionar elementos posteriormente, porém, deve-se tomar um cuidado. Conjuntos vazios devem ser criados com a função `set` sem passar qualquer parâmetro. O uso de chaves sem conteúdo criará um dicionário vazio, e não um conjunto, como já mostrado.

Conteúdos de listas e tuplas podem ser utilizados para produzir conjuntos, e vice-versa. O Exemplo 6.3 mostra a conversão de uma lista para conjunto. Note que, ao fazer essa operação, os elementos repetidos presentes na lista foram eliminados no conjunto. Essa é uma maneira conveniente de eliminar repetições de valores em listas e tuplas, pois é possível converter de volta o conjunto para uma lista ou tupla.

Exemplo 6.3 Conversão entre listas, tuplas e conjuntos

```
>>> L = [3, 7, 9, 16, 3, 8, 14, 9, 25] # lista c/ elem. repetidos
```

```
>>> a = set(L) # conversão da lista em conjunto
```

```
>>> a
```

```
{3, 7, 8, 9, 14, 16, 25} # apenas uma ocorrência de cada valor
```

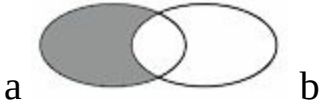
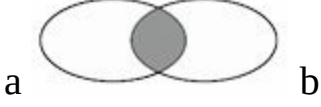
```
>>> L = list(a) # converte de volta o conjunto para
```

```
>>> L # lista
```

```
[3, 7, 8, 9, 14, 16, 25]
```

6.2.1 Operações com conjuntos

O Quadro 6.1 mostra os operadores aplicáveis aos conjuntos e o Exemplo 6.4 ilustra seu uso.

Operação	Nome	O que retorna (hachurado)
$a - b$	Diferença	
$a b$	União	
$a \& b$	Interseção	
$a \wedge b$	Diferença simétrica	
valor in a	Pertence	Valor está contido no conjunto a
valor not in a	Não pertence	Valor não está contido no conjunto a

Quadro 6.1 Operações aplicáveis aos conjuntos.

Exemplo 6.4 Operações com conjuntos

```
>>> a = set('abacaxi')
```

```
>>> a
```

```
{'a', 'x', 'i', 'b', 'c'} # não há objetos repetidos
```

```
>>> b = set('abacate')
```

```
>>> b
```

```
{'t', 'e', 'a', 'b', 'c'} # não há objetos repetidos
```

```
>>> a - b # elementos em a, porém, não em b
```

```
{'x', 'i'}
```

```
>>> a | b # elementos em a, em b ou em ambos
```

```
{'t', 'e', 'a', 'x', 'i', 'b', 'c'}
```

```
>>> a & b # elementos simultâneos em a e b
```

```
{'c', 'b', 'a'}
```

```
>>> a ^ b # elementos em a ou b, mas não em ambos
```

```
{'x', 'i', 't', 'e'}
```

```
>>> 'x' in a # 'x' está em a
```

```
True
```

```
>>> 'r' in a # 'r' não está em a
```

```
False
```

```
>>> 'r' not in a
```

```
True
```

6.2.2 Métodos disponíveis

Em adição aos operadores, os conjuntos contam com um significativo número de métodos, que são utilizados para efetuar diversas tarefas. Os conjuntos podem ter elementos adicionados e removidos, pode ser zerado e atualizado, e o Quadro 6.2 mostra alguns deles.

	Considere-se disponível a lista C
--	-----------------------------------

Método	Descrição
C.add(...)	Acrescenta um objeto ao conjunto.
C.clear()	Remove todos os elementos do conjunto.
C.copy()	Retorna uma cópia do conjunto.
C.difference(...)	Retorna um novo conjunto contendo a diferença de dois ou mais conjuntos.
C.difference_update(...)	Atualiza o conjunto C, removendo de seus elementos os que estejam no conjunto passado como parâmetro.
C.discard(...)	Se parâmetro estiver presente no conjunto, então o remove, caso não esteja não faz nada.
C.intersection(...)	Retorna um novo conjunto contendo a interseção de dois ou mais conjuntos.
C.intersection_update(...)	Atualiza o conjunto C com a interseção entre seus elementos e o conjunto passado como parâmetro.
C.isdisjoint(...)	Retorna <i>True</i> se os dois conjuntos têm interseção vazia e <i>False</i> , caso contrário.
C.issubset(...)	Retorna <i>True</i> se C é um subconjunto do conjunto passado como parâmetro.
C.issuperset(...)	Retorna <i>True</i> se C está contido no conjunto passado como parâmetro.
C.pop()	Remove e retorna um elemento arbitrário do conjunto C. Se o conjunto estiver vazio, gera uma exceção <i>KeyError</i> .
C.remove(...)	Remove do conjunto C um elemento que seja seu membro. Caso contrário, gera uma exceção <i>KeyError</i> .

	Considere-se disponível a lista C
Método	Descrição
C.symmetric_difference(...)	Atualiza o conjunto C com a diferença simétrica de seus elementos com o conjunto passado como parâmetro.

<code>C.symmetric_difference_update(...)</code>	Atualiza o conjunto C com a diferença simétrica de seus elementos com o conjunto passado como parâmetro.
<code>C.union(...)</code>	Retorna um novo conjunto com a união de C e o conjunto passado como parâmetro.
<code>C.update(...)</code>	Atualiza o conjunto C com a união de sim, mesmo com o conjunto passado como parâmetro.

Quadro 6.2 Métodos da classe “set” disponíveis ao programador.

O Exemplo 6.5 ilustra diversas situações de usos dos métodos listados no Quadro 6.2. Na maioria dos casos mostrados os métodos estão sendo usados com os conjuntos a e b. No entanto, os parâmetros passados para os métodos também podem ser listas e tuplas, como mostrado no final do exemplo.

Exemplo 6.5 Usos de conjuntos

```
>>> a = {1, 2, 3, 4, 5, 6} # define o conjunto a

>>> a.add(7) # adiciona elemento

>>> a.add(8) # adiciona outro elemento

>>> a.add(3) # esta adição não tem efeito, pois 3

>>> a # já está na lista

{1, 2, 3, 4, 5, 6, 7, 8} # conjunto ampliado

>>> b = {2, 4, 6}

>>> b.issubset(a) # o conjunto b é subconjunto de a
```

True

```
>>> a.issubset(b) # o conjunto a não é subconjunto de b
```

False

```
>>> a.issuperset(b) # b está contido em a
```

True

```
>>> b.add(10) # adiciona novo elemento em b
```

```
>>> b # não se tem controle sobre a posição
```

{2, 10, 4, 6} # onde o novo elemento é inserido

```
>>> a.issubset(b) # b não é mais subconjunto de a
```

False

```
>>> c = a.difference(b) # gera o conjunto c contendo a-b
```

```
>>> c
```

{1, 3, 5, 7, 8}

```
>>> c = b.difference(a) # gera o conjunto c contendo b-a
```

```
>>> c
```

{10}

```
>>> >>> a.difference_update(b) # atualiza a com a-b
```

```
>>> a
```

```
{1, 3, 5, 7, 8}
```

```
>>> x = 5
```

```
>>> a.remove(x) # remove de a o valor contido em x
```

```
>>> a
```

```
{1, 3, 7, 8}
```

```
>>> a.pop() # retorna e remove algum elemento de a
```

```
1
```

```
>>> a
```

```
{3, 7, 8}
```

```
>>> b
```

```
{2, 10, 4, 6}
```

```
>>> a.isdisjoint(b) # a interseção de a e b é vazia
```

```
True # e
```

```
>>> a.intersection(b) # este método comprova isso
```

```
set()
```

```
>>> a.issubset(range(1, 10)) # testa se a é subconjunto de um
```

```
True # range
```

```
>>> a = {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
>>> L = [5, 9, 'a']
```

```
>>> a.union(L) # é possível usar os métodos com listas
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 'a'} # e tuplas
```

Contando com esses recursos, os conjuntos da linguagem Python são poderosos e flexíveis, mas apresentam uma limitação. Só podem ser membros de um conjunto os tipos de objetos imutáveis, ou seja, listas e dicionários, por serem mutáveis, não podem fazer parte de conjuntos.

6.2.3 Uso de conjunto como iterador

Os conjuntos podem ser utilizados como iteradores, como ilustrado no Exemplo 6.6. A cada repetição, o objeto x assume um dos elementos de c.

Exemplo 6.6 Uso de conjunto como iterador

```
print("Início do Programa")
```

```
cont = 1
```

```
c = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} # c é um set
```

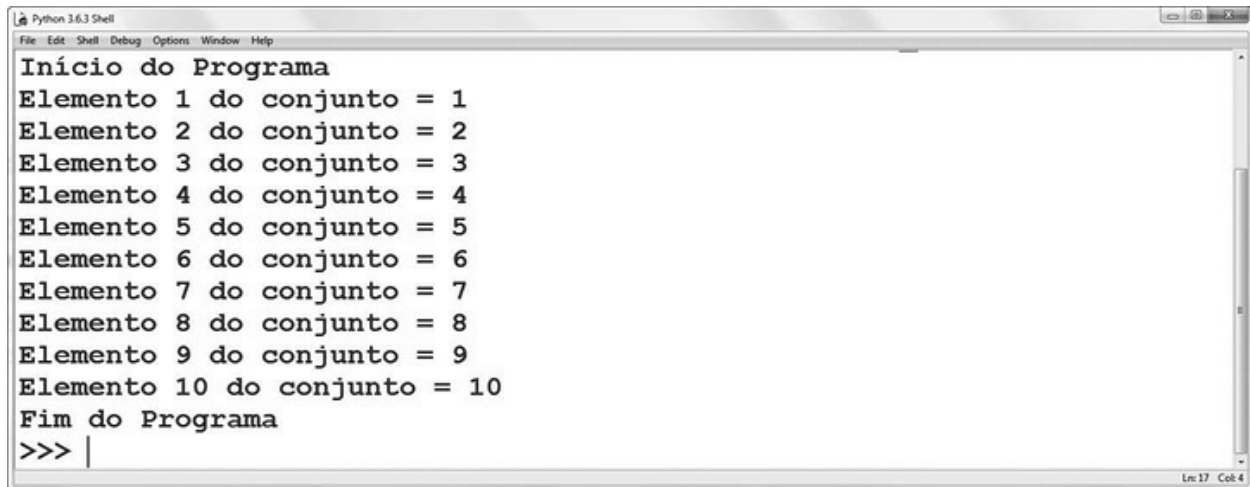
```
for x in c:
```

```
    print("Elemento {} do conjunto = {}".format(cont, x))
```

```
    cont += 1
```



```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Elemento 1 do conjunto = 1
Elemento 2 do conjunto = 2
Elemento 3 do conjunto = 3
Elemento 4 do conjunto = 4
Elemento 5 do conjunto = 5
Elemento 6 do conjunto = 6
Elemento 7 do conjunto = 7
Elemento 8 do conjunto = 8
Elemento 9 do conjunto = 9
Elemento 10 do conjunto = 10
Fim do Programa
>>> |
```

6.2.4 Exercícios resolvidos – conjuntos

1. Operações de união e interseção com conjuntos

Escreva um programa que leia do teclado dois conjuntos de números inteiros digitados pelo usuário. Exiba na tela a união e a interseção desses conjuntos.

Nessa solução, os conjuntos C1 e C2 foram carregados com os valores lidos do teclado, e ao término os operadores união (|) e interseção (&) foram utilizados para produzir o resultado pedido.

Exercício resolvido 6.1 – Operações com conjuntos

```
Msg = "Digite Valor: "
```

```
print("Dados do primeiro conjunto")
```

```
C1 = set()
```

```
x = int(input(Msg))
```

```
while x != 0:

C1.add(x)

x = int(input(Msg))

print("Dados do segundo conjunto")

C2 = set()

x = int(input(Msg))

while x != 0:

C2.add(x)

x = int(input(Msg))

print("Conjunto 1: {}".format(C1))

print("Conjunto 2: {}".format(C2))

print("União de C1 e C2")

print(C1 | C2)

print("Interseção de C1 e C2")

print(C1 & C2)

print("\nFim do programa")
```

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Dados do primeiro conjunto
Digite Valor: 2
Digite Valor: 10
Digite Valor: 15
Digite Valor: 18
Digite Valor: 23
Digite Valor: 25
Digite Valor: 0
Dados do segundo conjunto
Digite Valor: 10
Digite Valor: 15
Digite Valor: 20
Digite Valor: 25
Digite Valor: 0
Conjunto 1: {2, 10, 15, 18, 23, 25}
Conjunto 2: {25, 10, 20, 15}
União de C1 e C2
{2, 10, 15, 18, 20, 23, 25}
Interseção de C1 e C2
{25, 10, 15}

Fim do programa
>>> |
```

2. Conjuntos complementares

Considere que todos os valores no intervalo fechado $[1, 30]$ devam ser divididos em dois grupos, A e B, de 15 valores cada, de maneira aleatória. É importante que um valor que está em A não esteja em B, e vice-versa, bem como que todos os valores do intervalo estejam em algum grupo. Escreva um programa que use os recursos de conjuntos para atingir esse resultado.

Exercício resolvido 6.2 – Conjuntos complementares

```
from random import randint
```

```
A = set()
```

```
while len(A) < 15:
```

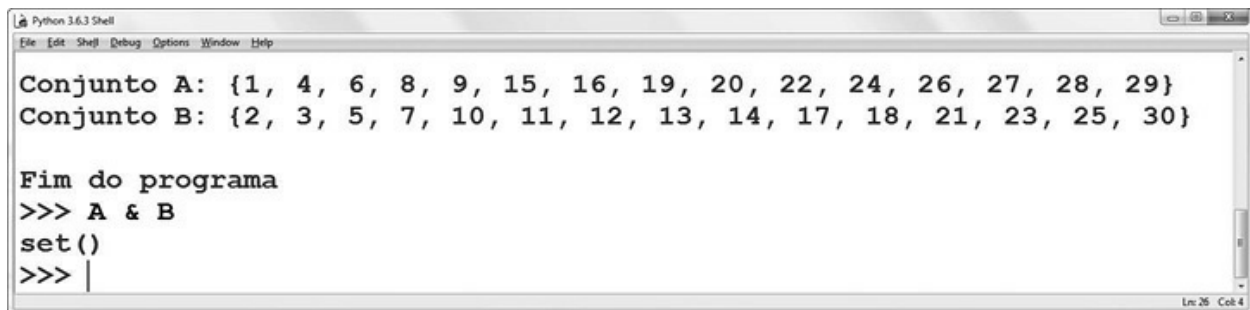
```
A.add(randint(1, 30))
```

```
B = set(range(1, 31)) - A

print("Conjunto A: {}".format(A))

print("Conjunto B: {}".format(B))

print("\nFim do programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Conjunto A: {1, 4, 6, 8, 9, 15, 16, 19, 20, 22, 24, 26, 27, 28, 29}
Conjunto B: {2, 3, 5, 7, 10, 11, 12, 13, 14, 17, 18, 21, 23, 25, 30}

Fim do programa
>>> A & B
set()
>>> |
```

Nessa solução, foi criado o conjunto vazio A e, em seguida, este foi preenchido com 15 valores no intervalo [1, 30].

Há alguns detalhes implícitos no laço while que são importantes para o correto funcionamento desse programa:

1. A função `randint(1, 30)` gera os números aleatórios nessa faixa, incluindo o 30. Essa função não é como *range*, que não inclui o valor final.
2. Eventualmente, a função `randint` pode gerar um valor repetido. Porém, como A é um conjunto, a tentativa de adicionar um valor repetido não é bem-sucedida.
3. Em virtude do aspecto do item 2, o conjunto não aumenta de tamanho quando é gerado um número repetido, de modo que o uso da função `len` para verificar o tamanho do conjunto A garante que o laço só termine quando o conjunto tiver 15 elementos.

Após a criação do conjunto A, é preciso criar o conjunto B contendo os valores que não estão em A. Para isso foi utilizado `set(range(1, 31))`, que gera todos os valores no intervalo [1, 30] já convertido para conjunto, do qual foi feita a diferença (operador “-”) com o conjunto A. No final é mostrado que $A \& B = ()$.

6.3 Dicionários

Os dicionários são, junto com as listas, os tipos de objetos mais flexíveis em Python. De um lado, as listas são sequenciais e o programador dispõe do índice para ter acesso individualizado a seus elementos. Por sua vez, os dicionários são não sequenciais, porém, permitem que um elemento individual seja acessado por meio de uma chave. Essa chave pode ser qualquer objeto *hashable*. A cada chave deve ser associado um conteúdo que pode ser qualquer objeto Python, imutável ou mutável.

O programador pode dispor dos dicionários da maneira que necessitar. É possível acrescentar e remover elementos, alterar o conteúdo de um elemento, verificar se um elemento está ou não presente, iterar sobre os membros do dicionário etc.

A sintaxe para uso dos dicionários assemelha-se muito à sintaxe utilizada para as listas, com a diferença de que, no lugar do índice, utiliza-se a chave. Diferentes membros de um dicionário podem ter diferentes tipos de chaves. No Exemplo 6.7 é criado um dicionário `d` contendo dois elementos: o primeiro tem chave numérica 442 e conteúdo string “Elemento 442”; o segundo tem chave numérica 513 e conteúdo string “Elemento 513”. Em seguida, é adicionado um novo membro com chave 377 e conteúdo “Elemento 377”. Para essa adição de novo elemento basta escrever uma linha com a seguinte estrutura: `ObjetoDicionário[chave] = conteúdo`.

Exemplo 6.7 Primeiros usos de dicionário

```
>>> d = {442:"Elemento 442", 513:"Elemento 513"}
```

```
>>> d
```

```
{442: 'Elemento 442', 513: 'Elemento 513'}
```

```
>>> d[442] # exhibe o conteúdo cuja chave é 442
```

```
'Elemento 442'
```

```
>>> d[513] # exhibe o conteúdo cuja chave é 513
```

```
'Elemento 513'
```

```
>>> d[377] = "Elemento 377" # adiciona um novo elemento
```

```
>>> d
```

```
{442: 'Elemento 442', 513: 'Elemento 513', 377: 'Elemento 377'}
```

```
>>> d['ab'] = (2, 4, 6)
```

```
>>> d
```

```
{442: 'Elemento 442', 513: 'Elemento 513', 377: 'Elemento 377', 'ab':  
(2, 4, 6)}
```

Por fim, no Exemplo 6.7 propositalmente foi adicionado um quarto membro com uma chave que foge ao padrão que vinha sendo usado, justamente para mostrar que não há padrão necessário. Nesse quarto elemento a chave é o string “ab” e o conteúdo é a tupla (2, 4, 6).

6.3.1 Métodos do tipo dicionário

O tipo dicionário apresenta diversos métodos que estão descritos no Quadro 6.3 e exemplificados em seguida.

	Considere-se disponível o dicionário D
Método	Descrição
D.clear()	Remove todos os elementos do dicionário.
D.copy()	Retorna uma cópia do dicionário.
D.fromkeys(i, v=None)	Recebe o iterável i como parâmetro e retorna um novo dicionário tendo os elementos do iterável como chave. Se o segundo parâmetro (opcional) for fornecido, cada valor será inicializado com ele.
D.get(k [,d])	Retorna o valor associado com a chave k passada como parâmetro. Caso a chave não esteja presente, retorna o segundo parâmetro (opcional) e, na ausência deste, retorna <i>None</i> (na prática não faz nada).
D.items()	Retorna um conjunto contendo os itens do dicionário (o par chave:valor). Esse retorno é do tipo dict_items e se assemelha ao tipo set, podendo ser utilizado como tal.
D.keys()	Retorna um conjunto contendo as chaves do dicionário. Esse retorno é do tipo dict_keys e se assemelha ao tipo set, podendo ser utilizado como tal.
D.pop(k [,d])	Remove a chave k e retorna o valor a ela associado. Caso k não esteja presente, o segundo parâmetro (opcional) é retornado e, na ausência deste, gera a exceção <i>KeyError</i> .
D.popitem()	Remove do dicionário e retorna um par chave:valor arbitrário. Se o dicionário estiver vazio, gera a exceção <i>KeyError</i> .
D.setdefault(k [,d])	Se a chave k estiver presente, seu retorno funciona como um get(k, d). Caso contrário, inclui o par k:d no dicionário, em que d é opcional e, em sua ausência, é utilizado <i>None</i> . Esse método representa uma maneira alternativa de inicializar um dicionário.

D.update(E)	Atualiza o dicionário D a partir dos itens contidos no dicionário E. Se algum item de E não estiver em D, então, será incluído, e caso esteja será atualizado. O parâmetro E também pode ser uma lista ou tupla que contenha seus elementos na forma (e1, e2).
D.values()	Retorna um conjunto contendo os valores do dicionário. Esse retorno é do tipo dict_values e se assemelha ao tipo set, podendo ser utilizado como tal.

Quadro 6.3 Métodos da classe “dict” disponíveis ao programador.

O modo mais elementar de inserir um par chave:valor em um dicionário é criar uma expressão `D[chave] = valor`. Se a chave não existir, ocorrerá a inserção. Essas operações estão feitas no bloco 1 do Exemplo 6.8. No bloco 2 foi utilizado o método `fromkeys` para gerar um dicionário a partir de uma tupla preexistente. Cada elemento da tupla foi utilizado como chave e, se o segundo parâmetro opcional for utilizado, ele será o valor atribuído a todos os membros. No exemplo, `fromkeys` é utilizado duas vezes, sem e com o segundo parâmetro.

Exemplo 6.8 Operações básicas com dicionários

```
# bloco 1
```

```
>>> D = {} # define o dicionário D vazio
```

```
>>> D['aa'] = 'Valor 1' # adiciona um par chave:valor
```

```
>>> D['ab'] = 'qq.coisa' # adiciona um par chave:valor
```

```
>>> D
```

```
{'aa': 'Valor 1', 'ab': 'qq.coisa'}
```



```
>>> D['ab'] = 'Valor 2' # altera o valor da chave = 'ab'
```

```
>>> D
```

```
{'aa': 'Valor 1', 'ab': 'Valor 2'}
```

```
# bloco 2
```

```
>>> T = (16, 12, 25, 14) # define-se uma tupla T
```

```
>>> A = dict.fromkeys(T) # o método fromkeys pode ser
```

```
>>> A # usado para gerar um dicionário
```

```
{16: None, 12: None, 25: None, 14: None} # sem o segundo parâmetro
```

```
>>> A = dict.fromkeys(T, 0)
```

```
>>> A
```

```
{16: 0, 12: 0, 25: 0, 14: 0} # com o segundo parâmetro
```

```
# bloco 3
```

```
>>> B = {'Nome': 'Pedro', 'Idade': 32, 'Profissão': 'Professor'}
```

```
>>> B
```

```
{'Nome': 'Pedro', 'Idade': 32, 'Profissão': 'Professor'}
```

```
>>> B.keys()
```

```
dict_keys(['Nome', 'Idade', 'Profissão'])
```

```

>>> B.values()

dict_values(['Pedro', 32, 'Professor']) >>> B.items()
dict_items([('Nome', 'Pedro'), ('Idade', 32), ('Profissão', 'Professor')])

# bloco 4

>>> E = {'Local': 'FATEC-SP', 'Cidade': 'São Paulo', 'Idade': 42}

>>> E

{'Local': 'FATEC-SP', 'Cidade': 'São Paulo', 'Idade': 42}

>>> B.update(E)

>>> B

{'Nome': 'Pedro', 'Idade': 42, 'Profissão': 'Professor', 'Local':
'FATEC-SP', 'Cidade': 'São Paulo'}

```

No bloco 3 foi definido um novo dicionário para exemplificar o uso dos métodos `keys`, `values` e `items`, que, respectivamente, retornam um conjunto de chaves, de valores e de tuplas formadas pelo par chave:valor. No bloco 4 foi definido um novo dicionário **E**, e este foi usado para atualizar o dicionário **B**. Note a atualização da Idade, que em **E** tem um valor diferente.

6.3.2 Uso de dicionário como iterador

Os dicionários também podem ser usados como iteradores em um comando `for`, sendo que, no caso dos dicionários, há diversas possibilidades para isso, conforme mostrado a seguir.

Caso 1: a cada repetição, o objeto de controle `x` assume a chave de um item

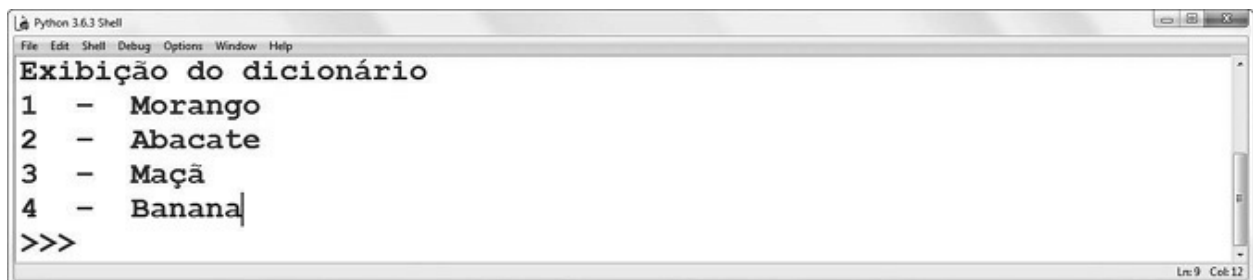
do dicionário. O valor pode ser acessado por meio de `D[x]`.

```
D = {1:'Morango', 2:'Abacate', 3:'Maçã', 4:'Banana'}
```

```
print("Exibição do dicionário")
```

```
for x in D: # iteração é feita com D
```

```
print(x, ' - ', D[x])
```

A screenshot of a Python 3.6.3 Shell window. The window title is "Python 3.6.3 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area displays the output of the code: "Exibição do dicionário" followed by four lines: "1 - Morango", "2 - Abacate", "3 - Maçã", and "4 - Banana". The prompt ">>>" is visible at the bottom left. The status bar at the bottom right shows "Ln: 9 Col: 12".

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Exibição do dicionário
1 - Morango
2 - Abacate
3 - Maçã
4 - Banana
>>>
```

Caso 2: as iterações são feitas com o retorno do método `keys`. O resultado é exatamente o mesmo exibido anteriormente.

```
D = {1:'Morango', 2:'Abacate', 3:'Maçã', 4:'Banana'}
```

```
print("Exibição do dicionário")
```

```
for x in D.keys(): # iteração é feita com D.keys()
```

```
print(x, ' - ', D[x])
```

Os casos 1 e 2, na prática, são um caso só. Quando o método `keys` é omitido o interpretador o assume por padrão.

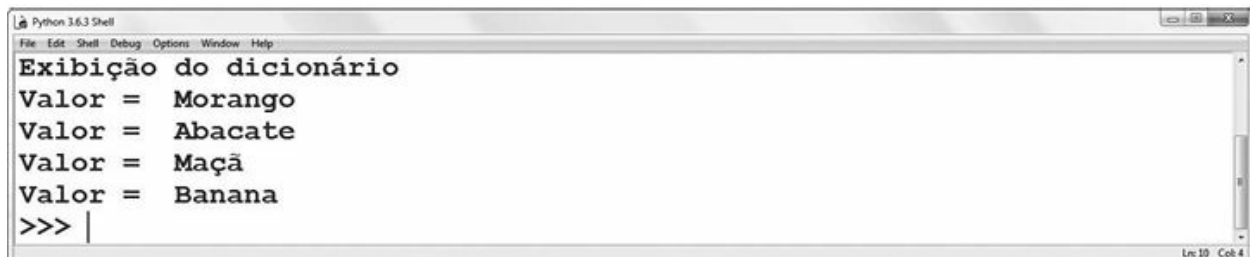
Caso 3: as iterações são feitas com o retorno do método `values`. Assim, cada repetição o objeto de controle `x` assume o valor de um item do dicionário. Neste caso, não é possível acessar a chave.

```
D = {1:'Morango', 2:'Abacate', 3:'Maçã', 4:'Banana'}
```

```
print("Exibição do dicionário")
```

```
for x in D.values():
```

```
    print('Valor = ', x)
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Exibição do dicionário
Valor = Morango
Valor = Abacate
Valor = Maçã
Valor = Banana
>>> |
```

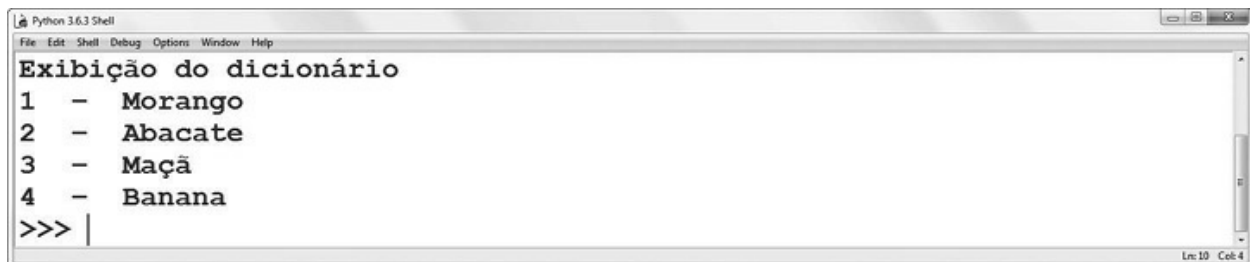
Caso 4: as iterações são feitas com o retorno do método items. Como esse método retorna tuplas contendo (chave, valor), então, podem-se utilizar dois objetos para receber cada um dos elementos da tupla. Esse programa utiliza um recurso diferente, porém, o resultado produzido é igual aos casos 1 e 2.

```
D = {1:'Morango', 2:'Abacate', 3:'Maçã', 4:'Banana'}
```

```
print("Exibição do dicionário")
```

```
for numero, nome in D.items():
```

```
    print(numero, ' - ', nome)
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Exibição do dicionário
1 - Morango
2 - Abacate
3 - Maçã
4 - Banana
>>> |
```

6.3.3 Exercícios resolvidos – dicionários

3. Construção de um dicionário e exibição de seus dados

Escreva um programa que leia do teclado o código de uma peça e a quantidade disponível no estoque. Esses dois dados de entrada são números inteiros. Acrescente o par código:quantidade em um dicionário apenas se o código não estiver presente. Caso esteja, dê uma mensagem informando essa situação e descarte os dados. O laço termina quando for fornecido 0 para o código. Exibir na tela os dados do dicionário, um membro por linha.

Exercício resolvido 6.3 – Construção de um dicionário e exibição de seus dados

```
pecas = {}

print("Leitura dos dados")

while True:

    cod = int(input(" Digite o código: "))

    if cod == 0:

        break # interrompe o laço se cod == 0

    elif cod in pecas:

        print(" A peça {} já está no cadastro".format(cod))

    continue # próxima iteração se cod in pecas

    qtde = int(input(" Digite a quantidade: "))

    pecas[cod] = qtde # acrescenta novo item ao dicionário
```

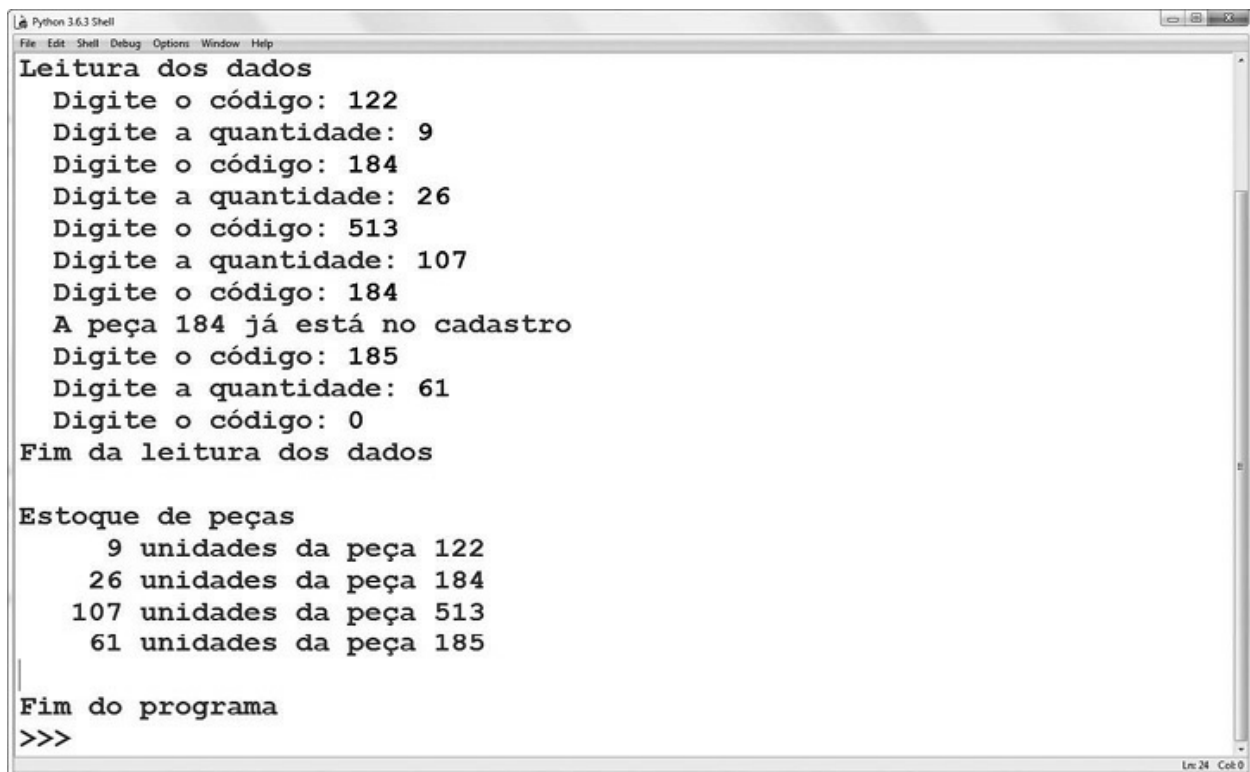
```
print("Fim da leitura dos dados\n")

print("Estoque de peças")

for c in pecas:

    print(" {1:4} unidades da peça {0}".format(c, pecas[c]))

print("\nFim do programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Leitura dos dados
  Digite o código: 122
  Digite a quantidade: 9
  Digite o código: 184
  Digite a quantidade: 26
  Digite o código: 513
  Digite a quantidade: 107
  Digite o código: 184
  A peça 184 já está no cadastro
  Digite o código: 185
  Digite a quantidade: 61
  Digite o código: 0
Fim da leitura dos dados

Estoque de peças
  9 unidades da peça 122
  26 unidades da peça 184
  107 unidades da peça 513
  61 unidades da peça 185

Fim do programa
>>>
```

Figura 6.1 Execução do Exercício resolvido 6.3.

Nessa solução há aspectos que merecem comentários. O dicionário `pecas` é inicializado vazio e ganha elementos a cada repetição do laço `while`. Esse laço foi construído de modo a ser sempre verdadeiro. O comando condicional `if cod == 0` dentro do laço garante que quando zero é digitado para o código o comando `break` o interrompe. Essa é uma técnica que foi apresentada no

Exemplo 3.7, muito prática em Python, porém, nem sempre é bem-vista por programadores que aprenderam a programar usando outras linguagens. Se este é o seu caso, sugere-se que o modifique para que fique mais ao seu estilo.

O `elif cod in pecas` verifica se o código digitado já está contido no dicionário. Caso esteja, emite a mensagem e prossegue para a próxima iteração com o comando `continue`.

Por fim, o laço `for` implementa o caso 1, visto na Seção 6.3.2, de iteração com o dicionário `pecas`, tendo como “`c`” objeto de controle, de modo que `c` assume o valor da chave, e `pecas[c]`, seu valor.

```
for c in pecas:
```

```
    print(“ {1:4} unidades da peça {0}”.format(c, pecas[c]))
```

4. Construção de um dicionário mais complexo

Primeira solução

Escreva um programa que permaneça em laço efetuando a leitura dos seguintes dados: número de matrícula, nome do aluno, idade e curso. O número de matrícula é a chave, e os demais dados constituem o valor. Faça a leitura desses dados e construa o dicionário enquanto não for digitado zero para o número de matrícula.

Nesse problema, têm-se múltiplos dados como valor para cada membro do dicionário, sendo necessário agrupá-los de alguma maneira. Para esse agrupamento pode-se utilizar uma lista, uma tupla, um conjunto ou outro dicionário aninhado.

No Exercício resolvido 6.4 será apresentada, inicialmente, uma solução na qual se utiliza uma tupla. No final apresenta-se a mudança necessária para utilizar lista no lugar da tupla e discute-se um pouco sobre a escolha de uma

ou outra alternativa.

No Exercício resolvido 6.5 será apresentada uma solução com dicionários aninhados.

Assim, nesta primeira solução a tupla será organizada de modo que seu primeiro elemento seja o Nome, o segundo seja a Idade e o terceiro seja o Curso do aluno. Veja o código do programa a seguir.

Exercício resolvido 6.4 – Construção de um dicionário mais complexo

```
Alunos = {} # cria o dicionário
```

```
print("Leitura dos dados")
```

```
while True: # laço da 1ª parte - leitura
```

```
    matr = int(input(" Digite a matrícula: "))
```

```
    if matr == 0:
```

```
        break
```

```
    elif matr in Alunos:
```

```
        print(" A matrícula {} já está no cadastro".format(matr))
```

```
    continue
```

```
    nome = input(" Nome: ")
```

```
    idade = int(input(" Idade: "))
```

```
    curso = input(" Curso: ")
```



```
Alunos[matr] = (nome, idade, curso) # linha 13

print("Fim da leitura dos dados\n")

print("Cadastro de Alunos") # 2ª parte – apresentação dos dados

for matricula, dados in Alunos.items():

    print(" Aluno {}".format(dados[0]))

    print(" nºmatr {}".format(matricula))

    print(" idade {}".format(dados[1]))

    print(" curso {}".format(dados[2]))

    print("\nFim do programa")
```

A execução desse programa é mostrada em duas etapas, nas Figuras 6.2 e 6.4. Na primeira parte o programa permanece em laço enquanto forem digitados valores diferentes de zero para o objeto matr, que é o número da matrícula. Cada conjunto de dados lido é armazenado em objetos temporários identificados por nome, idade e curso, e na linha 13 é feita a inserção no dicionário usando-se o par matr:(nome, idade, curso):

`matr: (nome, idade, curso)`

↑
Tupla como valor do membro do dicionário

Chave do dicionário – não pode ser repetida

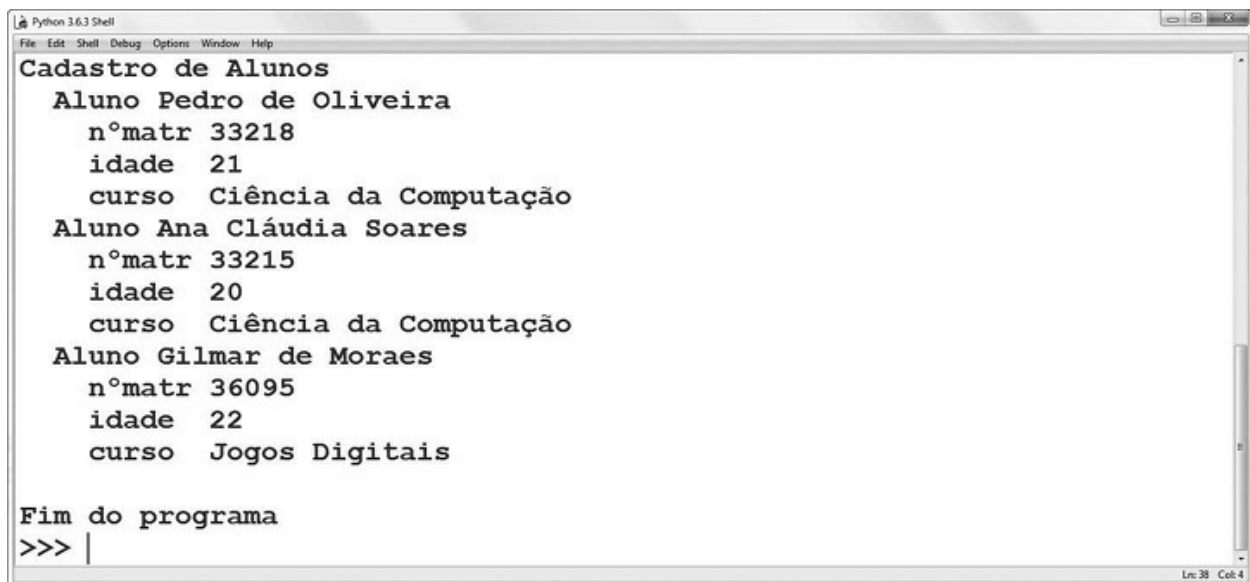
```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Leitura dos dados
Digite a matrícula: 33218
Nome: Pedro de Oliveira
Idade: 21
Curso: Ciência da Computação
Digite a matrícula: 33218
A matrícula 33218 já está no cadastro
Digite a matrícula: 33215
Nome: Gilmar de Moraes
Idade: 22
Curso: Jogos Digitais
Digite a matrícula: 0
Fim da leitura dos dados
Ln: 30 Col: 4
```

Figura 6.2 Execução do Exercício resolvido 6.4 – etapa de leitura dos dados.

Durante a leitura é feita a verificação se o número de matrícula digitado já está no dicionário. Em caso positivo, é dada uma mensagem informando a situação. Caso não esteja, os demais dados são lidos e é feita a inclusão no dicionário. Terminado o laço de leitura, o dicionário estará carregado com todos os dados fornecidos. Ilustrativamente, ele pode ser entendido como mostrado na Figura 6.3.

```
Alunos = {
    33218: ('Pedro de Oliveira', 21, 'Ciência da Computação'),
    33215: ('Ana Cláudia Soares', 20, 'Ciência da Computação'),
    36095: ('Gilmar de Moraes', 22, 'Jogos Digitais')
}
```

Figura 6.3 Ilustração de um dicionário carregado, no qual a chave é um número inteiro e o valor é uma tupla.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Cadastro de Alunos
  Aluno Pedro de Oliveira
    n°matr 33218
    idade  21
    curso  Ciência da Computação
  Aluno Ana Cláudia Soares
    n°matr 33215
    idade  20
    curso  Ciência da Computação
  Aluno Gilmar de Moraes
    n°matr 36095
    idade  22
    curso  Jogos Digitais

Fim do programa
>>> |
```

Figura 6.4 Execução do Exercício resolvido 6.4 – etapa de exibição dos dados lidos.

Na Figura 6.4 é mostrado o resultado da execução da segunda parte do programa, que se inicia na linha 15 e é responsável pela exibição em tela. Para conseguir tal resultado, deve-se ter acesso a cada parcela de informação contida no dicionário. Foi criado o laço iterador `for` destacado a seguir. Observe atentamente a estrutura desse comando. Nele foram utilizados dois objetos, `matricula` e `dados`, para receber o retorno do método `items` do dicionário, a cada repetição conforme ilustrado na Figura 6.5.

```
for matricula, dados in Alunos.items():
```

```
    print(" Aluno {}".format(dados[0]))
```

```
    print(" n°matr {}".format(matricula))
```

```
    print(" idade {}".format(dados[1]))
```

```
    print(" curso {}".format(dados[2]))
```

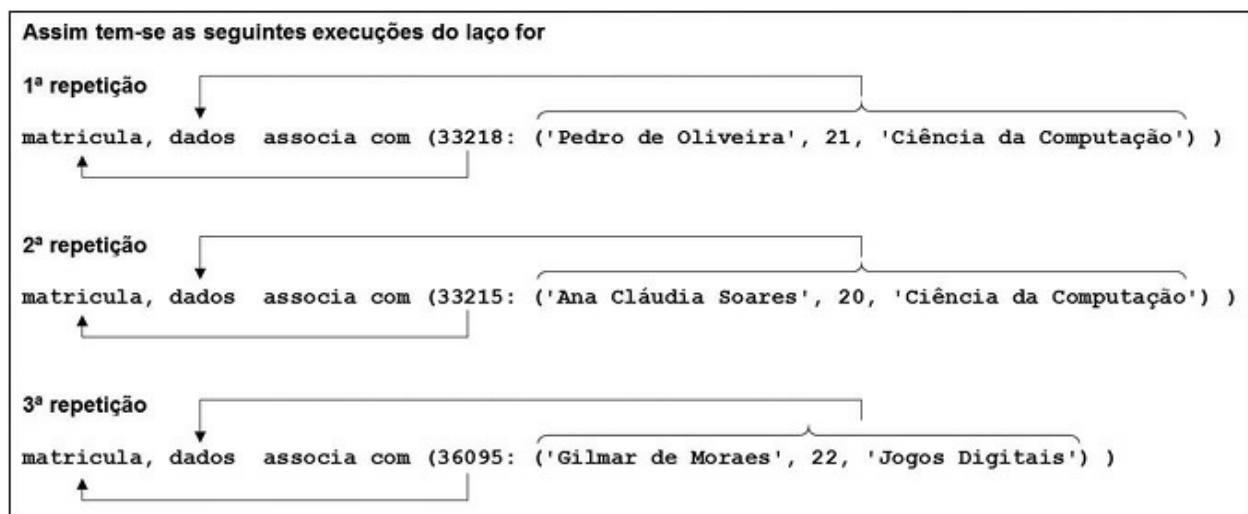


Figura 6.5 Esquema de atribuição do retorno produzido pelo método `items()` do dicionário.

Pode-se ver que o objeto `dados` é uma tupla que contém as três informações associadas ao número de matrícula do aluno, e o acesso individual é feito por meio de índices, sendo que:

`dados[0]` contém o nome do aluno

`dados[1]` contém a idade

`dados[2]` contém o curso

Nessa solução utilizaram-se um número inteiro para a chave do dicionário e uma tupla para o valor associado à chave. Caso necessite trabalhar com uma lista no lugar da tupla, basta fazer uma substituição direta na linha 13 do programa, substituindo

```
Alunos[matr] = (nome, idade, curso) # linha 13
```

por (note a sutil diferença da troca dos parênteses pelos colchetes).

```
Alunos[matr] = [nome, idade, curso] # linha 13
```

Efetuada essa pequena mudança, o programa do Exercício resolvido 6.4

funciona exatamente do mesmo modo. Mas há uma grande diferença potencial entre as duas versões.

Lembre-se que as listas são mutáveis, ao passo que as tuplas são imutáveis. Isso implica que, ao utilizar a lista, o programador tem a possibilidade de alterar o conteúdo dos elementos de dados. Com a tupla, essa possibilidade não existe.

Suponha o dicionário carregado com os dados utilizados nos exemplos trazidos até o momento. Utilizando o IDLE pode-se demonstrar essa possibilidade de alteração de dados. No Exemplo 6.9 é mostrada uma sequência de comandos em que o curso do aluno 33218 é alterado duas vezes.

Exemplo 6.9 Uso de lista como valor do membro do dicionário permite alteração de dados

```
>>> print(Alunos[33218])

['Pedro de Oliveira', 21, 'Ciência da Computação']

>>> dados = Alunos[33218]

>>> print(dados)

['Pedro de Oliveira', 21, 'Ciência da Computação']

>>> dados[2] = 'Administração de Empresas'

>>> print(Alunos[33218])

['Pedro de Oliveira', 21, 'Administração de Empresas']

>>> # ou diretamente
```

```
>>> Alunos[33218][2] = 'Engenharia de Produção'
```

```
>>> print(Alunos[33218])
```

```
['Pedro de Oliveira', 21, 'Engenharia de Produção']
```

Com tuplas, tais alterações não são possíveis. Porém, se o dicionário for passado como parâmetro para diversas funções ao longo do programa, o uso de tuplas pode garantir uma melhor proteção contra alterações acidentais. Em geral, as tuplas garantem mais integridade aos dados e as listas garantem mais flexibilidade. Não é correto afirmar taxativamente que tupla é melhor que lista, ou vice-versa. Ao modelar seu programa o programador precisa decidir qual utilizar em função das necessidades e requisitos do programa que se está desenvolvendo. A melhor opção dependerá de cada caso específico.

Segunda solução

Nessa segunda solução nada muda com relação ao número de matrícula, ao passo que os dados passam a ser armazenados em um dicionário aninhado ao dicionário Alunos.

Exercício resolvido 6.5 – Construção de uma estrutura de dados com dicionários aninhados

```
Alunos = {} # cria o dicionário principal
```

```
print("Leitura dos dados")
```

```
while True: # laço da 1ª parte - leitura
```

```
    matr = int(input(" Digite a matrícula: "))
```

```
    if matr == 0:
```

break

elif matr in Alunos:

print(" A matrícula {} já está no cadastro".format(matr))

continue

dicItem = {} # cria o dicionário aninhado

dicItem['nome'] = input(" Nome: ") # carrega os dados

dicItem['idade'] = int(input(" Idade: ")) # no dicionário

dicItem['curso'] = input(" Curso: ") # aninhado

Alunos[matr] = dicItem # linha 14

print("Fim da leitura dos dados\n")

print("Cadastro de Alunos") # 2ª parte – apresentação dos dados

for matricula, dados in Alunos.items():

print(" Aluno {}".format(dados['nome']))

print(" nºmatr {}".format(matricula))

print(" idade {}".format(dados['idade']))

print(" curso {}".format(dados['curso']))

print("\nFim do programa")

Nesta versão dispensam-se os objetos temporários nome, idade, curso, por não serem mais necessários. Em seu lugar é feita a atribuição direta do conteúdo lido do teclado a um membro do dicionário aninhado dicItem, conforme pode ser visto nas linhas destacadas:

```
dicItem['nome'] = input(" Nome: ") # carrega os dados
```

```
dicItem['idade'] = int(input(" Idade: ")) # no dicionário
```

```
dicItem['curso'] = input(" Curso: ") # aninhado
```

Após uma execução desses três comandos o dicionário dicItem tem o conteúdo em seguida, ou seja, três pares chave:valor, em que a chave é um string identificador e valor é o conteúdo digitado.

```
{ ' nome ' : 'Pedro de Oliveira', 'idade' : 21, 'curso' : 'Ciência da Computação' }
```

Ao executar a linha 14 (destacada a seguir) o dicionário dicItem é inserido no dicionário Alunos como valor associado ao número de matrícula, e o resultado é o mostrado na ilustração.

```
Alunos[matr] = dicItem # linha 14
```

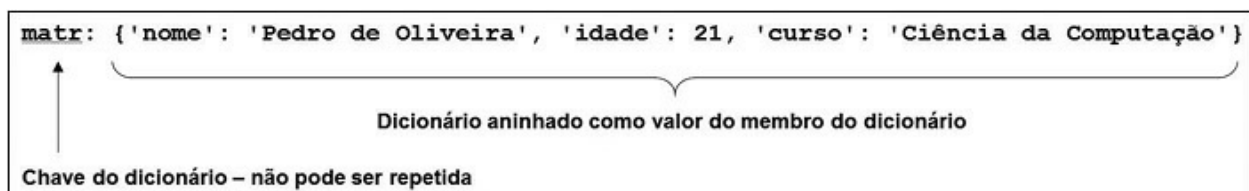


Figura 6.6 Ilustração de um membro do dicionário Alunos, no qual a chave é um número inteiro e o valor é um dicionário aninhado.

No final da leitura, o dicionário principal Alunos estará carregado da seguinte maneira:


```
Alunos = {  
    33218: {'nome': 'Pedro de Oliveira', 'idade': 21, 'curso': 'Ciência da Computação'},  
    33215: {'nome': 'Ana Cláudia Soares', 'idade': 20, 'curso': 'Ciência da Computação'},  
    36095: {'nome': 'Gilmar de Moraes', 'idade': 22, 'curso': 'Jogos Digitais'}  
}
```

Figura 6.7 Ilustração do dicionário Alunos completo contendo três membros, nos quais a chave é um número inteiro e o valor é um dicionário aninhado.

O resultado da execução da segunda parte do programa é exatamente o mesmo resultado da versão anterior exibido na Figura 6.4. A segunda parte dessa versão tem início na linha 16, em que foi criado o laço iterador `for` destacado. Observe atentamente a estrutura desse comando.

```
for matricula, dados in Alunos.items():  
  
    print(" Aluno {}".format(dados['nome']))  
  
    print(" nºmatr {}".format(matricula))  
  
    print(" idade {}".format(dados['idade']))  
  
    print(" curso {}".format(dados['curso']))
```

Nele foram usados dois objetos, `matricula` e `dados`, sendo que, neste caso, o objeto `dados` é o dicionário aninhado e o acesso ao seu conteúdo se faz pela chave desejada.

`dados['nome']` contém o nome do aluno

`dados['idade']` contém a idade

`dados['curso']` contém o curso

Considerando essa parte do código, faça uma comparação entre os

Exercícios resolvidos 6.4 e 6.5. Verifique, neste último caso, que a referência ao dado é feita por meio de um string. No caso das tuplas ou listas é preciso usar um índice numérico. Muitos programadores preferem esta última forma, pois é mais fácil lembrar onde está o dado pelo nome do que por um número. Dicionários aninhados serão muito utilizados no Capítulo 9.

Exercícios propostos

1. Escreva um programa que leia do teclado dois conjuntos de nomes de frutas. Em seguida, apresente na tela a união (operador $|$), a interseção ($\&$) e a diferença simétrica ($\wedge$). Utilizando a função `len`, verifique qual dos conjuntos tem mais elementos e apresente a diferença entre o maior e o menor ($-$).
2. Altere o programa do Exercício resolvido 6.2 da seguinte maneira: leia um número inteiro N ($N < 30$) e faça que o conjunto A tenha tamanho N e o conjunto B tenha tamanho $30-N$.
3. Escreva um programa que permaneça em laço lendo números inteiros do teclado. Esse laço termina quando for digitado zero ou qualquer valor negativo. O programa deve contar quantas vezes cada valor positivo foi digitado. Ao término do laço de leitura o programa deve mostrar quais valores foram digitados e quantas vezes cada um. Use um dicionário para resolver esse problema.
4. Considere o seguinte conjunto de dados: Nome + ($N1, N2, N3, N4$). Nome representa o nome de um aluno e deve ser usado como chave. $N1, N2, N3, N4$ representam as notas de provas desse aluno. Escreva um programa que leia os dados de Q alunos e apresente na tela se foram aprovados ou reprovados. O critério que garante a aprovação é

que a média aritmética das 4 notas seja maior ou igual 6,0. Q é a quantidade de alunos, e esse valor deve ser lido do teclado no começo do programa. As notas devem ser exibidas com uma casa decimal.

Sugestão de apresentação dos resultados:

Notas dos alunos

Cláudio dos Santos 6.5 3.5 5.0 6.5 5.4 Reprovado

Eduardo Falco 7.5 6.5 7.0 6.5 6.9 Aprovado

Lígia de Oliveira 8.5 7.5 8.0 7.0 7.9 Aprovado

5. Refaça o exercício 4 alterando o critério de aprovação para o seguinte: das quatro notas, despreze a menor e calcule a média aritmética das outras três. Será considerado aprovado o aluno que tiver essa média maior ou igual a 6,0.
6. Leia e armazene em um dicionário o nome, a idade e o número do telefone de seus contatos, sendo que a chave deve ser o nome. Ao digitar um string vazio para o nome, o programa interrompe a leitura. Apresente na tela os dados lidos em ordem alfabética pelo nome dos contatos. Em seguida, armazene os contatos em dois dicionários, utilizando como critério a idade: menores de 18 anos em um e os maiores em outro dicionário, eliminando o original. Apresente na tela os dois dicionários resultantes da separação.



Arquivos

Objetivos

Neste capítulo serão apresentados os conceitos relativos à gravação e à leitura de arquivos usando Python. Operações envolvendo arquivos são importantes porque eles representam uma das possíveis maneiras de armazenar dados permanentemente, gravando-os em disco.

Existem dois modos de se trabalhar com arquivos: o modo texto e o modo binário. Este segundo modo é adequado para trabalhar com arquivos que contêm imagens, sons, vídeos, arquivos compactados, entre outros tipos de conteúdo. Para trabalhar com arquivos assim, é necessário conhecer a estrutura de cada conteúdo específico, o que foge ao escopo deste capítulo.

Por outro lado, os arquivos texto podem ser abertos e editados com qualquer editor de texto básico, como o Bloco de Notas. Essa facilidade permite que os arquivos gerados com Python possam ser verificados no editor e arquivos digitados em um editor podem ser processados utilizando programas escritos em Python. Desse modo, este capítulo se concentra no trabalho com arquivos texto.

7.1 Arquivos – conceitos iniciais

Um arquivo de computador é um recurso de armazenamento de dados que está disponível em todo tipo de dispositivo computacional, seja um computador, dispositivo móvel, uma câmera fotográfica, entre outros. Os arquivos são utilizados para armazenar dados de maneira permanente, e para isso devem ser gravados em algum equipamento de hardware que não necessite estar ligado permanentemente em uma fonte de energia. Nos

tempos atuais, os equipamentos mais utilizados para esse fim são os discos magnéticos e os chips SSD.

O gerenciamento do armazenamento fica a cargo do sistema operacional (SO) instalado no dispositivo computacional. Isso permite maiores segurança, padronização e organização das unidades de armazenamento.

As linguagens de programação, por sua vez, contam com comandos e recursos próprios para realizar as operações de gravação e leitura dos arquivos. No entanto, não é a linguagem, qualquer que seja, que fará o acesso físico ao hardware. O que os comandos da linguagem fazem para manipular arquivos em disco é colocar em execução um conjunto de funções que fazem parte do sistema operacional. Assim sendo, o programador, ao utilizar os comandos necessários para efetuar operações de gravação e leitura, está indiretamente acessando serviços do SO.

Um grande benefício dessa abordagem para o programador é que ela permite separar, de um lado, os detalhes de implementação do sistema de arquivos que o SO utiliza e, de outro, os comandos da linguagem que o programador deve utilizar para executar as operações.

Uma vez gravado em disco, o arquivo pode ser entendido como um conjunto de bytes ao qual é atribuído um nome. Quando esse arquivo voltar a ser lido, haverá duas possíveis e distintas maneiras de interpretar seus bytes: o modo texto ou o modo binário. É possível abrir o mesmo arquivo tanto de um modo como de outro, porém, a interpretação que se terá dos bytes lidos difere, e é preciso haver coerência entre o que foi gravado e o que será lido.

- **Arquivos texto:** ao usar a linguagem Python, ao abrir um arquivo no modo texto e efetuar sua leitura, seus bytes serão lidos, decodificados e interpretados segundo uma tabela de caracteres, e o conjunto resultante será retornado como um objeto string. Por outro lado, ao realizar uma

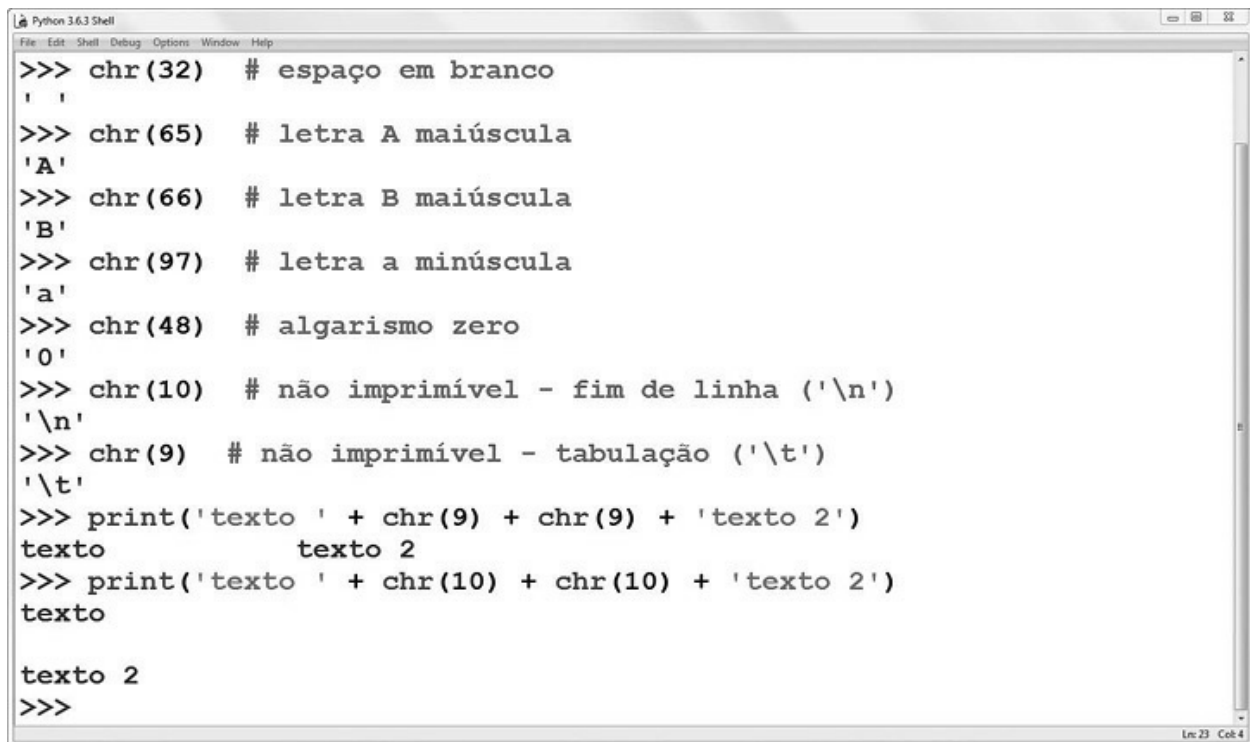
operação de gravação ocorrerá o processo inverso, no qual os caracteres de um string serão codificados, transformados em bytes e gravados no disco.

- Como exemplo de arquivos texto encontrados frequentemente tem-se: código-fonte de programas, arquivos HTML, CSS, JavaScript usados na web, arquivos XML e CSV (comma separated values) usados para intercâmbio de dados entre sistemas, arquivos TXT em geral.
- **Arquivos binários:** ao abrir um arquivo no modo binário, seus bytes são lidos e trazidos para a memória sem qualquer interpretação ou decodificação. São bytes em estado bruto, e caberá ao programador escrever o programa de maneira apropriada a fim de interpretar corretamente tais bytes.
- Como exemplo de arquivos binários comuns tem-se: arquivos de imagens, áudio e vídeo; arquivos de bancos de dados; arquivo executável de um programa compilado; arquivos compactados por meio de um algoritmo de compressão, entre outros.

O padrão ASCII (abreviação de *American Standard Code for Information Interchange*), criado na década de 1960, utiliza 7 bits para representar um caractere que é armazenado em 1 byte de 8 bits, sendo que oitavo bit não é efetivamente utilizado na codificação dos caracteres. Com esses 7 bits, é possível formar 128 números inteiros na faixa de valores de 0 a 127, sendo que cada um desses números equivale a um caractere diferente segundo a tabela padrão. Tais caracteres são letras, com diferenciação para maiúsculas e minúsculas, algarismos, pontuação e outros. São ao todo 95 sinais gráficos, conhecidos como *printables* (ou “*imprimíveis*”), e 33 sinais de controle que não têm uma aparência gráfica e, por isso, são conhecidos como *non-*

printables (“não imprimíveis”), sendo usados em dispositivos de comunicação e transferência de arquivos, bem como elementos que afetam o processamento do texto, como caractere de fim de linha (“\n”) ou tabulação (“\t”).

Em Python pode-se usar a função `chr` para converter um número inteiro para o caractere corresponde e a função `ord` para fazer a operação inversa. A Figura 7.1 ilustra alguns casos, mostrando, por exemplo, que o número 32 equivale a um espaço em branco e 65 equivale à letra “A” maiúscula. Os caracteres 9 (tabulação) e 10 (fim de linha) também são exemplificados.

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main area shows a series of Python commands and their outputs. The commands use the `chr` function to convert integers to characters. The outputs show the resulting characters, including spaces, letters, digits, and control characters like '\n' and '\t'. The window title is 'Python 3.6.3 Shell'. The status bar at the bottom right shows 'Lin 23 Col 4'.

```
>>> chr(32) # espaço em branco
' '
>>> chr(65) # letra A maiúscula
'A'
>>> chr(66) # letra B maiúscula
'B'
>>> chr(97) # letra a minúscula
'a'
>>> chr(48) # algarismo zero
'0'
>>> chr(10) # não imprimível - fim de linha ('\n')
'\n'
>>> chr(9) # não imprimível - tabulação ('\t')
'\t'
>>> print('texto ' + chr(9) + chr(9) + 'texto 2')
texto      texto 2
>>> print('texto ' + chr(10) + chr(10) + 'texto 2')
texto
texto 2
>>>
```

Figura 7.1 Exemplo de caracteres da tabela ASCII.

A tabela ASCII estruturada dessa maneira sempre foi muito apropriada para os textos no idioma inglês. No entanto, com o passar do tempo e o aumento da penetração dos computadores em todo o mundo, a tabela ASCII mostrou-se insuficiente para acomodar todos idiomas existentes e alternativas

começaram a ser buscadas.

Visando solucionar as novas demandas, o primeiro e natural passo dado foi utilizar o oitavo bit para ampliar a faixa de possibilidades, incorporando-se, assim, os códigos de 128 até 255. Este oitavo bit passou a ser usado de diversas formas distintas, com certo prejuízo de padronização: em alguns casos, era empregado para informar a paridade em transmissão assíncrona de dados; a Microsoft utilizou-o para criar seu sistema de páginas de codificação (*Windows Code Page*) entre os anos de 1980 e 1990 etc.

Um dos usos dados a esse oitavo bit foi sua incorporação à codificação, passando-se a denominá-la tabela ASCII estendida e tornando possível acomodar os caracteres acentuados típicos dos idiomas da Europa Ocidental e Américas do Sul e Central. Como esse uso foi muito intenso e relevante, erroneamente se difundiu a ideia de que a tabela ASCII foi ampliada para utilizar 8 bits. Fato este que, ao menos oficialmente, nunca ocorreu.

A busca de uma solução oficial para as limitações da tabela ASCII levou ao desenvolvimento do sistema de codificação Unicode, mantido pelo *Unicode Consortium* (ver referência UNICODE, 2017). Esse sistema permite a representação e a manipulação de texto de maneira consistente em qualquer sistema de escrita existente. Apenas 8 bits não eram suficientes para a representação de todos os caracteres de muitos idiomas, de modo que o Unicode trabalha com a opção de codificação usando 1 ou mais bytes por caractere. Em razão disso, cadeias de texto construídas utilizando-se a codificação Unicode são conhecidas no mundo da computação como *wide-character strings* ou *wide-strings*.

Para armazenar e manipular corretamente esse tipo de string são necessárias operações de codificação e decodificação que devem ser conhecidas pelos programadores. Assim, têm-se as definições:

- **Codificação de um string:** é a conversão de cada caractere do string para os bytes (de 1 a 4 bytes por caractere) que o compõem, segundo o tipo de codificação desejada.
- **Decodificação de um string:** é a conversão dos bytes que representam o caractere (de 1 a 4 bytes por caractere), gerando o caractere em si segundo o tipo de codificação desejada.

A codificação supramencionada refere-se a qual subconjunto de caracteres Unicode se deseja utilizar. No mundo ocidental, as codificações ASCII (sim, ela continua a existir e é um subconjunto do Unicode), Latin-1 e UTF-8 são as mais amplamente utilizadas.

Conhecer esses conceitos é importante para o programador, uma vez que, com frequência, precisa-se desse conhecimento para não incorrer em erros comuns. A título de exemplo, considere-se a Figura 7.2. Ela mostra a exibição de uma página html desenvolvida em duas situações de incoerência, com diferentes resultados errôneos. Do lado esquerdo, o código html especifica que a codificação usada é UTF-8, ou seja, Unicode e o arquivo texto foi salvo com codificação ANSI, que usa a tabela ASCII. Do lado direito, foi criada a situação inversa: ou seja, o html especifica que a codificação usada é “ANSI”, mas o arquivo foi com codificação UTF-8.

Ao utilizar os programas editores de texto, como *Notepad++*, no Windows, ou *Kwrite* e *Notepadqq*, no Linux, é possível especificar qual a codificação com que se quer gravar o arquivo. E, para evitar a ocorrência de erros assim, é necessário que o programador esteja atento e sempre tome o cuidado de manter a coerência entre a codificação do arquivo salvo em disco com a maneira como são interpretados os bytes lidos pelo programa que se está escrevendo.

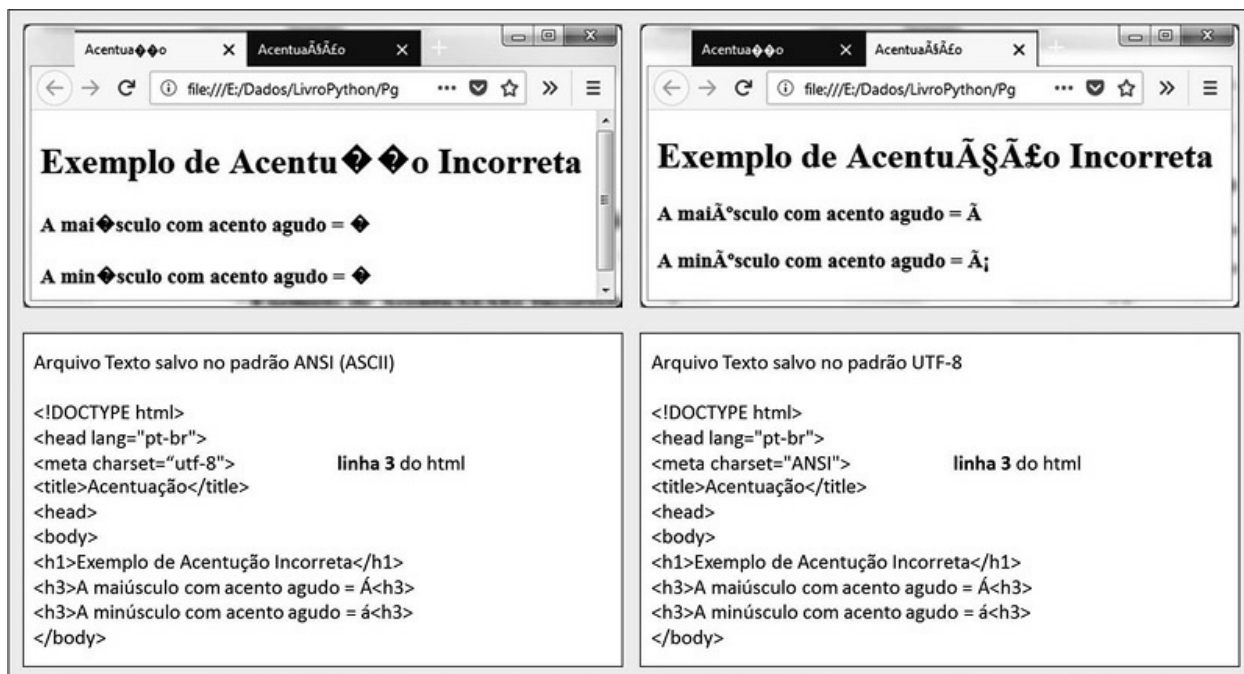


Figura 7.2 Erros frequentes relativos à codificação Unicode.

Se houver coerência entre o conteúdo do arquivo e a interpretação de seu conteúdo, então, tudo fica correto, como na Figura 7.3.

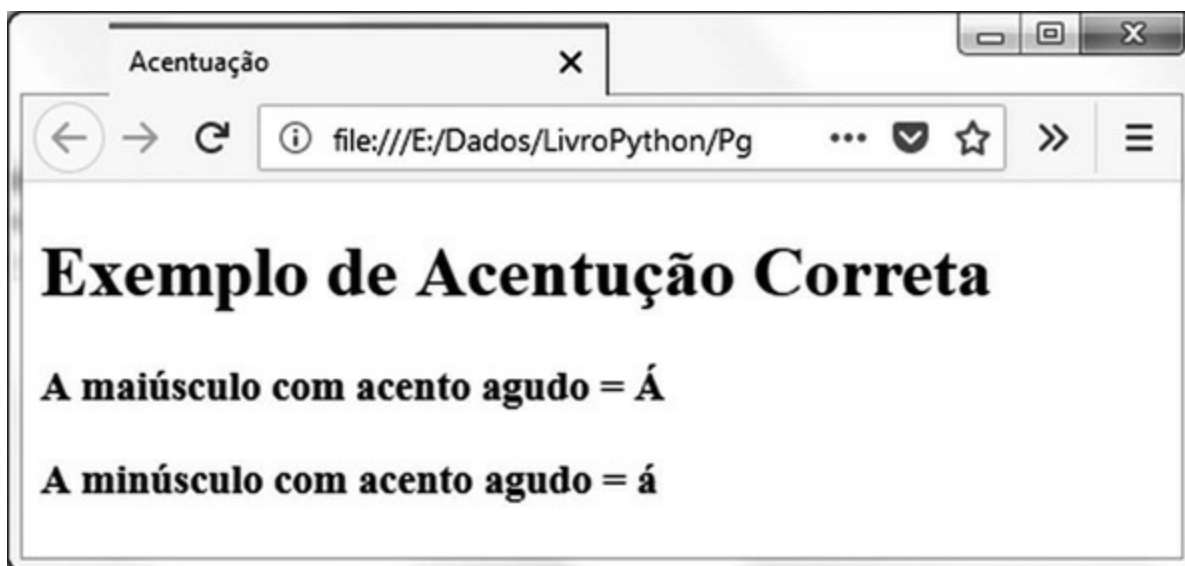


Figura 7.3 Coerência entre a codificação do arquivo e a interpretação de seu conteúdo.

A linguagem Python 3.0 oferece amplo suporte à codificação Unicode. Esse suporte está implementado na classe “str”, ou seja, nos strings já descritos no Capítulo 4. Naquele momento, nada foi dito sobre isso, pois faltavam as informações sobre codificação supra-apresentadas para tornar clara essa conceituação.

7.2 Arquivos em Python 3

Antes de começar, recomenda-se alguns cuidados. Tudo o que será tratado neste capítulo diz respeito à versão 3.x da linguagem Python. Nas versões anteriores (2.x), a forma de implementação é diferente em muitos casos. Na internet há muito material disponível sobre Python 2.x em fóruns, FAQs etc. Assim, se estiver fazendo buscas na internet com o propósito de complementar os conceitos e conteúdos aqui passados, verifique com atenção a versão de Python que está sendo discutida em cada material encontrado, pois há diferenças significativas entre Python 2.x e 3.x.

O Python conta com recursos voltados à gravação e à leitura de arquivos, sejam eles binários ou texto. Os arquivos binários não serão abordados neste livro, uma vez que demandam conhecimentos específicos de certos formatos de dados, tais como imagens ou áudio, que estão fora do escopo pretendido aqui. Todo este capítulo, daqui por diante, é dedicado aos arquivos texto.

Considere-se o Exemplo 7.1. Esse programa tem duas partes. Na primeira, grava-se o arquivo “Exemplo7_1.txt”, e na segunda parte o mesmo arquivo é lido.

A primeira providência é abrir o arquivo com o comando `open`. Esse comando recebe dois parâmetros: o nome do arquivo e o caractere indicador de modo de abertura. Para gravação, utiliza-se o caractere “w” (*write*). Neste caso, se o arquivo não existir, será criado, e caso exista, terá seu conteúdo zerado (é preciso ter cuidado com isso, pois os dados de um eventual arquivo

preexistente serão perdidos).

Como o nome do arquivo foi fornecido sem qualquer indicação de pasta, então, ele será gravado na mesma pasta onde está salvo o programa.

O comando open cria um objeto em memória e retorna seu id, que deve ser atribuído a um identificador. No caso do Exemplo 7.1, esse identificador foi denominado `arq`. A partir daí são empregados os métodos do objeto arquivo para executar as operações. O método `write` foi usado para executar a gravação do string `G` no arquivo e o método `close` foi usado para fechar o arquivo.

Na segunda parte do programa o mesmo arquivo foi aberto no modo de leitura, ou seja, passando-se “r” para o segundo parâmetro e utilizou-se o método `readline` para efetuar a leitura da primeira (e, neste caso, única) linha do arquivo.

Exemplo 7.1 Gravação e leitura de um arquivo texto

```
print("Inicio do Programa")
```

```
# Parte 1 - Gravação do arquivo
```

```
G = "Gravação e Leitura de Arquivo Texto" # carrega o string G
```

```
arq = open("Exemplo7_1.txt", "w") # abre o arquivo p/ gravar
```

```
arq.write(G) # executa a gravação
```

```
arq.close() # fecha o arquivo
```

```
# Parte 2 – Leitura do arquivo gravado na Parte 1
```

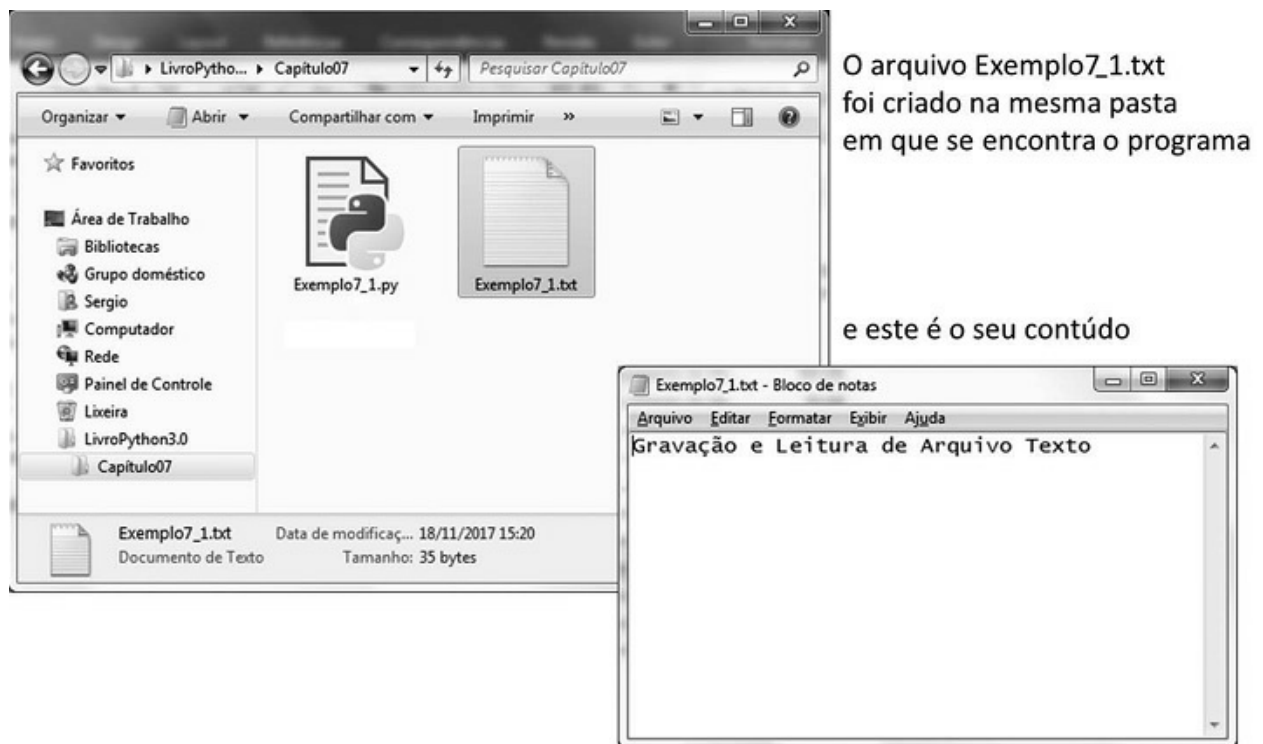
```
arq = open("Exemplo7_1.txt", "r") # abre o arquivo p/ ler
```

```
L = arq.readline() # executa a leitura
```

```
arq.close() # fecha o arquivo
```

```
print("String lido = {}".format(L)) # exhibe o string lido
```

```
print("Fim do Programa")
```



7.2.1 Abertura dos arquivos no Python 3

A forma mais simples de uso do comando `open` envolve o fornecimento apenas do primeiro parâmetro, o nome do arquivo. Neste caso, o interpretador Python assume que o arquivo será aberto para leitura e todos os demais parâmetros assumirão valores padrão.

No entanto, a abertura de arquivos na linguagem Python apresenta alguns detalhes relevantes que precisam ser conhecidos pelo programador. A forma completa do comando `open` é mostrada a seguir:

```
open(file, mode="r", buffering=-1, encoding=None,
      errors=None, newline=None, closefd=True, opener=None)
```

7.2.1.1 file

O primeiro parâmetro é o nome do arquivo que será lido e/ou gravado. Caso o programador queira criar o arquivo em outra pasta, então, deverá fornecer um nome qualificado, indicando a pasta de destino junto com o nome. O fornecimento do nome qualificado deve seguir as regras válidas para o sistema operacional em uso.

7.2.1.2 mode

Os modos de abertura de arquivos texto existentes em Python 3 não se limitam a “w” e “r”, vistos há pouco. O Quadro 7.1 ilustra as possibilidades para arquivos texto. Note-se que existe também o modo “b”, para arquivos binários, que não está presente neste quadro pois, como esclarecido anteriormente, arquivos binários não serão assunto deste livro.

Operações permitidas	r	r+	w	w+	a	a+	x	x+
Leitura de dados do arquivo	✓	✓		✓		✓		✓
Gravação de dados no arquivo		✓	✓	✓	✓	✓	✓	✓
Criação do arquivo			✓	✓	✓	✓	N ₁	N ₁
Zera o conteúdo de arquivo existente			✓	✓				
Posiciona cursor no início do arquivo (N ₂)	✓	✓	✓	✓			✓	✓
Posiciona cursor no fim do arquivo (N ₃ e 4)					✓	✓		

Quadro 7.1 Modos de abertura de arquivos em Python 3.

N₁ = Permite a criação do arquivo exclusivamente se este não existir. Caso exista, levanta a exceção “FileExistsError”.

N_2 = O cursor de um arquivo é um controle posicional que indica (ou aponta) o próximo byte a ser lido. Quando um arquivo é aberto, normalmente o cursor é posicionado em seu primeiro byte.

N_3 = Os modos **a** e **a+** de Python posicionam esse cursor no final do arquivo, pois seu pressuposto é que o arquivo foi aberto para acréscimos no final (append).

N_4 = O modo **a+** permite a leitura, porém, na abertura do arquivo o cursor estará posicionado no final. Caso o programador efetue a leitura nessas condições, obterá um resultado vazio. Para conseguir efetuar a leitura, deve, antes, reposicionar o cursor do arquivo com o método seek.

7.2.1.3 buffering

Especifica as características de buferização do arquivo. As opções são: 0, então, não será usado buffer (permitido apenas para arquivos binários); 1, só se aplica a arquivos texto e o buffer conterá uma linha do arquivo; número inteiro maior que 1, indica um buffer de tamanho fixo com o valor indicado. Caso não seja especificado, o valor -1 é assumido e o interpretador adotará um esquema--padrão de buffer.

7.2.1.4 encoding

Este parâmetro só se aplica a arquivos texto e diz respeito à codificação descrita na primeira parte deste capítulo. Existem muitas opções para uso, mas as mais frequentes no mundo ocidental são “ansi” e “utf-8”.

Os demais parâmetros – erros, newline, closed, opener – fogem ao escopo deste livro, de modo que basta dizer que seus valores-padrão atendem às necessidades dos exercícios e projetos que serão aqui desenvolvidos.

7.2.2 Métodos relativos à manipulação de arquivos em Python 3

O Quadro 7.2 apresenta e explica os comandos e métodos relativos à manipulação de arquivos.

Método	Descrição
close()	Fecha o arquivo que foi aberto com open. Se o arquivo foi aberto para gravação, primeiro descarrega seu buffer.
flush()	Descarrega o buffer de arquivo aberto para gravação, sem fechá-lo.
s = read()	Lê o arquivo inteiro e retorna-o como um único string. Se o arquivo contiver várias linhas, insere um caractere “\n” para cada quebra de linha.
S = readline()	Lê uma linha do arquivo e avança o cursor para o início da próxima. Retorna um string com o conteúdo da linha, incluindo o caractere “\n” se este estiver presente.
L = readlines()	Lê todas as linhas do arquivo e retorna-as como uma lista de strings, incluindo o “\n” no final de cada uma.
write(S)	Grava no arquivo um string de caracteres. O objeto “S” deve ser do tipo string.
writelines(L)	Grava no arquivo todos os strings contidos na lista L.
seek(N)	Altera a posição do cursor do arquivo, posicionando-o no N-ésimo caractere, contado a partir do início do arquivo.

Quadro 7.2 Comandos e métodos relativos a arquivos em Python 3.

Exercícios resolvidos

1. Sequência de números reais gravada em arquivo

Escreva um programa que permaneça em laço lendo números reais até que seja digitado 0. Todos os valores digitados, exceto o zero, devem ser gravados em um arquivo em disco, um por linha, com três casas

decimais.

Serão criadas duas soluções para esse exercício. Na primeira será utilizado o método `write`, e na segunda será utilizado o método `writelines`.

Na linha 2 o arquivo é aberto. Na linha 5 é empregado o método `write` para efetuar a gravação. Como esse método exige um string, então, foi escrito o string `"{0:.3f}\n"`, no qual `0:.3f` garante que o valor `x` seja formatado com três casas decimais, e o caractere de final de linha, `"\n"` garante que cada valor esteja salvo em uma linha diferente. Para verificar o resultado, pode-se abrir o arquivo gravado usando o Bloco de Notas.

Exercício resolvido 7.1a – Números reais gravados em arquivo com `write`

```
print("Início do Programa")

arqreais = open("ValsReais.txt", "w") # linha 2

x = float(input("Digite um número real: "))

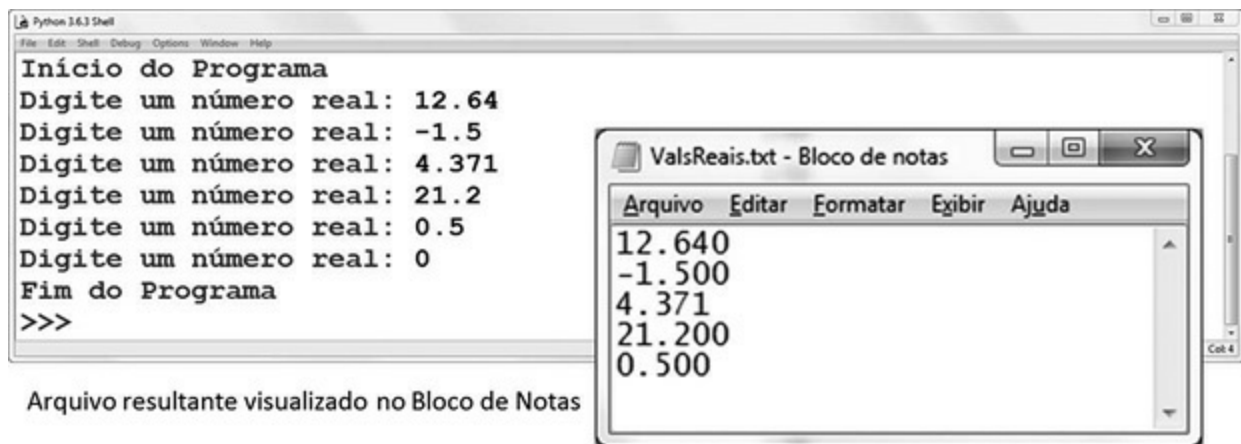
while x != 0:

    arqreais.write("{0:.3f}\n".format(x)) # linha 5

    x = float(input("Digite um número real: "))

arqreais.close() # linha 7

print("Fim do Programa")
```



A segunda solução para esse enunciado está no Exercício resolvido 7.1b. Como o objetivo é usar o método `writelines`, é necessário produzir uma lista contendo os strings que serão gravados no disco. Isso está feito a seguir, em que o objeto `L` é criado como uma lista vazia e dentro do laço é carregada com os strings formatados do mesmo modo como o feito no Exercício resolvido 7.1a. Ao término do laço, o arquivo é aberto, gravado e fechado. O resultado produzido é o mesmo do exercício anterior.

Exercício resolvido 7.1b – Números reais gravados em arquivo usando `writelines`

```
print("Início do Programa")
```

```
L = [] # inicia a lista vazia
```

```
x = float(input("Digite um número real: "))
```

```
while x != 0:
```

```
L.append("{:.3f}\n".format(x)) # inclui o string em L
```

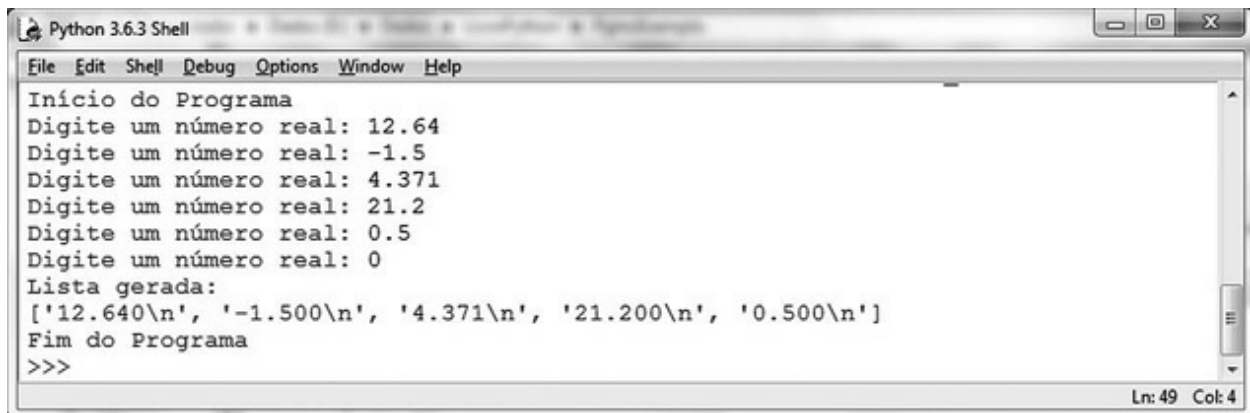
```
x = float(input("Digite um número real: "))
```

```
arcreais = open("ValsReais2.txt", "w") # abre o arquivo
```

```
arcreais.writelines(L) # grava a lista toda
```

```
arcreais.close() # fecha o arquivo
```

```
print("Fim do Programa")
```

A screenshot of a Python 3.6.3 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following output:

```
Início do Programa
Digite um número real: 12.64
Digite um número real: -1.5
Digite um número real: 4.371
Digite um número real: 21.2
Digite um número real: 0.5
Digite um número real: 0
Lista gerada:
['12.640\n', '-1.500\n', '4.371\n', '21.200\n', '0.500\n']
Fim do Programa
>>>
```

The status bar at the bottom right indicates 'Ln: 49 Col: 4'.

2.Leitura de arquivo e totalização de valores

Escreva um programa que leia um arquivo texto contendo um número inteiro em cada linha. Exiba na tela e faça a totalização dos valores lidos.

Esse exercício será resolvido de duas maneiras diferentes. Na primeira, Exercício resolvido 7.2a, será utilizado um laço while. A leitura da primeira linha do arquivo é feita fora do laço. Caso seu retorno seja diferente de string vazio, o laço é iniciado e prosseguirá até que o final do arquivo seja alcançado. O método readline retorna um string vazio quando a leitura do arquivo chega ao final. A cada repetição do laço o string lido é convertido para inteiro e totalizado no objeto Soma.

Na conversão de S para número inteiro pode ocorrer um erro se S for um

string vazio. Por isso, foi adotada a lógica de se fazer a primeira leitura fora do laço e as demais leituras como último comando dentro laço, evitando um if adicional.

Exercício resolvido 7.2a – Leitura de arquivo e totalização de valores – solução com while

```
print("Início do Programa")

Soma = 0

Cont = 0

arq = open("Inteiros.txt", "r")

S = arq.readline() # garante que o laço seja iniciado

while S != "":

    N = int(S) # converte o texto S para N

    Soma = Soma + N # totaliza o valor N em Soma

    Cont = Cont + 1 # conta mais um elemento

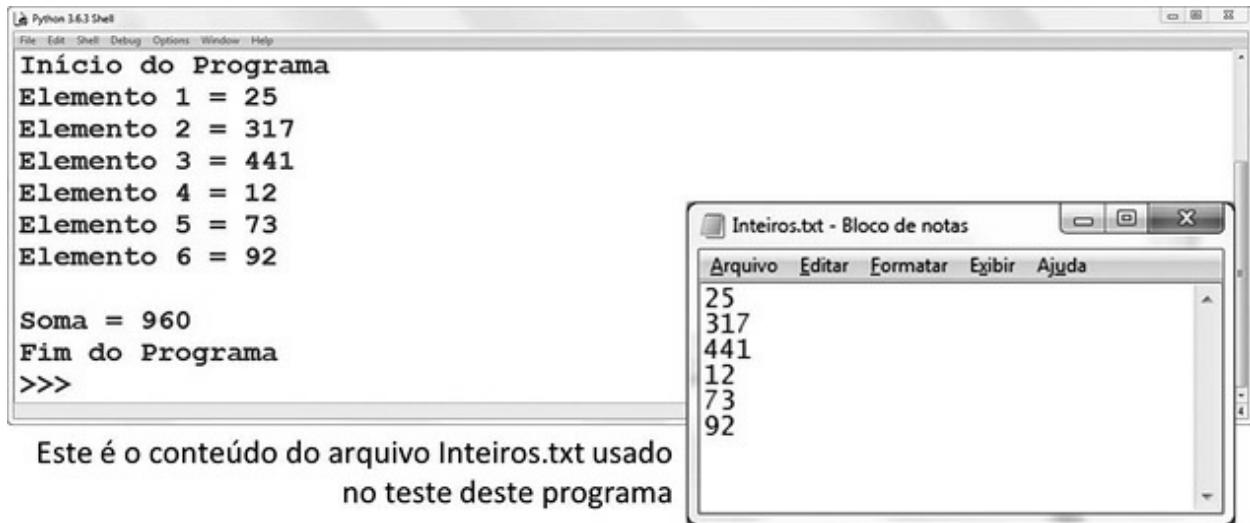
    print("Elemento {0} = {1}".format(Cont, N))

    S = arq.readline() # lê a próxima linha

arq.close()

print("\nSoma = {0}".format(Soma))
```

```
print("Fim do Programa")
```



Essa solução funcionará perfeitamente, porém, existe outra solução bem melhor no que diz respeito ao uso dos recursos da linguagem Python. O Exercício resolvido 7.2b mostra essa outra solução, na qual foi utilizado o conceito de **iterador de arquivo** (*file iterator*).

Quando o objetivo é percorrer todo o arquivo, linha a linha, recuperando e processando uma linha por vez, então, usar um iterador de arquivo é a melhor opção, pois o código fica mais enxuto, o interpretador gerencia melhor a memória e otimiza a execução, não sendo necessário fechar o arquivo, pois isso será automático ao fim do processo. Também se pode observar que não é necessário atribuir o retorno do comando `open` a um identificador. O iterador gerenciará um objeto temporário em memória, dispensando um identificador.

Exercício resolvido 7.2b – Leitura de arquivo e totalização de valores – solução com iterador de arquivo

```
print("Início do Programa")
```

```
Soma = 0
```

```
Cont = 0
```

```
for S in open("Inteiros.txt"): # note que "r" foi omitido, pois
```

```
N = int(S) # é default
```

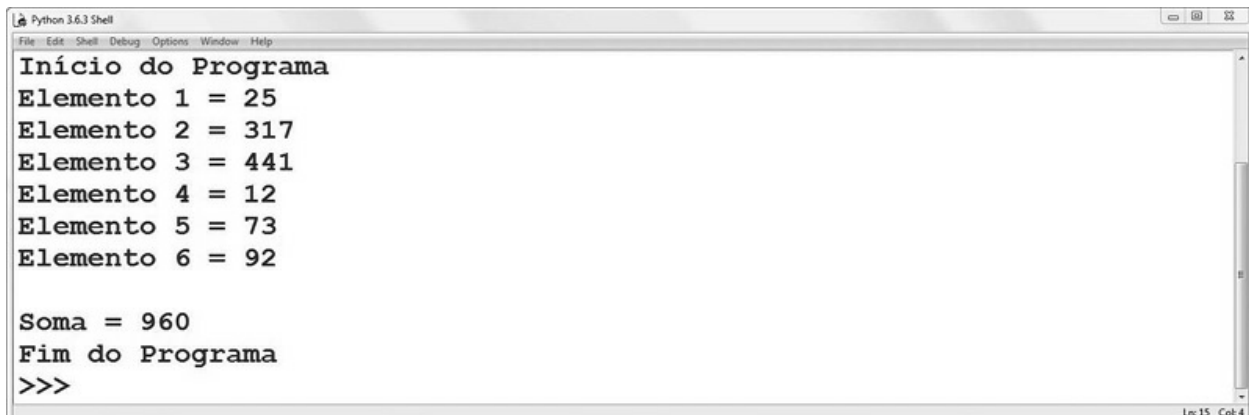
```
Soma = Soma + N
```

```
Cont = Cont + 1
```

```
print("Elemento {0} = {1}".format(Cont, N))
```

```
print("\nSoma = {0}".format(Soma))
```

```
print("Fim do Programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Início do Programa
Elemento 1 = 25
Elemento 2 = 317
Elemento 3 = 441
Elemento 4 = 12
Elemento 5 = 73
Elemento 6 = 92

Soma = 960
Fim do Programa
>>>
```

As duas soluções, 7.2a e 7.2b, produzem exatamente o mesmo resultado e, em linhas gerais, pode-se dizer que, enquanto a primeira talvez seja mais alinhada com o modo de pensar de um programador experiente em outras linguagens, a segunda reflete o jeito Python de ser. No jargão da comunidade Python, a segunda solução é *Pythonic*.

3. Ler um arquivo do tipo CSV¹

Escreva um programa que leia um arquivo texto que contém diversas linhas que representam uma lista de compras. Em cada linha há três informações: nome de um produto, quantidade e preço unitário, separados pelo caractere “;”. Pede-se que cada item da lista seja exibido na tela, incluindo o valor total do item. Ao final, exiba o total da compra.

Os valores devem ser exibidos com duas casas decimais. Um exemplo de arquivo de entrada é mostrado a seguir:

```
Leite,12,3.8  
Maçã,100,4.4  
Café,9,16.35  
Pão de Forma,41,5.9
```

Antes de chegar à solução final para esse problema, é necessário apresentar alguns recursos que serão utilizados. Por questões didáticas, o processo será apresentado e utilizado passo a passo. Para tanto, considere-se o Exemplo 7.2.

Na primeira linha é atribuído um string típico desse problema ao objeto `S`. Esse string contém um caractere “\n” no final, e a primeira tarefa é removê-lo. Isso poderia ser feito com o fatiamento `S[:-1]` (que significa todos os caracteres menos o último), porém, se o string não terminar em “\n”, haverá a remoção indevida do último caractere. Normalmente isso ocorre na última linha do arquivo, na qual pode não haver o caractere de fim de linha.

A solução, então, é usar o método `rstrip` do `string`, o qual remove caracteres à direita. Os caracteres a serem removidos devem ser especificados como parâmetro e, caso sejam omitidos, são removidos os espaços em branco e o “\n”.

Exemplo 7.2 Tratamento de um string CSV

```
>>> S = “prodA,12,3.8\n” # string S como virá do arquivo

>>> S[:-1] # remover o “\n” com fatiamento não é

‘prodA,12,3.8’ # a melhor opção

>>> S = “prodA,12,3.8” # se o “\n” não estiver presente, então

>>> S[:-1] # o último caractere será removido

‘prodA,12,3.’ # indevidamente

>>> S = “prodA,12,3.8\n”

>>> S.rstrip() # essa solução é melhor

‘prodA,12,3.8’ # o “\n” foi removido

>>> L = S.split(“,”) # separa S em uma lista de strings

>>> L # usando “,” como delimitador

[‘prodA’, ‘12’, ‘3.8’] # lista produzida

>>> L[1], L[2] = int(L[1]), float(L[2]) # converte L[1] e L[2]
```



```
>>> L # para int e float,
```

```
['prodA', 12, 3.8] # respectivamente
```

O passo seguinte é usar o método `split`, que retorna uma lista de strings separados a partir de `S`. O parâmetro passado é um substring empregado como delimitador para a separação.

O último passo é converter o elemento `L[1]` para número inteiro e o elemento `L[2]` para número real.

A partir daí, os dados estão prontos para processamento.

A solução implementada no Exercício resolvido 7.3 mostra, passo a passo, todos esses recursos sendo utilizados. Os totais pedidos são calculados e os resultados são exibidos por meio de um string de formatação convenientemente elaborado, que pode ser visualizado na Figura 7.4.

Exercício resolvido 7.3 – Ler um arquivo do tipo CSV

```
print("Lista de Compras")
```

```
TotGeral = 0
```

```
for S in open("dados.txt", "r"):
```

```
    S = S.rstrip()
```

```
    L = S.split(",")
```

```
    L[1], L[2] = int(L[1]), float(L[2])
```

```
TotProd = L[1] * L[2]
```

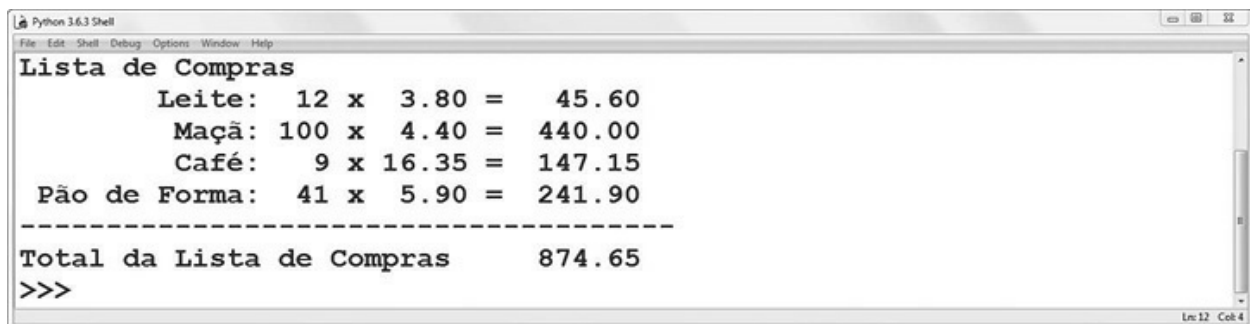
```
TotGeral += TotProd
```

```
print(" {0:>12}: {1:3} x{2:6.2f} = {3:7.2f}").
```

```
format(L[0], L[1], L[2], TotProd)) # cont. linha anterior
```

```
print("-" * 38)
```

```
print("Total da Lista de Compras {0:>10}".format(TotGeral))
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Lista de Compras
    Leite:  12 x  3.80 =  45.60
    Maçã: 100 x  4.40 = 440.00
    Café:   9 x 16.35 = 147.15
    Pão de Forma: 41 x 5.90 = 241.90
-----
Total da Lista de Compras      874.65
>>>
```

Figura 7.4 Execução do Exercício resolvido 7.3.

4. Gerador de dados em arquivo do tipo CSV

Escreva um programa que leia um número inteiro N ($10 < N < 10.000$) e grave um arquivo com N linhas com os dados listados na tabela seguinte. O arquivo deve ter o nome “Estoque.csv” e deve usar o caractere “;” (ponto e vírgula) como delimitador. Não é necessário que o arquivo esteja ordenado.

Campo	Descrição
Código do produto	Número inteiro entre 10000 e 50000. Não pode haver repetição desse código, e pede-se que não sejam sequenciais (aleatórios).
Quantidade	Número inteiro entre 1 e 3800.

em estoque	Gerar aleatórios.
Preço unitário compra	Número real entre 1.80 e 435.90. Gerar aleatórios.
Alíquota do ICMS	Alíquota do imposto ICMS. Essa alíquota deve ser 7%, 12% ou 18%. (Não colocar o caractere “%” no arquivo).

Quadro 7.3 Formato para o arquivo dos Exercícios resolvidos 7.4 e 7.5.

Para elaborar uma solução para esse problema nenhum conhecimento adicional é necessário. Podem ser elaboradas duas soluções com abordagens distintas, à semelhança dos Exercícios resolvidos 7.1a (uma linha é gravada a cada repetição do laço) e 7.1b (a lista toda é gerada e gravada no final).

A opção aqui será pela solução 7.1a, em que é gravada uma linha por vez no arquivo, e, se desejar, pode adaptar essa solução para outra que se assemelhe à 7.1b.

Para atender aos requisitos impostos para o código do produto, inicialmente é criada uma lista de códigos. Cada código é gerado utilizando a função `randint` e, caso ainda não esteja na lista, será adicionado. Após completada a lista de códigos, inicia-se a segunda parte do programa com a geração dos demais dados e a gravação destes no arquivo, linha a linha. Como as alíquotas de ICMS são três possíveis valores, os mesmos foram colocados em uma tupla, e gera-se um número aleatório para utilizar como seu indexador.

Exercício resolvido 7.4 – Gerador de dados em arquivo do tipo CSV

```
from random import randint
```

```
print("Início do Programa")
```

```
N = 0
```

```
while N < 10 or N > 10000: # leitura de N
```

```
N = int(input("Digite N entre [10, 10000]: "))
```

```
cod = []
```

```
cont = 0
```

```
while cont < N: # laço para gerar a lista de
```

```
a = randint(10000, 50000) # códigos únicos
```

```
if a not in cod: # este if garante a unicidade
```

```
cod.append(a)
```

```
cont += 1 # só conta 1 quando entrou em L
```

```
S = "{0};{1};{2:.2f};{3}\n" # string pré formatado
```

```
ICMS = (7, 12, 18) # tupla com os 3 valores de ICMS
```

```
arq = open("Estoque.csv", "w")
```

```
cont = 0
```

```
while cont < N:
```

```
Qtde = randint(1, 3800)
```

```
PcUn = randint(180, 43590) / 100

i = randint(0, 2)

arq.write(S.format(cod[cont], Qtde, PcUn, ICMS[i]))

cont += 1

arq.close()

print("Fim do Programa")
```

Tarefas adicionais

1. No processo de geração dos códigos, troque a lista por um conjunto (set). Dica: veja o Exemplo 7.5.
2. Use esse programa para gerar alguns arquivos com variadas quantidades de linhas. Abra-os em um programa de planilha eletrônica, calcule e anote os totais que são pedidos no próximo enunciado. Fazendo isso, será possível conferir as saídas produzidas pelo Exercício resolvido 7.5, que vêm na sequência.

Um arquivo gerado por este programa terá esta aparência.

```
12319;175;414.93;7
36770;1380;75.43;18
22571;3372;15.18;18
14558;749;45.84;12
19706;338;384.62;18
```

5. Lê e processa dados de um arquivo do tipo CSV

Escreva um programa que leia um arquivo com o layout especificado no Quadro 7.3 e apresente os seguintes resultados:

- valor total das mercadorias em estoque;

- valor total do imposto ICMS pago referente a essas mercadorias.

A solução desse problema implica o conhecimento da maneira como o imposto chamado ICMS é tratado. Esse é um imposto estadual que vem embutido no preço do produto, seja ele adquirido pelo consumidor final ou por uma loja que vai revendê-lo. No caso do comércio em geral, é importante conhecer qual é o valor de ICMS que está embutido nos produtos do estoque. Para chegar a esse valor, é preciso fazer o seguinte cálculo:

Valor pago	Alíquota ICMS	Valor imposto	Valor mercadoria
100,00	12%	$100,00 \times 0,12 = 12,00$	$100,00 - 12,00 = 88,00$

Assim, o programa deve ler o arquivo linha a linha, extrair os dados de cada linha, calcular os valores do imposto e da mercadoria e totalizá-los. Isso é o que está implementado no código do Exercício resolvido 7.5.

Exercício resolvido 7.5 – Lê e processa arquivo CSV

```
print("\nCálculo de Estoque\n")
```

```
saida = "{:>7}{:>13.2f}{:>10.2f}{:>12.2f}"
```

```
TotICMS = 0
```

```
TotMerc = 0
```

```
print("Produto Val.Compra ICMS Mercadoria")
```

```
arq = open("Estoque.csv", "r")
```

```
for s in arq.readlines():
```

```
s = s.rstrip()

L = s.split(";")

L[0] = int(L[0])

L[1] = int(L[1])

L[2] = float(L[2])

L[3] = float(L[3])/100

compra = L[1] * L[2]

icms = compra * L[3]

merc = compra - icms

TotICMS += icms

TotMerc += merc

print(saida.format(L[0], compra, icms, merc))

arq.close()

print(saida.format("Totais", TotMerc+TotICMS, TotICMS, TotMerc))

print("\n\nFim do Programa")
```

A Figura 7.5 mostra o resultado da execução desse programa para os dados exibidos aqui ao lado direito.

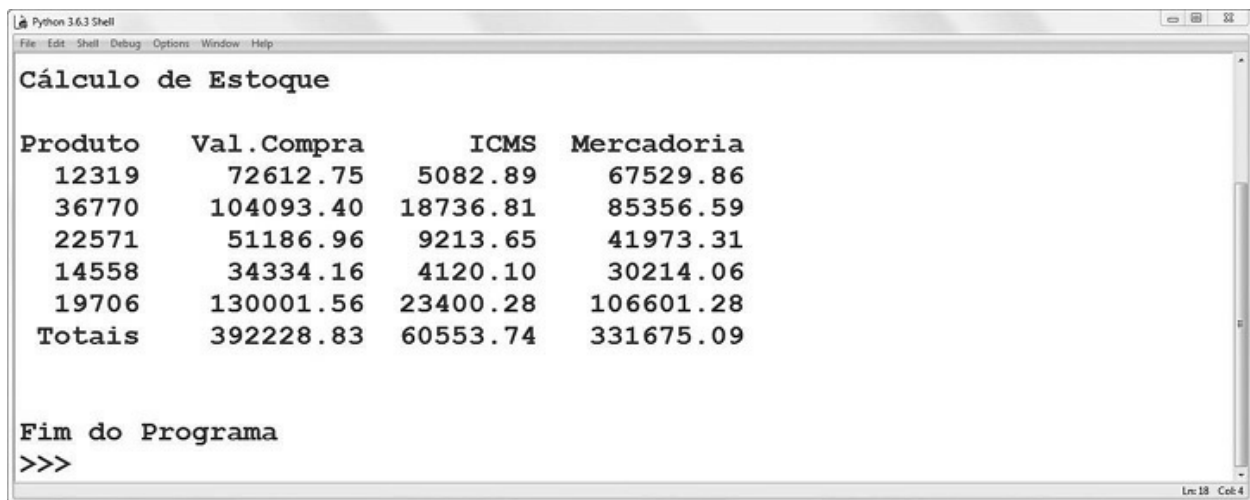
12319;175;414.93;7

36770;1380;75.43;18

22571;3372;15.18;18

14558;749;45.84;12

19706;338;384.62;18



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Cálculo de Estoque

Produto    Val.Compra    ICMS    Mercadoria
12319      72612.75      5082.89    67529.86
36770      104093.40     18736.81   85356.59
22571      51186.96      9213.65    41973.31
14558      34334.16      4120.10    30214.06
19706      130001.56     23400.28   106601.28
Totais     392228.83     60553.74   331675.09

Fim do Programa
>>>
```

Figura 7.5 Resultado da execução do Exercício resolvido 7.5.

Para verificar se os cálculos realizados pelo programa estão corretos, pode lançar os dados de entrada em uma planilha eletrônica e usar fórmulas para reproduzir os cálculos e confrontar os resultados.

6. Arquivos texto com codificação conflitante

O objetivo desse programa é mostrar como a codificação conflitante em arquivos texto pode causar problemas. É incorreto misturar ANSI com UTF-8, que são as codificações mais empregadas no Brasil, de modo que o programador deve estar atento a isso. No Exercício resolvido 7.6 é apresentado um menu com as quatro opções a seguir, além da opção de sair do programa (essa opção nem será usada, veja o porquê na Figura 7.6).

- 1.gravar ANSI e ler ANSI;
- 2.gravar UTF-8 e ler UTF-8;
- 3.gravar UTF-8 e ler ANSI;
- 4.gravar ANSI e ler UTF-8.

Foi criada uma função chamada GravaLe(grava, le) que recebe dois parâmetros aos quais são passados os textos “ANSI” ou “UTF-8”, conforme cada caso. Esses parâmetros são utilizados, respectivamente, na gravação e na leitura do arquivo. Foi usado um objeto S inicializado com um string que contém diversos caracteres acentuados para serem gravados no arquivo.

Exercício resolvido 7.6 – Arquivos texto com codificação conflitante

```
def GravaLe(grava, le):  
  
    print("\n--", grava, " para ", le, "--"*29)  
  
    arq = open("arquivo.txt", "w", encoding=grava)  
  
    arq.write(S)  
  
    arq.close()  
  
    arq = open("arquivo.txt", "r", encoding=le)  
  
    L = arq.readlines()  
  
    arq.close()
```

```
for x in L:
```

```
    print(x)
```

```
print("-"*50)
```

```
# A execução do programa começa por aqui
```

```
print("\nDemonstra os conflitos de codificação de arquivos\n\n")
```

```
S = """Uma boa porção de caracteres com acento
```

```
Maiúsculas: Á É Í Ó Ú Ã Õ Â Ê Ô À Ç
```

```
Minúsculas: á é í ó ú â ô â ê ô à ç"""
```

```
while True:
```

```
    print("O que deseja fazer?")
```

```
    print(" para gravar ANSI e ler ANSI digite 1")
```

```
    print(" para gravar UTF8 e ler UTF8 digite 2")
```

```
    print(" para gravar UTF8 e ler ANSI digite 3")
```

```
    print(" para gravar ANSI e ler UTF8 digite 4")
```

```
    print(" para sair digite 0")
```

```
    opc = int(input(" ...opc = "))
```

```
    if opc == 0:
```

```
break

elif opc == 1:

    GravaLe("ANSI", "ANSI")

elif opc ==2:

    GravaLe("UTF-8", "UTF-8")

elif opc == 3:

    GravaLe("UTF-8", "ANSI")

elif opc ==4:

    GravaLe("ANSI", "UTF-8")

print("Fim do Programa")
```

Como resultado da execução desse programa têm-se as telas da Figura 7.6. Podem-se ver, do lado esquerdo, todos os acentos corretos, pois ela mostra os casos 1 e 2, nos quais houve coerência entre gravação e leitura. Do lado direito são mostrados os casos 3 e 4. No caso 3 (grava UTF-8 e lê ANSI), os caracteres são exibidos incorretamente. No caso 4 (grava ANSI e lê UTF-8), a situação é pior: o codec UTF-8 não consegue interpretar apropriadamente a sequência de bytes ANSI, gerando erro de execução.

Bubble Sort.

4. Repita o exercício anterior trocando o método de ordenação, para o método recursivo *Quicksort* (faça uma pesquisa sobre esse algoritmo). O *Quicksort* é um algoritmo consideravelmente mais veloz, na maioria dos casos, que o *Bubble Sort*. Use os programas 3 e 4 para constatar esse fato.
5. Escreva um programa que leia um arquivo texto de entrada contendo os salários brutos dos funcionários de uma empresa e grave um arquivo de saída com os cálculos solicitados e descritos a seguir. O arquivo de entrada deve ter o nome “salario.txt” e conterá um valor em cada linha, sendo que, na primeira linha, estará escrito o título “Bruto”. Para cada salário devem ser calculados os descontos de INSS (previdência social), IR (imposto de renda) e salário líquido.

salario.txt	calculos.txt
Bruto	SalBruto;AliqINSS;VALINSS;AliqIR;DeduçãoIR;VALIR;SalLiq
1228.90	1228.90;8;98.31;0.00;0.00;0.00;1130.59;
2156.78	2156.78;9;194.11;7.50;142.80;4.40;1958.27;
2298.37	2298.37;9;206.85;7.50;142.80;14.06;2077.45;
3348.32	3348.32;11;368.32;15.00;354.80;92.20;2887.80;
4573.12	4573.12;11;503.04;22.50;636.13;279.64;3790.44;
4864.87	4864.87;11;535.14;22.50;636.13;338.06;3991.67;
5031.87	5031.87;11;553.51;22.50;636.13;371.50;4106.86;
8281.92	8281.92;TETO;570.88;27.50;869.36;1251.18;6459.86;
etc...	

Para obter cada valor aplicam-se as seguintes regras de cálculo:

a) $\text{SalLiquido} = \text{SalBruto} - \text{VALINSS} - \text{VALIR}$

$$b) VALINSS = SalBruto * AliqINSS$$

- AliqINSS é obtida na Tabela 1

$$c) VALIR = (SalBruto - VALINSS) * AliqIR - DeduçãoIR$$

- Note que a base de cálculo do IR é o salário bruto menos o valor do INSS
- AliqIR e DeduçãoIR são obtidos na Tabela 2

O arquivo de saída “calculos.txt” deve ser gravado contendo os campos listados a seguir, separados pelo caractere “;” como mostrado anteriormente.

- **SalBruto** é o valor oriundo do arquivo de entrada.
- **AliqINSS** é o percentual usado para cálculo do valor do INSS, ou a palavra TETO para os casos em que $SalBruto > 5.531,31$.
- **ValINSS** é o valor do INSS calculado segundo o item b das regras de cálculo.
- **AliqIR** é o percentual usado para cálculo do valor do IR obtido na Tabela 2 tendo como base $SalBruto - ValINSS$.
- **DeduçãoIR** é o valor dedutível de cada alíquota de IR obtido na Tabela 2.
- **ValIR** é o valor do IR calculado segundo o item c das regras de cálculo.
- **SalLiquido** é o salário líquido calculado segundo o item a das regras

de cálculo.

Tabela 1		Tabela 2		
Valor do Salário Bruto	Percentual	Salário Bruto – Val. INSS	Percentual	Dedução
até 1.556,94	8,00 %	Até 1.903,98	0,0%	0,00
de 1.556,95 até 2.594,92	9,00 %	De 1.903,99 até 2.826,65	7,5%	142,80
de 2.594,93 até 5.189,82	11,00 %	De 2.826,66 até 3.751,05	15,0%	354,80
A partir de 5.189,83	Teto = R\$ 570,88	De 3.751,06 até 4.664,68	22,5%	636,13
		Acima de 4.664,68	27,5%	869,36

Figura 7.7 Tabelas de INSS e Imposto de Renda de Pessoa Física em vigor em 2017.

6. Esse programa está baseado na leitura de um arquivo texto de entrada contendo números do registro de alunos. Esse arquivo deve ter o nome RA.TXT e conterá um RA em cada linha. Para cada número de RA presente no arquivo deve ser gerada uma senha, conforme as condições especificadas a seguir. Tanto o RA como a senha gerada devem ser gravados no arquivo de saída RASENHA.TXT, com o formato a seguir:

Condições para geração de senhas

No início do programa, antes de efetuar a leitura do arquivo de entrada RA.TXT, o programa deve pedir que o usuário informe:

- O tipo de senha: numérica (conterá apenas algarismos) ou alfanumérica (conterá letras maiúsculas e algarismos).
- O tamanho da senha: quantidade de caracteres que a mesma deve conter.

Exemplo: neste exemplo, foram geradas senhas alfanuméricas com sete caracteres:

RA.TXT	RASENHA.TXT
330019	330019;318A89P
414061	414061;E87H14M
109229	109229;019MKX9
827392	827392;313G093

etc...

¹ Arquivos assim também são conhecidos pelo termo “*comma separated values*” (CSV) e são muito utilizados para troca de dados entre diferentes sistemas.



Python 3 com Banco de Dados SQLite

Objetivos

O objetivo deste capítulo é destacar os recursos disponíveis na linguagem Python para se conectar e interagir com o sistema gerenciador de banco de dados SQLite.

Para atingir tal objetivo, inicialmente, serão apresentados conceitos essenciais sobre bancos de dados e sobre o SQLite. Na sequência, serão apresentados os recursos disponíveis em Python para manipular o SQLite.

Para ilustrar os conceitos e recursos apresentados e tornar seu uso bem claro e prático, serão desenvolvidos diversos programas-exemplo que vão, passo a passo, mostrando como usar a poderosa dobradinha Python e SQLite.

8.1 Gerenciadores de bancos de dados

No Capítulo 7 foi visto como gravar e ler arquivos texto para utilizá-los como forma de armazenamento permanente dos dados processados pelos programas. O uso de arquivos assim tem sua utilidade, porém, nem sempre é a melhor forma de implementar o armazenamento de dados.

O programador experiente já deve conhecer os conceitos que serão apresentados e pode pular diretamente para o próximo capítulo. Ao iniciante, convém ler atentamente e executar em seu computador as tarefas que serão sugeridas aqui.

8.1.1 Bancos de dados e bases de dados

Há muitas décadas tem-se desenvolvido a tecnologia de Sistemas Gerenciadores de Bancos de Dados (SGBD) ou, em inglês, *Data Base Management System* (DBMS), cujo objetivo maior é oferecer aos profissionais de desenvolvimento de sistemas uma base de armazenamento e recuperação de dados que seja ao mesmo tempo organizada, robusta, flexível, segura e escalável. Coloquialmente, um SGBD é conhecido como “o banco de dados”, e em uma conversa entre programadores é comum ouvir frases do tipo: “qual banco de dados vocês usam?” ou “onde está hospedado o seu banco de dados?”.

Ao usar frases assim, ocorre uma mistura de dois elementos distintos, que são simples e precisam ser compreendidos. O primeiro elemento é o SGBD em si, que é um software, ou seja, um conjunto de programas de computador que alguém – uma empresa, um consórcio ou uma comunidade virtual – desenvolve e mantém. O segundo elemento são os dados gerenciados por esse software.

Em um sistema computacional pode-se ter um software gerenciador de banco de dados, o qual é utilizado para gerenciar diversas **bases de dados**.

Assim, entenda-se este novo termo: base de dados é o conjunto de dados armazenados e utilizados pelos programas. Por exemplo, imagine-se um servidor de internet no qual estejam hospedados dez diferentes sites. Nesse servidor deve estar instalado um software de banco de dados, ou seja, um SGBD. Por outro lado, cada site tem sua base de dados, então, a conclusão imediata é que serão dez bases de dados nesse mesmo servidor. Em linhas gerais, é assim que as coisas acontecem.

Existem muitos SGBDs no mercado de computação: Oracle, Microsoft SQL Server, MySQL, PostgreSQL, MongoDB, DB2 etc. A lista é bem grande, e o SQLite faz parte dela. Nessa lista, há sistemas que são proprietários e licenças precisam ser adquiridas para que se possa utilizá-los, assim como há os que são softwares livres e podem ser usados sob determinadas condições, a depender da licença de software livre sob a qual é disponibilizado.

Dada a importância atual do uso desses gerenciadores de banco de dados, tomou-se a decisão de incluir este capítulo neste livro sobre Python. Referente a banco de dados, nos cursos de Ciência da Computação e Desenvolvimento de Sistemas há disciplinas exclusivamente dedicadas ao assunto, de modo que aqui não se tem a pretensão nem espaço para cobrir e extinguir o assunto. Ao contrário, será uma abordagem totalmente de natureza prática, mostrando como a linguagem Python pode ser utilizada para escrever programas que se conectam a bases de dados e delas recebem, ou a elas enviam, seus dados.

Nos sistemas que entrarão em produção, escolher uma dentre tantas possibilidades de sistemas gerenciadores é tarefa para profissionais experientes e envolve a análise de muitos fatores.

A escolha por usar o SQLite foi em função de sua simplicidade e disponibilidade. Ele e a biblioteca necessária ao seu uso estão contidos nas

instalações-padrão de Python, de modo que nenhum software adicional é necessário para que se utilize o par Python + SQLite.

8.1.2 Organização dos bancos de dados

Antes de iniciar o trabalho com o par Python + SQLite, é necessário apresentar alguns conceitos.

Em um SGBD, o ponto de partida conceitual é a forma de organização dos dados. A isso se dá o nome de **modelo** de banco de dados, e os quatro principais modelos são: hierárquico, rede, relacional e orientado a objetos. O modelo relacional é um dos mais utilizados, e toda a sua organização está fundamentada em tabelas, como a mostrada a seguir.

Nome da tabela: CADALUNOS

Colunas são Campos

Nomes de Campos →

Linhas são Registros

MATRICULA	NOME	DATANASC	ALTURA	PESO	CURSO	EMAIL
1728	Carlos de Oliveira	12/04/1997	1,68	73,1	33	carlos@dominio.com
1944	Diogo Sales	07/09/1996	1,82	86,3	29	diogo@dominio.com
2093	Michele Sanches	14/08/1998	1,65	58,5	37	michele@provedor.com
2155	Ana Paula Miranda	26/01/2997	1,73	69,6	33	ana.paula@algo.com.br

Chave primária

Figura 8.1 Elementos de uma tabela de banco de dados.

A Figura 8.1 contém os elementos essenciais de uma tabela de banco de dados do modelo relacional, a saber:

- **Nome da tabela:** toda tabela registrada na base de dados deve ter um nome de acordo com as regras de nomes adotadas no SGBD escolhido. Normalmente se utilizam letras, números e o caractere underline “_”. Na Figura 8.1 a tabela se chama CADALUNOS.
- **Registros:** cada linha da tabela é denominada registro e representa uma

coleção heterogênea de dados interligados. Os registros são subdivisíveis em campos.

- **Campos:** é o elemento no qual um dado é armazenado. Um conjunto de campos forma um registro, ou linha da tabela. Os campos apresentam tipo – e, quando pertinente – tamanho definidos. Embora haja certa variação entre os diversos sistemas, os tipos básicos de campos são: texto, número inteiro, número real, data, hora, date e hora juntos, lógico (true/false), entre outros. Campos são identificados por nomes que seguem regras com as de nomes de tabelas. No exemplo, MATRICULA, NOME, DATANASC etc. são campos.
- **Chave primária:** é responsável pela identificação do registro no banco de dados. Pode ser constituída por um ou mais campos, e é obrigatório que seja única e não nula. No exemplo, a chave primária é a MATRICULA.

As bases de dados não têm apenas uma tabela, ao contrário, costumam ter muitas. Cada tabela pode ter centenas de campos com milhões de registros. E, além desses elementos citados no Exemplo 8.1, os bancos de dados também têm outros que não foram citados, como índices secundários, chaves estrangeiras, *views*, *stored procedures*, direitos de acesso (*grants*) etc. Ao modo como os dados são estruturados nas tabelas aplicam-se os conceitos que levam à modelagem de dados, à organização das relações entre as tabelas, à normalização etc. E há, ainda, outras tantas coisas mais a se aprender. Como pôde perceber da leitura deste parágrafo, tal volume de conceitos tem mesmo de ser objeto de aprofundado estudo, o que justifica a existência das disciplinas de Banco de Dados citadas anteriormente.

O mais importante de tudo que foi exposto é que o básico contido no

Exemplo 8.1 é o que basta, e com esse básico poderemos resolver muitos exercícios envolvendo a linguagem Python e o gerenciador de banco de dados SQLite.

8.1.3 A sigla SQL

Por que essa sigla está sempre presente?

Até existe uma tradução para o português desse termo, que é “linguagem de consulta estruturada”, mas ela não “pegou”, e os profissionais de computação brasileiros sempre usam “SQL”. Abreviação de *Structured Query Language*, o termo refere-se à linguagem utilizada para criar, armazenar, recuperar e atualizar dados em um banco de dados relacional. Foi desenvolvida no início dos anos 1970 pela IBM como parte do projeto que levou à criação do modelo relacional de bancos de dados (CHAMBERLIN, 1981).

SQL não é uma linguagem genérica, como C, Java ou Python. SQL é exclusivamente utilizada para interagir com bancos de dados relacionais e tem seus comandos divididos em grupos, segundo as operações que realizam:

- **DQL (*Data Query Language*)**: este não é propriamente um grupo de comandos, pois aqui se conta apenas com o comando `select`. No entanto, é o mais usado e tem tantas variações que muitos programadores e gerentes de bancos de dados o classificam dessa maneira. O `select` é usado para buscar dados de tabelas, conforme mostrado nestes exemplos:

<code>select * from cadaluno</code>	Retorna todos os registros contidos na tabela CADALUNO.
<code>select nome, email from cadaluno where curso = 33</code>	Retorna os campos nome e email da tabela CADALUNO apenas para os registros em que o campo curso seja igual a 33.

- DML (*Data Manipulation Language*)**: são os comandos usados para realizar inclusões, alterações e exclusões de dados nas tabelas. As palavras-chave desses comandos são: insert, update e delete. Exemplos de uso desses comandos serão vistos adiante.
- DDL (*Data Definition Language*)**: são os comandos empregados para criar e excluir tabelas e seus elementos associados. As palavras-chave desses comandos são: create, alter e drop. Exemplos de uso desses comandos serão vistos adiante.
- DCL (*Data Control Language*)**: são os comandos utilizados para controlar o acesso de usuários aos dados das tabelas, definindo “quem pode ver o quê”. Esses comandos não serão utilizados neste livro.
- DTL (*Data Transaction Language*)**: são os comandos relacionados ao controle de transações no banco de dados, indicando quando e como os dados devem ser fisicamente salvos ou descartados. Existem com a finalidade de garantir a integridade de dados inter-relacionados. As palavras-chave desses comandos são: commit e rollback. Exemplos de uso desses comandos serão vistos adiante.

De tudo o que foi apresentado, espera-se que compreenda que há muito a ser aprendido sobre bancos de dados. No entanto, tal volume não impede que se compreenda e utilize o básico de BDs para a implementação de diversos programas escritos em Python que vão se conectar ao SQLite.

8.1.4 O Banco de dados SQLite

Há muitas opções de SGBDs, foi escolhido o SQLite para este livro simplesmente porque está disponível. O Python está instalado em seu computador? Se a resposta é sim, então, o SQLite está instalado também e o

que é melhor: pronto para uso. Não requer download, não requer instalação, não requer configurações difíceis de entender, não requer um DBA (*Database Administrator*).

O SQLite é uma biblioteca que implementa as funções de gerenciamento de banco de dados de maneira autossuficiente, sem a necessidade de um computador servidor rodando um software servidor de banco de dados. Por isso, ele não requer qualquer configuração. Escrito em linguagem C, seu código foi colocado em domínio público por seus autores, ou seja, é um software livre e de código-fonte aberto. No entanto, existe uma versão melhorada que é paga e contém recursos para trabalhar com bancos de dados compactados e criptografados.

Estão disponíveis binários executáveis para diversos ambientes, como Windows, Linux, macOS, Android, iOS, entre outros.

O SQLite está distribuído em bilhões¹ de dispositivos diferentes e é muito leve, compacto e confiável, além de ser muito útil como repositório de dados para aplicações diversas. A versão mais recente disponível quando este livro foi escrito é a 3.21.0, de 24 de outubro de 2017.

Ao usá-lo neste texto, propicia-se ao iniciante a oportunidade de conhecer o mundo dos bancos de dados relacionais, e o programador experiente pode travar um primeiro contato com esse pequeno notável, caso ainda não o tenha utilizado em alguma outra oportunidade.

Sendo um produto de software tão bom e bem-aceito, ele tem aplicação em qualquer projeto de tecnologia da informação? Não necessariamente. Em sistemas cliente-servidor e aplicações web com muitos usuários simultâneos, muitos acessos concorrente e volumes de dados grandes o uso de SQLite não é recomendável. Em pequenos websites, aplicações que rodam em uma rede com poucos usuários ou em um único computador, aplicações para dispositivos móveis são o foco do SQLite.

8.2 Python + SQLite

Agora que se tem uma ambientação apropriada sobre bancos de dados e sobre o SQLite, está na hora de iniciar a parte prática.

8.2.1 Criação do primeiro banco de dados com Python + SQLite

Para os exemplos daqui por diante, suponha que está sendo construído um software para auxiliar no cadastro e no controle dos alunos de uma academia esportiva. Observe no Exemplo 8.1 o primeiro programa em que a dupla Python e SQLite é usada.

Na linha 1 é feita a importação da biblioteca `sqlite3`, para que o interpretador carregue os elementos que serão utilizados para conexão e interação com o SQLite. O próximo passo é estabelecer a conexão do Python com o banco de dados a ser aberto. Isso é feito com o método `connect` na linha 2. Esse método requer o nome do arquivo que contém o banco de dados. Caso esse banco de dados não exista, então, o método `connect` o criará como um banco de dados vazio. É exatamente isso que acontece na primeira vez em que o programa do Exemplo 8.1 for executado. O nome usado para o banco de dados é qualquer nome válido no sistema de arquivos do sistema operacional em que se está trabalhando. Como não foi especificada nenhuma pasta no nome do arquivo, então, ele será criado na mesma pasta em que está o programa. A extensão `“.db”` é uma mera escolha do programador, que pode escolher qualquer outra, ou nenhuma extensão, conforme julgue apropriado.

Exemplo 8.1 Primeiro programa usando Python + SQLite

```
import sqlite3 # linha 1
```

```
conector = sqlite3.connect("academia.db") # linha 2
```



```
cursor = conector.cursor() # linha 3
```

```
sql = ""
```

```
create table cadastro
```

```
(codigo integer, nome text, idade integer)
```

```
""
```

```
cursor.execute(sql) # linha 8
```

```
sql = ""
```

```
insert into cadastro
```

```
(codigo, nome, idade) values (1284, 'Pedro de Oliveira', 32)
```

```
""
```

```
cursor.execute(sql) # linha 13
```

```
sql = ""
```

```
insert into cadastro
```

```
(codigo, nome, idade) values (1309, 'Maria Lúcia Machado', 37)
```

```
""
```

```
cursor.execute(sql) # linha 18
```

```
conector.commit() # linha 19
```

```
cursor.close() # linha 20
```

```
conector.close() # linha 21
```

```
print("Abra a pasta do programa e veja se o arquivo está lá")
```

```
print("Fim do programa")
```

Na linha 3 é criado o objeto que foi identificado como “cursor”. Esse nome poderia ser qualquer outro válido, como x ou abc. No contexto de banco de dados, um cursor é um objeto utilizado pelo programador para se comunicar com o SGBD, enviando e recebendo comandos e dados. E o primeiro comando é enviado nas linhas 4 e 8. Nas linhas 4 a 7 é utilizado um *docstring* para preparar um comando (create table) que é atribuído ao objeto “SQL”, e na linha 8 é utilizado o método `cursor.execute` para enviar o SQL ao banco de dados. Observe que nas linhas 8 a 13 e 14 a 18 esse processo é repetido, porém, com outros tipos de comandos (insert into). Você não deve se preocupar, por enquanto, com os detalhes dos comandos create table e insert into, pois eles são explicados no Quadro 8.1.

Na linha 19 é usado o método `conector.commit`, que pode ser entendido como o comando necessário para efetivamente salvar em disco os comandos enviados para o banco de dados. Na verdade, uma operação de commit encerra uma transação de banco de dados, e isso será abordado posteriormente. Antes de finalizar o programa, é preciso encerrar o cursor e a conexão com o banco de dados. Isso é feito nas linhas 20 e 21 com os métodos `close` dos objetos cursor e conector.

A execução desse programa pode parecer estranha ao programador iniciante, pois não há interação do programa com o usuário, e seu resultado é apenas uma mensagem exibida no final, como mostrado na Figura 8.2.

Porém, ao abrir a pasta onde o programa está salvo, constatará que ali está gravado o arquivo “academia.db” criado nessa execução.

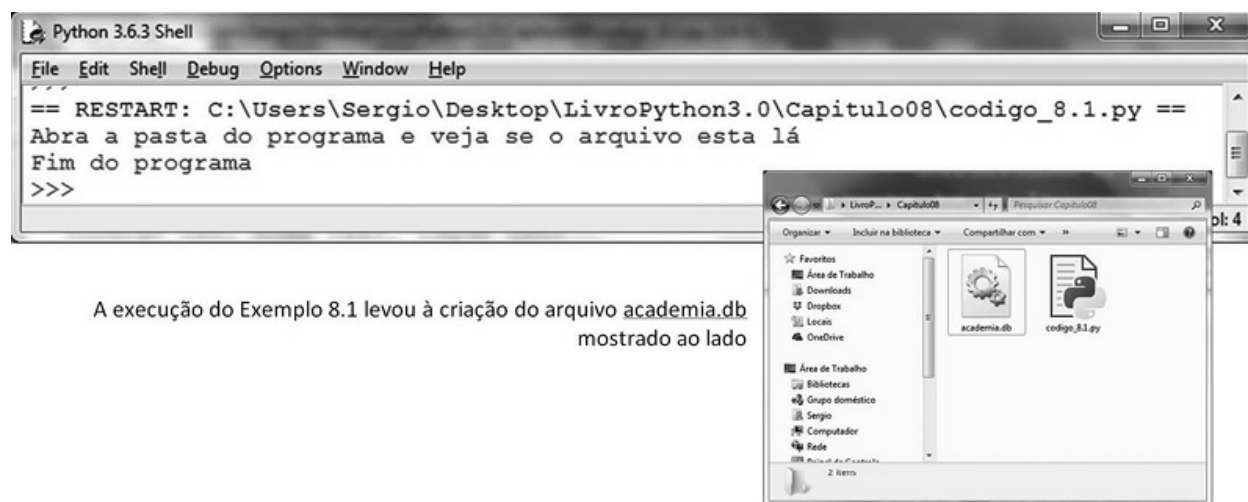


Figura 8.2 Resultado visível da execução do Exemplo 8.1.

Se executar esse programa novamente, o resultado será uma mensagem de erro, pois o programa se conectará ao banco de dados “academia.db” agora existente e nele tentará criar a tabela “cadastro”, também existente e levando à exibição do erro mostrado na Figura 8.3. Para rodar esse programa novamente sem obter a mensagem de erro, o arquivo do banco de dados deve ser excluído.

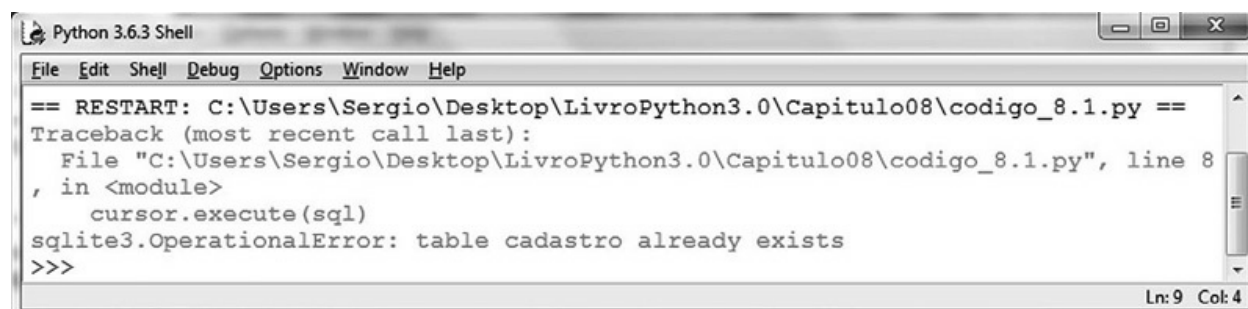


Figura 8.3 Resultado da segunda execução do Exemplo 8.1 – ocorre erro, a tabela cadastro já existe.

Comando

Descrição

create table cadastro (codigo integer, nome text, idade integer)	Este é um comando SQL do grupo DDL (<i>Data Definition Language</i>) visto no Item 8.1.3. Ele é usado para criar uma tabela. Se a tabela já existir, ocorre um erro. Deve-se fornecer o nome da tabela: “cadastro”. Deve-se fornecer o nome e o tipo de dados de cada campo que essa tabela conterà. Neste exemplo, são três campos: “código” e “idade” são números inteiros (integer) e “nome” é texto (text).
insert into cadastro (codigo, nome, idade) values (1284, 'Pedro', 32)	Este é um comando SQL do grupo DML (<i>Data Manipulation Language</i>) visto no Item 8.1.3. Ele é utilizado para inserir dados em uma tabela. O nome da tabela deve ser fornecido seguido dos campos que receberão dado. Não é obrigatório que todos os campos da tabela estejam presentes. E na sequência da cláusula <i>values</i> colocam-se os dados que vão preencher os campos especificados.

Quadro 8.1 Explicação dos comandos SQL usados no Exemplo 8.1.

Em síntese, o programa do Exemplo 8.1 faz as seguintes tarefas:

- 1.Carrega a biblioteca sqlite3 – linha 1.
- 2.Conecta-se com um BD existente ou cria o BD, caso não exista – linha 2.
- 3.Define um cursor para envio de comandos – linha 3.
- 4.Cria a tabela “Cadastro” – linhas 4 a 8.
- 5.Insere dois registros de dados na tabela “Cadastro” – linha 9 a 18.
- 6.Salva (commit) tudo no disco e encerra a transação – linha 19.

7. Encerra o cursor e a conexão – linha 20 e 21.

O resultado final é o arquivo do banco de dados gravado no disco com o nome especificado e dentro da pasta especificada.

8.2.2 Inspeccionando o banco de dados com o SQLite Studio

Qualquer banco de dados do SQLite é um arquivo gravado em disco. Como é um arquivo gravado em formato binário, não é possível abri-lo com um editor de textos, mas há várias opções para inspecionar seu conteúdo. Uma dessas opções é escrever um programa em Python para extrair os dados de uma tabela e exibí-los na tela. Isso será feito em seguida.

Existem diversos softwares que permitem acessar e manipular bancos de dados do SQLite. Muitos deles são gratuitos e de código livre, e alguns são pagos. Aqui será utilizado o SQLite Studio, que pode ser encontrado em <<https://sqlitestudio.pl>> e é um software livre, multiplataforma e que não requer instalação para ser usado. Basta acessar o endereço indicado, baixar a versão apropriada para seu computador, descompactar o arquivo e usar. A Figura 8.4 mostra a aparência dele. Para começar a usá-lo, primeiro é preciso adicionar a base de dados. Para isso, clique no menu *Database* → *Add a Database*, selecione o banco de dados criado com o Exemplo 8.1 na caixa *File* e clique no botão OK. O nome do BD aparecerá no painel à esquerda. Dê um duplo clique sobre ele e pronto. A tabela criada estará disponível e poderá ser aberta, visualizada, editada etc.

Esse programa é uma ferramenta simples, porém, muito útil para verificar se os programas desenvolvidos em Python e SQLite estão gerando os resultados esperados.

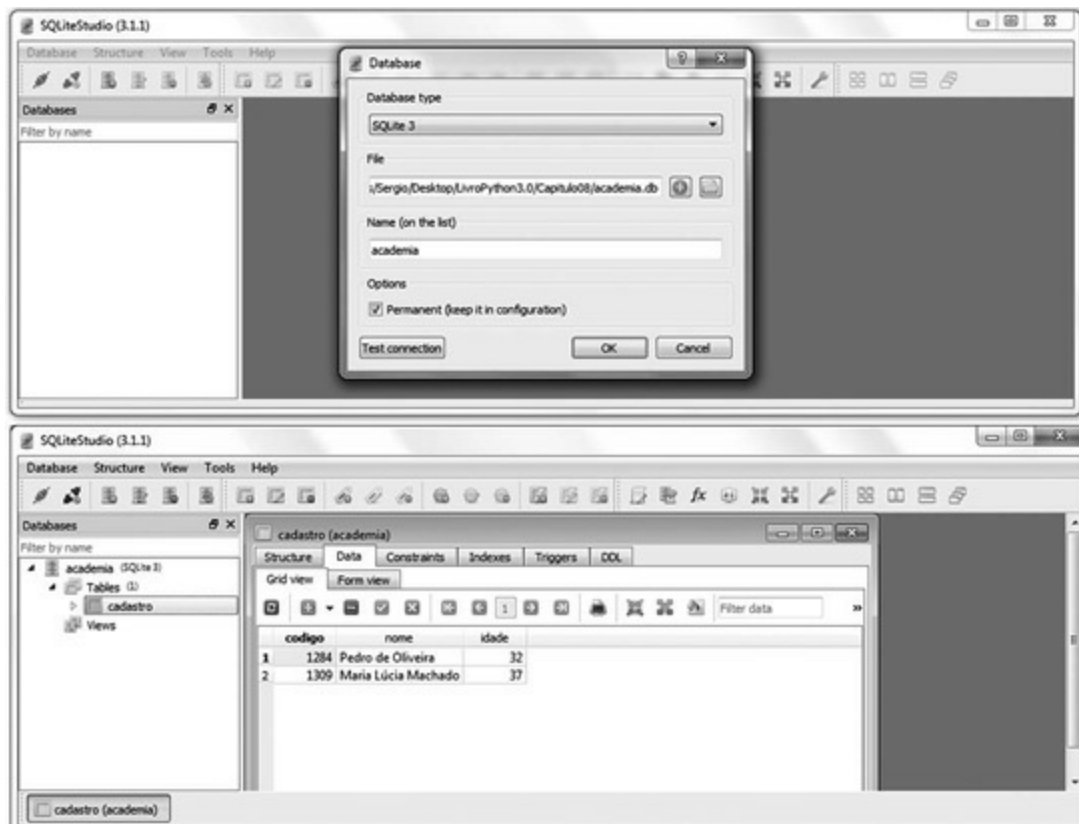


Figura 8.4 SQLite Studio – utilitário para inspecionar um banco de dados do SQLite.

8.2.3 Como acessar os dados inseridos usando Python

Agora que o banco de dados “academia.db” já está criado, contém uma tabela e esta contém dois registros, o próximo passo é escrever um programa que acesse tais dados e os exiba na tela. O Exemplo 8.2 executa essa tarefa.

Exemplo 8.2 Exemplo de programa que acessa os dados presentes em uma tabela

```
import sqlite3 # linha 1
```

```
conector = sqlite3.connect("academia.db") # linha 2
```

```
cursor = conector.cursor() # linha 3
```

```
sql = "select * from cadastro" # linha 4

cursor.execute(sql) # linha 5

dados = cursor.fetchall() # linha 6

cursor.close() # linha 7

conector.close() # linha 8

print("\nConsulta ao Banco de Dados 'academia.db' \n")

print("Dados da tabela 'cadastro'")

print("-" * 35)

print("{:7} {:20} {:>6}".format("Código", "Nome", "Idade"))

print("- " * 18)

for d in dados:

    print("{:<7} {:20} {:>6}".format(d[0], d[1], d[2]))

    print("-" * 35)

    print("Encontrados {} registros".format(len(dados)))

print("\n\nFim do programa")
```

```

Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Consulta ao Banco de Dados 'academia.db'

Dados da tabela 'cadastro'
-----
Código  Nome                Idade
-----
1284    Pedro de Oliveira          32
1309    Maria Lúcia Machado        37
-----
Encontrados 2 registros

Fim do programa
Ln: 19 Col: 4

```

Figura 8.5 Resultado da execução do Exemplo 8.2.

Nesse exemplo, as linhas 1, 2 e 3 têm a mesma função de suas correspondentes explicadas no exemplo anterior. A linha 4 define o comando SQL select, explicado no Quadro 8.2, de acesso aos dados da tabela “Cadastro” e a linha 5 executa o comando. Após essa execução, tem-se a linha 6, que é a chave desse exemplo. Nela foi usado o método `cursor.fetchall`, cujo efeito é retornar todos os registros produzidos pelo select em um objeto lista do Python. Uma vez que a lista “dados” já esteja carregada, a conexão com o BD pode ser encerrada por meio das linhas 7 e 8. Após isso, o conteúdo lista pode ser utilizado da maneira que for necessária ao programa. No caso do exemplo, foi elaborada uma saída formatada, que pode ser vista na Figura 8.5.

Comando	Descrição
select * from cadastro	Este é um comando SQL do grupo DQL (<i>Data Query Language</i>), visto no Item 8.1.3. Ele é usado para acessar os dados existentes em uma tabela. Essa é a forma mais simples desse comando. O “select *” significa que todos os campos devem ser selecionados e retornados. Alternativamente ao asterisco, poderia ser colocada uma lista de campos da tabela e, nesse caso, apenas os campos listados são retornados. A cláusula from determina qual tabela será acessada.

O comando select é o mais versátil e variado comando SQL. Outras formas deste serão vistas em outros exemplos.

Quadro 8.2 Explicação do comando SQL usado no Exemplo 8.2.

8.2.4 Inserindo mais registros na tabela “Cadastro”

No próximo exemplo mais registros serão inseridos na tabela “Cadastro” desse banco de dados. Para isso, o programa permanecerá em laço até que seja digitada uma linha nula, ou seja, o usuário apertar *Enter* sem digitar nada. Os dados de Código, Nome e Idade deverão ser digitados pelo usuário separados por vírgulas. Para cada linha válida digitada os dados devem ser inseridos na tabela.

A solução para essa nova situação está apresentada no Exemplo 8.3. Nessa solução, parte do código foi separado na função `ExibeDados`. Basicamente, o código dessa função é a parte final do código presente no Exemplo 8.2. Ela recebe o parâmetro `L`, que será a lista produzida pelo retorno dos dados do BD.

Esse programa inicia sua execução na linha 15. Nessa linha e na seguinte é feita a conexão com o banco e a criação do cursor. Na linha 17² o objeto `sql` recebe o string com o comando `insert into`, que, neste caso, é diferente do Exemplo 8.1. Note que a cláusula `values` faz referência a um conjunto de interrogações (`?, ?, ?`). Tais interrogações são **parâmetros** que indicam ao SQLite que os dados serão passados à parte, e seu modo de funcionamento é ilustrado na Figura 8.6. Observe no código do programa que a linha 27 contém o método `cursor.execute` com dois parâmetros: o SQL e um objeto lista identificado como `D`. Essa abordagem garante flexibilidade ao programa, uma vez que o mesmo SQL poderá ser executado várias vezes, cada uma com um conjunto de dados diferente carregado no objeto `D`.

Regras quanto ao uso de parâmetros passados do Python para o SQLite:

1. O comando SQL deve ser escrito com o caractere interrogação “?” no local onde deve entrar o dado real. Podem ser usadas tantas interrogações quantas necessário.
2. No método `cursor.execute(SQL, Objeto)` devem estar presentes, além do comando SQL, um segundo parâmetro (Objeto), que fornecerá os dados para que seja feita a substituição das interrogações por dados reais.
3. O segundo parâmetro deve ser uma tupla ou uma lista, mesmo que só exista uma interrogação no SQL. Tipos simples, conjuntos não são aceitos. Dicionários são aceitos, mas isso será visto no item 8.2.8.
4. A quantidade de interrogações e a quantidade de elementos no Objeto passado devem ser as mesmas. Caso contrário, ocorrerá erro.

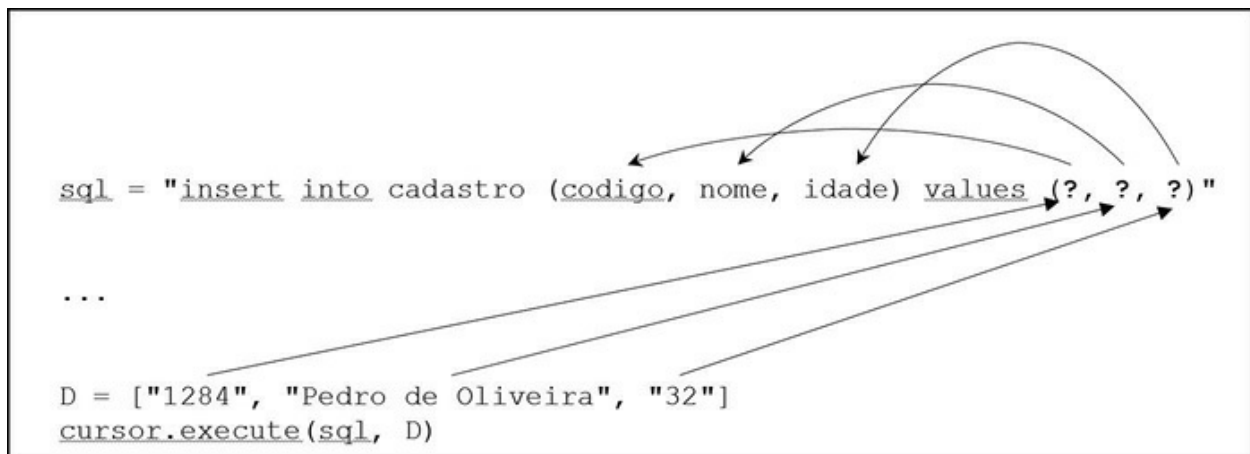


Figura 8.6 Ilustração de como são processados os comandos SQL com parâmetros.

Há, ainda, mais um aspecto importante relacionado à passagem de parâmetros do Python ao SQLite. É possível observar, na Figura 8.6, que o objeto `D` foi carregado com três elementos do tipo `string`. Porém, o primeiro e

o terceiro, “1284” e “32”, serão, respectivamente, associados aos campos “codigo” e “idade”, que são campos numéricos inteiros. Assim sendo, o SQLite está recebendo do Python dados com formatos em desacordo com o tipo dos campos que devem ser preenchidos.

Essa é uma situação que deve ser evitada. Em outros gerenciadores de bancos de dados, uma situação assim geraria um erro e a execução do SQL falharia. No entanto, o SQLite é um SGBD mais flexível que a maioria dos outros, e quando isso acontece ele tenta acomodar a situação verificando se os dados passados como string podem ser convertidos para os tipos numéricos dos campos de destino. Se essa conversão for possível, ele a executa e não gera qualquer mensagem de erro. Para mostrar que isso funciona, no Exemplo 8.3 a lista D foi mantida com três elementos string, como poderá constatar. Em outros programas deste livro os dados sempre serão convertidos para o tipo correto no Python antes do envio ao SQLite.

Prosseguindo com a descrição do exemplo, a carga de dados no objeto D é feita a partir da leitura do teclado. Os três dados: Código, Nome e Idade, deverão ser digitados separados por vírgulas. O comando input na linha 23 carregará o objeto Ler com o string lido do teclado e o método split será usado para separar as três partes usando o caractere “,” como delimitador. Se for necessário relembrar o uso do método split, reveja o Exercício resolvido 4.1. Com o objeto D carregado, executa-se o comando SQL e é feito o commit, linhas 27 e 28. Essas duas linhas foram protegidas com o comando try e, caso algum erro ocorra, o programa será desviado para a cláusula except, que exibirá uma mensagem de erro. As cláusulas else e finally também são usadas: a primeira para exibir a mensagem de que o registro foi inserido, caso tudo tenha dado certo, e a segunda exibe mensagem pedindo a próxima entrada de dados. Esse laço permanecerá em execução enquanto algo diferente de nulo for digitado.

Na parte final desse programa é utilizado um select para ler todos os dados da tabela “Cadastro” e a função ExibeDados é chamada para executar a exibição. A Figura 8.7 mostra a execução desse programa e a Figura 8.8 mostra, por meio do SQLite Studio, que os novos dados realmente estão contidos na nova tabela.

Exemplo 8.3 Inserção de mais registros na tabela “Cadastro”

```
import sqlite3

def ExibeDados(L): # linha 3

    """Exibe uma saída formatada dos dados contidos em L"""

    print("\nConsulta ao Banco de Dados 'academia.db' \n")

    print("Dados da tabela 'cadastro'")

    print("-" * 35)

    print("{:7} {:20} {:20>6}".format("Código", "Nome", "Idade"))

    print("- " * 18)

    for d in L:

        print("{:<7} {:20} {:>6}":20.format(d[0], d[1], d[2]))

    print("-" * 35)

    print("Encontrados {} registros".format(len(dados)))

conector = sqlite3.connect("academia.db") # linha 15
```

```
cursor = conector.cursor() # linha 16

sql = """

insert into cadastro

(codigo, nome, idade) values (?, ?, ?)

"""

print("Digite os dados separados por vírgulas")

print("Codigo,Nome,Idade") # linha 22

Ler = input() # linha 23

while Ler != "": # linha 24

D = Ler.split(",") # linha 25

try: # linha 26

cursor.execute(sql, D) # linha 27

conector.commit() # linha 28

except:

print("{} Dados inválidos".format(D))

else:

print(" *30, "...dados inseridos com sucesso")
```

finally:

```
print("Codigo,Nome,Idade")
```

```
Ler = input() # linha 35
```

```
sql = "select * from cadastro" # linha 37
```

```
cursor.execute(sql) # linha 38
```

```
dados = cursor.fetchall() # linha 39
```

```
cursor.close() # linha 40
```

```
conector.close() # linha 41
```

```
ExibeDados(dados) # linha 42
```

```
print("\n\nFim do programa")
```

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Digite os dados separados por vírgulas
Codigo, Nome, Idade
1193, José Carlos Siqueira, 25
..... dados inseridos com sucesso
Codigo, Nome, Idade
1450, Marta Silva Gaspar, 44
..... dados inseridos com sucesso
Codigo, Nome, Idade
1169, Luiz Sérgio Mendes, 21
..... dados inseridos com sucesso
Codigo, Nome, Idade
1185, Tereza Cavalcante, 36
..... dados inseridos com sucesso
Codigo, Nome, Idade

Consulta ao Banco de Dados 'academia.db'

Dados da tabela 'cadastro'
-----
Código  Nome  Idade
-----
1284    Pedro de Oliveira    32
1309    Maria Lúcia Machado  37
1193    José Carlos Siqueira  25
1450    Marta Silva Gaspar    44
1169    Luiz Sérgio Mendes    21
1185    Tereza Cavalcante     36
-----

Encontrados 6 registros

Fim do programa
>>> |
```

Figura 8.7 Resultado da execução do Exemplo 8.3.

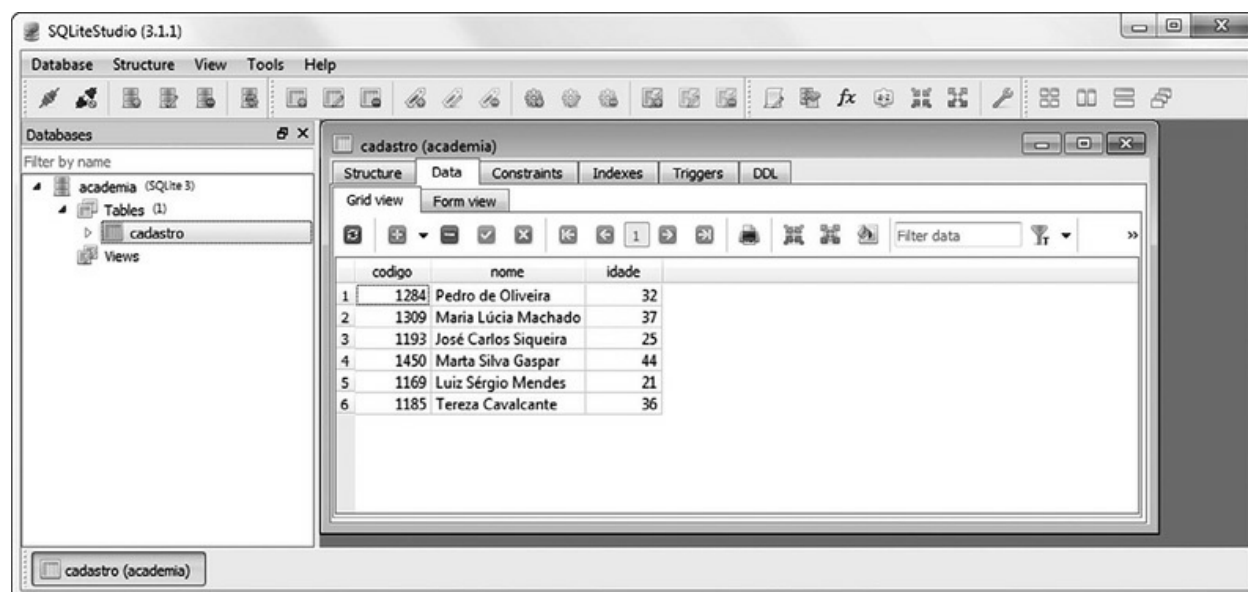


Figura 8.8 Visualização do BD com SQLite Studio, após a execução do Exemplo 8.3.

8.2.5 Alteração de tabelas

Muitas vezes é necessário alterar a estrutura de tabelas já existentes, incluindo ou excluindo campos, acrescentando índices ou até mesmo alterando sua chave primária. Em um projeto de um sistema novo benfeito isso não deve ocorrer. Porém, ao longo da vida útil de um sistema mudanças assim podem acontecer em virtude de alterações nas regras de negócio impostas por fatores externos ao sistema, tais como mudanças no negócio, implantação de novas funcionalidades, mudanças de legislação, adoção de novas tecnologias, entre outras motivações. Serão vistos a seguir três exemplos em que isso é realizado.

No Exemplo 8.4 são incluídos quatro novos campos na tabela “Cadastro”. Um campo inteiro que será utilizado para armazenar o número do curso que a pessoa frequenta na academia, a data de ingresso, que é uma data, e o peso e a altura do aluno, que são números reais. Depois de incluídos, esses campos precisarão ser carregados com conteúdo. Os campos curso e data de ingresso serão respectivamente 10 (musculação) e 01/07/2017 (quando a academia iniciou as atividades e os alunos já cadastrados a frequentam desde o início).

Para efetuar a inclusão de novos campos, é usado um comando SQL do grupo DDL alter table, e para atualizar o conteúdo dos campos é utilizado update, um comando do grupo DML. Ambos são explicados no Quadro 8.3.

Nesse programa não há nenhum aspecto novo no que diz respeito ao Python. São os mesmos comandos utilizados nos exemplos anteriores para: conexão com o banco de dados, criação do cursor, execução dos comandos DDL e DML, commit (necessário apenas por causa do update) e encerramento da conexão. Esse programa não interage com o usuário, e ao

seu término uma mensagem é mostrada na tela indicando que o BD foi atualizado.

Comando	Descrição
alter table cadastro add curso integer	Este é um comando SQL do grupo DDL (<i>Data Definition Language</i>) visto no Item 8.1.3. Ele é usado para acrescentar campos a uma tabela já existente. Os novos campos inseridos não terão valor e estarão com conteúdo nulo (<i>NULL</i>). Deve-se usar um comando deste para cada campo a ser incluído.
update cadastro set curso = 16, dtingr = '01/07/2017'	Este é um comando SQL do grupo DML (<i>Data Manipulation Language</i>) visto no Item 8.1.3. Este comando atualiza todos os registros da tabela colocando em cada campo os valores que lhes foram atribuídos por meio da cláusula set.

Quadro 8.3 Explicação do comando SQL utilizado no Exemplo 8.2.

Exemplo 8.4 Inclusão de campos em tabelas

```
import sqlite3

conector = sqlite3.connect("academia.db")

cursor = conector.cursor()

sql = "alter table cadastro add curso integer"

cursor.execute(sql)

sql = "alter table cadastro add dtingr date"

cursor.execute(sql)

sql = "alter table cadastro add peso double"
```

```
cursor.execute(sql)
```

```
sql = "alter table cadastro add altura double"
```

```
cursor.execute(sql)
```

```
sql = "update cadastro set curso = 10, dtinigr = '01/07/2017'"
```

```
cursor.execute(sql)
```

```
conector.commit()
```

```
cursor.close()
```

```
conector.close()
```

```
print("\n\nBanco de dados atualizado com sucesso")
```

```
print("\n\nFim do programa")
```



Na Figura 8.9 pode constatar que a tabela está alterada.

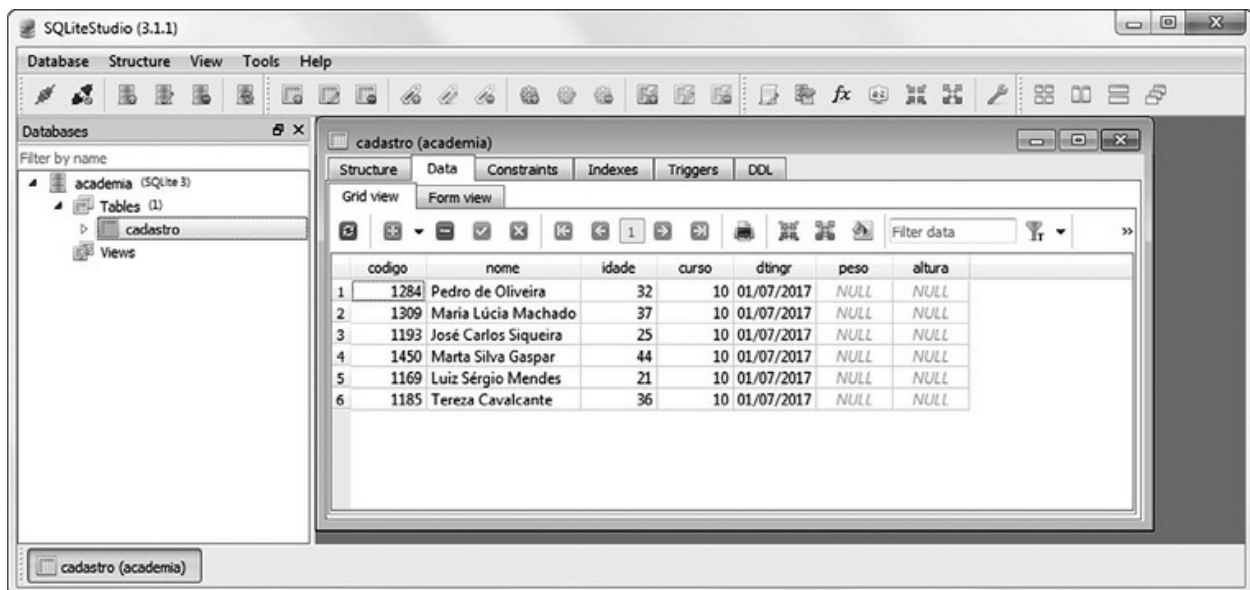


Figura 8.9 Resultado da execução do Exemplo 8.4 com os novos campos incluídos.

O Exemplo 8.5 é utilizado para atualizar os campos de peso e altura. Como esses dados variam para cada pessoa, é preciso fazer um update em separado usando a cláusula where para limitar os registros que serão afetados. O Quadro 8.4 mostra o formato deste SQL.

Comando	Descrição
update cadastro set peso = ?, altura = ? where codigo = ?	Este é um comando SQL do grupo DML (<i>Data Manipulation Language</i>) visto no Item 8.1.3. Este comando atualiza apenas os registros que tiverem o código que foi usado na cláusula where. Se nenhum registro satisfizer o critério, nada acontece. Neste caso, são utilizadas as interrogações porque serão feitas várias atualizações passando os dados uma pessoa por vez.

Quadro 8.4 Explicação do comando SQL usado no Exemplo 8.2.

Para escrever esse programa, os dados de peso e altura de cada pessoa devem estar disponíveis. Seria possível fazer como no Exemplo 8.3 e pedir

para o usuário fazer a digitação. Porém, aqui será lido um arquivo texto no qual cada linha contém código, peso e altura de uma pessoa separados com “;”, como no exemplo a seguir:

1284;93.5;1.74

1309;53.6;1.62 ... etc.

Como o arquivo é pequeno, utilizou-se o método `readlines` para carregar uma lista com o arquivo inteiro e, em seguida, a lista é processada, fazendo-se o processamento de cada um de seus elementos.

Exemplo 8.5 Atualização do banco de dados a partir da leitura de um arquivo em disco

```
import sqlite3
```

```
arq = open("PesoAltura.txt", "r")
```

```
L = arq.readlines()
```

```
arq.close()
```

```
print("\nLinhas do arquivo")
```

```
sql = """update cadastro set peso = ?, altura = ?
```

```
where codigo = ?"""
```

```
conector = sqlite3.connect("academia.db")
```

```
cursor = conector.cursor()
```

```
for s in L:
```

```
d = s.rstrip()
```

```
d = d.split(";")
```

```
cursor.execute(sql, (d[1], d[2], d[0]))
```

```
conector.commit()
```

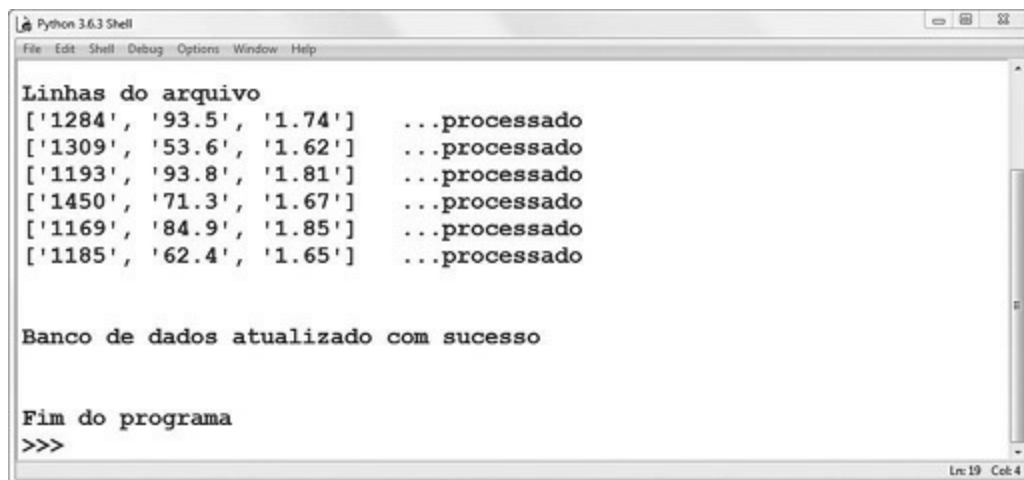
```
print(d, " ...processado")
```

```
cursor.close()
```

```
conector.close()
```

```
print("\n\nBanco de dados atualizado com sucesso")
```

```
print("\n\nFim do programa")
```



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Linhas do arquivo
['1284', '93.5', '1.74'] ...processado
['1309', '53.6', '1.62'] ...processado
['1193', '93.8', '1.81'] ...processado
['1450', '71.3', '1.67'] ...processado
['1169', '84.9', '1.85'] ...processado
['1185', '62.4', '1.65'] ...processado

Banco de dados atualizado com sucesso

Fim do programa
>>>
```

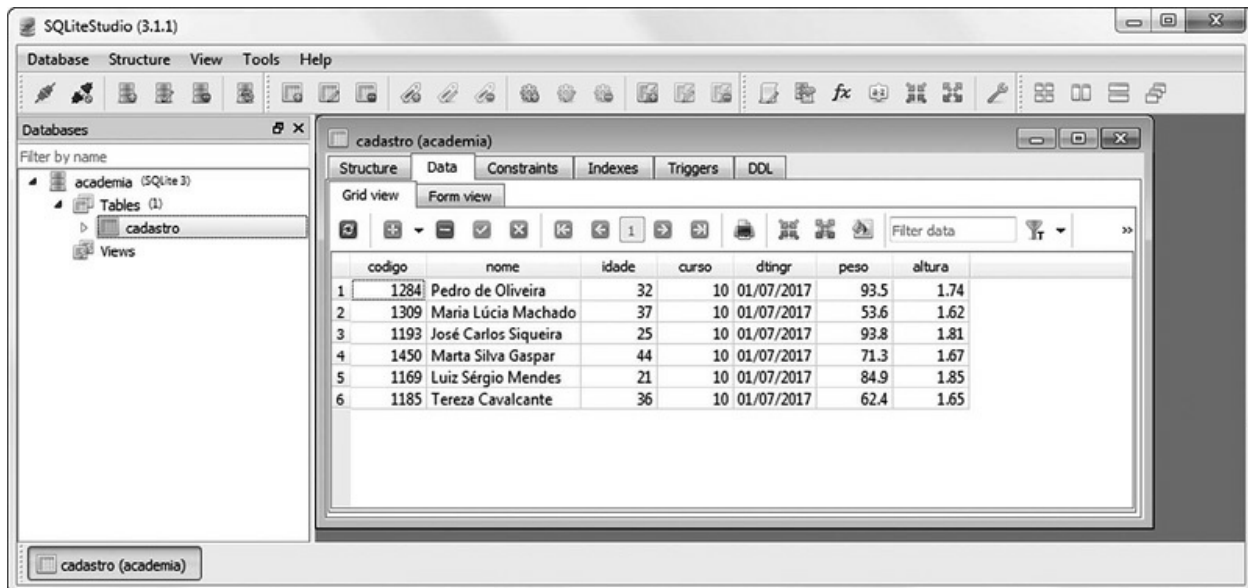


Figura 8.10 Resultado da execução do Exemplo 8.5 com os campos peso e altura atualizados.

Na Figura 8.10, pode-se verificar que de fato cada registro está atualizado com os valores corretos de peso e altura. No caso desse exemplo, o campo “codigo” foi usado como critério de seleção para a atualização dos registros. É frequente em bancos de dados que isso aconteça. Na verdade, um campo como esse é tão importante que recebe tratamento especial.

Na tabela “Cadastro” o campo “codigo” é uma **chave primária**. Porém, quando a tabela foi criada isso não foi especificado. Campos-chave são indexados e, por isso, permitem acesso muito mais rápido aos dados. Eles também devem obrigatoriamente ser preenchidos, não podendo ser *NULL*, e não pode haver repetição, ou seja, cada registro tem sua chave primária com valor único.

O que será feito no Exemplo 8.6 é transformar o campo “código” em chave primária. No entanto, há um problema: o SQLite não permite que essa transformação seja feita diretamente. Para realizar essa transformação, é preciso fazer os seguintes passos:

1. Clonar a tabela “cadastro” para uma tabela temporária.
2. Eliminar a tabela “cadastro”.
3. Criar uma nova tabela “cadastro”, dessa vez, com chave primária e todos os demais campos.
4. Copiar os dados da tabela temporária para a tabela “cadastro”.
5. Eliminar a tabela temporária.

É isso que está implementado no código a seguir.

Exemplo 8.6 Transformação do campo “codigo” em chave primária

```
import sqlite3

print("\nCriação de Chave Primária na tabela 'cadastro'")

conector = sqlite3.connect("academia.db")

cursor = conector.cursor()

sql = "create table temp as select * from cadastro" # passo 1

cursor.execute(sql)

sql = "drop table cadastro" # passo 2

cursor.execute(sql)

sql = ""

create table cadastro (
```

codigo integer NOT NULL PRIMARY KEY,

nome text,

idade integer,

curso integer,

dtinigr date,

peso double,

altura double) “””

cursor.execute(sql) # passo 3

sql = “””

insert into cadastro

(codigo, nome, idade, curso, dtinigr, peso, altura)

select codigo, nome, idade, curso, dtinigr, peso, altura

from temp

“””

cursor.execute(sql) # passo 4

conector.commit() # commit necessário para o insert into

sql = “drop table temp”


```

cursor.execute(sql) # passo 5

cursor.close()

conector.close()

print("\n\nBanco de dados atualizado com sucesso")

print("\n\nFim do programa")

```

O resultado do processamento desse exemplo pode ser visto em seguida. Usando o SQLite Studio, clicando na aba *Structure*, pode-se verificar que agora o campo “codigo” está identificado como Not NULL e Primary Key.

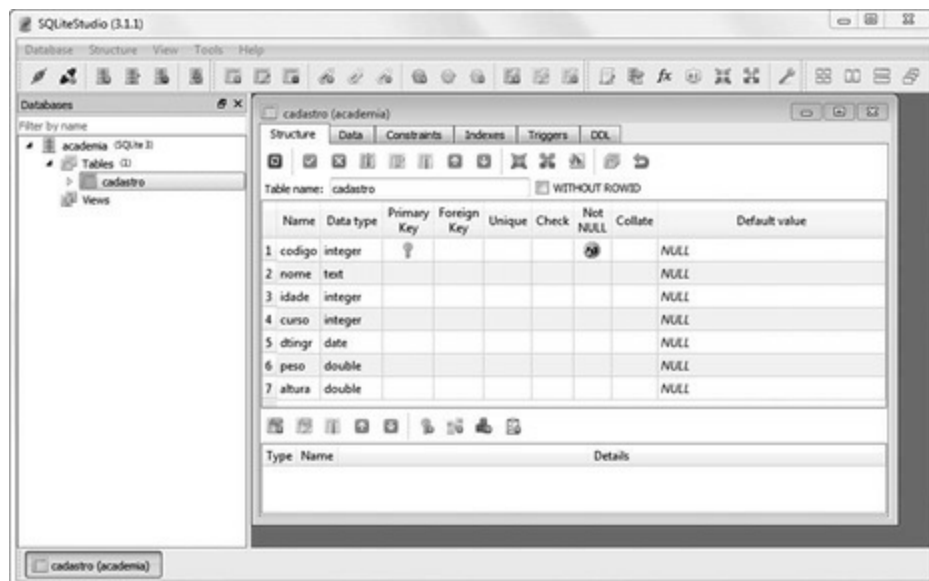


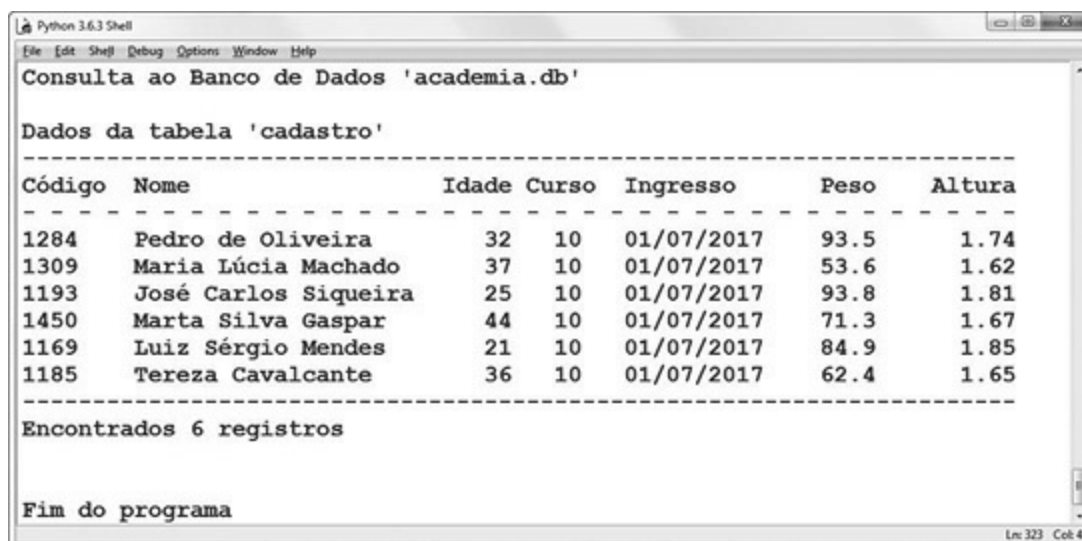
Figura 8.11 Resultado do Exemplo 8.6 que transforma o campo “codigo” em chave primária.

8.2.6 Alteração dos Exemplos 8.2 e 8.3

Tarefa adicional

Agora, a tabela “Cadastro” tem diversos campos a mais do que tinha quando foi feito o Exemplo 8.2, que exibe em tela todos os seus registros. Adapte

aquele exemplo para exibir a tabela ampliada com a formatação mostrada na Figura 8.12.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Consulta ao Banco de Dados 'academia.db'

Dados da tabela 'cadastro'

-----
Código  Nome                Idade  Curso  Ingresso    Peso    Altura
-----
1284    Pedro de Oliveira          32    10    01/07/2017  93.5    1.74
1309    Maria Lúcia Machado       37    10    01/07/2017  53.6    1.62
1193    José Carlos Siqueira      25    10    01/07/2017  93.8    1.81
1450    Marta Silva Gaspar        44    10    01/07/2017  71.3    1.67
1169    Luiz Sérgio Mendes        21    10    01/07/2017  84.9    1.85
1185    Tereza Cavalcante         36    10    01/07/2017  62.4    1.65
-----

Encontrados 6 registros

Fim do programa
```

Figura 8.12 Exibição ampliada dos dados da tabela “Cadastro”.

Tarefa adicional

O Exemplo 8.3 também pode ser adaptado para ler as novas informações para o cadastro do aluno da academia (todas em uma linha e separadas por vírgulas) e, após a leitura, fazer a inserção no banco de dados. Se desejar, você pode fazer essa alteração. Ao fazer a inserção de novos alunos, forneça variados cursos e diferentes datas de ingresso.

8.2.7 Criação e preenchimento de nova tabela

O banco de dados da academia já conta com a tabela “Cadastro”. Agora, será criada uma nova tabela para conter dados dos cursos oferecidos. Essa tabela terá os campos descritos no Quadro 8.5 e os dados do Quadro 8.6.

Campo	Tipo	Observações
codcurso	Número inteiro (integer)	Não pode ser nulo e é chave primária.
nomecurso	Texto	Será usado para conter o nome do curso.
valores	Número real (double)	Será usado para conter o valor da mensalidade do curso.

Quadro 8.5 Campos da tabela “Cursos”.

Código	Nomecurso	Valores
10	Musculação	110,00
11	Treino aeróbico	110,00
12	Combo 1: musculação + aeróbico	180,00
15	Natação	180,00
22	Pilates	165,00
25	Combo 2: pilates + aeróbico	214,00
30	Crossfit	180,00
41	Muay Thai	140,00
42	Jiu Jitsu	140,00
43	Boxe	140,00

Quadro 8.6 Campos da tabela “Cursos”.

Esse exemplo tem por objetivo demonstrar o uso do método `cursor.executemany`, que pode ser utilizado no lugar do método `cursor.execute` para os casos em que se necessita trabalhar com múltiplos conjuntos de dados. O objeto `DadosCursos` é uma lista e foi carregado com diversas tuplas, cada uma contendo os três dados referentes a um curso. Essa lista de tuplas é passada para o método `cursor.executemany` em conjunto com o comando SQL definido nas linhas 14 a 17, o qual se encarregará de executar o SQL uma vez para cada tupla contida na lista `DadosCursos`.

Exemplo 8.7 Criação de nova tabela e carga de dados usando `executemany`

```
import sqlite3

print("\n\nCria e carrega tabela 'cursos'")

conector = sqlite3.connect("academia.db")
```

```
cursor = conector.cursor()

#Cria a nova tabela

sql = """

create table cursos

(codcurso integer not NULL Primary Key,

nomecurso text, valormes double)

"""

cursor.execute(sql)

print("\n ...tabela cursos criada")

#Carrega com dados dos cursos # linha 13

sql = """

insert into cursos (codcurso, nomecurso, valores)

values(?, ?, ?)

""" # linha 17

DadosCursos = [ # linha 18

(10, "Musculação", 110.00),

(11, "Treino Aeróbico", 110.00),
```

(12, “Combo 1: Musculação + Aeróbico”, 180.00),

(15, “Natação”, 180.00),

(22, “Pilates”, 165.00),

(25, “Combo 2: Pilates + Aeróbico”, 240.00),

(30, “Crossfit”, 180.00),

(41, “Muay Thai” , 140.00),

(42, “Jiu Jitsu”, 140.00),

(43, “Boxe”, 140.00)]

print(“\n ...dados a serem carregados”)

for dados in DadosCursos:

print(“ “, dados)

cursor.executemany(sql, DadosCursos) # linha 32

conector.commit()

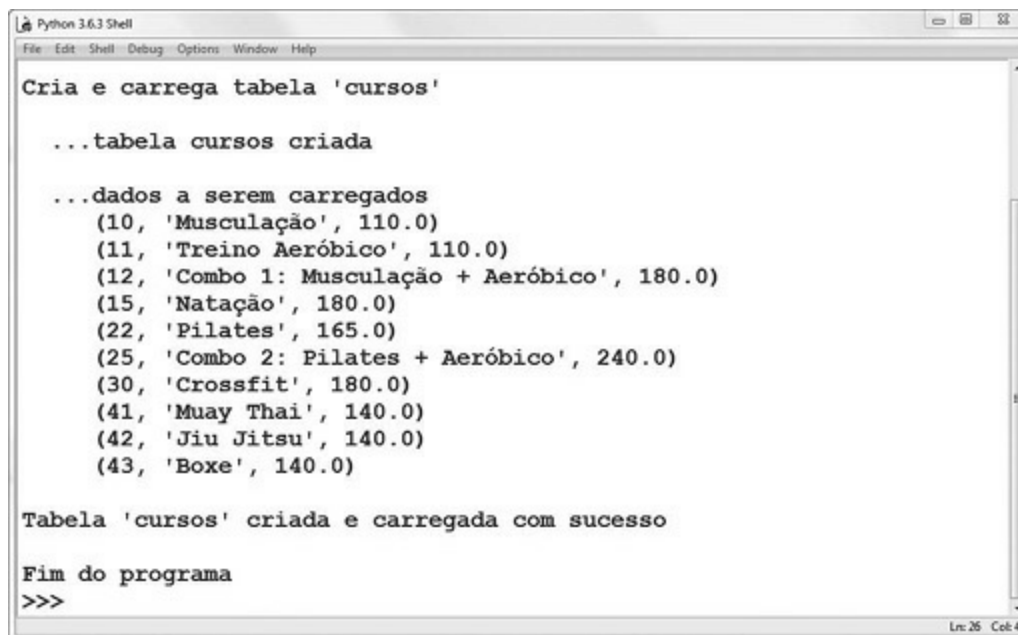
cursor.close()

conector.close()

print(“\nTabela ‘cursos’ criada e carregada com sucesso”)

print(“\nFim do programa”)

Como resultado da execução desse exemplo, a tabela “cursos” será criada no banco de dados da academia. As Figuras 8.13 e 8.14 mostram esse resultado. Uma observação sobre o SQLite Studio: caso tenha executado esse programa com o Studio aberto e conectado ao banco de dados, ao alternar do programa Python para o Studio, a nova tabela não aparecerá automaticamente. Para que apareça, deve-se acionar o comando de menu *Database → Refresh selected database scheme* ou pressionar a tecla F5.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Cria e carrega tabela 'cursos'

...tabela cursos criada

...dados a serem carregados
(10, 'Musculação', 110.0)
(11, 'Treino Aeróbico', 110.0)
(12, 'Combo 1: Musculação + Aeróbico', 180.0)
(15, 'Natação', 180.0)
(22, 'Pilates', 165.0)
(25, 'Combo 2: Pilates + Aeróbico', 240.0)
(30, 'Crossfit', 180.0)
(41, 'Muay Thai', 140.0)
(42, 'Jiu Jitsu', 140.0)
(43, 'Boxe', 140.0)

Tabela 'cursos' criada e carregada com sucesso

Fim do programa
>>>
```

Figura 8.13 Resultado da execução do Exemplo 8.7 que cria e carrega a tabela “Cursos”.

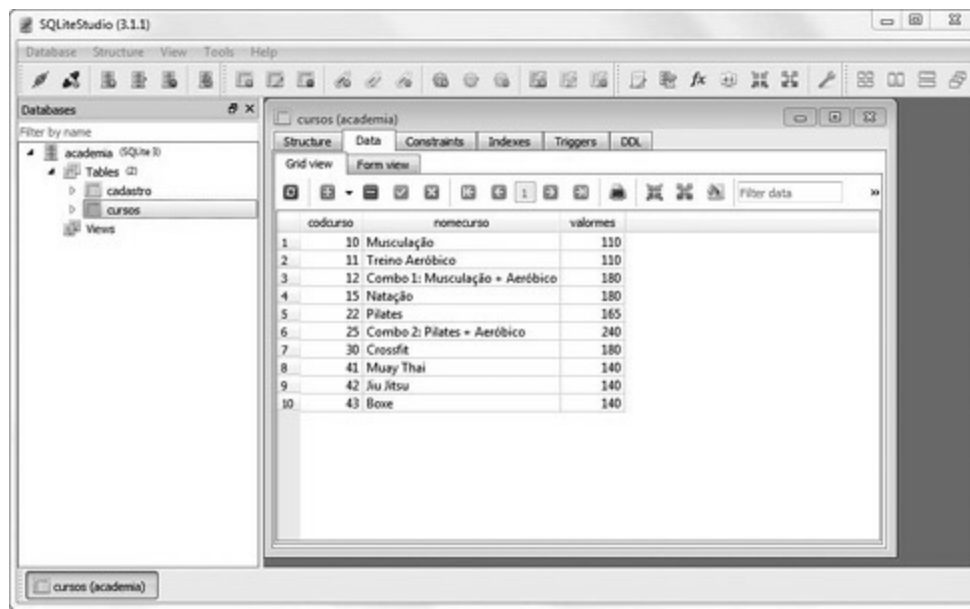


Figura 8.14 Como fica nova tabela “Cursos” no SQLite Studio.

Tarefa adicional

Escreva um programa para inserir novos alunos na tabela cadastro, usando o método executemany. Faça esse programa de modo que os dados dos alunos a serem inseridos no cadastro sejam lidos de um arquivo em disco, a exemplo do que foi feito no Exemplo 8.5. Será necessário criar esse arquivo para testar o programa. O Quadro 8.5 contém dados sugeridos para realizar os testes.

```
1227;Maria do Carmo Sila;54;12;16/07/2017;88.2;1.59
1377;Onofre Vilasboas;52;15;14/10/2017;108.7;1.78
1453;Takeshi Yamazaki;19;12;09/08/2017;71.1;1.75
1073;Lucas Marcolino;43;15;01/09/2017;84.9;1.80
1487;João Silva Ponte;23;30;10/07/2017;66.7;1.77
1002;Kaique Guimarães;19;30;10/07/2017;70.7;1.83
1230;Luciana Cardal Pó;46;41;14/10/2017;84.8;1.67
1103;Giulliana Trovatto;22;25;15/09/2017;85.1;1.69
1233;Piera Trovatto;46;25;15/09/2017;84.8;1.62
1046;Vinícius Catagallo;31;41;16/07/2017;98.0;1.79
```

Quadro 8.7 Dados de novos alunos para a tarefa adicional referente ao Item 8.2.7.

8.2.8 Exclusão de registros em uma tabela

No próximo exemplo serão apresentados alguns novos aspectos da programação para banco de dados com Python e SQLite. Esse programa contém duas funções, além da parte principal:

- A função `ExibeCursos` é usada para produzir uma saída em tela formatada com aspecto de tabela que exhibe todos os cursos cadastrados.
- A função `ExcluiCurso(Codigo)` recebe o parâmetro `Codigo` e verifica se existe um curso com o código que é passado. Caso não exista, ela retorna uma mensagem de texto informando que não existe. Caso exista, é feita a exclusão e retorna uma mensagem de texto informando que a exclusão ocorreu.
- A parte principal do programa contém um laço que só terminará quando for digitado zero (a estratégia utilizada foi laço infinito interrompido por `break` quando a opção digitada for igual a zero). Dentro desse laço, é feita uma chamada da função `ExibeCursos`, é apresentado um texto solicitando a entrada do usuário e, uma vez que algo tenha sido digitado, é feita uma chamada à função `ExcluiCurso` para processar a entrada do usuário.

Antes que o laço termine, a instrução

```
dummy = input("Pressione Enter para prosseguir...")
```

foi acrescentada para que o programa dê uma parada nesse ponto e o usuário tenha condição de ler a mensagem exibida. Após pressionar *Enter*, a tela é redesenhada (rola para cima) e o laço inicia uma nova

repetição.

Exemplo 8.8 Exclusão de registro em tabela

```
import sqlite3

def ExibeCursos():

    """Exibe os cursos existentes em uma saída formatada"""

    sql = "select * from cursos"

    cursor.execute(sql)

    dados = cursor.fetchall()

    print("\nConsulta ao Banco de Dados 'academia.db' \n")

    print("Dados da tabela 'cursos'")

    print("-" * 49)

    print("{:7} {:30} {:>11}".format(

        "Código", "Nome do Curso", "Val./Mês"))

    print("- " * 25)

    for d in dados:

        print("{:<7} {:30} {:>10.2f}".format(d[0], d[1], d[2]))

    print("-" * 49)
```

```
print("Encontrados {} registros\n".format(len(dados)))
```

```
def ExcluiCurso(Codigo):
```

```
    """Verifica se o curso existe e o exclui."""
```

```
    sql = """select Count(codcurso) from cursos
```

```
    where codcurso = :param"""
```

```
    cursor.execute(sql, {'param' : Codigo})
```

```
    x = cursor.fetchone()
```

```
    print(x[0])
```

```
    if x[0] == 0:
```

```
        return "Curso {} não Existe".format(Codigo)
```

```
    else:
```

```
        sql = "delete from cursos where codcurso = :param"
```

```
        cursor.execute(sql, {'param' : Codigo})
```

```
        conector.commit()
```

```
        return "Curso {} Excluído".format(Codigo)
```

```
# O programa começa a executar por aqui
```

```
conector = sqlite3.connect("academia.db")
```

```
cursor = conector.cursor()

while True:

    ExibeCursos()

    print("Para excluir um curso digite o Código")

    print("Para sair do programa digite 0 (zero)")

    Opc = int(input("sua escolha >> "))

    if Opc == 0:

        break

    else:

        Msg = ExcluiCurso(Opc)

        print(Msg)

        dummy = input("Pressione Enter para prosseguir...")

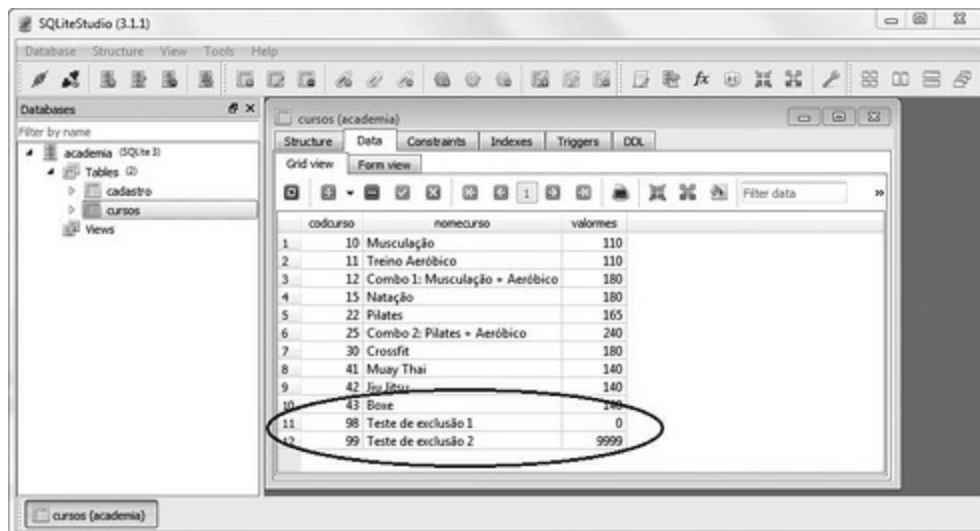
        cursor.close()

        conector.close()

        print("\n\nFim do programa")
```

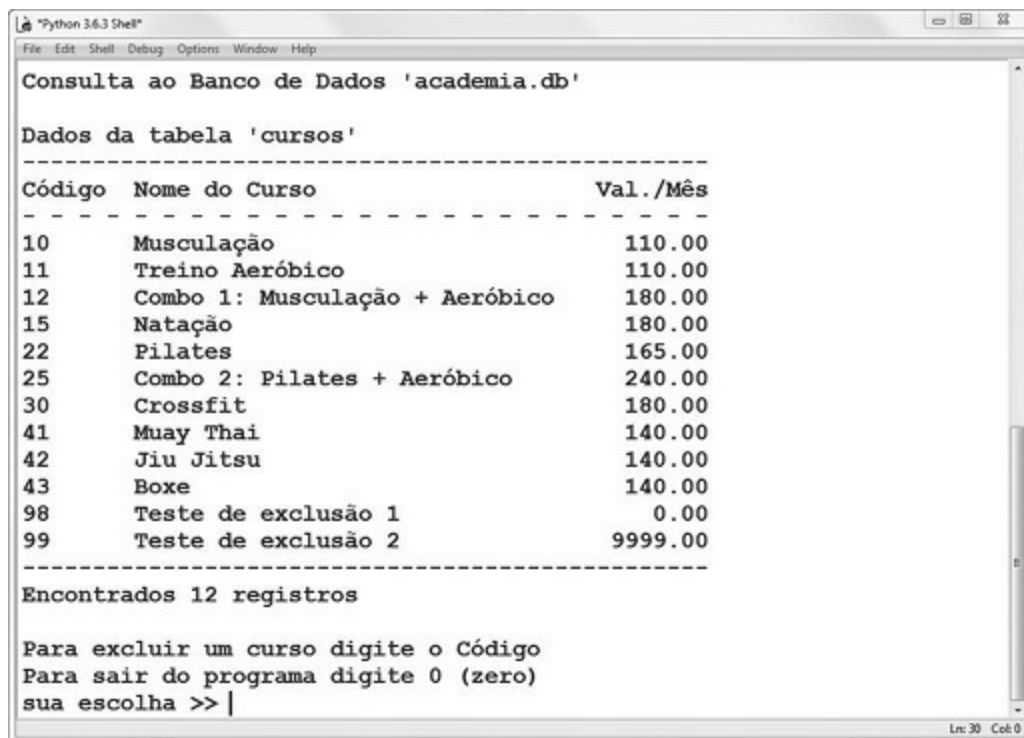
Os cursos 98 e 99 mostrados na Figura 8.15 serão utilizados como exemplo para exclusão. Para testar esse programa, esses dois registros adicionais foram incluídos usando o SQLite Studio, e a Figura 8.16 mostra que esses

dois cursos aparecem na tela de execução do Python. Isso mostra que não importa qual software seja utilizado para manipular os dados. Por estarem em um local de armazenamento centralizado (o arquivo “academia.db”), todos os programas que acessarem o banco de dados terão acesso aos registros das tabelas.



	codcurso	nomecurso	valores
1	10	Musculação	110
2	11	Treino Aeróbico	110
3	12	Combo 1: Musculação + Aeróbico	180
4	15	Natação	180
5	22	Pilates	165
6	25	Combo 2: Pilates + Aeróbico	240
7	30	Crossfit	180
8	41	Muay Thai	140
9	42	Jiu Jitsu	140
10	43	Bose	140
11	98	Teste de exclusão 1	0
12	99	Teste de exclusão 2	9999

Figura 8.15 Tabela “cursos” contendo mais dois cursos inseridos diretamente no SQLite Studio.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Consulta ao Banco de Dados 'academia.db'

Dados da tabela 'cursos'
-----
Código  Nome do Curso                Val./Mês
-----
10      Musculação                      110.00
11      Treino Aeróbico                  110.00
12      Combo 1: Musculação + Aeróbico  180.00
15      Nataação                       180.00
22      Pilates                        165.00
25      Combo 2: Pilates + Aeróbico     240.00
30      Crossfit                       180.00
41      Muay Thai                      140.00
42      Jiu Jitsu                      140.00
43      Boxe                          140.00
98      Teste de exclusão 1             0.00
99      Teste de exclusão 2            9999.00
-----

Encontrados 12 registros

Para excluir um curso digite o Código
Para sair do programa digite 0 (zero)
sua escolha >> |
```

Figura 8.16 Os dados digitados no SQLite Studio estão disponíveis para o Python.

Ao digitar 98 no programa, o registro correspondente acaba sendo excluído.

```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Consulta ao Banco de Dados 'academia.db'

Dados da tabela 'cursos'
-----
Código  Nome do Curso                Val./Mês
-----
10      Musculação                    110.00
11      Treino Aeróbico                110.00
12      Combo 1: Musculação + Aeróbico 180.00
15      Nataação                     180.00
22      Pilates                      165.00
25      Combo 2: Pilates + Aeróbico 240.00
30      Crossfit                     180.00
41      Muay Thai                    140.00
42      Jiu Jitsu                     140.00
43      Boxe                         140.00
99      Teste de exclusão 2          9999.00
-----

Encontrados 11 registros

Para excluir um curso digite o Código
Para sair do programa digite 0 (zero)
sua escolha >> |
```

Figura 8.17 A tabela “cursos” após a exclusão do registro 98.

Por fim, nesse exemplo foi usada uma forma diferente de passagem de parâmetro para o comando SQL, a passagem com parâmetro nomeado. Nos dois SQLs utilizados dentro da função ExcluiCurso e destacados a seguir, aparece a construção `codcurso = :param`, em que `:param` é um parâmetro nomeado e a palavra usada poderia ser qualquer uma no lugar do “param”. Veja em seguida como seria a alternativa no caso de ser utilizado um parâmetro posicional.

foi usado assim – parâmetro nomeado

```
sql = “select Count(codcurso) from cursos where codcurso = :param”
```

mas poderia ter sido assim – parâmetro posicional

```
sql = “select Count(codcurso) from cursos where codcurso = ?”
```

foi usado assim – parâmetro nomeado

```
sql = "delete from cursos where codcurso = :param"
```

mas poderia ter sido assim – parâmetro posicional

```
sql = "delete from cursos where codcurso = ?"
```

```
cursor.execute(sql, {"param" :Codigo})
```

Quando é utilizado o parâmetro nomeado, os dados devem ser fornecidos por meio de um dicionário do Python. No caso desse exemplo, o dicionário foi construído da seguinte maneira: {"param" :Codigo}, em que a chave "param" deve coincidir com o parâmetro nomeado existente no comando SQL, e o valor associado a essa chave será usado em sua execução.

Exercícios propostos

1. Escreva um programa que crie um banco de dados chamado "agenda.db". Nesse BD deve existir uma tabela de contatos com os seguintes campos:

Nome do campo	Tipo	Descrição
NumContato	integer (usar autonumeração)	Chave primária. Número inteiro de identificação gerado automaticamente no momento do cadastro.
Nome	Text	Nome da pessoa.
Cel	Text	Número do celular.
Tel	Text	Número do telefone fixo.
Email	Text	Endereço de e-mail.
Aniver	Text	Data de aniversário da pessoa (armazenar como texto porque nem sempre se sabe o ano).

2. Escreva um programa que exiba a agenda o exercício 1 e permita efetuar as seguintes ações:

- cadastrar novas pessoas;
- alterar os telefones e o e-mail;
- excluir pessoas da agenda.

3. Escreva um programa que crie um banco de dados chamado “musicas.db”. Esse BD deve conter três tabelas, mostradas a seguir.

Tabela “Musicas” contém a relação de músicas disponíveis no computador.

Nome do campo	Tipo	Descrição
nummusica	integer (usar autonumeração)	Chave primária. Número inteiro de identificação gerado automaticamente no momento do cadastro.
nomemus	text	Nome da música.
artista	text	Usado para inserir o nome do artista ou banda.
album	text	Nome do álbum em que foi lançada.
ano	integer	Ano de lançamento.
arquivo	text	Nome completo (inclui pasta) onde está salvo o arquivo .mp3.

Tabela “Nomespl” permite cadastrar o nome da *playlist* e sua data de criação.

Nome do campo	Tipo	Descrição
nomepl	text	Chave primária.

		Nome da <i>playlist</i> .
data	date	Data de criação da <i>playlist</i> .

Tabela “Playlist” permite relacionar as músicas cadastradas na tabela “Musicas”, com os nomes de *playlists* cadastrados na tabela “Nomespl”.

Nome do campo	Tipo	Descrição
nomepl	text	Compõe a chave primária. Nome da <i>playlist</i> deve ser um dos nomes previamente cadastrados na tabela Nomespl.
nummusica	integer	Compõe a chave primária. Número inteiro de identificação da música. Deve ser um número que conste da tabela “Musicas”, para estabelecer o vínculo entre a música cadastrada e a <i>playlist</i> dada pelo campo nomepl.

Essa tabela terá a chave primária formada por dois campos. Isso é comum em bancos de dados, e nesses casos o SQL utilizado deve ser escrito da seguinte maneira:

```
create table playlist (
```

```
nomepl Not NULL,
```

```
nummusica Not NULL,
```

```
PRIMARY KEY (nomepl, nummusica)
```

```
)
```

4. Escreva um programa que permita exibir, inserir, alterar e excluir músicas no banco de dados do exercício 3.

5. Escreva um programa que permita exibir, inserir, alterar e excluir *playlists* (o nome e as músicas relacionadas) no banco de dados do exercício 3. Na inclusão de músicas, o programa só deve permitir inserir na tabela “Playlist” números de músicas que constarem da tabela “Musicas”.

6. Escreva um programa que leia um arquivo texto do tipo CSV, com os dados separados pelo caractere ponto e vírgula “;” contendo

codigo;qtde;pccompra;pcvenda

e importe esses dados na tabela vendas com a estrutura dada a seguir. Para testar seu programa, crie um arquivo de entrada com os dados anteriores, utilizando um editor de textos comum. O banco de dados deve ter o nome “loja.db” e o nome da tabela deve ser “vendas”.

Nome do campo	Tipo	Descrição
codigo	integer	Chave primária.
qtde	integer	Quantidade do produto em estoque.
pccompra	double	Preço de compra do produto.
pcvenda	double	Preço de compra do produto.

7. Refaça o programa do Exercício proposto 5 do Capítulo 7, gravando os resultados em um banco de dados. O BD e a tabela devem ser criados por esse programa, com nomes de sua escolha. A tabela deve conter os campos: SalBruto, AliqINSS, ValINSS, AliqIR, DeducIR, ValIR, SalLiquido, todos do tipo double. No caso dos salários que atingem o teto do INSS, carregue o campo AliqINSS com 11% em vez da palavra TETO.

8. Refaça o programa do Exercício proposto 6 do Capítulo 7, gravando

os RAs e as senhas dos alunos em um banco de dados. O BD e a tabela devem ser criados por esse programa, com nomes de sua escolha.

¹ Caro leitor, você não leu errado nem se trata de erro de impressão. Bilhões. Esta cifra é real, pois o SQLite está presente em todos os dispositivos Android, iOS e macOS e Windows 10, pois é parte integrante desses sistemas. Acompanha a instalação dos navegadores Chrome, Firefox e Safari. É distribuído junto com PHP e Python, com o Skype, com o iTunes, com o DropBox, e esta é apenas uma lista parcial. Mais informações podem ser encontradas em <<https://www.sqlite.org/mostdeployed.html>>.

² O rótulo ‘# linha 17’ não está escrito no exemplo de propósito. Isso porque nessa linha inicia-se um docstring com o comando sql, e o rótulo não pode fazer parte do sql. Caso faça parte, ocorrerá erro em sua execução.



Projeto 1: Demanda de Mercadorias e Rentabilidade de Vendas

Objetivos

Neste capítulo será apresentado um projeto contemplando um exercício de programação cujo problema consiste em apurar a necessidade de compra de produtos para atender às entregas futuras de mercadoria demandada por pedidos de clientes pré-programados, bem como avaliar a rentabilidade

média referente à venda de cada produto.

Para isso, serão elaborados dois programas. O primeiro será usado para gerar os dados de pedidos programados. O segundo programa gerará os resultados esperados.

9.1 O problema

Uma distribuidora de equipamentos e peças industriais trabalha com um esquema de carteira de pedidos. Isso significa que seus clientes enviam uma programação de pedidos a serem entregues no futuro. O conjunto de itens que constituem esses pedidos compõe a chamada carteira de pedidos.

Com base na programação de entrega, a empresa precisa manter um controle de estoque que permita avaliar as necessidades de compras futuras, de modo a garantir que não falem mercadorias e os pedidos programados possam ser entregues. Assim, rotineiramente, o controlador desse estoque emite um relatório de controle que, com base no cadastro de produtos e nos pedidos programados, mostra dois conjuntos de informações:

1. Posição do estoque e demanda de mercadorias no período.
2. Totais da carteira, incluindo preço médio de venda e margem média de cada produto.

Este projeto consiste em elaborar um programa que gere tais informações a partir dos dados disponíveis na empresa, que são:

1. Cadastro com dados dos produtos que são necessários a esse programa (este será o arquivo PRODUTOS.TXT).
2. Registro da programação de entregas contendo as quantidades e preços unitários de venda de cada pedido programado (este será o

arquivo VENDAS.TXT).

Uma questão adicional que se apresenta é o fato de que os dados reais da empresa não estão disponíveis para o programador. Como este necessita testar o programa que escreverá, será necessário gerar arquivos de testes.

O primeiro é algo simples, não é necessário ter muitos produtos cadastrados para realização dos testes, sendo que algo em torno de 10 a 20 produtos devem bastar. Um arquivo assim pode ser digitado em um editor de textos, como o programa Bloco de Notas.

Já o segundo arquivo não é tão simples. É necessário ter um volume maior de informações. Supondo que o período de avaliação seja de um mês (22 dias úteis) e que em cada dia haja em torno de 25 entregas, serão 550 registros a serem digitados. Além do grande volume, é necessário que haja coerência e consistência nesses dados, de modo que sua produção manual é inviável.

Portanto, neste projeto serão desenvolvidos dois programas:

1. **gerador.py**: servirá para gerar o arquivo VENDAS.TXT.
2. **apurador.py**: gerará os resultados de estoque, demanda de compras e margem média dos produtos. Esses resultados serão gravados em um arquivo de saída chamado APURA.TXT.

9.1.1 Dados de entrada

Nesta etapa são descritos os dados de entrada que devem estar gravados nos dois arquivos texto: PRODUTOS.TXT e VENDAS.TXT.

9.1.1.1 PRODUTOS.TXT

Este arquivo está organizado de modo que cada linha contém os dados de um produto, delimitados pelo caractere ‘;’, e tem suas linhas organizadas em ordem crescente de código do produto.

O primeiro campo é o código do produto, e não pode haver repetição deste.

O segundo campo é a quantidade em estoque no início do período de apuração. O terceiro campo é a quantidade mínima do produto que deve haver em estoque. Essa quantidade mínima é mantida para que a empresa possa atender a demandas urgentes e não programadas que às vezes ocorrem (embora tais demandas não programadas não sejam consideradas neste projeto).

O quarto campo contém o preço unitário de compra do produto. O último campo é a margem mínima de venda, porcentagem que é aplicada ao preço de custo para se obter o preço de venda. A margem real aplicada a uma venda específica pode ser um pouco maior que essa margem mínima. Isso será explicado quando for definida a geração do arquivo de vendas.

Este arquivo tem os seguintes campos:

Posição	Informação	Formato	Observações
1	Código do produto	5 dígitos numéricos	
2	Quantidade em estoque	Nº inteiro	Quantidade de produtos em estoque no início do período.
3	Quantidade mínima	Nº inteiro	Quantidade mínima que a empresa deve ter em estoque
4	Custo unitário	Nº real	Preço unitário que a empresa paga quando adquire o produto.
5	Margem de venda	Nº real	Margem mínima que é aplicada ao preço de custo para gerar o preço de venda.

O Quadro 9.1 mostra um exemplo desse arquivo com quatro produtos.

```
12100;1417;500;2.30;38.80
12200;725;100;23.70;13.58
15100;618;500;5.60;34.90
```

15200;223;50;61.90;8.15

Quadro 9.1 Exemplo de arquivo PRODUTOS.TXT.

9.1.1.2 VENDAS.TXT

Este arquivo está organizado de modo que cada linha contém os dados de uma venda, delimitados pelo caractere ‘;’, sendo organizado por data (Ano/Mês/Dia), e as vendas da mesma data não seguem uma ordem específica.

Os três primeiros campos são Ano (com 4 dígitos), Mês e Dia (com DOIS dígitos cada). Em seguida, vem o Código do Produto. Todos esses dados são números inteiros.

O quinto campo é a quantidade vendida. O histórico de vendas da empresa mostra que em 60% das vendas essa quantidade fica entre 1 e 10 peças, em 25% fica entre 11 e 25 peças e em 15% fica entre 26 e 400 peças.

O preço unitário de venda é o último campo da linha.

Cada linha contém o registro de venda de um produto com o layout:

Posição	Informação	Formato	Observações
1	Ano	Nº inteiro	Os campos Ano, Mês e Dia em conjunto definem a data da venda.
2	Mês	Nº inteiro	
3	Dia	Nº inteiro	
4	Código do Produto	Nº inteiro	
5	Quantidade Vendida	Nº inteiro	
6	Preço unitário	Nº real	Preço unitário de venda.

2017;5;1;12100;475;3.42

```

2017;5;1;15200;87;68.18
2017;5;2;12100;254;3.19
2017;5;2;12200;37;27.39
2017;5;2;12100;251;3.26
2017;5;3;15200;47;71.90
2017;5;3;15200;59;68.18
2017;5;3;12100;364;3.19
2017;5;5;15200;48;69.42

```

Quadro 9.2 Exemplo de arquivo VENDAS.TXT.

9.1.2 Saída – primeira parte: demanda e compras

A saída a ser produzida neste projeto contém duas partes, que serão descritas nesta e na próxima etapa. Conforme mencionado anteriormente, essa saída será gravada em um arquivo texto chamado APURA.TXT.

A primeira parte da saída serão a demanda por produtos e a necessidade de compras para atender a essa demanda. O Quadro 9.3 exibe a saída que será gerada a partir dos dados contidos nos Quadros 9.1 e 9.2.

NECESSIDADE DE ESTOQUE NO PERÍODO					
Prod.	Estoque Inicial	Demanda	Estoque Final	Estoque Mínimo	Neces. Compra
12100	1417	1344	73	500	427
12200	725	37	688	100	0
15100	618	0	618	500	0
15200	223	241	0	50	68

Quadro 9.3 Saída – primeira parte: demanda e compras.

O Estoque Inicial e o Estoque Mínimo vêm do cadastro de produtos (veja o Quadro 9.1 e compare com o Quadro 9.3). A Demanda é calculada a partir dos pedidos programados para o período. As colunas Estoque Final e Necessidade de Compras são calculadas como indicado a seguir:

$$\text{Estoque Final} = \text{Estoque Inicial} - \text{Demanda}$$

Caso resulte negativo, então $\text{Estoque Final} = 0$

$$\text{Neces.Compras} = \text{Demanda} - \text{Estoque Inicial} + \text{Estoque M\u00ednimo}$$

O trabalho maior, neste caso, \u00e9 calcular a Demanda. Para isso, as quantidades constantes nos pedidos devem ser agrupadas por produto e somadas. Desse agrupamento e totaliza\u00e7\u00e3o resulta a Demanda, que \u00e9 a quantidade de produtos que deve ser entregue no per\u00edodo. Isto est\u00e1 demonstrado no Quadro 9.4, no qual os dados foram reordenados, alinhados, e os totais, calculados. Isso foi feito com o \u00fanico prop\u00f3sito de demonstrar o c\u00e1lculo necess\u00e1rio para gerar o resultado da sa\u00edda. No programa a solu\u00e7\u00e3o \u00e9 um pouco diferente, pois n\u00e3o s\u00e3o feitos esse agrupamento e essa ordena\u00e7\u00e3o.

```
2017 5 1 12100 475 3.42
2017 5 2 12100 254 3.19
2017 5 2 12100 251 3.26
2017 5 3 12100 364 3.19
Demanda 1344
2017 5 2 12200 37 27.39
Demanda 37
2017 5 1 15200 87 68.18
2017 5 3 15200 47 71.90
2017 5 3 15200 59 68.18
2017 5 5 15200 48 69.42
Demanda 241
```

Quadro 9.4 Exemplo de arquivo VENDAS.TXT.

9.1.3 Sa\u00edda – segunda parte: totais e margem m\u00e9dia

A segunda parte da sa\u00edda cont\u00e9m, para cada produto, a demanda (cujo

cálculo foi exemplificado no Quadro 9.4) e o valor total calculado multiplicando-se a quantidade da venda pelo preço unitário e totalizando esse valor de maneira semelhante ao que foi feito com a quantidade. Os valores totais dos produtos são totalizados para obtenção do Total Geral.

Para cada produto é possível calcular o preço médio praticado. Isso porque cada venda individual terá o próprio valor unitário. Esse preço médio, comparado ao preço de custo do produto, produz a margem média que a empresa ganha referente a cada produto. Essas duas colunas são calculadas como se segue. O Quadro 9.5 ilustra esses cálculos.

$$\text{Preço Médio} = \text{Valor Total} / \text{Qtde}$$

$$\text{Margem Média} = \text{Preço Médio} / \text{Preço Custo} * 100\%$$

TOTAIS DE PEDIDOS EM CARTEIRA					
Prod.	Valor Tot	Qtde	Pç Médio	Pç Custo	Margem Méd
12100	4414.18	1344	3.28	2.30	42.8%
12200	1013.43	37	27.39	23.70	15.6%
15100	0.00	0	0.00	5.60	0.0%
15200	16665.74	241	69.15	61.90	11.7%

Total	22093.35				

Quadro 9.5 Exemplo de arquivo VENDAS.TXT.

9.2 A solução – o programa apurador.py

Agora que o problema já está bem definido, é possível começar a tratar da solução. O primeiro passo é dispor dos arquivos de entrada PRODUTOS.TXT e VENDAS.TXT. Para isso, pode digitar os dados dos

Quadros 9.1 e 9.2 e salvá-los com os respectivos nomes. Mais à frente será mostrado um programa capaz de gerar dados de vendas em grande volume.

9.2.1 Parte principal

Esse programa foi implementado utilizando-se diversas funções. A parte principal do programa, que é o ponto de início do programa, está mostrada no Exemplo 9.1.

Exemplo 9.1 Programa apurador.py – parte principal

```
# Ponto de início da execução

ExibeApresentacao() # 1 exibe a tela inicial

Prods = LeArqProdutos() # 2 lê e retorna o arquivo de produtos

Vendas = LeArqVendas() # 3 lê e retorna os dados de vendas

# abaixo abre o arquivo de saída

arqsai = open("APURA.TXT", "w", encoding="UTF-8") # 4

ApuraDemandaEstoque() # 5 apura a demanda e neces. compras

ApuraTotaisPorProduto() # 6 apura os totais vendidos

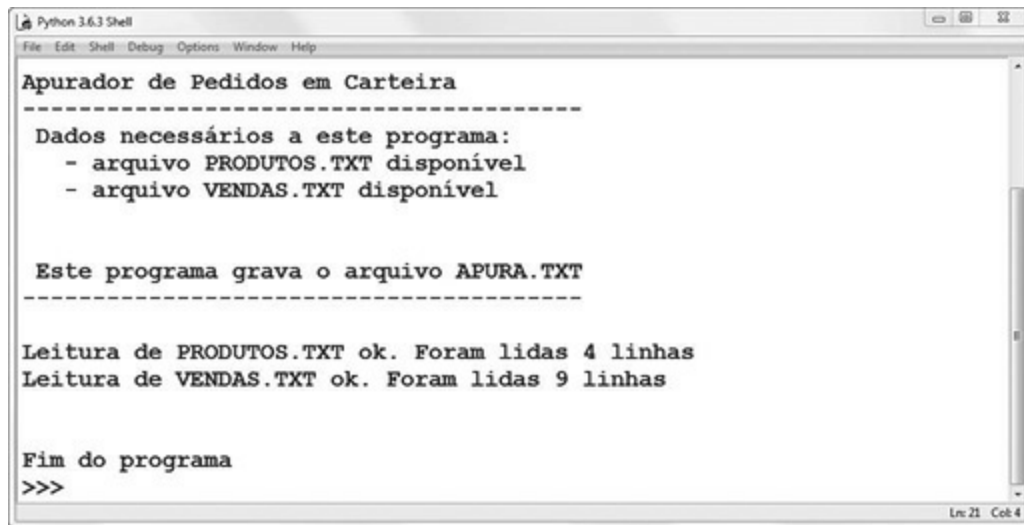
arqsai.close() # 7 fecha o arquivo de saída

print("\n\nFim do programa")
```

Essa parte principal consiste em sete linhas de código nas quais são chamadas diversas funções, explicadas na sequência. Uma tela inicial é exibida na função ExibeApresentacao. Em seguida, nas linhas 2 e 3, são

feitas as leituras dos arquivos de entrada, cujos dados ficarão armazenados nos objetos globais Prods e Vendas. Essas variáveis, depois, serão utilizadas dentro de duas funções.

Na linha 4 o arquivo de saída é aberto, sendo fechado apenas na linha 7. Este arquivo será gravado dentro das funções `ApuraDemandaEstoque` e `ApuraTotaisPorProduto`.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Apurador de Pedidos em Carteira
-----
Dados necessários a este programa:
- arquivo PRODUTOS.TXT disponível
- arquivo VENDAS.TXT disponível

Este programa grava o arquivo APURA.TXT
-----

Leitura de PRODUTOS.TXT ok. Foram lidas 4 linhas
Leitura de VENDAS.TXT ok. Foram lidas 9 linhas

Fim do programa
>>>
Lin: 21 Col: 4
```

Figura 9.1 Execução do programa `apurador.py`.

A Figura 9.1 exibe o resultado da execução desse programa. Não há interação com o usuário. O programa simplesmente lê os arquivos de entrada, processa e encerra.

A função `ExibeApresentacao` é composta por diversos prints e mais nada. Ela monta um visual básico para informar ao usuário do programa que a execução ocorreu.

Exemplo 9.2 Programa `apurador.py` – função `ExibeApresentacao`

```
def ExibeApresentacao():
```

```
    print("\nApurador de Pedidos em Carteira")
```

```

print("-" * 40)

print(" Dados necessários a este programa:")

print(" - arquivo PRODUTOS.TXT disponível")

print(" - arquivo VENDAS.TXT disponível")

print("\n")

print(" Este programa grava o arquivo APURA.TXT")

print("-" * 40, "\n")

```

9.2.2 Função LeArqProdutos

O objetivo desta função é ler o arquivo PRODUTOS.TXT e gerar o dicionário Prods contendo os dados dos produtos. Esse é um dicionário que contém dicionários aninhados, no qual a chave é o código do produto e o valor associado é outro dicionário contendo os strings estq, qmin, pcunit e margem como chaves às quais estarão associados os valores lidos do arquivo.

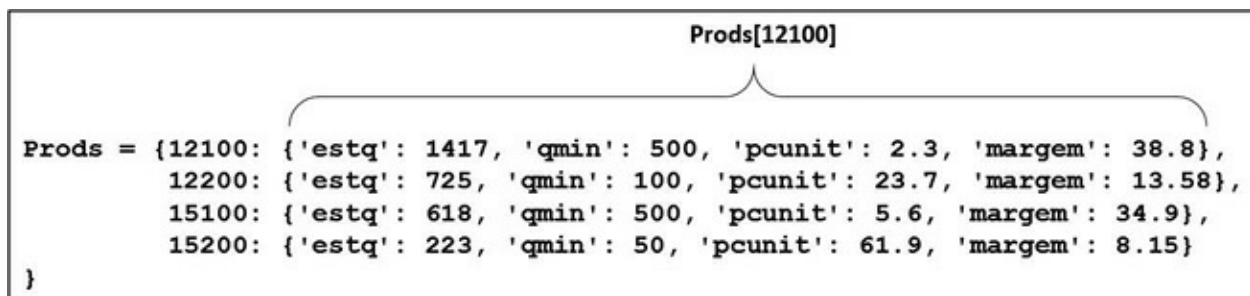


Figura 9.2 Estrutura do dicionário Prods.

A Figura 9.2 mostra como ficará o dicionário de produtos. Para se chegar a isso, a função do Exemplo 9.3 define o objeto dicProd como um dicionário vazio. Em seguida, abre o arquivo e usa um laço iterador que lê uma linha por vez, carregando no objeto S o string lido (linha 3).

Cada string é processado como descrito nos comentários do código. Em especial, tem-se que na linha 7 o dicionário aninhado é criado e nas linhas 8 a 11 ele é carregado. Feito isso, na linha 12 o dicionário de produtos é carregado com o item dicItem como valor vinculado ao código do produto.

Exemplo 9.3 Programa apurador.py – Função LeArqProdutos

```
def LeArqProdutos():

    dicProd = {} # cria o dicionário vazio

    arq = open("PRODUTOS.TXT") # abre o arquivo de produtos

    for S in arq.readlines(): # linha 3

        S = S.rstrip() # remove o '\n' do string lido

        L = S.split(";") # separa o string em uma lista

        codigo = int(L[0]) # obtém o cód. do produto

        dicItem = {} # linha 7

        dicItem['estq'] = int(L[1]) # linha 8

        dicItem['qmin'] = int(L[2]) # linha 9

        dicItem['pcunit'] = float(L[3]) # linha 10

        dicItem['margem'] = float(L[4]) # linha 11

        dicProd[codigo] = dicItem # linha 12

    arq.close() # fecha o arquivo
```

```
print("Leitura de PRODUTOS.TXT ok. Foram lidas {} \
linhas".format(len(dicProd)))

return dicProd # retorna o dicionário
```

9.2.3 Função LeArqVendas

As vendas já não podem ser armazenadas na forma dicionários aninhados, pois não há um campo que possa servir como chave do dicionário. Com isso, elas serão armazenadas como uma lista de tuplas. Dentro de cada tupla os dados de vendas estarão exatamente na mesma ordem em que aparecem no arquivo de entrada: ano, mês, dia, código produto, quantidade vendida e preço unitário de venda.

```
[ (2017, 5, 1, 12100, 475, 3.42) ,
  (2017, 5, 1, 15200, 87, 68.18) ,
  (2017, 5, 2, 12100, 254, 3.19) ,
  (2017, 5, 2, 12200, 37, 27.39) ,
  ...
  (2017, 5, 5, 15200, 48, 69.42) ]
```

Figura 9.3 Estrutura da lista Vendas.

Essa função faz a leitura do arquivo de vendas, sendo que para cada linha elimina o ‘\n’, faz um split no string, separando os dados na lista L, executa a conversão dos dados de string para inteiro ou real, no caso do preço, e, por fim, à lista V é adicionada a lista L convertida para tupla. Essa lista V é retornada pela função LeArqVendas.

Exemplo 9.4 Programa apurador.py – função LeArqVendas

```
def LeArqVendas():

    V = [] # define a lista
```

```

    arq = open("VENDAS.TXT") # abre o arquivo

    for s in arq.readlines(): # para cada linha do arquivo

        s = s.rstrip() # elimina '\n' do final

        L = s.split(';') # separa o string em uma lista

        for i in range(6): # converte os dados para int

            if i < 5: # ou float no caso do último

                L[i] = int(L[i])

            else:

                L[i] = float(L[i])

        V.append(tuple(L)) # acrescenta a tupla à lista.

    arq.close()

    print("Leitura de VENDAS.TXT ok. Foram lidas {} \
linhas".format(len(V)))

    return V # retorna V

```

9.2.4 Função ApuraDemandaEstoque

Esta é a função responsável pela apuração da demanda de quantidades e da eventual necessidade de compras. Na linha 2 são declaradas as três variáveis globais que serão utilizadas. O elemento-chave desta função é o dicionário

aninhado dicEstq, construído nas linhas de 3 a 9 e que tem a seguinte estrutura:

```
dicEstq = {cód.produto:{‘estq’:0, ‘qmin’:0, ‘demanda’:0}, ...}
```

Nela, o código do produto é chave para o dicionário aninhado, que conta com as chaves estq, qmin e demanda. Os dois primeiros são obtidos diretamente do cadastro de produtos, e a demanda é iniciada com zero.

Nas linhas seguintes, de 11 a 13, é feita a interação com o dicionário Vendas e as quantidades são totalizadas na chave demanda (linha 13). Depois, a função implementa os cálculos já explicados no Item 9.1.2 (linhas 25 a 32) e grava cada linha no arquivo arqsai.

Exemplo 9.5 Programa apurador.py – função ApuraDemandaEstoque

```
def ApuraDemandaEstoque():  
  
    global Prods, Vendas, arqsai # linha 2  
  
    dicEstq = {} # linha 3  
  
    for codprod in Prods.keys():  
  
        dicItem = {}  
  
        dicItem[‘estq’] = Prods[codprod][‘estq’]  
  
        dicItem[‘qmin’] = Prods[codprod][‘qmin’]  
  
        dicItem[‘demanda’] = 0  
  
        dicEstq[codprod] = dicItem # linha 9  
  
    for v in Vendas: # linha 11
```

```

codprod = v[3]

dicEstq[codprod]['demanda'] += v[4] # linha 13

arqsai.write("-"*52 + " Início de Bloco -" + "\n")

arqsai.write("NECESSIDADE DE ESTOQUE NO PERÍODO\n")

arqsai.write("-"*70 + "\n")

arqsai.write(" "*9 + "Estoque "+ " "*14 + "Estoque" + " "*4 +
"Estoque" + " "*6 + "Neces.\n")

arqsai.write("Prod. Inicial Demanda" + " "*6 +
"Final Mínimo Compra\n")

sgrava = "{:<5} {:>10d} {:>10d} {:>10d} {:>10d} {:>10d}\n"

for codprod, dados in dicEstq.items():

# bloco de cálculo desta função # linha 25

EstqFinal = dados['estq'] - dados['demanda']

if EstqFinal < 0:

EstqFinal = 0

NecCompra = dados['demanda'] - dados['estq'] + dados['qmin']

if NecCompra < 0:

```

```

NecCompra = 0

# fim do bloco de cálculo desta função # linha 32

arqsai.write(sgrava.format(

codprod,

dados['estq'],

dados['demanda'],

EstqFinal,

dados['qmin'],

NecCompra))

arqsai.write("-"*55 + " Fim de Bloco -" + "\n\n\n")

```

9.2.5 Função ApuraTotaisPorProduto

Nesta função o objetivo é totalizar, para cada produto, as quantidades e os valores das vendas. A lógica é muito semelhante à da função anterior. Foi criado o dicionário dicTotais, que terá como chave os códigos de produtos e como valor um dicionário aninhado, o qual tem dois campos: totval, para o total do pedido, e totqtd, para o total da quantidade (linhas de 3 a 8).

```
dicTotais = {cód.produto:{'totval':0, 'totqtd':0}, ...}
```

Na sequência, as linhas 9 a 12 implementam o laço de iteração com a lista de vendas, realizando as totalizações necessárias nas linhas 11 e 12.

Exemplo 9.6 Programa apurador.py – função ApuraTotaisPorProduto

```
def ApuraTotaisPorProduto():

    global Prods, Vendas, arqsai # linha 2

    dicTotais = {} # linha 3

    for codprod in Prods.keys():

        dicItem = {}

        dicItem['totval'] = 0

        dicItem['totqtd'] = 0

        dicTotais[codprod] = dicItem # linha 8

    for v in Vendas: # linha 9

        codprod = v[3]

        dicTotais[codprod]['totval'] += v[4] * v[5]

        dicTotais[codprod]['totqtd'] += v[4] # linha 12

    arqsai.write("-"*52 + " Início de Bloco -" + "\n")

    arqsai.write("TOTAIS DE PEDIDOS EM CARTEIRA\n")

    arqsai.write("-"*70 + "\n") # linha 16

    arqsai.write("Prod. Valor Tot Qtde " +

        "Pç Médio Pç Custo Margem Méd\n")
```

```
sgrava = "{:<5} {:>11.2f} {:>8d} {:>10.2f} {:>10.2f} \n  
{:>10.1f}%\n"
```

```
TotVendas = 0 # linha 21
```

```
for codprod, dados in dicTotais.items():
```

```
try:
```

```
TotVendas += dados['totval']
```

```
pcmedio = dados['totval'] / dados['totqtd']
```

```
lucrat = (pcmedio / dados['pcunit'] - 1) * 100
```

```
except:
```

```
pcmedio = lucrat = 0
```

```
arqsai.write(sgrava.format(
```

```
codprod,
```

```
dados['totval'],
```

```
dados['totqtd'],
```

```
pcmedio,
```

```
Prods[codprod]['pcunit'],
```

```
lucrat))
```

```
arqsai.write("-"*70 + "\n")
```

```
arqsai.write("Total {:>11.2f}\n".format(TotVendas))
```

```
arqsai.write("-"*55 + " Fim de Bloco -" + "\n\n\n")
```

Nota

Observe que, no código anterior, algumas linhas (17 e 19) ficaram muito extensas e foi necessário fazer uma quebra de linha. Para que o interpretador Python entenda essa quebra de linha, é necessário usar o caractere ‘\’ no final da linha quebrada.

Nas demais linhas dessa função é feita a gravação da saída no arquivo em disco, sendo que para isso são implementados os cálculos explicados no Item 9.1.3.

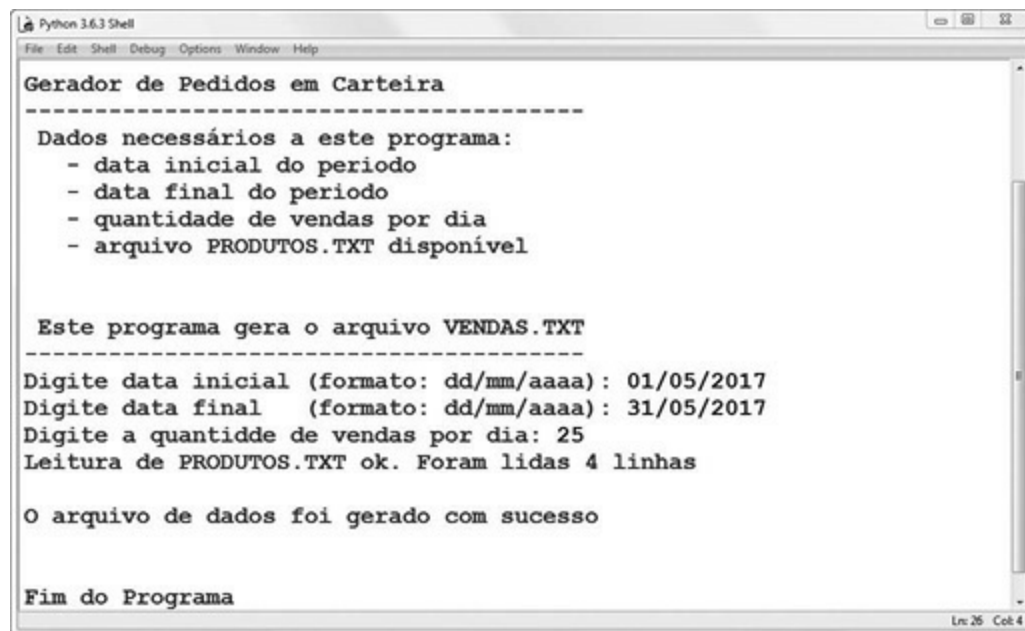
Para que consiga executar este programa, é necessário escrever todo o código, programa principal e todas as funções, em um único arquivo, sendo que as funções devem estar posicionadas antes do programa principal.

9.3 A solução – o programa gerador.py

Na descrição deste problema, Item 9.1, foi utilizado um arquivo VENDAS.TXT com apenas 9 linhas. Isso foi feito para ser possível explicar o problema e mostrar com um exemplo numérico o que deve ser calculado. Porém, convém testar o programa com diversos conjuntos de dados, que devem abranger períodos bem maiores, com muitas vendas diárias e, além disso, ser consistentes.

A melhor opção para dispor de arquivos assim é escrever um programa capaz de criá-los. Esta é a finalidade do programa gerador.py. Ao executá-lo, o usuário precisa fornecer três informações: as datas inicial e final do período

e a quantidade de vendas por dia (essa quantidade varia em uma empresa real, mas aqui é razoável trabalhar com uma quantidade fixa).



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

Gerador de Pedidos em Carteira
-----
Dados necessários a este programa:
- data inicial do periodo
- data final do periodo
- quantidade de vendas por dia
- arquivo PRODUTOS.TXT disponível

Este programa gera o arquivo VENDAS.TXT
-----
Digite data inicial (formato: dd/mm/aaaa): 01/05/2017
Digite data final (formato: dd/mm/aaaa): 31/05/2017
Digite a quantidade de vendas por dia: 25
Leitura de PRODUTOS.TXT ok. Foram lidas 4 linhas

O arquivo de dados foi gerado com sucesso

Fim do Programa
Ln: 26 Col: 4
```

Figura 9.4 Resultado da execução do programa gerador.py.

A Figura 9.4 exibe a tela de execução desse programa e o Exemplo 9.7 é o programa completo. Esse programa também foi escrito com o uso de funções, e a seguir é feito o detalhamento dele.

9.3.1 Descrição do programa – parte principal

O programa principal inicia exibindo uma tela de apresentação por meio da função `ExibeApresentacao` e é igual ao que foi feito no primeiro programa.

A função `ObtemEntradas` efetua a leitura dos dados a serem digitados pelo usuário e os retorna em uma tupla com a data inicial, a data final e a quantidade de vendas por dia, que são associadas aos objetos `DtIni`, `DtFim` e `Qtde`.

O próximo passo é efetuar a leitura do arquivo de produtos e a consequente carga do dicionário `Prods`. É exatamente igual ao que foi feito no primeiro programa e está descrito no Item 9.2.2.

Agora começa a parte essencial do programa. Trata-se de um laço que percorrerá todos os dias, desde a data inicial até a data final. Isso é feito com o auxílio dos objetos e funções da biblioteca `datetime` importada no início do código. Observe a linha. Para somar um dia em uma data e, com isso, obter a data do dia seguinte, é preciso usar o método `datetime.timedelta(days = 1)`. Para simplificar o código, foi criado o objeto `UmDia` já carregado com o `timedelta` de um dia. Dentro do laço controlado pelas datas, é feita a verificação se é dia útil ou não. Para isso, utiliza-se o método `datetime.weekday`, que retorna 0 para segunda-feira, 1 para terça, e assim por diante. Portanto, quando esse retorno for 5 (sábado) ou 6 (domingo), o programa não deve gerar vendas.

E assim chega-se à função `GeraDadosDia`, que é a responsável pela efetiva geração dos dados. Ela utiliza, basicamente, a geração de números aleatórios para:

1. Sortear um produto gerando um número inteiro aleatório é utilizado como índice da lista de códigos de produtos. Essa lista de códigos de produto foi gerada a partir das chaves do dicionário `Prods`.
2. Gerar a quantidade vendida, segundo as proporções descritas no Item 9.1.1.
3. Gerar o valor unitário de venda a partir do preço unitário de compra e da margem mínima que constam do arquivo de produtos. Além disso, há uma variação entre 0% e 10% adicionais que são somados à margem, aumentando-a. Esse dado também é gerado aleatoriamente.

Após tudo ter sido gerado, a função efetua a gravação no disco.

Esse programa pode ser usado para gerar dados referentes a quaisquer

períodos e para qualquer quantidade de vendas por dia. E esses dados podem ser usados para testar o programa **apurador.py**.

Exemplo 9.7 Programa gerador.py – programa completo

```
import datetime

from random import randint

def ExibeApresentacao():

    print("\nGerador de Pedidos em Carteira")

    print("-" * 40)

    print(" Dados necessários a este programa:")

    print(" - data inicial do período")

    print(" - data final do período")

    print(" - quantidade de vendas por dia")

    print(" - arquivo PRODUTOS.TXT disponível")

    print("\n")

    print(" Este programa gera o arquivo VENDAS.TXT")

    print("-" * 40)

    def ConverteData(d):

        d = d.split("/")
```

```
data = datetime.date(int(d[2]), int(d[1]), int(d[0]))

return data

def ObtemEntradas():

s = input("Digite data inicial (formato: dd/mm/aaaa): ")

ini = ConverteData(s)

s = input("Digite data final (formato: dd/mm/aaaa): ")

fim = ConverteData(s)

q = int(input("Digite a quantidade de vendas por dia: "))

return ini, fim, q

def LeArqProdutos():

dicProd = {}

arq = open("PRODUTOS.TXT")

for S in arq.readlines():

S = S.rstrip()

L = S.split(";")

codigo = int(L[0])

dicItem = {}
```

```
dicItem['estq'] = int(L[1])

dicItem['qmin'] = int(L[2])

dicItem['pcunit'] = float(L[3])

dicItem['margem'] = float(L[4])

dicProd[codigo] = dicItem

arq.close()

print("Leitura de PRODUTOS.TXT ok. Foram lidas { } linhas".

format(len(dicProd)))

return dicProd

def GeraQtdeVenda(codprod):

global Prods

sorteio = randint(1, 100)

if sorteio <= 60:

q = randint(1, 10)

elif sorteio <= 85:

q = randint(11, 25)

else:
```

```
q = randint(26, 400)

return q

def GeraPcUnitVenda(codprod):

    global Prods

    pccompra = Prods[codprod]['pcunit']

    margem = Prods[codprod]['margem'] / 100

    variacao = randint(0, 10) / 100

    pcvenda = pccompra * (1 + margem + variacao)

    return pcvenda

def GeraDadosDia(dia, qtvendas):

    global Prods, arq

    L = list(Prods.keys()) # carrega a lista L com os cód. prod.

    for x in range(qtvendas):

        # sorteia produto, gera um índice e o utiliza na lista

        iprod = randint(0, len(Prods)-1)

        codprod = L[iprod]

        # randomiza a quantidade vendida
```

```
qtitem = GeraQtdeVenda(codprod)

# randomiza o preco de venda

pcunit = GeraPcUnitVenda(codprod)

a = str(dia.year)+';' +str(dia.month)+';' +str(dia.day)

a = a + ';' + str(codprod)

a = a + ';' + "{:d}".format(qtitem)

a = a + ';' + "{:.2f}".format(pcunit)

a = a + "\n"

arq.write(a)

# Ponto de início da execução

ExibeApresentacao()

DtIni, DtFim, Qtde = ObtemEntradas()

Prods = LeArqProdutos()

arq = open("VENDAS.TXT", 'w', encoding="UTF-8")

UmDia = datetime.timedelta(days=1)

Cont = DtIni

while Cont <= DtFim:
```

```
if Cont.weekday() < 5: # só permite dia útil

GeraDadosDia(Cont, Qtde)

Cont = Cont + UmDia

arq.close()

print("\nO arquivo de dados foi gerado com sucesso")

print("\n\nFim do Programa")
```

Exercícios propostos

1. Faça uma alteração no programa gerador.py de modo que ele crie um número variável de vendas por dia que gire em torno do valor fornecido pelo usuário para o objeto Qtde.
2. No programa gerador.py, elabore um método para que, opcionalmente, os sábados possam ser considerados dia nos quais ocorram vendas. O usuário escolherá se quer ou não gerar vendas aos sábados.

TOTAIS DE VENDAS POR DIA

Data	Valor Total
01/05/2017	7556,16
02/05/2017	2641,95
03/05/2017	6981,28
04/05/2017	3332,16



Projeto 2: Controle de Torneios Esportivos

Objetivos

O objetivo deste exercício de programação é didático, ou seja, o foco maior é na lógica que leva à solução do problema e nos algoritmos implementados com o uso dessa lógica. Para desenvolver a solução serão utilizados praticamente todos os recursos vistos nos demais capítulos deste livro.

Os algoritmos são escritos de maneira mais tradicional, ou seja, menos *pythonica*. Essa opção foi escolhida porque o que se quer aqui é mostrar ao iniciante como desenvolver tais algoritmos. Para isso, é preciso escrevê-los de uma maneira mais fácil de ser compreendida, e as estruturas caracterizadas como *pythonicas* muitas vezes são difíceis de entender. Por outro lado, o programador que já tem experiência com outras linguagens não ficará na mão, pois muitos recursos de Python são amplamente utilizados e exemplificados.

O programa resultante está todo organizado na forma de funções. São utilizados objetos globais e locais. São amplamente utilizados strings e seus recursos de construção e formatação, tuplas, listas e dicionários. O SQLite3 será empregado como repositório dos dados do programa.

O código-fonte desse programa e todos os arquivos relacionados estão disponíveis no site da editora.

10.1 Problema

10.1.1 Motivação

Pouco tempo depois de ter concluído o curso de Processamento de Dados

da Fatec São Paulo, um amigo perguntou ao autor deste capítulo se seria fácil desenvolver um programa de computador para controlar a classificação dos times participantes de um campeonato de futebol de salão, que utilizava o sistema de pontos corridos. O ponto de partida para a obtenção da classificação seriam os resultados dos jogos já realizados. À época estudante de Educação Física, ele gerenciava dois torneios de futsal cuja regra de disputa era essa de pontos corridos, semelhante ao que atualmente se pratica no campeonato brasileiro. Naquela época ele fazia todo o trabalho à mão e sabia bem do que precisava. O autor topou, o amigo explicou as regras e o resultado foi um programa escrito em linguagem Pascal que fazia tudo de que ele precisava.

A proposta deste capítulo é implementar um programa semelhante, porém, escrito em Python.

10.1.2 Descrição do torneio de pontos corridos

Em um torneio de pontos corridos, todos os times jogam entre si em rodadas sucessivas, podendo existir um ou dois turnos. No caso de apenas um turno, há apenas uma partida entre dois times. Por exemplo, isso ocorre na fase de grupos da Copa do Mundo, em que, dentro de cada grupo, quatro seleções jogam entre si. Quando há dois turnos, como no caso do Brasileirão, dois times se enfrentam duas vezes, sendo que em uma o time é o mandante, dono do campo e conta com maior torcida a favor, e na outra é visitante. Como o amigo do autor deste capítulo gerencia torneios dos dois tipos, isso precisa estar previsto no programa.

Em torneios assim há alguns números que são relevantes à estrutura do programa. Todos esses números dependem da quantidade de times participantes do torneio, que será denominada N . O Quadro 10.1 exemplifica esses números.

Sendo N o número de times	N é par	N é ímpar	ex. N = 8	ex. N = 9
Quantidades de jogos por rodada	$\frac{N}{2}$	$\frac{(N - 1)}{2}$	4	4 (um time folga)
Quantidade de rodadas por turno (no caso de 2 turnos, dobrar esse nº)	N - 1	N	7	9
Quantidade de jogos por turno (no caso de 2 turnos, dobrar esse nº)	$\frac{(N^2 - N)}{2}$	$\frac{(N^2 - N)}{2}$	28	36

Quadro 10.1 Números principais de um torneio por pontos corridos.

Em campeonatos de grande visibilidade, nos quais muitos times querem participar, é praxe que o número de times participantes seja par, pois equilibra o emparelhamento em cada rodada. Nos torneios gerenciados pelo amigo do autor, nem sempre estão inscritos times em número par, mas os torneios serão disputados de qualquer maneira. Isso não chega a ser um problema, não havendo nada que impeça um torneio com um número ímpar de times. O único detalhe a ser considerado em casos assim é que, a cada rodada, um dos times não joga, fica de folga.

Neste tipo de campeonato, à medida que os jogos transcorrem, os times são classificados de acordo com a quantidade de pontos conquistados. A vitória confere 3 pontos ao vencedor e o derrotado não pontua. Em caso de empate, cada time recebe 1 ponto. Quanto mais pontos um time tiver, mais bem colocado ele estará, e em caso de dois ou mais times com a mesma quantidade de pontos são adotados outros critérios de classificação, na seguinte ordem: maior número de vitórias, maior saldo de gols, maior número de gols marcados e confronto direto. O campeonato brasileiro ainda conta com critérios adicionais, como menor número de cartões vermelhos e

amarelos, mas esse tipo de parâmetro não será considerado aqui.

10.1.3 Requisitos do programa

Essa é uma visão geral do tipo de torneio ao qual está dirigido este projeto. Para atendê-lo, o programa a ser desenvolvido deve contar com as funcionalidades descritas no Quadro 10.2.

Funcionalidade	Detalhes
Criação de torneio	1. Obtém o nome do torneio e a quantidade de turnos. 2. Os nomes dos times devem ser lidos do teclado. 3. Todos os jogos devem ser gerados e organizados em rodadas. 4. Cada jogo deve ser identificado por um número único. 5. Pode haver qualquer número de times maior que 2.
Exclusão de torneio	1. Torneios passados, sobre os quais já não há mais interesse, podem ser excluídos. 1. Para cada time o programa deve computar os pontos, o número de vitórias, empates e derrotas, gols pró e contra e o saldo de gols. 2. O programa deve mostrar a classificação geral, que deve levar em conta o conjunto de critérios a seguir. Fica na frente quem tiver: • maior número de pontos; • maior número de vitórias;

Gestão de
torneio

- melhor saldo de gols;
- maior número de gols marcados;
- confronto direto.

3. Deve ser possível informar o resultado de cada jogo, via teclado.

4. Deve ser possível eliminar um resultado de jogo, para ser utilizado em caso de erro de digitação.

5. O programa deve gerar um arquivo HTML com a classificação para que seja publicado em um website.

Quadro 10.2 Funcionalidades que devem estar disponíveis no programa deste projeto.

10.2 A solução

A solução desse projeto está implementada em um programa chamado “torneio.py” e será descrita em três partes:

1. Modo de armazenamento dos dados do programa.
2. As telas e suas funcionalidades.
3. A implementação do programa, contendo a descrição do código desenvolvido.

10.2.1 Apresentação das telas do programa

Para iniciar, a apresentação do programa, será feita a partir da descrição das telas que este contém, dando uma visão geral de seu uso. São telas simples, construídas para serem visualizadas em modo console, ou seja, nada mais são

do que telas em modo texto, formatadas exclusivamente com o uso do comando print.

A Figura 10.1 exibe a tela inicial, na qual é possível criar um novo torneio, escolher um torneio para gerenciar e sair do sistema. A escolha da opção é feita digitando-se a letra ou número que aparece entre parênteses à esquerda de cada opção. No caso de letras, pode-se digitar maiúsculas ou minúsculas que o resultado é o mesmo. E, se for digitado algo inválido, nada acontece.

Observe na figura que há seis torneios cadastrados. Nestes exemplos os torneios 1, 2, 3 e 6 estão em andamento e os torneios 4 e 5 estão encerrados, com todos os jogos já realizados.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
-----
                          Programa Torneio
                          Menu Principal
-----
Opções:
(N) Criar Novo Torneio
(.) Gerenciar Torneio Existente
    (1) Colégio Luchinni Handebol
    (2) Colégio Luchinni Infantil
    (3) Colégio Luchinni Juvenil
    (4) SP Oeste 2015
    (5) SP Oeste 2016
    (6) SP Oeste 2017
(S) Sair do programa

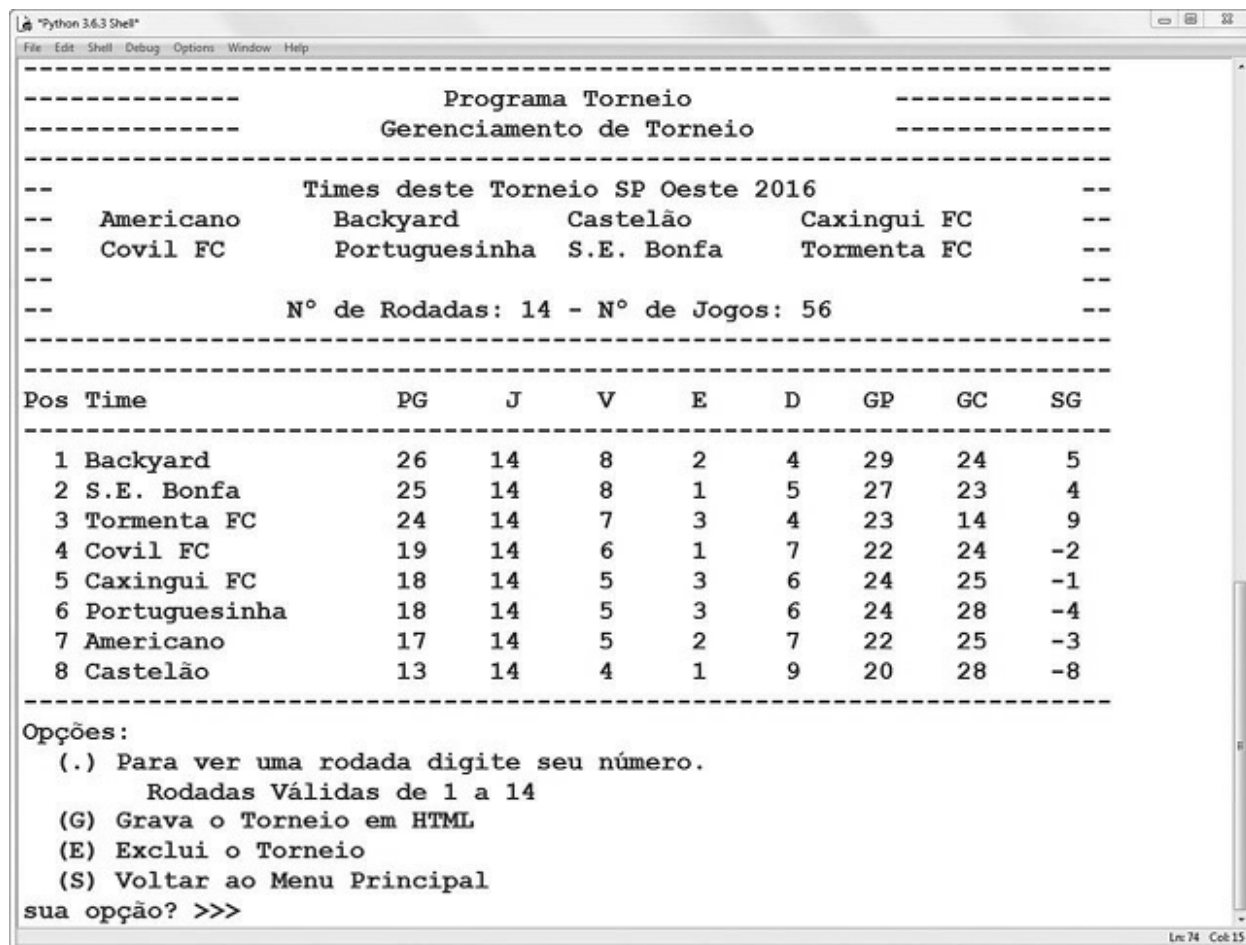
para escolher digite o que está entre parênteses
sua opção? >>>
```

Figura 10.1 Tela inicial do programa torneio.py – menu principal do programa.

A Figura 10.2 mostra a tela de gerenciamento de torneio. Escolheu-se, aqui, exibir o torneio denominado “SP Oeste 2016”. Nessa tela podem ser vistos os nomes dos times participantes, as quantidades de rodadas e de jogos e a classificação das equipes.

A tabela de classificação mostra a posição e os nomes dos times, os pontos

ganhos (PG), o número de jogos (J) já realizados, as vitórias (V), empates (E) e derrotas (D), os números de gols pró (GP), contra (GC) e o saldo de gols (SG).



```

-----
                        Programa Torneio
                        Gerenciamento de Torneio
-----

--          Times deste Torneio SP Oeste 2016          --
--  Americano  Backyard  Castelão  Caxingui FC  --
--  Covil FC   Portuguesinha  S.E. Bonfa  Tormenta FC  --
--
--          N° de Rodadas: 14 - N° de Jogos: 56          --
-----

Pos Time                PG    J    V    E    D    GP    GC    SG
-----
1 Backyard              26    14    8    2    4    29    24    5
2 S.E. Bonfa            25    14    8    1    5    27    23    4
3 Tormenta FC           24    14    7    3    4    23    14    9
4 Covil FC              19    14    6    1    7    22    24   -2
5 Caxingui FC           18    14    5    3    6    24    25   -1
6 Portuguesinha         18    14    5    3    6    24    28   -4
7 Americano             17    14    5    2    7    22    25   -3
8 Castelão              13    14    4    1    9    20    28   -8
-----

Opções:
  (.) Para ver uma rodada digite seu número.
      Rodadas Válidas de 1 a 14
  (G) Grava o Torneio em HTML
  (E) Exclui o Torneio
  (S) Voltar ao Menu Principal
sua opção? >>>

```

Figura 10.2 Tela de gerenciamento de torneio.

Essa tela também contém um menu com novas opções. Pode-se escolher uma rodada para visualizar, pode-se gravar um arquivo HTML com a tabela de classificação e pode-se excluir um torneio que já não é mais necessário.

Para ver uma rodada deve-se escolher seu número. A Figura 10.3 mostra a tela da rodada, na qual foi escolhida a última rodada, de número 14. Os resultados de cada partida já estão lançados. No caso de uma rodada ainda não jogada, nenhum número seria exibido.

Nessa tela é possível lançar o resultado de um jogo. Para isso, deve-se digitar o número do jogo e as quantidades de gols de cada time, separados por vírgulas. Por exemplo: ao digitar 53,2,1 significa que o jogo 53 teve resultado 2 a 1, ou seja, dois gols para o time à esquerda e um gol para o time à direita na tabela. Também é possível digitar: 53,limpa, o que significa que o resultado cadastrado para o jogo 53 deve ser apagado. Isso serve para corrigir eventuais erros de digitação.

```

Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
-----
                        Programa Torneio
                        Gerenciamento de Torneio
-----
-- Times deste Torneio SP Oeste 2016 --
-- Backyard      S.E. Bonfa    Tormenta FC    Covil FC      --
-- Caxingui FC   Portuguesinha  Americano     Castelão      --
--
-- N° de Rodadas: 14 - N° de Jogos: 56 --
-----
-- *** Rodada 14 *** --
-- 53  Castelão      2   x   1      Americano      --
-- 54  Caxingui FC   1   x   1      Covil FC       --
-- 55  Portuguesinha 2   x   4      Backyard       --
-- 56  S.E. Bonfa    0   x   3      Tormenta FC    --
-----
Opções:
(.) Para atualizar o placar de um jogo digite:
    N°Jogo,GolsA,GolsB exemplo: 12,2,1
(.) Para limpar o placar de um jogo digite:
    N°Jogo,limpa      exemplo: 12,limpa
(S) Voltar ao Menu do Torneio
sua opção? >>>
Linha 99 Col 15

```

Figura 10.3 Tela de visualização da rodada.

Voltando à Figura 10.2, ao usar a funcionalidade de excluir o torneio (opção “E”), o programa exibe um pedido de confirmação dessa exclusão e, uma vez confirmada, todos os seus dados são eliminados e o torneio desaparece da tela principal.

```

Python 3.6.3 Shell
File Edit Shell Debug Options Window Help

----- Confirma Exclusão do Torneio SP Oeste 2016 -----
Opções:
(C) para Confirmar
qualquer outra tecla para retornar
sua opção? >>>
Ln: 139 Col: 15

```

Figura 10.4 Confirmação da exclusão de torneio.

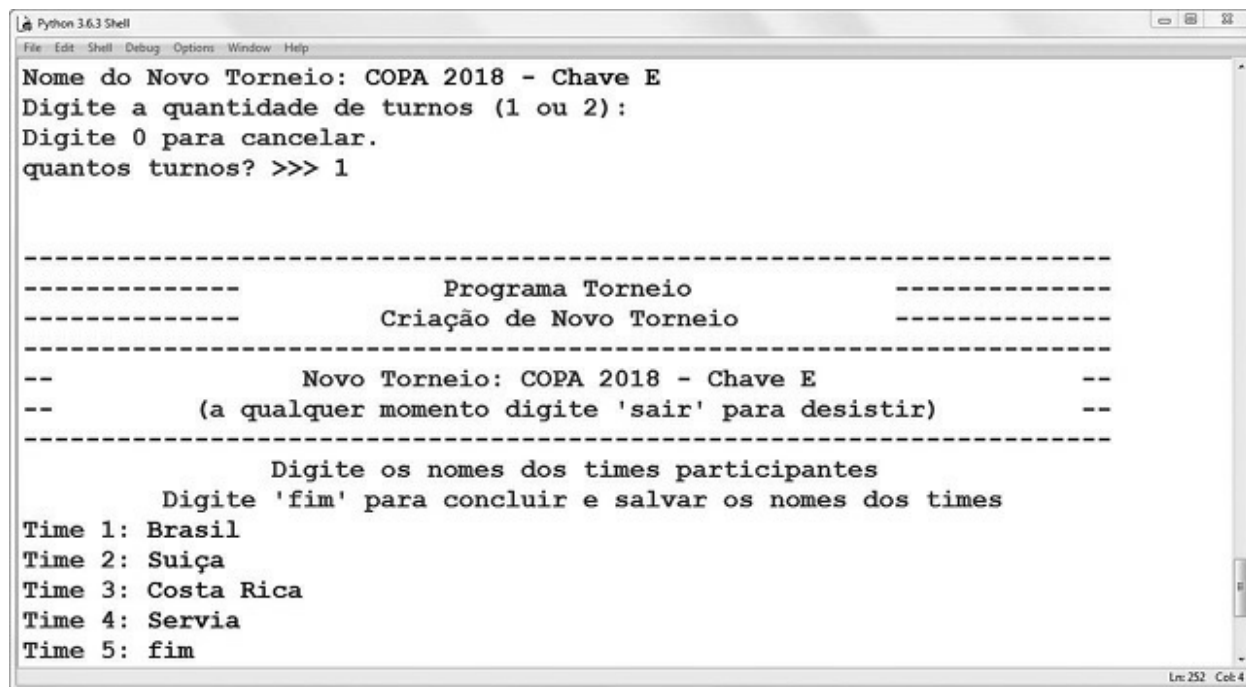
Por fim, nessa tela de gerenciamento, está disponível a opção de gravação da classificação do torneio em um arquivo HTML. Na versão original desse programa, citada no início do capítulo, era feita a impressão da tabela e esta era enviada via fax para os líderes dos times. Com a internet, tudo ficou mais fácil e o amigo do autor deste capítulo sabe fazer o upload de arquivos HTML no provedor de internet que utiliza. Assim, no programa basta gravar o arquivo no disco do computador local. O upload para o local de hospedagem da página é feito manualmente. O HTML gravado tem a aparência mostrada na Figura 10.5.

The screenshot shows a web browser window with the address bar displaying a file path. The page content includes a header with the tournament name and a date, followed by a table titled 'Classificação' containing 8 rows of team performance data.

Torneio: SP Oeste 2016		Acompanhamento de Torneio atualizado em: 6/4/2018							
Classificação									
Pos	Time	PG	J	V	E	D	GP	GC	SG
1	Backyard	26	14	8	2	4	29	24	5
2	S.E. Bonfa	25	14	8	1	5	27	23	4
3	Tormenta FC	24	14	7	3	4	23	14	9
4	Covil FC	19	14	6	1	7	22	24	-2
5	Caxingui FC	18	14	5	3	6	24	25	-1
6	Portuguesinha	18	14	5	3	6	24	28	-4
7	Americano	17	14	5	2	7	22	25	-3
8	Castelão	13	14	4	1	9	20	28	-8

Figura 10.5 Arquivo HTML com a classificação do torneio gerado pelo programa.

Voltando às opções do menu principal, a criação de um novo torneio é feita digitando-se “N”, e a Figura 10.6 mostra um exemplo no qual foi criado um torneio para a chave E da Copa do Mundo de 2018.



```
Python 3.6.3 Shell
File Edit Shell Debug Options Window Help
Nome do Novo Torneio: COPA 2018 - Chave E
Digite a quantidade de turnos (1 ou 2):
Digite 0 para cancelar.
quantos turnos? >>> 1

-----
-----
                          Programa Torneio
-----
                          Criação de Novo Torneio
-----
--
                          Novo Torneio: COPA 2018 - Chave E
--
                          (a qualquer momento digite 'sair' para desistir)
--
-----
                          Digite os nomes dos times participantes
                          Digite 'fim' para concluir e salvar os nomes dos times
Time 1: Brasil
Time 2: Suíça
Time 3: Costa Rica
Time 4: Servia
Time 5: fim
```

Figura 10.6 Criação de um novo torneio.

A criação do torneio começa pela digitação de um nome para ele, seguido do número de turnos. Em seguida, digitam-se os nomes das equipes participantes, um a um. A qualquer momento, se for digitada a palavra “sair”, o processo é cancelado. Ao digitar “fim”, o programa entende que todos os times já foram inseridos, gera as rodadas de jogos e as salva no banco de dados. Depois disso, o novo torneio passa a estar presente no menu da tela inicial.

10.2.2 Armazenamento dos dados do programa

Esse programa manipula certa quantidade de dados que precisa ser armazenada permanentemente. Foi escolhido o SQLite para essa finalidade. Esse armazenamento pode ser pensado e estruturado de diversas maneiras

distintas, e a maneira escolhida visa à simplicidade e a um fácil entendimento por parte do leitor iniciante.

O primeiro aspecto é que ao usar esse programa o usuário tem a opção de criar e gerenciar vários torneios ao mesmo tempo. Assim, os nomes dos torneios devem ser armazenados. Para isso, é mantido um banco de dados chamado “torneios.db”, dentro do qual há uma tabela que contém dois campos: o nome do torneio e a quantidade de turnos do mesmo. Quando o programa é executado pela primeira vez em um computador esse arquivo é criado. Daí para a frente, ele sempre é lido logo no início e utilizado para montar o menu de torneios da tela inicial.

Além disso, para cada torneio cadastrado é criado um arquivo de banco de dados do SQLite com o nome “<nometorneio>.db”, em que o identificador <nometorneio> é substituído pelo nome escolhido pelo usuário. Todos os arquivos de banco de dados são criados e mantidos na mesma pasta na qual se encontra o programa. Na Figura 10.7 podem ser vistos todos os arquivos “.db” mencionados, em conjunto com o arquivo do programa “torneio.py”.

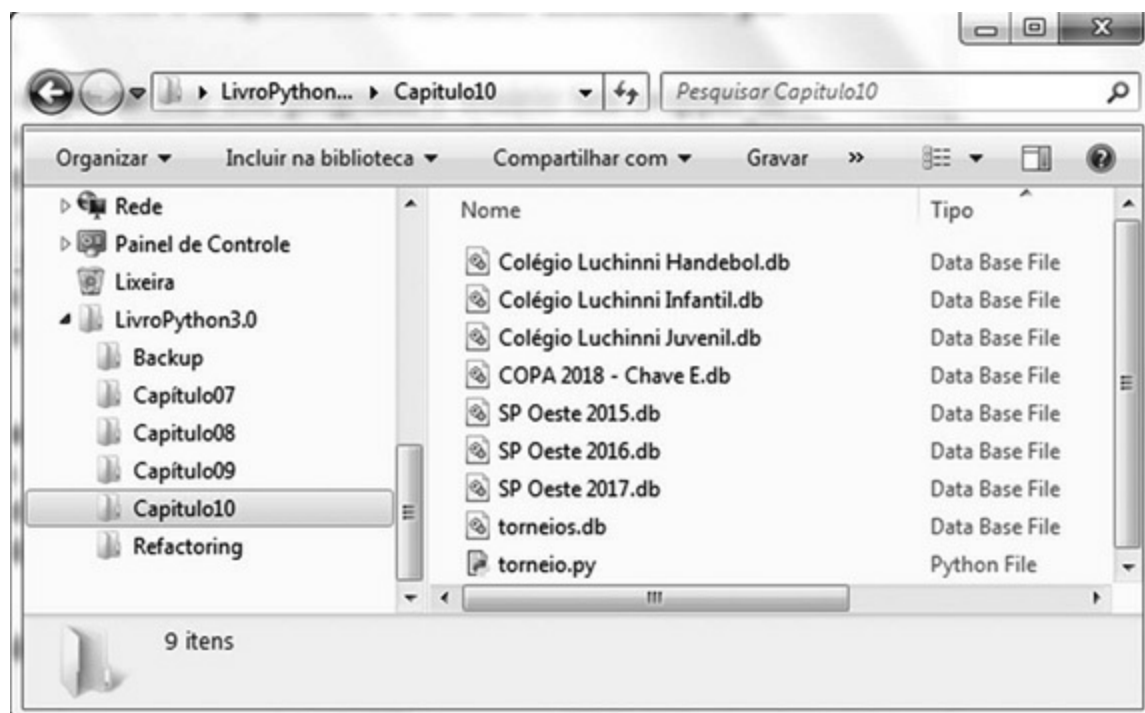


Figura 10.7 Armazenamento de dados do programa torneio.py.

O banco de dados “torneios.db” contém apenas a tabela “Torneios”, que, por sua vez, tem dois campos: um para o nome do torneio e outro para a quantidade de turnos, conforme definido no Quadro 10.3 e ilustrado na Figura 10.8.

Campo	Tipo	Observações
nometorneio	Texto	Contém o nome do torneio.
turnos	Número inteiro	Contém 1 ou 2 indicando a quantidade de turnos do torneio.

Quadro 10.3 Campos da tabela “Torneios”.

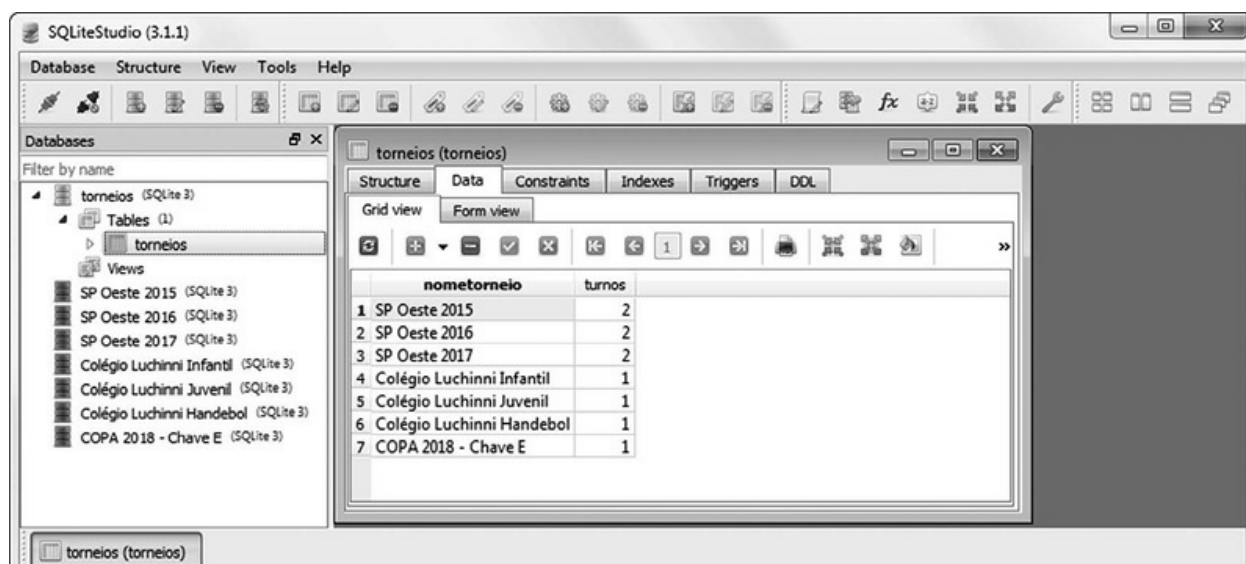


Figura 10.8 Banco de dados torneios.db visualizado por meio do SQLite Studio.

Por sua vez, os bancos de dados de torneios têm duas tabelas: a tabela “Times”, para armazenar os nomes dos times participantes, e a tabela “Jogos”, para armazenar as rodadas e os resultados dos jogos.

Campo	Tipo	Observação
nometime	Texto	Contém o nome do time.

Quadro 10.4 Campos da tabela “Times”

Campo	Tipo	Observações
numjogo	Número inteiro	Número do jogo.
numrod	Número inteiro	Número da rodada.
time1	Texto	Nome do primeiro time.
gol1	Número inteiro	Gols marcados pelo time1.
time2	Texto	Nome do segundo time.
gol2	Número inteiro	Gols marcados pelo time2.

Quadro 10.5 Campos da tabela “Jogos”.

A Figura 10.9 ilustra, por meio do SQLite Studio, a organização do banco do torneio, exibindo dados armazenados na tabela “Jogos”.

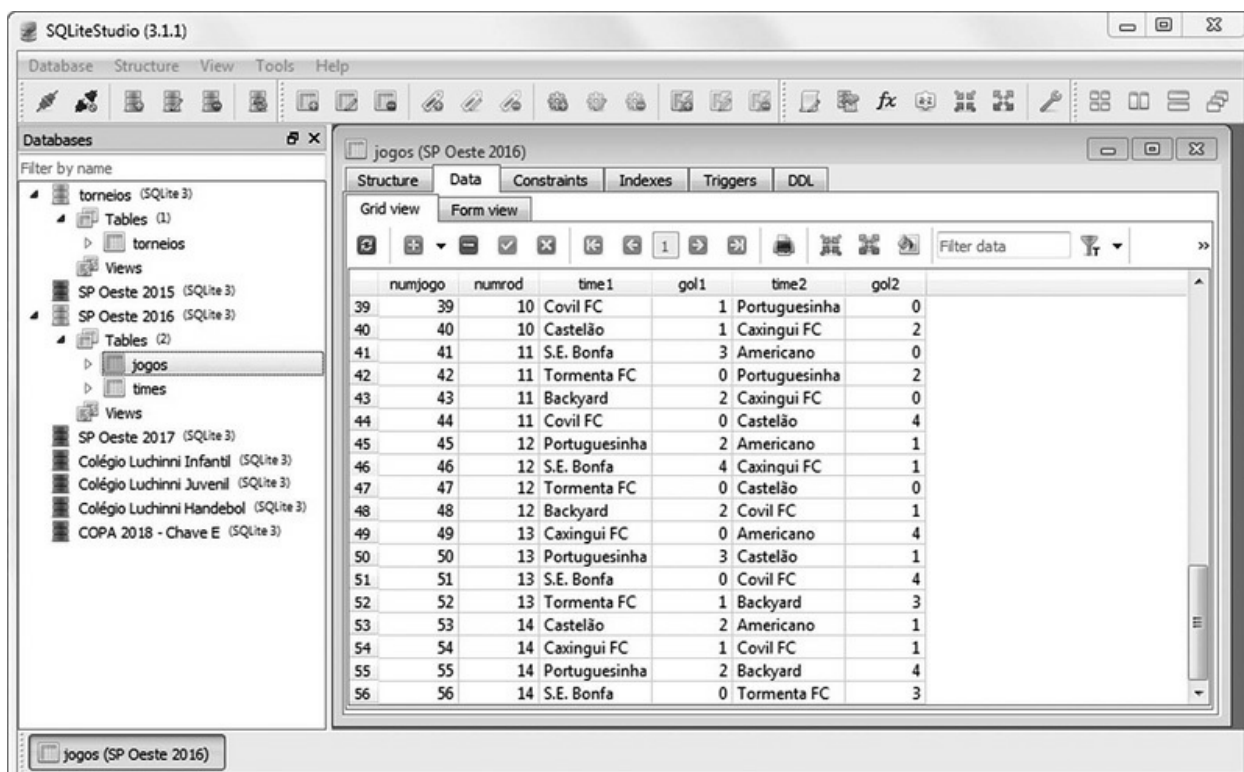


Figura 10.9 Banco de dados “SP Oeste 2016” visualizado por meio do SQLite Studio.

Deste ponto em diante, tem início a descrição do programa. Tudo o que

será apresentado a partir daqui diz respeito ao código-fonte do programa e é necessário que esteja presente no arquivo “torneio.py” para que este funcione corretamente. O programa está organizado em 33 funções que devem estar reunidas em um único código-fonte. Tais funções serão apresentadas agrupadas segundo a finalidade a que se destinam dentro do programa e podem ser divididas em três grupos.

Grupo	Descrição
1 Funções gerais	Pequeno número de funções que não se enquadram em nenhum dos outros dois próximos grupos.
2 Criação de torneio	Funções necessárias às funcionalidades de criação de um novo torneio.
3 Gerenciamento de torneio	Funções necessárias às funcionalidades relativas ao gerenciamento de um torneio.

10.2.3 Objetos globais e início do programa

Nessa implementação foram utilizados quatro objetos de escopo global para conter informações que, dada sua relevância, são necessárias em muitas funções do programa. Assim, em vez de passá-los como parâmetro a cada chamada de função, optou-se pelo escopo global, que faz que sejam visíveis e possam ser alteradas em todas as funções do programa.

Três desses objetos são empregados para gerenciamento de torneio em andamento, são criados dentro da função GerenciaTorneio e eliminados ao seu final, por não serem necessários em outras partes do programa. O quarto objeto global é utilizado na composição das telas.

Identificador	Tipo	Observações
Torneio	String	Contém o nome do torneio escolhido no menu principal para ser gerenciado.
	Número	Contém a quantidade de turnos do torneio escolhido

Turnos	inteiro	no menu principal.
Times	Lista de listas	Contém os dados necessários para exibição da tabela de classificação do torneio. Cada sublista contida em “Times” é formada por um string, que é o nome do time e oito números inteiros que são: quantidade de pontos, jogos, vitórias, empates, derrotas, gols pró, gols contra e saldo de gols. Exemplo de uma dessas sublistas: [“Americano”, 5, 4, 1, 2, 1, 4, 4, 0]
LargTela	Número inteiro	Usado para definir a largura das saídas formatadas que compõem o visual das telas do programa.

Quadro 10.6 Objetos globais usados no programa.

No início do código-fonte deve haver a importação dos módulos indicados no Exemplo 10.1, pois algumas de suas funções são utilizadas no programa, bem como é definido o objeto LargTela.

Exemplo 10.1 Programa torneio.py – módulos importados

```
from datetime import date

import os

import sqlite3

LargTela = 70
```

O ponto de partida do programa está no Exemplo 10.2, que contém a chamada à função MenuPrincipal e, ao término desta, faz o fechamento do programa exibindo mensagem de encerramento.

Exemplo 10.2 Programa torneio.py – ponto de partida do programa

```
# Ponto de início da execução

MenuPrincipal()

print("\n"*2)

ExibeLinha("Programa Torneio encerrado", 30)

print("\n")

Pausa("Pressione Enter para sair.", 30)
```

10.2.4 Grupo de funções gerais

A função MenuPrincipal é a primeira dessas funções gerais. Seu propósito é permanecer em laço até que o usuário decida sair do programa. Nesse laço é feita a exibição da tela inicial mostrada na Figura 10.1 e é lida e processada a opção do usuário. Para exibir a lista de torneios cadastrados no programa é carregado o objeto local “Torneios” (não confundir com o objeto global “Torneio” mencionado no item anterior) com o uso da função PreparaAmbiente. Caso haja torneios cadastrados, o comando for é usado para iterar sobre o objeto e montar a lista de torneios.

Quanto às opções disponíveis ao usuário, “N” leva à chamada da função CriaNovoTorneio descrita no Item 10.2.5 e o número do torneio mostrado na tela leva à chamada da função GerenciaTorneio descrita no Item 10.2.6. “S” encerra o laço dessa função e leva ao término do programa.

Exemplo 10.3 Função MenuPrincipal

```
def MenuPrincipal():

    while True:
```

```
Torneios = PreparaAmbiente()

TopoTela("Menu Principal")

print("Opções: ")

print(" (N) Criar Novo Torneio")

print(" (.) Gerenciar Torneio Existente")

if len(Torneios) == 0:

    print(" { não há torneios cadastrados }")

else:

    for i, t in Torneios.items():

        print(" ({} {})".format(i, t["nome"]))

    print(" (S) Sair do programa")

    print("\npara escolher digite o que está entre parênteses")

    opc = input("sua opção? >>> ")

    opc = opc.upper()

    if opc == "N":

        CriaNovoTorneio()

    elif opc.isnumeric():
```

```
n = int(opc)
```

```
if n in Torneios:
```

```
GerenciaTorneio(Torneios[n])
```

```
elif opc == "S":
```

```
break
```

A função `PreparaAmbiente` verifica se o banco de dados “torneios.db” existe. Em caso afirmativo, lê os torneios cadastrados, carrega e retorna um dicionário. Caso contrário, cria o banco de dados e a tabela de torneios dentro deste e retorna um dicionário vazio. Essa criação ocorrerá apenas na primeira vez em que o programa for executado em um computador.

Exemplo 10.4 Função `PreparaAmbiente`

```
def PreparaAmbiente():
```

```
    """ Se o B.D. torneios.db existe, então, lê os torneios.
```

```
    Caso contrário, cria o B.D. Necessário no primeiro
```

```
    uso do programa, """
```

```
    conector = sqlite3.connect("torneios.db")
```

```
    cursor = conector.cursor()
```

```
    sql = """
```

```
    select name from sqlite_master
```


where type='table' and name = 'torneios'

“””

cursor.execute(sql)

R = {}

N = 1

if cursor.fetchone() == None:

sql = “create table torneios (nometorneio text, turnos int)”

cursor.execute(sql)

else:

sql = “select * from torneios”

cursor.execute(sql)

for x in sorted(cursor.fetchall()):

item = {}

item[“nome”] = x[0]

item[“turnos”] = x[1]

R[N] = item

N+=1

```
cursor.close()
```

```
conector.close()
```

```
return R
```

O Exemplo 10.5 exibe três funções criadas para auxiliar na exibição das telas e para pausar o programa ao exibir alguma mensagem para o usuário. São funções auxiliares que ajudam a organizar o código eliminando redundância desnecessária.

Exemplo 10.5 Funções auxiliares de exibição em tela

```
def ExibeLinha(msg, tam = 0, alinha = “^”):
```

```
    borda = (LargTela - tam - 2) // 2
```

```
    sfmt = “{:” + alinha + str(tam) + “}”
```

```
    print(“-” * borda, sfmt.format(msg), “-” * borda)
```

```
def Pausa(msg, tam=64):
```

```
    if msg != “”:
```

```
        ExibeLinha(msg, tam)
```

```
    input()
```

```
def TopoTela(msg = “”):
```

```
    print(“\n” * 2, “-” * LargTela, sep = “”)
```

```
    ExibeLinha(“Programa Torneio”, 40, “^”)
```

```
if msg != "":
```

```
ExibeLinha(msg, 40, "^")
```

```
print("-" * LargTela)
```

10.2.5 Grupo de funções de criação de torneio

Quando o usuário deseja criar um novo torneio, ocorre a chamada da função `CriaNovoTorneio`, mostrada no Exemplo 10.6. Essa é uma função concentradora, ou seja, ela existe para organizar a sequência lógica das tarefas que serão efetivamente realizadas por outras funções. Em linhas gerais, por meio dela é feita a leitura do nome do novo torneio e de sua quantidade de turnos, dos times participantes, e é gerado o banco de dados do torneio.

Exemplo 10.6 Função `CriaNovoTorneio`

```
def CriaNovoTorneio():
```

```
    NomeTorneio = input("\nNome do Novo Torneio: ")
```

```
    if not ValidaTorneio(NomeTorneio):
```

```
        return None
```

```
    QtdeTurnos = ObtemQtdeTurnos()
```

```
    if QtdeTurnos == 0:
```

```
        return None
```

```
    TopoTela("Criação de Novo Torneio")
```

```
ExibeLinha("Novo Torneio: " + NomeTorneio, 64)
```

```
ExibeLinha("(a qualquer momento digite 'sair'
```

```
para desistir)", 64)
```

```
print("-" * LargTela)
```

```
ExibeLinha("Digite os nomes dos times participantes", 70)
```

```
ExibeLinha("Digite 'fim' para concluir e salvar
```

```
os nomes dos times", 70)
```

```
ListaTimes = ObtemNomesTimes()
```

```
if ListaTimes:
```

```
    CriaBDTorneio(NomeTorneio, ListaTimes)
```

```
    GravaNomeTorneio(NomeTorneio, QtdeTurnos)
```

```
    GeraGravaJogos(NomeTorneio, QtdeTurnos, ListaTimes)
```

A seguir são apresentadas as funções chamadas por `CriaNovoTorneio`, na ordem em que ocorrem.

A função `ValidaTorneio` (Exemplo 10.7) recebe o nome digitado e retorna *False* caso este seja nulo ou já esteja cadastrado. Neste último caso, faz uma pausa para mostrar uma mensagem. Caso essas duas situações não ocorram, então, retorna *True*.

A função `ObtemQtdeTurnos` (Exemplo 10.7) foi criada para ler e tratar a quantidade de turnos. É possível que o usuário digite qualquer coisa,

inclusive texto, porém, o que se espera é que digite apenas os algarismos 1 ou 2, então, foram implementados um tratamento de exceção e um laço que se repetirá até que um valor apropriado seja fornecido. Como o usuário pode desistir de cadastrar o novo torneio, também é permitida a digitação do algarismo 0. No retorno da chamada, se a quantidade de turnos for zero, sua criação é interrompida (ver Exemplo 10.6).

Exemplo 10.7 Funções ValidaTorneio e ObtemQtdeTurnos

```
def ValidaTorneio(NomeTorneio):  
  
    if NomeTorneio == "":  
  
        return False  
  
    elif ExisteTorneio(NomeTorneio):  
  
        Pausa(NomeTorneio + " já existe (pressione Enter)")  
  
        return False  
  
    else:  
  
        return True  
  
def ObtemQtdeTurnos():  
  
    while True:  
  
        print("Digite a quantidade de turnos (1 ou 2): ")  
  
        print("Digite 0 para cancelar.")
```

```
Qtde = input("quantos turnos? >>> ")

try:

    Qtde = int(Qtde)

except:

    print("Entrada inválida {}".format(Qtde))

    print("Digite qtde 1 ou 2. Digite 0 para desistir.")

else:

    if 0 <= Qtde <= 2:

        return Qtde
```

A próxima função, `ObtemNomesTimes`, permanece em laço fazendo a leitura dos nomes dos times participantes. Esse laço termina com a digitação de uma destas palavras: “sair” e “fim”. No caso de “fim”, o laço termina e a função retorna uma lista com os nomes digitados. No caso de “sair”, a função retorna *None*, indicando que o usuário desistiu do processo.

Exemplo 10.8 Função `ObtemNomesTimes`

```
def ObtemNomesTimes():

    L = []

    Cont = 1

    while True:
```

```
s = input("Time {:d}: ".format(Cont))
```

```
if s.upper() == "SAIR": return None
```

```
if s.upper() == "FIM": break
```

```
L.append(s)
```

```
Cont += 1
```

```
return L
```

Por fim, nas últimas linhas da função `CriaNovoTorneio` é processada a criação do torneio. A função `CriaBDTorneio` gera a estrutura do banco de dados necessária ao armazenamento dos dados, criando o BD fisicamente, e nele, as tabelas “Times” e “Jogos”. A tabela “Times” recebe todos os nomes digitados pelo usuário contidos no parâmetro “ListaTimes”.

A função `GravaNomeTorneio` insere o nome do novo torneio na tabela do banco de dados “torneios.db”, e a partir daí esse novo torneio aparecerá no menu da tela principal.

Exemplo 10.9 Funções `CriaBDTorneio` e `GravaNomeTorneio`

```
def CriaBDTorneio(NomeTorneio, ListaTimes):
```

```
    # Cria o BD do torneio
```

```
    conector = sqlite3.connect(NomeTorneio + “.db”)
```

```
    cursor = conector.cursor()
```

```
    # Cria a tabela times
```

```
sql = "create table times (nometime text)"
```

```
cursor.execute(sql)
```

```
# Insere os times na tabela
```

```
sql = "insert into times (nometime) values (?)"
```

```
for nome in ListaTimes:
```

```
    cursor.execute(sql, (nome,))
```

```
conector.commit()
```

```
# Cria a tabela de Jogos
```

```
sql = ""
```

```
create table jogos (
```

```
    numjogo int NOT NULL PRIMARY KEY ASC,
```

```
    numrod int,
```

```
    time1 text, gol1 int,
```

```
    time2 text, gol2 int)""
```

```
cursor.execute(sql)
```

```
cursor.close()
```

```
conector.close()
```



```
def GravaNomeTorneio(NomeTorneio, QtdeTurnos):  
  
    # Insere o nome do torneio no BD de torneios  
  
    conector = sqlite3.connect("torneios.db")  
  
    cursor = conector.cursor()  
  
    sql = "insert into torneios (nometorneio, turnos) values (?, ?)"  
  
    cursor.execute(sql, (NomeTorneio, QtdeTurnos))  
  
    conector.commit()  
  
    cursor.close()  
  
    conector.close()
```

10.2.5.1 Algoritmo de criação dos jogos

Por fim, a função GeraGravaJogos é a responsável pela criação dos jogos e rodadas, fazendo o emparelhamento dos times.

Em termos de lógica e algoritmos, essa é a rotina mais interessante e a grande novidade desse projeto. A grande questão que se coloca neste ponto é: dados os nomes dos times, como escrever um algoritmo que produza todos os jogos de um turno sem que falte algum jogo ou haja repetição? A resposta é utilizar um algoritmo conhecido com **Round-Robin Tournament**. Mas cuidado, não o confunda com o algoritmo Round-Robin sem a palavra "Tournament". Este último é um algoritmo muito conhecido empregado em sistemas operacionais e não guarda qualquer relação com o Round-Robin Tournament, que será utilizado aqui.

Dica

Você poderá encontrar na internet diversos sites que implementam esse algoritmo para gerar campeonatos. Uma das possibilidades está no endereço, [disponível em: <https://www.printyourbrackets.com/generator.php>](https://www.printyourbrackets.com/generator.php).

Esse algoritmo é exemplificado na Figura 10.10. Ele consiste em fazer o emparelhamento dos times o primeiro com o último, o segundo com o penúltimo, e assim por diante, de modo que isso produz todos os jogos da primeira rodada. Caso o número de times seja ímpar, acrescenta-se um time fictício (no algoritmo chamado de FOLGA) e procede-se da mesma maneira.

Para produzir a segunda rodada, deve-se mover o time B para a última posição da lista, adiantando os times de C até H uma posição para a esquerda. Para a terceira rodada, transfere-se o time C para a última posição e adiantam-se os demais. Durante todo o processo, o time A não deve ser movido, e quando o time B chegar à sua posição original, todos os jogos de um turno estarão montados. Para dois turnos, é só usar o mesmo conjunto de jogos já gerados e, se for o caso, trocá-los de posição, fazendo que o mandante de campo apareça do lado esquerdo, e o visitante, do lado direito na tabela de jogos.

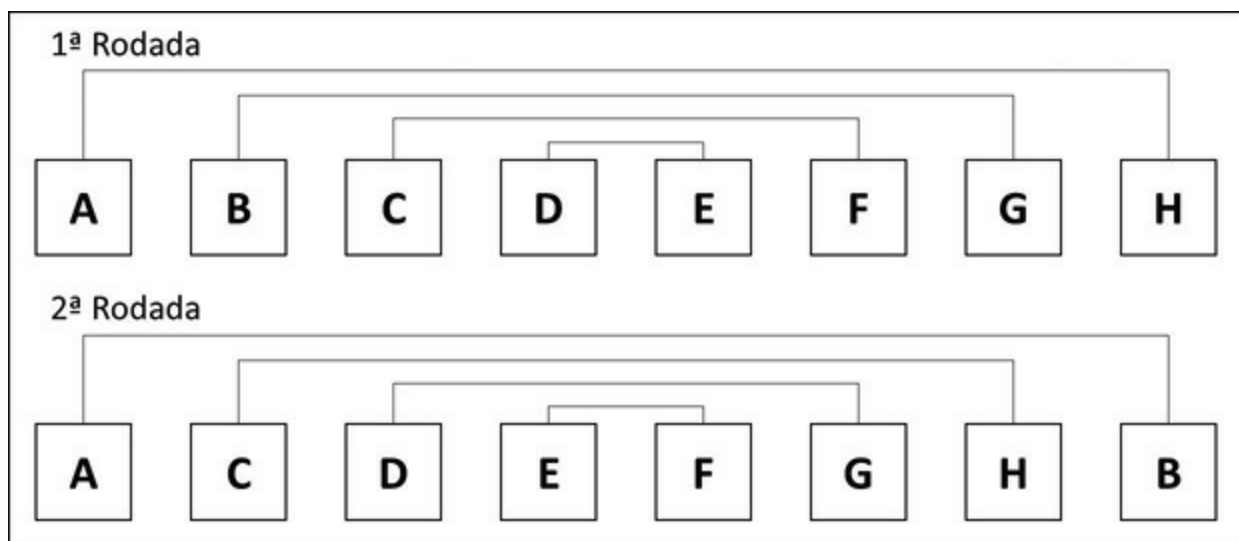


Figura 10.10 Ilustração do algoritmo Round-Robin Tournament.

A função `GeraGravaJogos` recebe como parâmetros de entrada o nome do torneio, a quantidade de turnos e a lista de times. As partidas são geradas no dicionário **Jogos**, cuja chave é o número do jogo e o valor associado é um dicionário aninhado contendo o número da rodada, o time 1 e o time 2.

Após a geração do dicionário **Jogos**, todas as partidas são gravadas no banco de dados, para 1 ou 2 turnos, conforme o caso.

Exemplo 10.10 Função `GeraGravaJogos`

```
def GeraGravaJogos(NomeTorneio, QtdeTurnos, ListaTimes):
```

```
    # Gera os Jogos
```

```
    Jogos = {}
```

```
    NJogo = 1
```

```
    Qtde = len(ListaTimes)
```

```
    if Qtde % 2 == 1:
```

```
        ListaTimes.append("FOLGA")
```

```
    Qtde += 1
```

```
    # Implementa o algoritmo Round-Robin Tournament
```

```
    for r in range(Qtde-1):
```

```
        for i in range(Qtde//2):
```

```
            if ListaTimes[i] == "FOLGA" or \
```

```
ListaTimes[Qtde-1-i] == "FOLGA":
```

```
continue
```

```
umJogo = {}
```

```
umJogo["rodada"] = r+1
```

```
umJogo["time1"] = ListaTimes[i]
```

```
umJogo["time2"] = ListaTimes[Qtde-1-i]
```

```
Jogos[NJogo] = umJogo
```

```
NJogo += 1
```

```
aux = ListaTimes[1]
```

```
del(ListaTimes[1])
```

```
ListaTimes.append(aux)
```

```
# Insere no banco de dados os jogos gerados
```

```
conector = sqlite3.connect(NomeTorneio + ".db")
```

```
cursor = conector.cursor()
```

```
sql = ""
```

```
insert into jogos (numjogo, numrod, time1, time2)
```

```
values (?, ?, ?, ?)
```

“””

```
for NJogo, Jogo in Jogos.items():  
  
    cursor.execute(sql, (NJogo, Jogo[“rodada”], Jogo[“time1”],  
  
Jogo[“time2”]))  
  
    if QtdeTurnos == 2:  
  
        for NJogo, Jogo in Jogos.items():  
  
            cursor.execute(sql, (NJogo+(Qtde-1)*Qtde/2,  
  
Jogo[“rodada”]+Qtde-1,  
  
Jogo[“time2”], Jogo[“time1”]))  
  
        conector.commit()  
  
        cursor.close()  
  
        conector.close()
```

10.2.6 Grupo de funções de gerenciamento de torneio

Neste grupo está concentrada a maior parte das funções do programa, e isso ocorre porque aqui se concentra a maior parte de suas funcionalidades. O Exemplo 10.11 exhibe a função GerenciaTorneio, que é a responsável pelo desenho da tela mostrada na Figura 10.2 e concentra a chamada às demais funções de gerenciamento. Nessa função são criados os objetos globais mencionados no Item 10.2.3. Os nomes de times são carregados e é gerada a lista com os campos necessários à montagem da tabela de classificação, são

exibidos os nomes de times e a classificação. Em seguida, é exibido o menu de opções e é lida e processada a opção do usuário.

Exemplo 10.11 Função GerenciaTorneio

```
def GerenciaTorneio(t):  
  
    global Times, Torneio, Turnos  
  
    Torneio = t["nome"]  
  
    Turnos = t["turnos"]  
  
    Times = CarregaTimes()  
  
    Qtde, NRod, NJog = CalcPramsTorneio()  
  
    while True:  
  
        TopoTela("Gerenciamento de Torneio")  
  
        ExibeTimes()  
  
        ExibeClassificacao()  
  
        print("Opções: ")  
  
        print(" (.) Para ver uma rodada digite seu número.")  
  
        print(" Rodadas Válidas de 1 a {}".format(NRod))  
  
        print(" (G) Grava o Torneio em HTML")  
  
        print(" (E) Exclui o Torneio")
```

```
print(" (S) Voltar ao Menu Principal")
```

```
opc = input("sua opção? >>> ")
```

```
opc = opc.upper()
```

```
if opc == "N":
```

```
    NovoTorneio()
```

```
elif opc.isnumeric():
```

```
    n = int(opc)
```

```
    if 1 <= n <= NRod:
```

```
        GerenciaRodada(n)
```

```
elif opc == "G":
```

```
    GravaHTML()
```

```
elif opc == "E":
```

```
    if ExcluiTorneio(Torneio):
```

```
        break
```

```
elif opc == "S":
```

```
    break
```

```
del(Times, Torneio, Turnos)
```

A função CarregaTimes é a responsável pela geração do objeto global Times a partir da leitura do banco de dados.

Exemplo 10.12 Função CarregaTimes

```
def CarregaTimes():  
  
    global Torneio  
  
    T = []  
  
    conector = sqlite3.connect(Torneio + “.db”)  
  
    cursor = conector.cursor()  
  
    sql = “select nometime from times order by nometime”  
  
    cursor.execute(sql)  
  
    for time in cursor.fetchall():  
  
        t = [time[0]] + [0]*8  
  
        T.append(t)  
  
    cursor.close()  
  
    conector.close()  
  
    return T
```

A função ExibeTimes é responsável pela exibição dos nomes dos times participantes do torneio escolhido e também dos números de rodadas e jogos, sendo que estes são calculados em outra função, CalaParamsTorneio.

Exemplo 10.13 Funções CalcParamsTorneio e ExibeTimes

```
def CalcParamsTorneio():  
  
    global Times, Turnos  
  
    Qtde = len(Times)  
  
    if Qtde % 2 == 0:  
  
        NRod = (Qtde - 1) * Turnos  
  
    else:  
  
        NRod = Qtde * Turnos  
  
        NJog = (Qtde - 1) * Qtde // 2 * Turnos  
  
    return Qtde, NRod, NJog  
  
def ExibeTimes():  
  
    global Times  
  
    Qtde, NRod, NJog = CalcParamsTorneio()  
  
    ExibeLinha("Times deste Torneio " + Torneio, 64)  
  
    cont = 1  
  
    s = ""  
  
    for t in Times:
```

```

s = s + "{:<15}".format(t[0])

if cont % 4 == 0:

    ExibeLinha(s, 64)

    s = ""

    cont += 1

    if s != "":

        ExibeLinha(s, 64)

    ExibeLinha("", 64)

    s = "Nº de Rodadas: {} - Nº de Jogos: {}"

    ExibeLinha(s.format(NRod, NJog), 64)

    print("-" * LargTela)

```

Na função `ExibeClassificacao` é feita a exibição da tabela de classificação. Porém, para que isso seja possível, primeiro é preciso que sejam apurados os dados de classificação do torneio a partir dos resultados dos jogos, bem como seja definida a ordem do melhor para o pior colocado. Todas essas tarefas são executadas pela função chamada `MontaClassificacao`, e o restante de `ExibeClassificacao` apenas gera as saídas em tela.

Exemplo 10.14 Função `ExibeClassificacao`

```
def ExibeClassificacao():
```

```

global Times, Torneio

MontaClassificacao()

sfmt = "{:>3} {:<16}" + "{:>6}"*8

print("-" * LargTela)

s = "Pos;Time;PG;J;V;E;D;GP;GC;SG"

print(sfmt.format(*s.split(";")))

print("-" * LargTela)

Pos = 1

for time in Times:

    dados = (Pos,) + tuple(time)

    print(sfmt.format(*dados))

    Pos += 1

print("-" * LargTela)

```

Por sua vez, a função `MontaClassificacao` tem duas partes distintas e bem definidas. Na primeira, é feita a apuração dos resultados como pontos, número de jogos, vitórias, empates, derrotas e totais de gols marcados e sofridos. A função `ZeraDadosTimes` é chamada no início para zerar todos esses campos. Em seguida, a função `LeJogos` é chamada e retorna uma lista com todos os jogos que já tenham placar registrado (os jogos em que os campos `gols1` e `gols2` sejam nulos ficam de fora). Lidos os jogos, é iniciado

um laço que itera pelos Jogos comparando os gols marcados pelos times e computando os resultados de maneira apropriada por meio de chamadas à função ComputaResultado. Essas quatro funções citadas estão no Exemplo 10.15.

Exemplo 10.15 Função MontaClassificacao e suas associadas

```
def MontaClassificacao():

    global Times, Torneio

    # Primeira parte – Computa os resultados

    ZeraDadosTimes()

    Jogos = LeJogos()

    for jogo in Jogos:

        if jogo[3] == jogo[5]:

            ComputaResultado(jogo[2], "E", jogo[3], jogo[5])

            ComputaResultado(jogo[4], "E", jogo[5], jogo[3])

        elif jogo[3] < jogo[5]:

            ComputaResultado(jogo[2], "D", jogo[3], jogo[5])

            ComputaResultado(jogo[4], "V", jogo[5], jogo[3])

        elif jogo[3] > jogo[5]:

            ComputaResultado(jogo[2], "V", jogo[3], jogo[5])
```

```
ComputaResultado(jogo[4], "D", jogo[5], jogo[3])
```

```
# Segunda parte – Ordena a tabela de classificação
```

```
OrdenaTimes()
```

```
def ZeraDadosTimes():
```

```
    global Times
```

```
    for time in Times:
```

```
        for i in range(1, 9):
```

```
            time[i] = 0
```

```
def LeJogos():
```

```
    global Torneio
```

```
    conector = sqlite3.connect(Torneio + ".db")
```

```
    cursor = conector.cursor()
```

```
    sql = ""
```

```
    select * from jogos
```

```
    where gol1 is not null order by numjogo
```

```
    ""
```

```
    cursor.execute(sql)
```

```
J = cursor.fetchall()
```

```
cursor.close()
```

```
conector.close()
```

```
return J
```

```
def ComputaResultado(QualTime, Res, GP, GC):
```

```
    global Times
```

```
    for time in Times:
```

```
        if time[0] == QualTime:
```

```
            time[2] += 1 # qtde jogos
```

```
            time[6] += GP # gols pro
```

```
            time[7] += GC # gols contra
```

```
            time[8] += GP-GC # saldo gols
```

```
        if Res == "V":
```

```
            time[1] += 3 # ptos ganhos
```

```
            time[3] += 1 # qtde vitorias
```

```
        elif Res == "E":
```

```
            time[1] += 1 # ptos ganhos
```

```
time[4] += 1 # qtde empates
```

```
elif Res == "D":
```

```
time[5] += 1 # qtde derrotas
```

Após apurados os resultados dos jogos, é necessário ordenar a lista Times. Isso é feito na função `OrdenaTimes`, chamada na última linha de `MontaClassificacao` e na qual é implementado um algoritmo de ordenação *Bubble Sort*. Esse é um algoritmo simples e já foi explicado no Exercício resolvido 4.11. Trata-se de um algoritmo de simples entendimento, porém, lento para grandes quantidades de dados. Por grandes quantidades entenda-se algo acima de 10^5 elementos. No caso de um torneio esportivo, tem-se no máximo duas dezenas de elementos, de modo que o *Bubble Sort* será suficientemente rápido.

Por outro lado, não será utilizada nenhuma função de ordenação disponível em Python, porque o critério de comparação de dois times é muito complexo, uma vez que envolve cinco diferentes parâmetros de comparação e desempate, conforme listado no Quadro 10.2. Como pode ser visto no Exemplo 10.16, a função `OrdenaTimes` propriamente dita é simples. O laço externo depende de haver ocorrido troca de posição de elementos da lista, e o laço interno percorre toda a lista, comparando um elemento a seu vizinho subsequente, e caso o elemento i seja menor – porque a ordenação é decrescente – que o elemento $i+1$, ocorre a troca de posições. A complexidade desse processo fica por conta da função `Compara`, que deve atender a todos os critérios. Essa função recebe os parâmetros a e b , que são sublistas da lista de times. Ela retorna -1 caso a seja menor que b ; 0 caso a seja igual a b e 1 caso a seja maior que b .

Exemplo 10.16 Função `OrdenaTimes` e suas associadas

```

def OrdenaTimes():

    # usa BubbleSort para ordenar os times

    global Times

    Trocou = True

    while Trocou:

        Trocou = False

        i = 0

        while i < len(Times)-1:

            if Compara(Times[i], Times[i+1]) < 0:

                Times[i], Times[i+1] = Times[i+1], Times[i]

                Trocou = True

            i += 1

    def Compara(a, b):

        # 1º Crit. Pontos

        if a[1] < b[1]:

            return -1

        elif a[1] > b[1]:

```



```
return 1
```

```
# 2º Crit. Vitórias
```

```
if a[3] < b[3]:
```

```
return -1
```

```
elif a[3] > b[3]:
```

```
return 1
```

```
# 3º Crit. Saldo Gols
```

```
if a[8] < b[8]:
```

```
return -1
```

```
elif a[8] > b[8]:
```

```
return 1
```

```
# 4º Crit. Gols Pró
```

```
if a[6] < b[6]:
```

```
return -1
```

```
elif a[6] > b[6]:
```

```
return 1
```

```
return ConfrontoDireto(a, b)
```

```

def ConfrontoDireto(a, b):

    global Torneio

    d = {"timeA":a[0], "timeB":b[0]}

    ptoA = ptoB = 0

    conector = sqlite3.connect(Torneio + ".db")

    cursor = conector.cursor()

    sql = """

    select * from jogos where gol1 is not null and

    time1 = :timeA and time2 = :timeB

    """

    cursor.execute(sql, d)

    J = cursor.fetchone()

    if J:

        if J[3] == J[5]:

            ptoA += 1

            ptoB += 1

        elif J[3] > J[5]:

```

ptoA += 3

elif J[5] < J[3]:

ptoB += 3

sql = “”

select * from jogos where gol1 is not null and

time1 = :timeB and time2 = :timeA

“”

cursor.execute(sql, d)

J = cursor.fetchone()

if J:

if J[3] == J[5]:

ptoA += 1

ptoB += 1

elif J[3] > J[5]:

ptoB += 3

elif J[5] < J[3]:

ptoA += 3

```
cursor.close()

conector.close()

if ptoA > ptoB:

    return -1

elif ptoA < ptoB:

    return 1

else:

    return 0
```

Por fim, há o confronto direto, que exige a recuperação dos jogos com placar já registrado e a verificação dos resultados para definir entre os times a e b qual deverá ficar na frente caso todos os demais critérios estejam empatados. Isso é executado na função `ConfrontoDireto`. Devem ser verificadas duas possibilidades: o time a como primeiro time e b como segundo, e o inverso. Para isso, executa-se o comando `sql`, que busca o jogo em cada situação, e comparam-se os gols (que estarão em `J[3]` e `J[5]`) para a atribuição de pontos.

Na tela de gerenciamento do torneio, já exibida, estão disponíveis ao usuário três funcionalidades. Ao digitar o número da rodada, o usuário tem acesso ao lançamento de resultados de jogos, conforme apresentado na tela da Figura 10.3. No Exemplo 10.17 tem-se a função `GerenciaRodada`, responsável pela implementação da tela exibida na Figura 10.3. Essa função permanece em laço até que o usuário opte por sair, e dentro dele é feita a exibição dos jogos da rodada por meio da função `ExibeJogos`, bem como se pode lançar ou apagar o resultado de um jogo.

Exemplo 10.17 Função GerenciaRodada

```
def GerenciaRodada(NRod):  
  
    global Times, Torneio, Turnos  
  
    Jogos = ObtemJogosRodada(NRod)  
  
    while True:  
  
        TopoTela("Gerenciamento de Torneio")  
  
        ExibeTimes()  
  
        JogosRodada = ExibeJogos(Jogos)  
  
        print("Opções: “)  
  
        print(" (.) Para atualizar o placar de um jogo digite:")  
  
        print(" N°Jogo,GolsA,GolsB exemplo: 12,2,1")  
  
        print(" (.) Para limpar o placar de um jogo digite:")  
  
        print(" N°Jogo,limpa exemplo: 12,limpa")  
  
        print(" (S) Voltar ao Menu do Torneio")  
  
        opc = input("sua opção? >>> “)  
  
        opc = opc.upper()  
  
        if opc == "S":
```

```

break

else:

    msg = TrataEntrRodada(opc, JogosRodada)

    if msg != None:

        Pausa(msg + “ (pressione Enter)”)

    else:

        Jogos = ObtemJogosRodada(NRod)

```

A função `ObtemJogosRodada` é executada uma vez antes do início do laço e, depois, é executada novamente dentro deste. Essa função busca no banco de dados os jogos da rodada desejada, independentemente de o jogo ter sido jogado ou não, e retorna uma lista de listas. Cada sublista é um jogo e seus elementos são: [nº jogo, nº rodada, time1, gols time1, time2, gols time2]. Na função `ExibeJogos`, caso os elementos 3 e 5 (os gols) sejam nulos, são omitidos da exibição; caso contrário, são exibidos. Isso evita que a palavra *None* seja mostrada na tela quando um jogo ainda não tem o resultado registrado.

Essa função `ExibeJogos` também gera uma lista `JRod` que é retornada ao seu final. Tal lista contém os números dos jogos da rodada. No retorno dessa função em `GerenciaRodada` a lista é atribuída ao objeto `JogosRodada`, que será utilizado para validar o lançamento dos resultados de um jogo. O motivo disso é explicado na descrição da função `TrataEntrRodada`.

Exemplo 10.18 Funções `ObtemJogosRodada` e `ExibeJogos`

```

def ObtemJogosRodada(NRod):

```

```
global Torneio
```

```
conector = sqlite3.connect(Torneio + “.db”)
```

```
cursor = conector.cursor()
```

```
sql = “select * from jogos where numrod = ? order by numjogo”
```

```
cursor.execute(sql, (NRod,))
```

```
J = cursor.fetchall()
```

```
cursor.close()
```

```
conector.close()
```

```
return J
```

```
def ExibeJogos(Jogos):
```

```
ExibeLinha(“*** Rodada {} ***”.format(Jogos[0][1]), 64)
```

```
s = “{:<6}{:<16}{:<5}x{:>5}{:>16}”
```

```
JRod = []
```

```
for J in Jogos:
```

```
JRod.append(J[0])
```

```
if J[3] == J[5] == None:
```

```
ExibeLinha(s.format(J[0], J[2], “ “, “ “, J[4]), 64)
```

else:

ExibeLinha(s.format(J[0], J[2], J[3], J[5], J[4]), 64)

print("-" * LargTela)

return JRod

Quando o usuário digita o resultado de um jogo, a função `TrataEntrRodada` é chamada. Nessa função são feitas a interpretação e a validação do que foi digitado pelo usuário e, em seguida, a ação apropriada é tomada. É necessário lembrar que o usuário pode digitar qualquer coisa que queira, então, a validação deve ser criteriosa e a entrada só será aceita caso siga o padrão esperado. Caso não siga, a função retornará uma mensagem “Entrada Inválida”. Essa função retorna um string com uma mensagem caso a entrada seja inválida. Caso seja válida, retorna *None*.

A primeira providência dentro dessa função é remover eventuais espaços em branco. Como o padrão da entrada requer dados separados por vírgulas, é empregado o método `split()` do string para separação das partes. O `split()` deve gerar uma lista com dois (no caso de limpar o resultado) ou três (no caso de registrar resultado) elementos. Qualquer quantidade diferente invalida a entrada.

Se essa quantidade for dois, o primeiro elemento deve ser numérico (o nº do jogo) e o segundo elemento deve ser a palavra “LIMPA”. Caso isso ocorra, é chamada a função `LimpaJogo`; caso contrário, a entrada é inválida.

Se essa quantidade for 3, os três elementos devem ser numéricos (o nº do jogo, gols do time 1, gols do time 2). Nesse caso, optou-se por usar um bloco `try-except-finally` para tratar as conversões de string para inteiro. Caso algumas das conversões falhe, no bloco `except` retorna a mensagem de

entrada inválida. Se tudo correr bem, é feita a chamada à função *AtualizaJogo*, e o bloco *finally* garante o retorno de *None*.

Exemplo 10.19 Função *TrataEntrRodada*

```
def TrataEntrRodada(opc, JogosRodada):

    opc = opc.replace(" ", "") # remove espaços em branco

    L = opc.split(",")

    if len(L) != 2 and len(L) != 3:

        return "Entrada Invalida"

    if len(L) == 2:

        if L[0].isnumeric() and L[1] == "LIMPA":

            jogo = int(L[0])

            if jogo in JogosRodada:

                LimpaJogo(jogo)

            return None

        else:

            return "Jogo de outra rodada"

    else:

        return "Entrada Invalida"
```

```
if len(L) == 3:

    try:

        jogo = int(L[0])

        gols1 = int(L[1])

        gols2 = int(L[2])

        if jogo in JogosRodada:

            AtualizaJogo(jogo, gols1, gols2)

        else:

            return "Jogo de outra rodada"

    except:

        return "Entrada Inválida"

    finally:

        return None
```

Por fim, as funções `AtualizaJogo` e `LimpaJogo` são responsáveis por executar *updates* na tabela de jogos, de modo a registrar ou limpar as quantidades de gols marcados pelos times do jogo. A existência da função `LimpaJogo` justifica-se pelo fato de que não é raro o erro de lançamento por parte do usuário, e o programa deve contar com algum recurso que permita a correção.

É possível verificar na função supracitada que essas duas funções só são

chamadas se o nº do jogo digitado estiver contido na lista JogosRodada criada por ExibeJogos. Se isso não for feito, o usuário poderá alterar jogos (talvez acidentalmente) de rodadas diferentes da que está sendo visualizada, causando uma situação confusa e levando a erro de resultado.

Exemplo 10.20 Funções LimpaJogo e AtualizaJogo

```
def LimpaJogo(numjogo):

    global Torneio

    conector = sqlite3.connect(Torneio + ".db")

    cursor = conector.cursor()

    sql = """

    update jogos set gol1 = NULL, gol2 = NULL

    where numjogo = ?

    """

    cursor.execute(sql, (numjogo,))

    conector.commit()

    cursor.close()

    conector.close()

def AtualizaJogo(numjogo, gol1, gol2):

    global Torneio
```

```
conector = sqlite3.connect(Torneio + “.db”)
```

```
cursor = conector.cursor()
```

```
sql = “”
```

```
update jogos set gol1 = ?, gol2 = ?
```

```
where numjogo = ?
```

```
“”
```

```
cursor.execute(sql, (gol1, gol2, numjogo))
```

```
conector.commit()
```

```
cursor.close()
```

```
conector.close()
```

De volta à tela de gerenciamento do torneio, o Exemplo 10.21 exhibe a função `ExcluiTorneio`, que é chamada quando o usuário deseja descartar um torneio cadastrado. Essa função pede que a ação seja confirmada e, caso seja, elimina o nome do torneio do banco de dados central “torneios.db” e usa a função `os.remove` (a biblioteca `os` contém funções que permitem a interação com o sistema operacional) para excluir o arquivo de banco de dados do torneio.

Exemplo 10.21 Função `ExcluiTorneio`

```
def ExcluiTorneio(Torneio):
```

```
    print(“\n”*3)
```

```
ExibeLinha("Confirma Exclusão do Torneio " + Torneio, 40)
```

```
print("Opções: ")
```

```
print(" (C) para Confirmar")
```

```
print(" qualquer outra tecla para retornar")
```

```
opc = input("sua opção? >>> ")
```

```
opc = opc.upper()
```

```
if opc == "C":
```

```
conector = sqlite3.connect("torneios.db")
```

```
cursor = conector.cursor()
```

```
sql = "delete from torneios where nometorneio = " + \
```

```
Torneio + ""
```

```
print(Torneio)
```

```
print(sql)
```

```
Pausa("-" * 58)
```

```
cursor.execute(sql)
```

```
conector.commit()
```

```
cursor.close()
```

```

conector.close()

os.remove(Torneio + “.db”)

return True

else:

return False

```

E a última funcionalidade é a gravação do torneio em arquivo HTML. Para realizar essa tarefa foi implementada a função GravaHTML. Nessa tarefa serão utilizados HTML5 para exibição do conteúdo e CSS3 para formatação visual da página. O arquivo CSS3 está pronto e disponível (veja o Exemplo 10.24).

Dica

Se você não estiver familiarizado com HTML e CSS3, sugere-se uma visita ao portal disponível em: <www.w3schools.com>, no qual há um excelente conjunto de tutoriais sobre o assunto.

O arquivo HTML precisa ser gravado pelo programa. A técnica empregada foi a de deixar um arquivo HTML preparado previamente como um gabarito (veja com atenção o Exemplo 10.23). Nesse código há três elementos que serão substituídos. São eles:

<...NomeTorneio...>	Será substituído pelo nome do torneio
<...Hoje...>	Será substituído pela data em que o arquivo foi gravado.
<...Tabela...>	Será substituído pela tabela de classificação do torneio.

Quadro 10.7 Strings a serem substituídos no arquivo gabarito.html.

A ideia central é montar os strings que serão usados para as substituições listadas no Quadro 10.7, ler o arquivo “gabarito.html” em um objeto string do Python, usar o método `replace()` para efetivar as substituições e, por fim, gravar o HTML em um novo arquivo com o nome do torneio.

Na função `GravaHTML` foi utilizado o objeto `date` da biblioteca `datetime` para a montagem da data de criação do arquivo. O nome do torneio está disponível no objeto `Global Torneio`. A tabela HTML de classificação foi montada a partir do objeto `global Times`, que é construído usando-se a função `MontaClassificacao`, já descrita no Exemplo 10.15. Um comando `for` faz a iteração com o objeto `Times`, e para cada um de seus elementos é feita a montagem do string “tabela”.

No final dessa função é implementado o código que faz a leitura do gabarito, executa as substituições e grava o arquivo de saída “nome_do_torneio.html”.

Exemplo 10.22 Função `GravaHTML`

```
def GravaHTML():
```

```
    global Times, Torneio
```

```
    hoje = date.today()
```

```
    hoje = “{}/{}/{”}.format(hoje.day, hoje.month, hoje.year)
```

```
    MontaClassificacao()
```

```
    sfmt = “<tr><td>{</td><td style=’text-align:left’>{</td>” + \
```

```
    “<td>{</td>”*8 + “</tr>”
```

```

Pos = 1

tabela = ""

for time in Times:

    dados = (Pos,) + tuple(time)

    tabela = tabela + sfmt.format(*dados)

Pos += 1

arq = open("gabarito.html", "r", encoding="UTF-8")

html = arq.read()

arq.close;

html = html.replace("<...NomeTorneio...>", Torneio)

html = html.replace("<...Hoje...>", hoje)

html = html.replace("<...Tabela...>", tabela)

arq = open(Torneio+".html", "w", encoding="UTF-8")

arq.write(html)

arq.close()

```

Exemplo 10.23 Gabarito em HTML (arquivo “gabarito.html”)

```
<!DOCTYPE html>
```


<html>

<head>

<meta charset="UTF-8">

<title>Gerenciador de Torneios - <...NomeTorneio...></title>

<link rel="stylesheet" type="text/css" href="torneio.css">

</head>

<body>

<div class="titulo">

<div class='nometorneio'>Torneio: <...NomeTorneio...></div>

<div class='dettitulo'>Acompanhamento de Torneio</div>

<div class='dettitulo'>atualizado em: <...Hoje...></div>

</div>

<div class="separador">Classificação</div>

<div class="tabclassifica">

<table id="jogos">

<th style="width:10px">Pos</th>

<th style="width:150px">Time</th>

<th>PG</th><th>J</th><th>V</th><th>E</th><th>D</th>

<th>GP</th><th>GC</th><th>SG</th>

<...Tabela...>

</table>

</body>

</html>

Exemplo 10.24 Arquivo CSS utilizado na formatação do HTML

```
body{font-family:sans-serif;}
```

```
div.titulo{height:32px;width:1000px;padding:10px;
```

```
background-color:#DDEEFF;border:1px solid #AABBBF;}
```

```
div.nometorneio{font-size:28px;width:650px;float:left;}
```

```
div.dettitulo{font-size:14px;width:340px;height:20px;float:right;
```

```
text-align:right;font-weight:bold;}
```

```
div.separador{width:1000px;font-size:20px;text-align:center;
```

```
margin: 20px 0px 0px 0px;padding:0px 10px 0px 10px;}
```

```
div.tabclassifica{width:1000px;padding:10px;
```

```
border:1px solid #AABBBF;}
```

```
#jogos {font-family:Helvetica,sans-serif;border-collapse:collapse;
margin: 0 auto;text-align:center;}

#jogos td, #jogos th {border: 1px solid #D0D0D0;padding: 8px;}

#jogos tr:nth-child(even){background-color: #E0E0E0;}

#jogos tr:nth-child(odd){background-color: #FFFFFF;}

#jogos tr:hover {background-color: #D0D0D0;}

#jogos th {padding-top:12px;padding-bottom:12px;text-align:center;
background-color:#5599FF;color:white;width:50px;}
```

Exercícios propostos

1. Desenvolva uma nova funcionalidade que permita clonar um torneio já cadastrado, para ser repetido em uma nova temporada. Essa funcionalidade deve ler os nomes do novo torneio e do torneio a ser clonado. O novo torneio deve ser incluído no banco de dados “torneios.db”, e para gerar o banco de dados do torneio novo há duas possíveis opções:

- a)** gerar o novo banco com comandos sql;
- b)** copiar o arquivo .db usando função de cópia de arquivo da biblioteca denominada os (consulte a página da biblioteca em: <https://docs.python.org/3.6/library/os.html>).

2. Desenvolva uma nova funcionalidade que permita exibir em tela todas

as rodadas do torneio.

3. Crie uma segunda opção para o arquivo HTML, incluindo, após a tabela de classificação geral, todas as rodadas do torneio. As que já foram jogadas devem conter os resultados, e aquelas em aberto devem conter os nomes dos times sem o registro de gols.
4. Inclua na tabela de jogos um campo para a data da realização da partida e faça uma alteração no registro de jogo para incluir essa informação. Sugestão de validação: faça de um modo que o usuário tenha de digitar o nº do jogo, gols do time 1, gols do time 2, data (formato dd/mm/aaaa). Por exemplo: 14,3,1,17/10/2017 – significa que o jogo 14 teve resultado 3×1 e foi jogado em 10/outubro/2017.

Bibliografia

Chamberlin, D. et al. A history and evaluation of System R.

Communications ACM, v. 24, n. 10, 1981, 632-646. Disponível em:

<<http://doi.acm.org/10.1145/358769.358784>>. Acesso em: 23 out 2017.

DB-API 2.0 interface for SQLite database. **Python Software Foundation**,

2017. Disponível em: <<https://docs.python.org/3.6/library/sqlite3.html>>.

Acesso em: 15 out. 2017.

DODIS, Y. et al. Security analysis of pseudo-random number generators with input: /dev/random is not robust. In: CCS '13 – ACM SIGSAC

CONFERENCE ON COMPUTER & COMMUNICATIONS SECURITY,

2013, Berlin. **Proceedings...** Berlin: Association for Computing Machinery, 2013. p. 647-658.

GENERATE pseudo-random numbers. **Python Software Foundation**, 2017.

Disponível em: <<https://docs.python.org/3/library/random.html>>. Acesso em:

14 set. 2017.

GOODRICH, M. T.; TAMASSIA, R.; GOLDWASSER, M. H. **Data structures and algorithms in Python**. Hoboken: Wiley, 2013.

GUO, P. Python is now the most popular introductory teaching language at top U.S. universities. **Communications of the ACM**, 7 jul. 2014. Disponível em: <<https://cacm.acm.org/blogs/blog-cacm/176450-python-is-now-the-most-popular-introductory-teaching-language-at-top-u-s-universities/fulltext>>. Acesso em: 7 set. 2017.

LICENSE agreement for Python 3.6.3. **Python Software Foundation**, 2017. Disponível em: <<https://docs.python.org/3/license.html>>. Acesso em: 16 set. 2017.

LUTZ, M. **Learning Python**. 4. ed. Sebastopol: O'Reilly Media, 2009.

NEUMANN, J. Various techniques used in connection with random digits. **Applied Mathematics Series**, Washington, D.C., v. 12, p. 36-38, 1951.

PAYNE, J. **Beggining Python: using Python 2.6 and Python 3.1**. Indianapolis: John Wiley & Sons, 2010.

PYTHON DATA MODEL. **Python Software Foundation**, 2017. Disponível em: <<https://docs.python.org/3/reference/datamodel.html>>. Acesso em: 14 set. 2017.

PYTHON FORMATTED Output. **Python Course**. Disponível em: <www.python-course.eu/python3_formatted_output.php>. Acesso em: 16 ago. 2017.

Python Software Foundation, 2017. Disponível em: <<https://docs.python.org/3/glossary.html>>. Acesso em: 16 set. 2017.

PYTHON SOFTWARE FOUNDATION. Disponível em: <www.python.org>. Acesso em: 26 set. 2017.

RAMALHO, L. **Fluent Python**. Sebastopol: O'Reilly Media, 2015.

ROSSUM, G. **Computer programming for everybody, a funding proposal sent to DARPA**, 1999. Disponível em:

<[http://citeseerx.ist.psu.edu/viewdoc/download?](http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.123.6836&rep=rep1&type=pdf)

[doi=10.1.1.123.6836&rep=rep1&type=pdf](http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.123.6836&rep=rep1&type=pdf)>. Acesso em: 4 set. 2017.

_____. Python 3000 status update (long!). **Artima**, 29 jun. 2007. Disponível em: <<http://www.artima.com/weblogs/viewpost.jsp?thread=208549>>. Acesso em: 27 set. 2017.

ROSSUM, G. **PEP315**, 2003. Disponível em:

<www.python.org/dev/peps/pep-0315/>. Acesso em: 18 set. 2017.

SQLite. Disponível em: <<https://www.sqlite.org/>>. Acesso em: 25 set. 2017.

SQLite STUDIO. Disponível em: <<https://sqllitestudio.pl/index.rvt>>. Acesso em: 25 set. 2017.

TERMINOLOGIA Python em português: um guia para a tradução de termos específicos da linguagem Python para português. Disponível em: <<http://turing.com.br/pydoc/2.7/tutorial/TERMINOLOGIA.html>>. Acesso em: 16 set. 2017.

THE PYTHON LANGUAGE Reference. **Python Software Foundation**, 2017. Disponível em: <<https://docs.python.org/3/reference/index.html>>. Acesso em: 19 out. 2017.

THE PYTHON STANDARD LIBRARY – BUILT-IN **Types**. **Python Software Foundation**, 2017. Disponível em: <<https://docs.python.org/3/library/functions.html>>. Acesso em: 19 out. 2017.

THE PYTHON STANDARD LIBRARY – GENERAL **Index**. **Python Software Foundation**, 2017. Disponível em: <<https://docs.python.org/3/library/index.html>>. Acesso em: 19 out. 2017.

THE UNICODE CONSORTIUM. 2017. Disponível em: <www.unicode.org/>. Acesso em: 15 out. 2017.

UNICODE HOWTO. 2017. Disponível em:

<<https://docs.python.org/3/howto/unicode.html>>. Acesso em: 14 out. 2017.

VAZIRANI, U. V. Efficient and secure pseudo-random number generation.

In: 25TH Annual Symposium on Foundations of Computer Science, 25, 1984, Singer Island. **Proceedings...** Singer Island, 1984. p. 458-463.